

原版技术性开发系列教程

第 1 册

命令系统 Ciallo!

徐木弦 主编
(急招：编写组成员)

第一版序言

Minecraft 拥有多种多样的玩法，生存、PVP、PVE、模组、建筑、红石电路这些为众多玩家所熟知的玩法自成体系，玩家可以自由选择其中的某一方面深入研究。在这些玩法中，比较默默无闻的一种玩法可能便是广义上的命令，即包含了 MC-CMD（命令）、资源包和数据包的系统。

在没有系统地接触命令系统前，玩家对于这个系统不了解、甚至完全没有意识到这个系统的存在都是有可能的。对于一些仅对命令系统做出更新的快照版本，一些玩家可能认为它们是“无用的”，即使这些更新在大部分情况下丰富了命令系统，使得服务器的运营、冒险地图及原版模组的制作更方便。在较为早期的版本中，命令更多依靠命令方块运行，这使得一些玩家错误地认为冒险地图或原版模组中的机关是由纯粹的红石电路运行的，并将这些机关的巧妙之处完全归结于红石电路的运作而完全忽略命令的作用。

这是因为命令系统更多地呈现一种“在后台运行”的状态，玩家无法在明面上查看它们的运行逻辑，更多地只是感受它们带来的效果。其次，命令系统是一个“入门难、精通容易”的板块，学习的门槛较高，一些基本的概念，如权限等级、命名空间 ID、加载等级、坐标、目标选择器、JSON、NBT、记分板等，都是难于理解而对命令学习至关重要的知识点。再者，普通玩家在接触到命令系统时，更多地是熟悉诸如 `/kill`、`/gamemode` 这类带有简单参数的命令写法，而不会深究命令中各参数的意义，浅尝辄止的行为让玩家不大可能形成命令系统是一个完备体系的认识。最后，玩家在游戏时一般不会启用命令，而大部分命令所需的权限等级较高（即需要启用命令才能使用），因此一般玩家很少会与命令产生接触。

不过，命令系统应用范围极广，它对游戏的作用也是显而易见的，服务器的运行、冒险地图的制作、原版模组的制作均需要使用命令系统。对于游戏原有内容的修改和新内容的制作，如游戏外观的修改、粒子动画效果、自定义战利品、自定义物品等，均可以由命令系统来完成。命令系统提供了一个在不修改源代码情况下的开发环境，使得 Minecraft 几乎成为了一款“无限”的游戏，唯一有限的也许便是玩家的想象力。同时，命令系统也极大地丰富了游戏社区、论坛。

在历代版本更新中，命令系统也随之有过大大小小的改善。在早期的版本中，命令一般只能借由聊天栏和命令方块执行，而命令方块是一种红石元件，因此彼时命令的自动化执行对于红石电路的依赖性较强，只能将红石信号转化为命令执行的信号或条件。然而在后续的版本中，函数及数据包的问世使得命令能够脱离红石电路单独运行，并随着数据包的不断完善，命令系统的研究和应用则更加方便。一些冒险地图真正做到了零命令方块化，命令的执行全部转交给数据包完

成。因此命令方块及红石电路的教程则放在了附录中,不使用数据包仅使用命令方块时可以参阅这些内容。

对于这样一个较为完备的体系,笔者在前人既有教程的基础上,理清思路,加以整理,为之编写完整的命令教程。在内容安排上,优先讲述基本概念,如权限等级、命令执行上下文、命名空间 ID、数据包标签、参数、区块及加载等级等,这些概念是能够执行命令的最基本条件。在绪论之后是命令系统的五大重要基础板块:坐标、目标选择器、JSON、NBT 和记分板,这些内容是需要熟练掌握的基本功。在第七章,我们主要学习命令 `/execute`,这条命令能够修改命令执行上下文,一般可以认为是整个命令系统中最强大的命令。一些零散的命令则放到了第八章中讲解。资源包和数据包则被安排到了其他教程中,这些内容是命令系统的进阶部分,一定程度上脱离了狭义的命令体系,却也是命令系统的精华部分。学有余力的读者可进行《资源包》和《数据包》两本教程的学习。

在系统学习命令系统前,读者需要了解到的一点是,其他玩法领域可能更多地追求稳定的游戏环境,因此可能会选择旧的游戏版本。而命令系统随着版本更新不断完善,使得命令玩家不自觉地追求新版本,因此有关命令的讨论一般都基于最新版游戏。一些问题,诸如“在 1.8 版本中这个命令怎么写”,则可能得不到答复,因为命令系统在扁平化时经历了巨大的改动,使得改动前后几乎是两个完全不同的系统。同时,不同平台的命令系统也是不互通的,本教程仅适用于 Java 版,部分内容与基岩版不同,因此无法作为基岩版命令教学的参考。

那么在学习命令之前需要掌握哪些预备知识呢?首先需要对原版的 Minecraft 有足够的了解,因为命令基本上涉及了游戏中的方方面面。其次可以适当学习一些英语,命令的参数中有大量的英语单词,如果熟悉它们的意思将会极大提升命令编写的效率。本教程在论述的过程中,会经常使用高等数学、线性代数和概率论与数理统计的语言,读者可先行了解与命令系统紧密相关的一些数学知识,如向量代数与空间解析几何、矩阵与变换、随机事件及概率分布等。计算机图形学的一些内容也包括在内。此外,大家还可以提前学习一些编程思想,建立严密的逻辑思维能力,懂得如何指挥电脑(游戏)进行工作。没有相关编程基础并不意味着完全不能学好命令,编程思维也可以在命令学习的过程中逐步建立。

本教程仅作为基础教程使用,即仅介绍命令系统中所有的基本概念,并在此基础上进行一定的应用、扩展,其中应用层面主要面向冒险地图和原版模组的制作。一些较为高级的算法则不在本教程范围之内。其中带“*”号的是选择性学习内容,它们所述的一些是不常用的内容,一些则是《资源包》或《数据包》两本教程的相关内容,想进修《资源包》和《数据包》的读者需注意这些带“*”号知识点。

限于笔者的知识水平,本教程必有一定疏漏之处,望广大读者斧正!

编者

2024 年 1 月 24 日

第二版序言

自 2023 年发布的 1.19.4 起，Minecraft 的技术性开发板块进入了又一个新时代，这可谓是自 2012 年骇人更新加入命令方块、2018 年水域更新扁平化以来第三次革命性的巨变。这次“巨变”的主要体现在于：逐步开放了很多 API，即将很多固有注册表转为可写注册表，使得原先需要大量命令模拟的效果用数据包自定义即可，这无疑大大省去了技术性开发的成本。从这两年 Minecraft Java 版的更新迭代可以看出，Minecraft 的开发重点逐渐地由加入新的实质性游戏内容转为对数据包、资源包的支持。新游戏内容更新的占比下降，技术性更新的占比上升。数据包、资源包的版本号更迭已由先前的一个大版本一次变为一个快照一次，迭代速度越来越快。以数据包为例，2022 年数据包的版本号为 10，彼时距数据包的加入仅过了四年多；而截止至 2024 年底，数据包的版本号已达到了 61。

许多玩家对自 1.17 以来的 Minecraft 开发极不满意，有些人认为 Minecraft 近几年并没有革命性的更新出现，每个版本的更新内容也极少，对游戏深度的挖掘不够，“敷衍了事”。对于每个快照更新介绍中大篇幅的技术性更新内容，大量玩家则视若无睹。这个现象直接反映了普通玩家对技术性开发这一板块认识的欠缺，正如第一版序言所说，“玩家对于这个系统不了解、甚至完全没有意识到这个系统的存在”。为此，对玩家进行技术性开发板块知识的普及很有必要。

不得不承认的是，技术性开发确实是一种门槛较高的玩法，其学习成本较高，容错率低，开发周期长，且开发过程不可见，因此难于在社区中引起较大的讨论度。目前，各大社区平台仍然缺乏专门的技术性开发论坛，现有的教程也零碎且不完整。社区的讨论主要集中于原版游戏内容、Mods 玩法等，鲜有技术性开发有关的资料，这使其搜索难度较大。技术性开发玩家通常“单打独斗”，缺少交流，进一步限制了有开发想法的玩家学习进步。为此，为已入门的技术性开发玩家编写系统性的、完整的教程很有必要。

原版技术性开发的命令、资源包、数据包三个板块在本系列教程中分作三部分别讲述。对于其中的一些基本概念，如注册表、坐标、目标选择器、文本组件、NBT、记分板、属性、存档格式等，为了便于教程编纂，将其放在《命令系统》一书中与命令语法一起说明。命令是数据包的一个子集，用于函数中，数据包为它提供了程序化运行的环境。在早先的版本中，命令通常是红石电路的一个子集，用于命令方块中，由红石信号控制命令的执行。虽然现在主流的开发环境是数据包，但仍有玩家使用命令方块红石电路进行开发，因此本系列教程中仍保留对命令方块红石电路的讲解，《命令系统》书中的一些例子则提供了使用数据包和使用命令方块红石电路的两种解

法，但侧重于使用数据包，而在《数据包》中则仅提供使用数据包的解法。资源包是相对较独立的一个板块，其内容中仅少部分与数据包有关联，对美术设计的要求较高，部分开发资源包的玩家专注于资源包，不会去开发数据包。然而在开发实践中数据包与资源包的联系却越来越紧密，仅依靠数据包能够实现的效果有限，与资源包配合开发能得到更好的效果。

这一版的《命令系统》教程相较上一版而言，对调了一些章节使得教程整体的逻辑更合理。同时新增了一些章节，新增的内容大多拆分自原本的章节，但对这些被拆分的内容有了更详尽的叙述。原本教程中零散的内容在这一版中均尽可能地归到一类下。经过这些变动后，教程的整体章节较先前的版本有了很大的不同。产生这些变动的主要原因是文本组件的存储格式由 JSON 变为了 SNBT，即使在数据包中文本组件仍使用 JSON 格式。从教程逻辑上讲，文本组件应合并至“NBT 格式”一章，但上一版的“NBT 格式”一章已有 11 个小节，内容量极其庞大，若再塞入文本组件的全部内容，只会使“NBT 格式”一章显得更臃肿。况且属性、状态效果这些内容依照存储格式也归到了实体格式或物品堆叠组件格式之下，将这些归类的内容全部放入“NBT 格式”一章更不适宜。于是，原本经典的“绪论——坐标——目标选择器——原始 JSON 文本——NBT 格式——记分板——命令 `/execute`——杂项命令”的教程章节顺序已不适用。经过考虑，这一版的教程章节变动安排如下：

1. “绪论”、“坐标与区块”、“UUID 与目标选择器”三章的顺序不变，仅调整这三章内部小节的顺序及编排。
2. 删去原本的“NBT 格式”一章，并将其重命名为“NBT 与 JSON 格式”，新的章节保留原本“NBT 格式”一章的概述、NBT 路径、命令 `/data` 的语法。同时将原本“原始 JSON 文本”一章的“JS 对象表示法”一节移入新的“NBT 与 JSON 格式”。随后将概述一节中涉及 SNBT 和 JSON 相互转换的内容移出，单独设置一个小节。
3. 将新的“NBT 与 JSON 格式”提前至第四章，将“原始 JSON 文本”改名为“文本组件”并后移至第五章。“文本组件”一章将同时讲解其 SNBT 和 JSON 形式。
4. 对于从原本“NBT 格式”一章中移出的其他内容，结合属性、状态效果，单独开设新的一章“存档格式”作为第六章，同时合并原本附录 II “游戏文件”的内容至这一章的概述小节。新的“存档格式”包含命令存储格式、区域文件格式、方块实体格式、实体格式、属性格式、状态效果格式、标记格式、展示实体格式、交互实体格式、玩家格式和物品堆叠组件格式。
5. “记分板”、“命令的执行”、“其他分类的命令”分别顺延至第七、八、九章。“命令的执行”更名为“命令 `/execute`”。
6. 由于章节合并，“其他分类的命令”中“状态效果”、“属性”两节被删除。
7. 删除原本的附录 II “游戏文件”。

受限于编者的知识水平，本教程一定有不足之处，望广大读者斧正！

编者

2025 年 2 月 7 日

目 录

第一章 绪论	1
1.1 注册表与数据值	2
1.1.1 注册表 *	2
1.1.2 扁平化 *	8
1.1.3 命名空间 ID	9
1.1.4 数据包标签	12
1.2 命令	12
1.2.1 命令参数	12
1.2.2 命令的输入	15
1.2.3 权限等级与限制条件	16
1.2.4 命令的解析 *	18
1.2.5 命令上下文	19
1.3 JSON 格式	21
1.3.1 JSON 数据类型	21
1.3.2 JSON 的转义序列	24
1.4 游戏文件	26
1.4.1 常用文件格式	26
1.4.2 .minecraft 文件夹 *	27
1.5 数据包	33
1.5.1 数据包的基本结构	36
1.5.2 实验性内容 *	45
1.5.3 数据包标签定义格式	46
1.6 资源包	49
1.6.1 资源包的基本结构	49
1.6.2 GPU 警告列表 *	54
1.6.3 地区合规性警告 *	55
1.7 游戏机制	56

1.7.1	端	56
1.7.2	游戏刻	58
1.7.3	区块加载	61
1.7.4	区块刻 随机刻 计划刻 红石刻	66
1.7.5	方块	67
1.7.6	实体	69
1.7.7	难度 游戏模式 游戏规则	72
1.8	服务器管理	76
1.8.1	server.properties *	76
1.8.2	仅在多人游戏可用命令	80
1.8.3	其他与服务器相关的命令	83
	第一章思考题与习题	83
第二章	坐标	87
2.1	坐标系与坐标	88
2.1.1	坐标的命令参数类型	88
2.1.2	相对坐标 局部坐标	91
2.1.3	调试屏幕的坐标	92
2.2	朝向	93
2.2.1	偏航角	93
2.2.2	俯仰角	94
2.2.3	命令/rotate	95
2.2.4	朝向与局部坐标的关系 *	97
2.3	坐标参数在命令中的应用	100
2.3.1	使用方块坐标的命令	100
2.3.2	使用三维坐标的命令	106
2.3.3	使用二维坐标的命令	107
2.4	区块	111
2.4.1	区块坐标和区段坐标	111
2.4.2	命令/forceload	113
	第二章思考题与习题	114
第三章	UUID 与目标选择器	117
3.1	UUID	118
3.2	目标选择器	119
3.2.1	目标选择器变量	119
3.2.2	目标选择器参数	120
	第三章思考题与习题	132
第四章	NBT 格式	134
4.1	概述	135
4.1.1	SNBT 的概念	135
4.1.2	数据类型与数据树	135
4.1.3	SNBT 操作	143
4.1.4	SNBT 转换为 NBT *	143

4.1.5	二进制格式 *	144
4.2	测试 NBT 标签	147
4.3	NBT 路径	150
4.3.1	节点	150
4.3.2	路径	154
4.4	命令/data 的语法	155
4.4.1	get 子命令	156
4.4.2	remove 子命令	157
4.4.3	merge 子命令	157
4.4.4	modify 子命令	157
4.4.5	命令/data 的语法树	160
4.5	NBT 与 JSON	161
4.5.1	NBT 和 JSON 格式的转换	161
4.5.2	SNBT 和 JSON 的嵌套	163
	第四章思考题与习题	165
第五章	文本组件	168
5.1	概述	169
5.2	文本组件类型	172
5.2.1	纯文本组件	174
5.2.2	翻译文本组件	174
5.2.3	按键绑定组件	177
5.2.4	记分板分数组件	180
5.2.5	实体名称组件	181
5.2.6	NBT 组件	181
5.2.7	精灵图组件	182
5.2.8	组件解析	185
5.3	文本组件样式	187
5.3.1	样式与字体	187
5.3.2	格式化代码	190
5.3.3	交互事件	193
5.4	子组件	198
5.4.1	数组	198
5.4.2	字段 extra	200
5.4.3	继承	201
5.4.4	组件序列化	204
5.5	应用实例	204
5.6	聊天类型	207
5.6.1	基本聊天类型	207
5.6.2	聊天类型的定义	209
	第五章思考题与习题	211
第六章	存档格式	213
6.1	概述	214

6.2	存档数据	217
6.2.1	命令存储	218
6.2.2	Boss 栏	221
6.2.3	随机序列	223
6.2.4	秒表	225
6.2.5	天气	226
6.3	维度与区域文件	227
6.3.1	Anvil 文件	228
6.3.2	维度数据 *	230
6.4	方块实体	236
6.4.1	方块实体共通标签 *	236
6.4.2	方块实体数据的应用实例	236
6.5	实体	241
6.5.1	处理实体的命令	241
6.5.2	实体共通标签	243
6.5.3	实体数据的应用实例	245
6.6	状态效果	251
6.7	属性	254
6.7.1	属性修饰符	261
6.7.2	命令/attribute 的语法	264
6.7.3	属性 NBT 格式	265
6.8	技术性实体	266
6.8.1	标记	266
6.8.2	展示实体	267
6.8.3	交互实体	272
6.9	玩家	272
6.10	物品堆叠	272
6.11	数据组件	272
6.12	教程：NBT Studio 的使用 ×	272
第七章	记分板	273
7.1	队伍与标签	273
7.2	记分板的基本概念	273
7.2.1	记分板 NBT 格式	273
7.3	记分板命令 触发器	273
7.3.1	触发器	273
第八章	命令/execute	274
附录 I	命令方块与红石电路	275
I.1	红石电路基础	276
I.1.1	红石信号	276
I.1.2	充能与激活理论	276
I.1.3	红石元件	277
I.2	结构方块	279

附录 II	数据库	280
II.1	数据包和资源包版本号	280
II.2	方块状态	292
II.3	方块实体数据	292
II.4	实体数据	292
II.5	数据包标签	292
附录 III	索引	293
III.1	本书命令	293
III.2	专有名词（汉语拼音顺序）	296
III.3	重要方法	301
参考文献		302

第一章 绪论

原版技术性开发, Minecraft Wiki 称为“Java 版可自定义内容”, 是由命令、资源包、数据包及相关的组件附件组合成的一个板块。技术性开发成果丰富, 这些成果即是社区玩家常用的 Mods、冒险地图、数据包、资源包、服务器等。Minecraft 的技术性开发大致分为 Mods 开发和原版开发, 其区别在于是否对游戏的源代码进行了修改。

本系列教程针对的是原版技术性开发, 这一部分玩家的工作方向通常为制作冒险地图、开发原版模组、制作资源包或者管理服务器。

1.1 注册表与数据值

Minecraft 有许多不同的游戏资源，如草方块、石头、箭、铁锹、猪等，对这些游戏资源进行分类，可以将草方块、石头分为方块，箭、铁锹分为物品，猪划分至实体。方块、物品、实体显然是区分这些游戏资源的“大类”。**注册表 (Registry)** 就是对不同资源进行分类管理的机制。除分类管理外，还需要给予每种资源一个独特的“身份证”，目的是与别的资源区分开来，唯一地映射到注册表内的给定值。这些“身份证”被称为游戏资源的数据值，或称 ID。

1.1.1 注册表 *

注册表可分为以下两类：**固有注册表 (Built-in registry)** 和**可写注册表 (Writable registry)**。无论资源位于什么类型的注册表，**游戏都只会识别已被注册的资源**。

下面列举了 Minecraft 中所有的资源类型：

一、固有注册表

固有注册表存储硬编码的游戏内容，除非修改源代码，否则其中的内容不可更改。

表 1.1 固有注册表

注册表	资源类型	数据包路径	默认值
ACTIVITY	生物 AI	activity	
ATTRIBUTE	属性	attribute	
ATTRIBUTE_TYPE	环境属性类型	attribute_type	
BIOME_SOURCE	生物群系源	worldgen/biome_source	
BLOCK	方块	block	air
BLOCK_ENTITY_TYPE	方块实体类型	block_entity_type	furnace
BLOCK_PREDICATE_TYPE	方块谓词类型	block_predicate_type	
BLOCK_STATE_PROVIDER_TYPE	方块状态提供器类型	worldgen/block_state_provider_type	
BLOCK_TYPE	方块类型	block_type	
CARVER	雕刻器类型	worldgen/carver	
CHUNK_GENERATOR	区块生成器类型	worldgen/chunk_generator	
CHUNK_STATUS	区块状态	chunk_state	empty
COMMAND_ARGUMENT_TYPE	命令参数类型	command_argument_type	
CONSUME_EFFECT_TYPE	消耗使用效果类型	consume_effect_type	
CREATIVE_MODE_TAB	创造标签页	creative_mode_tab	
CUSTOM_STAT	统计信息	custom_stat	
DATA_COMPONENT_PREDICATE_TYPE	数据组件谓词类型	data_component_predicate_type	

(续表)

注册表	资源类型	数据包路径	默认值
DATA_COMPONENT_TYPE	数据组件类型	data_component_type	
DEBUG_SUBSCRIPTION	调试订阅	debug_description	
DECORATED_POT_PATTERN	饰纹陶罐图案类型	decorated_pot_pattern	
DENSITY_FUNCTION_TYPE	密度函数类型	worldgen/density_function_type	
DIALOG_ACTION_TYPE	对话框操作类型	dialog_action_type	
DIALOG_BODY_TYPE	对话框主体类型	dialog_body_type	
DIALOG_TYPE	对话框类型	dialog_type	
ENCHANTMENT_EFFECT_COMPONENT_TYPE	魔咒效果组件类型	enchantment_effect_component_type	
ENCHANTMENT_ENTITY_EFFECT_TYPE	魔咒实体效果类型	enchantment_entity_effect_type	
ENCHANTMENT_LEVEL_BASED_VALUE_TYPE	魔咒等级依赖函数类型	enchantment_level_based_value_type	
ENCHANTMENT_LOCATION_BASED_EFFECT_TYPE	魔咒位置依赖函数类型	enchantment_location_based_effect_type	
ENCHANTMENT_PROVIDER_TYPE	魔咒提供器类型	enchantment_provider_type	
ENCHANTMENT_VALUE_EFFECT_TYPE	魔咒值效果类型	enchantment_value_effect_type	
ENVIRONMENT_ATTRIBUTE	环境属性	environment_attribute	
ENTITY_SUB_PREDICATE_TYPE	实体子谓词类型	entity_sub_predicate_type	
ENTITY_TYPE	实体类型	entity_type	pig
FEATURE	地物类型	worldgen/feature	
FEATURE_SIZE_TYPE	树木生成的最小空间要求类型	worldgen/feature_size_type	
FLOAT_PROVIDER_TYPE	浮点数提供器类型	float_provider_type	
FLUID	流体类型	fluid	empty
FOLIAGE_PLACER_TYPE	树叶放置器类型	worldgen/foilage_placer_type	
GAME_EVENT	游戏事件	game_event	step
GAME_RULE	游戏规则	game_rule	step
HEIGHT_PROVIDER_TYPE	高度提供器类型	height_provider_type	
INCOMING_RPC_METHOD	服务端管理协议中的请求方法	incoming_rpc_methods	
INPUT_CONTROL_TYPE	对话框输入控件类型	input_control_types	
INT_PROVIDER_TYPE	整数提供器类型	int_provider_type	
ITEM	物品	item	air
SLOT_SOURCE_TYPE	槽位源类型	slot_source_type	
LOOT_CONDITION_TYPE	战利品表谓词类型	loot_condition_type	
LOOT_FUNCTION_TYPE	物品修饰器类型	loot_function_type	

(续表)

注册表	资源类型	数据包路径	默认值
LOOT_NBT_PROVIDER_TYPE	战利品表相关 NBT 源类型	loot_nbt_provider_type	
LOOT_NUMBER_PROVIDER_TYPE	值提供器类型	loot_number_provider_type	
LOOT_POOL_ENTRY_TYPE	抽取项类型	loot_pool_entry_type	
LOOT_SCORE_PROVIDER_TYPE	分数提供器类型	loot_score_provider_type	
MAP_DECORATION_TYPE	地图图标类型	map_decoration_type	
MATERIAL_CONDITION	地表规则条件	worldgen/material_condition	
MATERIAL_RULE	地表规则	worldgen/material_rule	
MEMORY_MODULE_TYPE	生物记忆	memory_module_type	dummy
MENU	屏幕类型	menu	
MOB_EFFECT	状态效果	mob_effect	
NUMBER_FORMAT_TYPE	记分板分数显示样式类型	number_format_type	
OUTGOING_RPC_METHOD	服务端管理协议中的通知方法	outgoing_rpc_methods	
PARTICLE_TYPE	粒子类型	particle_type	
PLACEMENT_MODIFIER_TYPE	放置修饰器类型	worldgen/placement_modifier_type	
PERMISSION_CHECK_TYPE	命令权限检查类型	permission_check_type	
PERMISSION_TYPE	命令权限类型	permission_type	
POINT_OF_INTEREST_TYPE	兴趣点类型	point_of_interest_type	
POOL_ALIAS_BINDING	模板池映射	worldgen/pool_alias_binding	
POSITION_SOURCE_TYPE	位置源类型	position_source_type	
POS_RULE_TEST	位置规则测试类型	pos_rule_test	
POTION	药水效果	potion	
RECIPE_BOOK_CATEGORY	配方书分类标签	recipe_book_category	
RECIPE_DISPLAY	预览配方类型	recipe_display	
RECIPE_SERIALIZER	配方序列化/反序列化器	recipe_serializer	
RECIPE_TYPE	配方类型	recipe_type	
REGISTRY	注册表	root	
ROOT_PLACER_TYPE	树根放置器类型	worldgen/root_placer_type	
RULE_TEST	规则测试	rule_test	
RULE_BLOCK_ENTITY_MODIFIER	方块实体数据修饰器	rule_block_entity_modifier	
SENSOR_TYPE	感受器类型	sensor_type	
SLOT_DISPLAY	预览槽位类型	slot_display	

(续表)

注册表	资源类型	数据包路径	默认值
SLOT_SOURCE_TYPE	槽位源类型	slot_source_type	
SOUND_EVENT	声音事件	sound_event	
SPAWN_CONDITION_TYPE	通用变种选择器类型	spawn_condition_type	
STAT_TYPE	统计类型	stat_type	
STRUCTURE_PIECE	结构片段	worldgen/structure_piece	
STRUCTURE_PLACEMENT	结构放置方式	worldgen/structure_placement	
STRUCTURE_POOL_ELEMENT	结构模板池元素	worldgen/structure_pool_element	
STRUCTURE_PROCESSOR	结构处理器	worldgen/structure_processor	
STRUCTURE_TYPE	结构类型	worldgen/structure_type	
TEST_ENVIRONMENT_DEFINITION_TYPE	测试环境类型	test_environment_definition_type	
TEST_FUNCTION	硬编码测试函数	test_function	
TEST_INSTANCE_TYPE	测试实例类型	test_instance_type	
TICKET_TYPE	加载标签及计算标签类型	ticket_type	
TREE_DECORATOR_TYPE	树额外装饰器类型	worldgen/tree_decorator_type	
TRIGGER_TYPE	触发器类型	trigger_type	
TRUNK_PLACER_TYPE	树干放置器类型	worldgen/trunk_placer_type	
VILLAGER_PROFESSION	村民职业	villager_profession	none
VILLAGER_TYPE	村民类型	villager_type	plain

二、可写注册表

可写注册表允许数据包通过数据反序列化器向其中添加自定义（或称数据驱动）的游戏内容。

表 1.2 可写注册表

注册表	资源类型	数据包路径
ADVANCEMENT	进度	advancement
BANNER_PATTERN	旗帜图案	banner_pattern
BIOME	生物群系	worldgen/biome
CAT_SOUND_VARIANT	猫音效变种	cat_sound_variant
CAT_VARIANT	猫的变种	cat_variant
CHAT_TYPE	聊天类型	chat_type
CHICKEN_SOUND_VARIANT	鸡音效变种	chicken_sound_variant
CHICKEN_VARIANT	鸡的变种	chicken_variant
CONFIGURED_CARVER	已配置的雕刻器	worldgen/configured_carver

(续表)

注册表	资源类型	数据包路径
CONFIGURED_FEATURE	已配置的地物	worldgen/configured_feature
COW_SOUND_VARIANT	牛音效变种	cow_sound_variant
COW_VARIANT	牛的变种	cow_variant
DAMAGE_TYPE	伤害类型	damage_type
DENSITY_FUNCTION	密度函数	worldgen/density_function
DIALOG	对话框	dialog
DIMENSION	维度	dimension
DIMENSION_TYPE	维度类型	dimension_type
ENCHANTMENT	魔咒	enchantment
ENCHANTMENT_PROVIDER	魔咒提供器	enchantment_provider
FLAT_LEVEL_GENERATOR_PRESET	超平坦世界生成预设	worldgen/flat_level_generator_preset
FROG_VARIANT	青蛙的变种	frog_variant
INSTRUMENT	山羊角乐器	instrument
ITEM_MODIFIER	物品修饰器	item_modifier
JUKEBOX_SONG	唱片机曲目	jukebox_song
LEVEL_STEM	维度	dimension
LOOT_TABLE	战利品表	loot_table
MULTI_NOISE_BIOME_SOURCE_PARAMETER_LIST	多噪声参数列表	worldgen/multi_noise_biome_source_parameter_list
NOISE	噪声	worldgen/noise
NOISE_SETTINGS	噪声设置	worldgen/noise_settings
PAINTING_VARIANT	画的变种	painting_variant
PIG_SOUND_VARIANT	猪音效变种	pig_sound_variant
PIG_VARIANT	猪的变种	pig_variant
PLACED_FEATURE	已放置的地物	worldgen/placed_feature
PREDICATE	谓词	predicate
PROCESSOR_LIST	处理器列表	worldgen/processor_list
RECIPE	配方	recipe
STRUCTURE	已配置的结构地物	worldgen/structure
STRUCTURE_SET	结构集	worldgen/structure_set
TEMPLATE_POOL	结构池	worldgen/template_pool
TEST_ENVIRONMENT	测试环境	test_environment
TEST_INSTANCE	测试实例	test_instance
TIMELINE	时间线	timeline
TRADE_SET	交易集	trade_set

(续表)

注册表	资源类型	数据包路径
TRIAL_SPAWNER_CONFIG	试炼刷怪笼配置	trial_spawner
TRIM_MATERIAL	盔甲纹饰材料	trim_material
TRIM_PATTERN	盔甲纹饰图案	trim_pattern
VILLAGER_TRADE	村民交易	villager_trade
WOLF_SOUND_VARIANT	狼音效变种	wolf_sound_variant
WOLF_VARIANT	狼的变种	wolf_variant
WORLD_CLOCK	世界时钟	world_clock
WORLD_PRESET	世界预设	worldgen/world_preset
ZOMBIE_NAUTILUS_VARIANT	僵尸鹦鹉螺变种	zombie_nautilus_variant

三、不属于任何注册表的游戏资源

有一些游戏资源不属于任何注册表, 这些资源包括数据包内的函数、结构模板以及资源包内的所有内容。这些资源类型中部分都与可写注册表的性质类似, 即可以自定义写入资源; 部分则不能增添新的资源, 但可以修改已有资源的配置文件。

表 1.3 其他游戏资源

类别	游戏资源	说明
数据包内容	函数	可写, <code>function</code> 路径下的内容
	结构模板	可写, <code>structure</code> 路径下的内容
资源包内容	纹理图集	位于资源包内路径 <code>atlases</code>
	方块状态	位于 <code>blockstates</code>
	纹饰图案	位于 <code>equipment</code>
	字体	可写, 位于 <code>font</code>
	物品模型映射	位于 <code>items</code>
	模型	包括方块模型和物品模型, 位于 <code>models</code>
	粒子	位于 <code>particles</code>
	着色器	可写, 位于 <code>shaders</code>
	后处理管线	位于 <code>post_effect</code>
	声音	可写, 位于 <code>sounds</code>
	纹理	可写, 位于 <code>textures</code>
属性修饰符		可写, 存储于所属物品的堆叠组件内
命令存储		可写, 存储于存档文件夹中的  <code>data > <命名空间> > command_storage.dat</code>
Boss 栏		可写, 存储于存档文件夹中的  <code>data > minecraft > custom_boss_events.dat</code>
随机序列		可写, 存储于存档文件夹中各自维度的  <code>data > minecraft > random_sequences.dat</code> 文件内

(续表)

类别	游戏资源	说明
秒表		可写，存储于存档文件夹中各自维度的  <code>data > minecraft > stopwatches.dat</code> 文件内

1.1.2 扁平化*

Minecraft 的历次版本更新都会对某一些特定的系统进行优化和更改，比如：战斗更新对 PVP 机制进行了颠覆性的更改，使得 1.9 之前和之后的 PVP 是两个完全不同的系统。命令系统也经历过类似的大幅度更改，这便是随着水域更新进行的**扁平化（The flattening）**。

在 Minecraft 开发之初，由于游戏资源的数量有限，只需要使用 1 字节就可以设置所有游戏资源的 ID。在历次版本更新中，Minecraft 的方块、物品数量越来越多，特别是自缤纷更新以来，方块的数量呈爆炸式增长。在扁平化之前，为了应对这些不断增多的游戏资源，一种解决办法是将一大类全部收归到某一个特定的 ID 中，用这个 ID 来表示这一类方块，然后在后面附加一个 Damage 值来表示这一类方块中的某一种。比如花岗岩属于石头一类，石头的数字 ID 为 1，而花岗岩的 Damage 值为 1，所以在旧版本中给予玩家一块花岗岩的命令为：

```
give @p 1 1 1
```

1.1

可以看到这条命令中有三个参数，第一个 `1` 为石头的 ID，第二个 `1` 为数量，第三个 `1` 为 Damage 值。这种表示方式的底层逻辑是：在访问花岗岩时，必须先访问上一级的数字 ID，再访问 Damage 值，如此才能映射至花岗岩这个值。

缤纷更新做了一个小修改：即启用了部分英文 ID，于是在 1.8 中给予玩家一块花岗岩的命令变为：

```
give @p stone 1 1
```

1.2

但是缤纷更新做出的这种更改是不完全的，虽然在命令的主体部分将数字 ID 替换成了英文 ID，但是方块的 Damage 值仍然存在，这种一级 ID——Damage 值的映射方式没有改变。

时间来到了 2017 年，水域更新加入了大量的游戏内容，原先的映射方式已不能适应新版本。于是，**Java 版 1.13 的更新基本上删除了所有的数字 ID，使得每一个游戏资源都有其独立的英文 ID。同时也删除了用 Damage 值映射方块的办法，去除了中间层，使得资源的映射方式只需要一个键名即可，这一过程便被称作“扁平化”^①**。比如，在 1.13 中给予玩家一块花岗岩的命令为：

```
give @p granite 1
```

1.3

其中参数 `granite` 为花岗岩的键名，`1` 为物品数量。扁平化对所有需要 ID 的对象都进行了修改，包括但不限于方块、物品、实体、生物群系、粒子、声音事件和画。其具体内容可分为以下几类：

1. 拆分

拆分是最能体现扁平化过程的一类。此举移除了用于指定大类下某一种方块的 Damage 值，使得一类方块下的每一种方块都有其独立的 ID。举例：`stone` 一类共有七种不同的方块：石头、花

^① 并非所有数字 ID 都被移除，至今 Minecraft 仍保留了部分需要使用数字 ID 的地方。

岗岩、磨制花岗岩、闪长岩、磨制闪长岩、安山岩和磨制安山岩，分别对应 0~6 的 Damage 值。拆分后这七种方块都被给予了独立的 ID: `stone`、`granite`、`polished_granite`、`diorite`、`polished_diorite`、`andesite` 和 `polished_andesite`。

2. 重命名

顾名思义，该类即对原有的英文 ID 进行重命名。有相当一部分重命名是为了迎合方块或物品的英文名称。举例：草方块在扁平化前的 ID 为 `grass`，扁平化后被重命名为 `grass_block`。

3. 重新分类

这种操作常见于台阶和由双台阶组成的完整方块。扁平化前的台阶和双台阶有两种不一样的 ID，扁平化后取消双台阶的 ID，并对台阶的下属分类进行拆分，同时取消相关的 Damage 值。举例：双木台阶被取消，统一更换为木制台阶，同时 ID 拆分为橡木、云杉、白桦、丛林木、金合欢和深色橡木。

4. 合并

合并常见于有不同方块状态的方块。举例：燃烧的熔炉和熔炉有不同的 ID，现合并为熔炉一种，同时将是否燃烧设定为方块状态。

1.1.3 命名空间 ID

游戏资源的指定有一个前提是这些对象的 ID 相互之间不能混淆。数字 ID 及使用 Damage 值作区分的指定方法能够避免对象之间的冲突，但扁平化后这种指定方法便无效了。为此在当前的版本中统一使用 **（赋）命名空间 ID（Namespaced identifier）** 来映射注册表内的值。

命名空间 ID，又称 **（赋）命名空间标识符、资源路径（Resource location）、资源标识符（Resource identifier）** 或 **命名空间字符串（Namespaced string）**，是字符串化的映射方式。无论命名空间 ID 用于映射何种对象，它们都具有同一的表达方式：

```
<namespace>:<path>
```

1.4

其中的参数：<namespace>——命名空间。

<path>——路径。

在写法上，除用于分割命名空间和路径的冒号 `:` 外，其中所有的字符都只能为合法字符。合法字符包含以下几类：

1. 数字：`0123456789`；
2. 小写字母：`abcdefghijklmnopqrstuvwxyz`；
3. 下划线：`_`；
4. 连字符：`-`；
5. 点：`.`。

小提示

大写字母在命名空间 ID 中为非法字符！

除此之外所有的字符均为非法字符，包括汉字、平假名、片假名、西里尔字母、希腊字母、制表符、几何图形符、`+` 等符号。斜杠 `/` 比较特殊，它在命名空间中为非法字符，但是在路径中可用于分割目录的不同层级，以下会有详细说明。

一、命名空间 ID 的实际意义

小提示

原版游戏中大部分对象都使用命名空间 `minecraft`，但是六种基本命令参数类型（见小节 1.2.1）却使用命名空间 `brigadier`。

命名空间 (Namespace) 是游戏资源的区界，它位于资源类型的父层级，所有来自 Minecraft 原版游戏的资源均位于命名空间 `minecraft`。通过不同的自定义命名空间可以将新增的内容和原版内容区分开来，以防止新内容和原版内容、新内容和其他新内容之间产生冲突。例如，有两个命名空间 ID `minecraft:something` 和 `custom:something`，它们指定的是两个不同的对象，因为它们的命名空间不同，前者为 `minecraft`，后者为 `custom`，即使两者拥有相同的路径（名称）`something`。

部分游戏资源在命名空间下有一定的文件路径，尤其是资源包、数据包内容，这时就可以使用 `/` 以表明它们的文件路径。这里命名空间实际上是一个文件夹，这个文件夹的结构一般是 `<命名空间>\<资源类型>\<文件夹1>\<文件夹2>\...<文件夹n>\<文件名>.<后缀>`。若要映射到这个文件，则命名空间 ID 的写法为

```
<命名空间>:<文件夹 1>/<文件夹 2>/...<文件名>
```

1.5

注意以下几点：

1. 资源类型不作为命名空间 ID 的一部分；
2. 资源路径的子文件夹至文件之间的一系列路径必须完整；
3. 文件名后缀不写。

下面举两个例子以说明之：

例 1.1

有数据包函数文件路径为 `minecraft\function\load.mcfunction`，试用命名空间 ID 指定之。

解

这里 `function` 为资源类型，`.mcfunction` 为文件的后缀。故命名空间 ID 为

```
minecraft:load
```

1.6

例 1.2

有资源包纹理文件路径为 `minecraft\textures\block\command_block_front.png`，试用命名空间 ID 指定之。

解

这里 `textures` 为资源类型，`.png` 是文件后缀，依照其路径将命名空间 ID 写为

```
minecraft:block/command_block_front
```

1.7

二、命名空间 ID 字符串的识别与一般写法

命名空间 ID 是形如 `<字符串>:<字符串>` 的字符串，游戏在识别、调用相关对象时，如果冒号 `:` 存在，会将冒号前的内容视作命名空间，其中不能出现斜杠 `/`；将冒号后的内容视作路径，可以出现斜杠。为了保证识别的正确性，整个字符串中最多只能出现一个冒号，否则识别会出现错

误。如果冒号不存在，字符串的形式为 `<字符串>`，则游戏会直接将整个字符串直接识别为路径，并默认命名空间为 `minecraft`，有些时候可以适当地减少书写命名空间，但省略命名空间的行为仍是不被建议的：虽然命名空间在原版大部分需要 ID 的地方上不是必须的，但在一些情况下命名空间是必须的，包括但不限于 NBT 路径中指定 ID 的节点。鉴于读者在实践的过程中可能会忘记必须添加命名空间的地方，或是混淆原版内容和自定义的内容，那么最好还是完整地书写命名空间 ID。

例 1.3

下列命名空间 ID 的识别结果为何？

something	1.8
minecraft:something	1.9
custom:something	1.10
minecraft/custom:something	1.11
minecraft/something	1.12
minecraft:Something	1.13
minecraft:custom:something	1.14

解

以上各命名空间 ID 的识别结果列于下表：

表 1.4 命名空间 ID 的识别结果

命名空间 ID	识别的命名空间	识别的路径	识别结果
1.8	minecraft	something	minecraft:something
1.9	minecraft	something	minecraft:something
1.10	custom	something	custom:something
1.11	-（含有非法字符 /）	-	识别失败
1.12	minecraft	minecraft/something	minecraft:minecraft/something
1.13	minecraft	-（含有非法字符 S）	识别失败
1.14	-（冒号数量大于 1）	-	识别失败

一般而言，命名空间和路径推荐的写法是**蛇形命名法（Snake case）**，即当名称中含有多个单字时，以下划线 `_` 取代每一个空格的写法。蛇形命名法的书写仍需遵守合法字符的规定，不能出现大写字母。例如，下面的命名空间 ID 在命名空间和路径上均使用了蛇形命名法：

ancient_city:get_out	1.15
----------------------	------

1.1.4 数据包标签

一个单独的命名空间 ID 只能映射至单独的一个对象，如果要同时映射多个对象，一般的做法是将对象分类，通过映射同一种类别的对象从而映射多个对象。这种将游戏资源分类的手段被称为**数据包标签（Tags in data packs）**，简称**标签（Tag）**。由于命令系统存在多个名为“标签”的概念，笔者不建议使用这样的简称以防止与其他概念的混淆。。原版游戏有一些既有数据包标签，数据包标签的名称大多拥有实际的意义：例如，数据包标签 `#fire` 映射至两种方块，即 `fire`（火焰）和 `soul_fire`（灵魂火焰）；`#mineable/axe` 映射至所有能被斧采集的方块。

数据包标签的表示方式类似于命名空间 ID，但需要在前面加上井号 `#`，写法为

```
#<namespace>:<id>
```

1.16

例如 `#minecraft:fire`。数据包标签映射的对象可以直接是一个游戏资源，如方块、实体等，也可以是另一个数据包标签。例如，数据包标签 `#minecraft:mineable/axe` 还包含了 `#minecraft:planks`（所有种类的木板）、`#minecraft:signs`（所有种类的告示牌）等数据包标签。指定某数据包标签时，其映射的其他数据包标签下的对象也会被选择。但是同一个数据包标签映射的资源类型必须相同，不能将不同类型的对象放入一个数据包标签中，例如，猪和石头不能被放在同一个数据包标签中。

数据包标签涵盖的对象类型非常广，包括方块、实体、物品、游戏事件、生物群系等。读者可以在既有数据包标签的基础上，使用数据包添加一些自定义的数据包标签。数据包标签的定义方式见小节 1.5.3。

1.2 命令

命令（Command），又称**控制台命令（Console command）**、**斜杠命令（Slash command）**或**MC-CMD**，是一种高级的、通过输入具有特定语法文本以实现控制游戏本身运行的功能。命令文本需要讲究严格的语法，不允许任何模糊的表达。目前 MC-CMD 已被正式确认为一种编程语言，名称为 mcfuction，与 C 语言、Java、Python 等并列——但这是一种只适用于游戏 Minecraft 内部的编程语言，无法与外部环境进行交互。

1.2.1 命令参数

参数是命令的组成部分，每一条命令都由一个命令头和若干参数组成，参数之间用空格分隔，由此得到命令的通用格式：

```
<命令名> [<参数 1>] [<参数 2>] ...
```

1.17

例如在下面的命令中，`say` 是该命令的命令名，`Hello World!` 是后续参数：

```
say Hello World!
```

1.18

在本教程中, 为了方便解释命令中各参数的语法, 会使用一系列括号或其他符号来表示这些参数的具体内容及可用性。为了使说明更加方便, 本教程采用了和游戏本体、Minecraft Wiki 一样的命令语法格式, 如下表所示。

表 1.5 语法指引格式

语法	含义	举例
字面量	按字面量原样输入	命令 <code>/locate biome</code> 中第 2 个参数 <code>biome</code> 即为字面量, 编写命令时不要更改这个参数的写法。
<参数>	需要使用一合适的值来替换该参数	命令 <code>/say <message></code> 的第 2 个参数需要用自定的值, 比如 <code>/say Hello World!</code> 。
[<参数>]	该参数是可选的, 如果使用该参数, 则需要使用一合适的值替换之	命令 <code>/summon <entity> [<pos>] [<nbt>]</code> 中第 3、4 个参数可选, 填写这些参数时需要用自定的值。
(参数 参数)	(必须的) 在显示的值中选择一个填写。语法介绍中这些值由竖线分隔	命令 <code>/time query (daytime gametime day)</code> 的第 3 个参数是必填的, 从 <code>daytime</code> 、 <code>gametime</code> 和 <code>day</code> 中选择一个填写。
[参数 参数]	(可选的) 在显示的值中选择一个填写。语法介绍中这些值由竖线分隔	命令 <code>/experience add <targets> <amount> [levels points]</code> 的第 5 个参数虽然不是必写的, 但仍可以从参数 <code>points</code> 和 <code>levels</code> 中选择一个填写。
-> 子命令	必须接入一条子命令	命令 <code>/execute positioned <pos> -> execute</code> 的第 6 个参数是必填的, 内容为命令 <code>/execute</code> 的一个子命令。
-> [子命令]	可以接入一条子命令	命令 <code>/execute (if unless) block <pos> <block> -> [execute]</code> 的第 7 个参数可以填写 <code>/execute</code> 的子命令, 也可以不填写。

下面举一实例以说明之:

例 1.4

命令 `/data` 的一种语法如下所示:

```
data get (block <targetPos>|entity <target>|storage <target>) [<path>]
[<scale>]
```

1.19

解

语法中 `data` 为命令名, `get` 是字面量, 这两者必须按语法指引中的字面量原样输入命令。第 3、4 个参数必须从 `block <targetPos>`、`entity <target>` 和 `storage <target>` 中选则一种, 且不得空缺。若使用 `block <targetPos>`, 则 `block` 按原样输入, 后续使用 `<targetPos>` 的自定义值, 但不得在 `block` 后续使用 `<target>` 参数, 因为 `<target>` 是 `entity` 或 `storage` 的后续参数。第 5、6 个参数 `[<path>]`、`[<scale>]` 可选并自定义值。使用该语法且实际可行的命令可以是:

```
data get entity @s SelectedItem
```

1.20

命令 1.20 使用了参数 `entity`, `@s` 是 `<target>` 使用的值, `SelectedItem` 是 `[<path>]` 使用的值, 参数 `[<scale>]` 未使用。

为了使不同的命令具有不同的功能, 它们使用的参数类型各不相同。有些命令作用的对象为实体, 它们则会使用指定实体的参数, 有些命令作用的对象为某一个坐标, 则其使用坐标参数。

游戏使用的命令参数有如布尔值、整型、函数、槽位值、坐标值、目标选择器、JSON、NBT 等。一些复杂参数会在本教程后文呈现，下面列举的是基本参数，即在 Brigadier 中使用的六种基本数据类型：

1. 布尔值 (Bool)

只有两种可用参数，为 `true` 和 `false`，分别代表“是”与“否”。

2. 整数 (Integer)

使用 32 位整型数值，是介于 `-2147483648` 和 `2147483647` 之间的整数值，如 `1`、`0`、`-1` 等。不同命令中使用整型的参数规定的最大可用值和最小可用值不一致。

3. 长整数 (Long)

使用 64 位整型数值，是介于 `-9223372036854775808` 和 `9223372036854775807` 之间的整数值。

4. 单精度浮点数 (Float)

使用占据 4 字节的浮点数，范围大约介于 -3.4×10^{38} 和 3.4×10^{38} 之间，在不同命令中使用单精度浮点数的参数规定的最大可用值和最小可用值不一致。一些单精度浮点数的示例有：`0`、`1.1`、`-1`、`.5` 等，小数形式的整数部分可以省略。在命令参数中使用的浮点数暂时不支持科学计数法^①。

5. 双精度浮点数 (Double)

使用占据 8 字节的浮点数，范围大约介于 -1.8×10^{108} 和 1.8×10^{108} 之间。可以表示比单精度浮点数绝对值更大的有效数字。

6. 字符串 (String)

字符串是由多个字符组成的序列，可用于表示单词、句子或其他符号组合。

(1) 单个词 (Single word)

即不含空格的字符串，如 `word`，若单个词的内容由多个词语组成，则一般使用下划线 `_` 连接相邻词，如 `word_with_underscores`。

(2) 词组 (Quotable phrase)

可以由双引号括起，如 `"quoted phrase"`，也可以使用单引号来定义，如 `'quoted phrase'`，此时单词之间可以有空格。

(3) 贪婪词组 (Greedy phrase)

这种形式的词组不带引号，任意使用空格。该形式的参数通常位于命令的末尾，将命令的剩余部分全部作为字符串参数。如：`words with spaces`。上文中命令 1.18 就使用了这种参数。

Minecraft 的命令有很多，可用 `/help` 命令查询任何可用命令的语法，`/help` 本身的语法为

```
help [<command>]
```

1.21

其中的参数：`[<command>]`（字符串 `brigadier:string`）——可选，若不指定，则在执行权限等级下列出所有可用的命令。若指定了该参数，则必须为可用的命令，比如 `help help` 会返回 `/help` 这条命令的语法指引。由于该参数是一个贪婪词组，因此也可以输入命令的多个参数，例如若要查询 `/execute` 命令下的 `if` 子命

^① 参见 MC-130925。

令用法，则可以输入 `help execute if`，然后会返回 `execute if` 的具体用法。

1.2.2 命令的输入

命令是一种文本输入，以下是可供命令输入的途径：

1. 使用聊天栏输入命令

为了和普通的聊天文本区分开来，在聊天栏中输入命令时会在命令前加一个前缀 `/`，此前缀必不可少。在不使用按键 `T` 召唤聊天栏时可以直接键入 `/` 输入命令，这是使玩家快速进入命令输入模式的一种办法。比如，玩家可以在聊天栏中输入 `/help` 以查询命令语法。

呼出聊天栏后，可以使用 `↑` 或 `↓` 键调用**命令历史（Command history）**，即先前键入的命令。如果之前输入的命令有语法错误的话，切换至该命令时依旧会有语法错误，不会自动更正，更不会因为含有语法错误就不显示该命令。这种快捷键在命令方块控制台中不适用。命令历史可以跨存档调用。

在聊天栏输入命令时，`Tab` 键可用于补全命令。未输入任何命令字符的时候，使用 `Tab` 键可以看到聊天栏上出现的一个命令列表（如图 1.1），鼠标滚轮有助于翻找需要的命令。

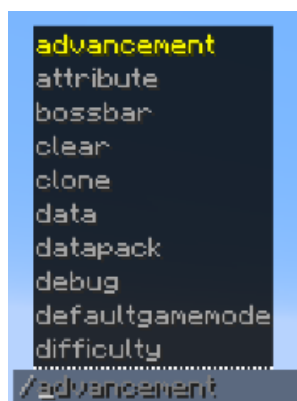


图 1.1 在聊天栏输入命令时使用 `Tab` 键

读者可以直接在命令列表中点击需要的命令，或者如图 1.2 (a) 所示，输入命令的前若干字符后使用 `Tab` 键补全。若这个命令后续还有其他参数，则也可以如图 1.2 (b) 所示用 `Tab` 键补全。



图 1.2 命令输入过程

聊天栏的字符数量被限制在 256 以内，命令开头的 `/` 也会被计入字符数。因此，聊天栏不能用于执行太长的命令。

2. 在命令方块或命令方块矿车内输入命令

命令方块控制台可输入的字符最多为 32500 个，较聊天栏的限制有很大提升。文本框长度有限，每次只能显示命令的其中一段，需要使用 `鼠标左键` 或 `←`、`→` 键移动光标调整命令显示的位置，且每次打开命令方块 GUI 时光标总显示在命令的末尾。

在命令方块或命令方块矿车内输入命令时，斜杠前缀 `/` 不是必须的。和在聊天栏中使用命令一样，当文本框中无任何内容时，下方会显示一个命令列表（如图 1.3），通过调整鼠标滚轮能够调整命令列表显示的位置，按 `Tab` 键能够在输入命令时自动补全或选择命令的部分。



图 1.3 命令方块 GUI

3. 在数据包函数文件中输入命令
- 这种编写方式需要使用一定的编译软件，常用的编译软件有 Windows 自带的记事本、Visual Studio Code 等。函数中的命令不能带有斜杠前缀。具体的内容可参阅《数据包》教程的描述。
4. 在服务器控制台中输入命令
5. 在带有 `run_command` 动作的点击事件的文本组件或对话框按钮中输入命令

1.2.3 权限等级与限制条件

命令功能强大、种类繁多，如果在任意情况下都能够随意使用，则很有可能会破坏玩家的游戏体验。因此，命令系统有一套专门的机制用于控制游戏内可用命令的情形，即权限等级。**权限等级（Permission level）** 用于决定命令执行者可以使用什么样的命令。所有命令都有一个所需的权限等级，如果命令执行者没有达到该有的权限等级，则无法执行该命令。例如：`/advancement` 需要的权限等级为 2，命令方块的权限等级也为 2，因此命令方块可以执行该命令；而关闭命令的单人游戏玩家的权限为 0，所以该玩家不能执行该命令。

权限等级共分为 0、1、2、3、4 级，表罗列了 Java 版所有可用命令需要的权限等级与限制条件。除权限等级之外，一些命令还对当前的游戏世界有限制：一些命令只能在专用服务器（以下简称多人游戏）中使用，另有只能在非专用服务器（以下简称单人游戏，无论是否对局域网开放）中使用的命令，然而大部分命令都是无此限制条件的。

表 1.6 Java 版可用命令列表

命令	权限等级	限制条件	命令	权限等级	限制条件
<code>/advancement</code>	2		<code>/raid</code>	3	仅调试工具
<code>/attribute</code>	2		<code>/random</code>	0（不使用 <code>sequence</code> ）或 2（使用 <code>sequence</code> ）	
<code>/ban</code>	3	仅多人游戏	<code>/recipe</code>	2	

(续表)

命令	权限等级	限制条件	命令	权限等级	限制条件
<code>/ban-ip</code>	3	仅多人游戏	<code>/reload</code>	2	
<code>/banlist</code>	3	仅多人游戏	<code>/return</code>	2	
<code>/bossbar</code>	2		<code>/ride</code>	2	
<code>/chase</code>	0	仅调试工具	<code>/rotate</code>	2	
<code>/clear</code>	2		<code>/save-all</code>	4	仅多人游戏
<code>/clone</code>	2		<code>/save-off</code>	4	仅多人游戏
<code>/damage</code>	2		<code>/save-on</code>	4	仅多人游戏
<code>/data</code>	2		<code>/say</code>	2	
<code>/datapack</code>	2		<code>/serverpack</code>	2	仅调试工具
<code>/debug</code>	3		<code>/schedule</code>	2	
<code>/debugconfig</code>	3	仅调试工具	<code>/scoreboard</code>	2	
<code>/debugmobspawning</code>	2	仅调试工具	<code>/seed</code>	0(单人游戏) 或2(多人游戏)	
<code>/debugpath</code>	2	仅调试工具	<code>/setblock</code>	2	
<code>/defaultgamemode</code>	2		<code>/setidletimeout</code>	3	仅多人游戏
<code>/deop</code>	3	仅多人游戏	<code>/setworldspawn</code>	2	
<code>/dialog</code>	2		<code>/spawn_armor_trims</code>	2	仅调试工具
<code>/difficulty</code>	2		<code>/spawnpoint</code>	2	
<code>/effect</code>	2		<code>/spectate</code>	2	
<code>/enchant</code>	2		<code>/spreadplayers</code>	2	
<code>/execute</code>	2		<code>/stop</code>	4	仅多人游戏
<code>/experience</code>	2		<code>/stopsound</code>	2	
<code>/fill</code>	2		<code>/stopwatch</code>	2	
<code>/fillbiome</code>	2		<code>/summon</code>	2	
<code>/fetchprofile</code>	2		<code>/swing</code>	2	
<code>/forceload</code>	2		<code>/tag</code>	2	
<code>/function</code>	2		<code>/team</code>	2	
<code>/gamemode</code>	2		<code>/teammsg</code>	0	
<code>/gamerule</code>	2		<code>/teleport</code>	2	
<code>/give</code>	2		<code>/tell</code>	0	
<code>/help</code>	0		<code>/tellraw</code>	2	
<code>/item</code>	2		<code>/test</code>	2	
<code>/jfr</code>	4		<code>/tick</code>	3	
<code>/kick</code>	3		<code>/time</code>	2	
<code>/kill</code>	2		<code>/title</code>	2	

(续表)

命令	权限等级	限制条件	命令	权限等级	限制条件
/list	0		/tm	0	
/locate	2		/tp	2	
/loot	2		/transfer	3	仅多人游戏
/me	0		/version	0	
/msg	0		/version	0 (单人游戏) 或 2 (多人游戏)	
/op	3	仅多人游戏	/w	0	
/pardon	3	仅多人游戏	/waypoint	2	
/pardon-ip	3	仅多人游戏	/warden_spawn_tracker	2	仅调试工具
/particle	2		/weather	2	
/perf	4		/whitelist	3	仅多人游戏
/place	2		/worldborder	2	
/playsound	2		/xp	2	
/publish	4	仅单人游戏			

权限等级和限制条件也会影响命令 `/help` 的行为。比如，若执行权限等级为 0，则 `/help` 仅列出所需权限等级为 0 的命令；若执行权限等级为 2，则仅列出所需权限等级小于等于 2 的命令。

1.2.4 命令的解析 *

游戏处理命令的过程可分为解析和执行两个阶段。Minecraft 使用 **Brigadier** 作为命令的解析器、派发器。

命令的实质是一个根命令节点的直接量分支，这就意味着所有的命令都是一个树状结构，命令中每一个参数都作为一个节点，而命令名作为根节点使用。显然，一个节点可能会有两种类型：字面量和变量，反映到命令文本语法中分别为字面量参数和需要自定义值的参数。

游戏在读取命令后，会首先解析根节点是否是已注册的命令，其次解析下一个参数即子节点是否可用，然后依次解析余下的节点。Brigadier 读取到某一个节点时，会枚举其子节点的所有可行节点，并在聊天栏或命令方块控制台内显示为可读性较强的可视化参数列表。

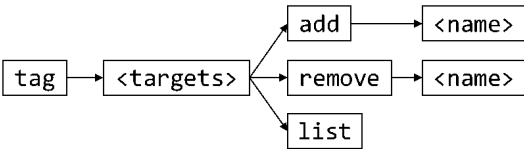


图 1.4 命令 `tag` 的所有结点和分支

以命令 `/tag` 为例，其命令树如图 1.4 所示。`tag` 是根命令，其子节点 `<target>` 是一个需要特定参数类型（这里是 `entity`）的节点，解析此节点的时候，会判断输入的参数是否为 `entity` 类型，若为否则解析异常，命令无法执行。2 级子节点是已注册的字面量 `add`、`remove` 和 `list`，解析该级节点的工作比较简单：只需读取该节点的文本是否与注册的字面量吻合。若 2 级子节点的

参数指定为 `add`、`remove`，则读取 3 级子节点 `<name>`，这个节点又是一个需要自定义的量；若 2 级子节点的参数指定为 `list`，则不能再添加后续参数。

1.2.5 命令上下文

当一条命令被执行时，该命令一定有一个调用者以及调用环境，这一系列调用者及调用环境构成的集合被称为**命令上下文 (Command context)**，或称**执行上下文 (Execution context)**、**命令源 (Command origin)**、**命令来源堆叠 (Command source stack)**。

命令上下文由以下参数构成：

1. 执行权限等级

2. **执行者 (Executor)**

由“执行者名称”和“执行者实体”两个参数构成，但执行者实体不一定存在，例如执行者为命令方块、命令方块矿车或服务端的时候。

3. **执行位置 (Execution position)**

这个参数是命令执行时所在的坐标，包含 x 、 y 、 z 三个坐标参数。

4. **执行朝向 (Execution rotation)**

这个参数是命令执行时面向的方向，包含偏航角和俯仰角两个参数。

5. **执行锚点 (Execution anchor)**

实体锚点 (Entity anchor) 是实体身上用于定位的**点**，有两个可用的实体锚点：脚部和眼部。故名思义，脚部位于实体碰撞箱的底部中心点，这个位置实际上就是实体本身的位置，也是**默认使用的实体锚点**。眼部位于实体眼睛高度处碰撞箱的中心点。眼部和脚部在水平方向上的位置是一样的，在 y 轴上，这个实体眼睛部位的高度就是眼部和脚部高度的差值。

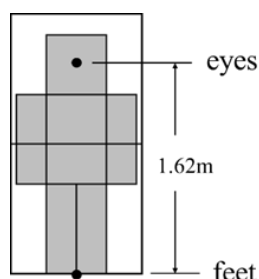


图 1.5 玩家的实体锚点

对于玩家而言，其眼部与脚部的高度差约为 1.62 格，如上图所示。但是不同实体的眼部高度实际上是不一致的，不能笼统地认为所有实体的眼部高度均为 1.62 格。

相应地，锚点也作为命令上下文参数的一部分，是为执行锚点。如果执行位置为一个实体所在的位置，则脚部与执行位置实际上是重合的。事实上在所有的命令中，**执行锚点默认都是脚部**。无论锚点是否被修改为眼部，执行位置一定是在实体碰撞箱的底部中心点，不会随着锚点的变化而发生变动。

6. **执行维度 (Execution dimension)**

这个参数是命令执行所在的维度，执行位置位于这个维度内。

7. 执行输出反馈

尝试执行命令会产生一定的执行效果，并在执行失败或执行成功时返回**成功次数 (Success)** 和**结果 (Result)** 两个返回值。其中成功次数总是为 0 或 1，结果一定为整数，遇到小数时则向下取整。下面讨论所有种类的命令执行效果：

(1) 无法解析

这种效果会在命令中存在无法解析的参数、输入的命令不完整或执行上下文不符合命令的限制条件，如执行者拥有的权限等级不够、超出游戏世界的限制时出现。此时命令没有返回值。在聊天栏或命令方块内输入的命令若无法解析，则会返回语法错误信息。在函数内的命令若无法解析，则该函数无法加载。

(2) 执行错误

出现这种效果说明命令中存在严重的漏洞。此时命令没有返回值。

(3) Void

当且仅当执行命令 `/function` 时会出现这种效果，说明 `/function` 调用了一个 Void 类型的函数，没有返回值。

(4) 执行中断

当且仅当执行命令 `/execute` 时会出现这种效果，此时 `/execute` 的分支数量为 0，在 `run` 子命令执行前执行就已经中止。此时命令没有返回值。

(5) 执行失败和**执行成功**

当命令执行效果不是上述 4 种中任意一种时，才能出现执行失败或执行成功。且只有当执行效果为执行失败或执行成功时，才能返回成功次数和结果。执行失败并不意味着命令没有起作用，执行成功也不意味着命令会对游戏做出更改。

不同情况下的命令上下文列举于下表：

表 1.7 不同情况的命令上下文

情况	执行权限等级	执行者	执行位置	执行朝向	执行锚点	执行维度
玩家	视情况而定	玩家	玩家的位置	玩家的朝向	脚部	玩家所在的维度
服务器控制台	4	Server	世界出生点方块的西北下角顶点	水平向南	脚部	主世界
服务端	2	Server	世界出生点方块的西北下角顶点	水平向南	脚部	主世界
命令方块	2	命令方块	命令方块的正中心	命令方块的朝向	脚部	命令方块所在维度
命令方块矿车	2	命令方块矿车	命令方块矿车的位置	命令方块矿车的朝向	脚部	命令方块矿车所在维度
函数	可修改	该函数的调用者	函数调用者的位置	函数调用者的朝向	函数调用者的锚点	函数调用者所在维度
告示牌	2	点击告示牌的玩家	告示牌所在方块正中心	水平向南	脚部	告示牌所在维度
<code>/execute</code>	-	修饰后的执行者	修饰后的执行位置	修饰后的执行朝向	修饰后的执行锚点	修饰后的执行维度

(续表)

情况	执行权限等级	执行者	执行位置	执行朝向	执行锚点	执行维度
自定义进度	2	获得进度的玩家	玩家的位置	玩家的朝向	脚部	玩家所在的维度
自定义魔咒	2	魔咒作用的实体	魔咒作用的位置	魔咒作用实体的朝向	脚部	魔咒作用的维度

小提示

- 玩家的权限等级与其游戏模式无关，需要分情况讨论：
- 1. 若该玩家是服务器管理员，则他的权限等级由 `ops.json` 中的值决定，默认为 4 级；
 - 2. 若该玩家处于启用命令的单人世界中或为启用命令的局域网世界所有者，则他的权限等级为 4 级；
 - 3. 若该玩家处于启用命令的局域网世界中，则他的权限等级为 4 级；
 - 4. 非上述情况者权限等级一律为 0 级。
- 函数的权限等级默认为 2 级，可在 `server.properties` 中修改。

1.3 JSON 格式

在讲述数据包和资源包这两种开发实例之前，有必要先介绍 Minecraft 使用的用于数据交换的格式，即 JSON 格式。事实上，这种数据格式的应用极其广泛，是当今互联网最通用的数据交换格式之一。在 Minecraft 中，为方便网络传输和数据存储，一些游戏内程序会被**序列化**为 JSON 格式。而对于数据包、资源包内容中的 JSON 格式而言，游戏会尝试将这些 JSON 数据**反序列化**为游戏内相应的程序。

游戏的一些数据以 `.json` 文件的格式存储在各文件夹中。这些文件有如：资源包的模型文件，数据包的进度、谓词文件，游戏版本信息文件，等等。下面展示的是金合欢木按钮的模型文件：

```
assets > minecraft > models > block > acacia_button.json
1 {
2   "parent": "minecraft:block/button",
3   "textures": {
4     "texture": "minecraft:block/acacia_planks"
5   }
6 }
```

1.3.1 JSON 数据类型

JSON (JavaScript Object Notation, JavaScript 对象表示法)是一种轻量级数据交换格式，独立于编程语言，是 JavaScript 的一个子集。其内容主要由键和值构成，即**键值对 (Name-value pair)**，这些键值对可认为是一个个**字段 (Field)**。这种格式主要有两个优点：第一，便于编写

者阅读和修改；第二，由于其轻量级的特点，其对环境的依赖程度较小，因此能用于存储大量不同种类的信息。Minecraft 使用的 JSON 标准为 ECMA-404。

JSON 格式键值对的基本语法为：

```
"<键>":<值>
```

1.22

对于一个键，可以给它定义一个值。在书写时，JSON 的所有键的键名必须用**双引号**引起。若有多个键值对，则需使用逗号将这些键值对分隔开来，最后一个键值对的后面不加逗号，如：

小提示

键名的两侧必须是**英文引号**，且不接受单引号！

```
"<键>":<值>,"<键>":<值>
```

1.23

在一个 `.json` 文件中，须使用花括号 `{}` 将所有的键值对封装包裹在一起，如：

```
{"<键>":<值>,"<键>":<值>}
```

1.24

对于值而言，每一个不同的键都需的值的类型不尽相同，比如键 `color` 可能需要的是颜色值，`bold` 可能需要的是布尔值，`text` 可能需要的是字符串，等等。JSON 一共使用六种不同的数据类型：

1. **字符串 (String)**

常见的数据类型，可以包含任意字符（如空格），字符串由一对**（英文）双引号**定义，**不接受单引号**，用法举例：

```
"description": "The default data for Minecraft"
```

1.25

也可以使用中文：

```
"description": "我的世界默认数据包"
```

1.26

JSON 同时也支持 Unicode，表示方式为 `\uxxxx`，其中每一个 `x` 都为十六进制数字。例如，符号★的 Unicode 为 `U2605`，则在字符串中输入★的方式可以为：

```
"text": "\u2605"
```

1.27

这样便可以在字符串中输入一些生僻字或是在键盘上无法直接打出来的字符。但是 Minecraft 的字库是有限的，并非所有的字符都可以在 Minecraft 中显示。

2. **布尔值 (Bool)**

由 `true`（真）或 `false`（假）定义，这两者是 JSON 中的字面量符号，不需要使用双引号引起，举例：

```
"bold": true
```

1.28

```
"italic": false
```

1.29

3. **数值 (Number)**

由数字定义，允许使用整数、浮点数或是科学计数法表示的数，举例：

```
"min": 1.0
```

1.30

在 JSON 中使用的数值不需要注明它们的数据类型。

4. 数组 (Array, 或称为列表)

由一对方括号定义, 数组中元素与元素之间使用逗号隔开, **最后一个元素后不能有逗号**。这些元素可以是其他的数据类型, 如字符串、布尔值、数值和对象, 数组中甚至能嵌套数组。在定义其他的数据类型时, 需注意这些数据类型的定义方法。以下为包含了数值的数组:

```
"frames": [1, 2, 3, 4, 5]
```

1.31

下面为包含了字符串的数组, 字符串均由一对双引号定义:

```
"text": ["A", "B", "C"]
```

1.32

对于数组内的元素, 其数据类型不必完全一致, 例如:

```
"extra": [1, {"text": "2"}, "3"]
```

1.33

5. 对象 (Object)

由一对花括号定义, 对象内字段与字段之间使用逗号隔开, **最后一个字段后不能有逗号**。对象中可以包含其他数据类型, 也可以在对象中嵌套对象。整个 `.json` 文件就可以看作是一个大的对象。在编写 JSON 的时候, 通常需要用到对象嵌套对象, 因此花括号一定要检查是否匹配。用法举例:

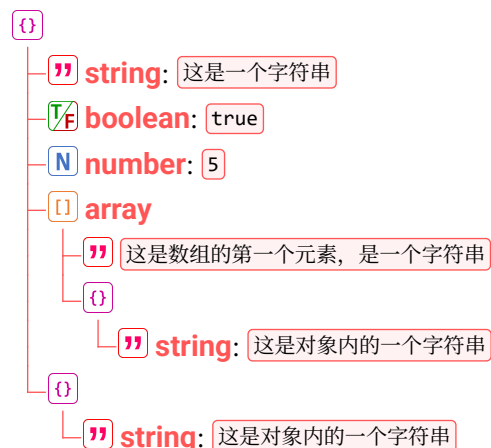
```
{
  "rolls": {
    "type": "minecraft:binomial",
    "n": 3,
    "p": 0.2
  }
}
```

1.34

6. Null

空值, 作为字面量符号使用。Minecraft 基本不使用这种数据类型。

由于 JSON 为多层级结构, 为了方便说明, 本系列教程会使用 and Minecraft Wiki 一样的树状图来表示。例如:



对应的 JSON 为：

```
{
  "string": "这是一个字符串",
  "boolean": true,
  "number": 5,
  "array": [
    "这是数组的第一个元素，是一个字符串",
    {
      "string": "这是对象内的一个字符串"
    }
  ],
  {
    "string": "这是对象内的一个字符串"
  }
}
```

1.35

小提示

如何看懂树状图？

1. 父节点和子节点的关系会以这样的方式表示：

这是父节点

└ 这是子节点

相同层级的节点会表示为相同的缩进。

2. 对于一个字段：

field: 这是一个字段

(1) 字段开头的 **field** 表示这个字段使用的数据类型。如果出现了多种数据类型，则表示这些数据类型均可使用。

(2) 加粗红色的字表示这个字段的键名。

(3) 冒号后面如果只有 **代码块**，表示此 **代码块** 是该字段使用的真实值。如果冒号后面是一段文字，则这是对于该字段的解释。如：

field: 这是对于这个字段的解释。

(4) 如果键名有下划线，则表示这个字段是必填项：

string: 此项为必选项。

1.3.2 JSON 的转义序列

使用 JSON 字符串时，如果字符串本身的内容中含有英文引号 **"**，如一个 JSON 字段 **text** 的值需要为 **"Hello World!"**，那该如何编写 JSON 呢？若使用如下的 JSON：

```
"text": "Hello World!"
```

1.36

这样通常会产生报错，这是由于用于定义字符串的引号和值中的英文引号发生了配对从而导致了错误，因此需要使用**转义字符 (Escape character)** **** 对文本引号进行转义。转义的作用为：将被转义的字符转换成字符，被转换的引号便不再与用于定义字符串的引号发生配对。除用于转义英文引号外，反斜杠还可以用于转义反斜杠以及创造一些特定的转义序列。JSON 中可用的转义序列如下：

1. `"`, 是半角双引号 (英文引号) `"` 的转义方式 (中文引号不需要转义);
2. `\\`, 是反斜杠 `\` 的转义方式, 已被转义的反斜杠被视为普通字符, 不再具有转义作用;
3. `\b`, 退格;
4. `\f`, 换页;
5. `\n`, 换行;
6. `\r`, 回车;
7. `\t`, 制表符;
8. `\u<unicode>`, 用四位十六进制表示 Unicode 字符。

小提示

`\b`、`\f`、`\n`、`\r`、`\t`

这些特殊的转义序列能在 JSON 中使用, 但这不代表这些转义序列能在相应的游戏实例中真正起作用。例如, 在物品修饰器中定义物品名称时, 虽然 JSON 支持输入换行符 `\n`, 但物品名称本身不支持换行。

由此可以得出字段 1.36 的正确写法:

```
"text": "\"Hello World!\""
```

1.37

同样地, 如果值为 `\Hello World!\`, 需要对反斜杠进行转义, 正确的写法为:

```
"text": "\\Hello World!\\"
```

1.38

例 1.5

判断下列 JSON 的转义是否有效, 若有效, 则写出其值。

```
"text": "\"\\Hello World\\\""
```

1.39

```
"text": "\"\\Hello World!\\\""
```

1.40

```
"text": "\"Hello World!\\\""
```

1.41

```
"text": "\"Hello World!\\\""
```

1.42

解

字段 1.39 的值出现了连续三个反斜杠 `\\` 的情况, 第一个反斜杠用于转义第二个反斜杠, 而第二个反斜杠不再具有转义作用; 而第三个反斜杠后面没有其他转义序列, 故值无效。

对于 1.40, 依次检验所有反斜杠: 第一个反斜杠用于转义引号, 第二个反斜杠用于转义第三个反斜杠, 第四个反斜杠用于转义第五个反斜杠, 第六个反斜杠用于转义引号。故值为 `"\Hello World!\`。

字符串两端的反斜杠数量不一定需要相等, 因此字段 1.41 是有效的, 输出结果 `"Hello World\"`。

注意, 1.42 的所有转义序列都书写正确, 但第三个反斜杠后面存在一个引号, 而第三个反斜杠已被第二个反斜杠转义而失去转义作用。因此用于定义字符串的引号配对混乱, 该值无效。判断一个值是否有效, 不仅需要有效的转义序列, 还要注意字符串本身的双引号是否正确配对。

特殊情况下，JSON 字段可能会以字符串类型包含另一个需要被解析的 JSON 数据而非直接嵌套为相应类型的值，例如：

” content: {"text": "Hello World!"}

字段 `content` 的值类型为字符串，两端一定需要一对引号。为了防止决定字符串的引号与值中的引号发生匹配混乱，会将值中的引号进行转义：

```
"content": "{ \"text\": \"Hello World!\" }"
```

1.43

如果 `text` 字段的值也帶有引号，如 `"Hello World!"`，则需要相应地添加反斜杠：

” content: {"text": "\"Hello World!\""}

这时如果把字段 `content` 写成如下的形式:

```
"content": "{ \"text\": \"Hello World!\" }"
```

1.44

现在来手动分析这个字段。把 `"content"` 的值拆出来，对所有的转义序列都去掉反斜杠。首先，键 `"text"` 两端的引号被转义，因此能够正常匹配。其次，冒号 `:` 之后、字符 `H` 之前有两个已被转义的引号；而字符感叹号 `!` 后又有两个已被转义的引号，一共有四个被转义的引号：

```
{"text": "Hello World!"}
```

1.45

所以不可避免地又发生了引号匹配混乱的情况。现在要解决的问题就是如何让这些引号不发生匹配混乱。最有效的写法就是**从里层向外层书写，每嵌套一层，就在上一层所有需要被转义的字符（如 `"` 和 `\`）前添加反斜杠**。因此，对于里层的 `{"text": "\"Hello World!\""}` ，需要在所有的 `"` 和 `\` 之前都添加一个反斜杠：

```
"content": "{\\"text\\":\\"\\\\\\"Hello World!\\\\\\"\\\"}"
```

1.46

1.46 才是正确的写法。如果更进一步，将 1.46 封装在对象中，让它作为另一个 `content` 的值，这样就又增加了一层嵌套，于是应在 1.46 的每一个 `"` 和 `\` 之前都添加一个反斜杠：

```
"content": "{\"content\": \"{\\\"text\\\":\\\"Hello World!\\\"}\\\"}\"
```

1.47

1.4 游戏文件

游戏的各项数据被零零散散地存放在各个游戏文件里，部分数据对于做技术性开发而言非常重要，因此有必要适当掌握游戏文件的结构。

1.4.1 常用文件格式

存储 Minecraft 数据的文件格式有很多种，下面介绍一些常见的文件格式。

1. .txt 文件

.txt 文件是非常常见的文本文件，用 Windows 自带的记事本即可打开。这种文件通常被用于存储一些简易的文本，如游戏标题画面上的闪烁标语，有时也被用于存储游戏中的设置，在这些 **.txt** 文件中更改的内容会在游戏本体上有相应的改动。有时候 **.txt** 文件也可用于记录一些自定义的、不作为游戏数据的文本。有效的 **.txt** 文件必须为无 BOM 的 UTF-8 格式。

2. .mcfunction 文件

.mcfunction 文件，即函数文件，同样必须为无 BOM 的 UTF-8 格式。函数文件可以用 Windows10 自带的记事本打开并编辑，默认的 Windows 10 记事本已经为无 BOM 的 UTF-8 格式，这点从记事本页面下方的状态栏就可以看到。记事本无法指出函数中的语法错误，必须得手动检查，笔者更推荐在编译软件中打开函数文件。本教程推荐的辅助工具是  Data-pack Helper Plus (DHP)，这是编译软件  Visual Studio Code (VS Code) 的一个扩展，可在  VS Code 的应用商店中找到。 DHP 是专门用于制作 Minecraft 数据包或资源包部分文件的辅助工具，在编写数据包或资源包的过程中， 提供了高亮显示，并为部分错误的语法提供解决方案。

《数据包》教程提供了该文件格式的具体编写规范。

3. .json 和 .mcmeta 文件

.json 和 **.mcmeta** 文件都是使用 JSON 格式的文件。这些文件中的 JSON 格式是允许换行的，且为了美观、可读性，编写者在习惯上会在所有的 **.json** 和 **.mcmeta** 文件中使用换行，并使得同一层级的字段在行前缩进上保持一致。**.json** 和 **.mcmeta** 文件没有专门用于注释的语法，若需要注释，则使用游戏不需要、不会被游戏识别的键，如 **_comment1**、**_comment2**。

4. .mca、 .dat、 .dat_old 和 .nbt 文件

.mca、**.dat**、**.dat_old** 和 **.nbt** 文件均是使用 NBT 格式的文件，通常用于存储世界的全局信息和结构信息。同样地，这两类文件不能用  DHP 在编译软件内进行编辑，但可以在 NBT 编辑器内编辑，本教程推荐的编辑器为  NbtStudio。一些无法由命令进行编辑的信息可以通过  NbtStudio 修改。

5. .png 文件

.png 文件是图片文件，被用于存储游戏中的绝大部分图像，包括但不限于图标、游戏截图、资源包纹理。可以使用 Windows 自带的  画图、 PS 或  GIMP 处理，但需要注意  画图不支持透明背景。

6. .ogg 文件

游戏中所有的声音文件都为 **.ogg** 格式，从外部导入声音时应注意格式转换。直接修改文件名后缀是无效的，可以使用

7. .zip 文件

压缩文件，即 **.zip** 文件，也是常用的文件格式，通常被用于数据包和资源包的压缩。读者可自行选择合适的压缩软件对数据包或资源包进行压缩。

8. 其他的文件格式

Minecraft 还使用其他一些文件格式，如 **.jfr** 文件、**.log** 文件等。具体见下文的说明。

1.4.2 .minecraft 文件夹 *

.minecraft 文件夹，macOS 上为  **minecraft**，是存储 Java 版所有游戏数据的文件夹。

对于 Windows 系统, 这个文件夹默认位于 `C:\Users\Admin\AppData\Roaming\.minecraft`, 其中 `AppData` 文件夹一般是隐藏的, 可以在文件资源管理器 `查看` 工具栏, 在 `显示 > 隐藏` 一项勾选 `隐藏的项目` 以显示这个文件夹。

对于 Mac 系统, 这个文件夹默认位于 `home\用户名\Library\Application Support\.minecraft`。对于 Linux 系统, 这个文件夹默认位于 `home\用户名\.minecraft`, 其中以 `.` 开头的文件夹默认是隐藏的, 需要使用 `Ctrl + H` 切换是否可见。

第三方启动器会有其特殊的文件夹路径, 具体见各启动器的设置。由官方启动器运行的游戏可以在启动器内手动修改存储路径, 或者在默认存储路径处使用快捷方式重定向至自定义路径下。

随着游戏内容的增多、各种其他资源(如光影、模组)不断被下载到游戏中, `.minecraft` 中的子文件(夹)可能会持续增多。鉴于无法讲到所有可能出现的文件(夹), 本节仅列举原版游戏使用文件(夹)。文件结构如下所示:

`.minecraft`

`assets`: 存放原版资源包部分游戏资源的文件夹, 如简体中文的语言文件、声音 `.ogg` 文件等, 其中这些文件被称为**散列资源文件**。

`indexes`

`<版本号>.json`: 该版本号用于映射散列资源的哈希表。

`log_configs`

`client.xml`

`objects`: 此文件夹专门用于存储声音、语言文件。

`<哈希值前两位>`

`<哈希值>`: 散列资源文件。

`skins`

`<哈希值前两位>`

`<哈希值>`: 散列资源文件。

`virtual`

小提示

`assets` 文件夹内的资源文件都是用**哈希值 (Hash value, 散列值)**加密的, 以哈希表的方式映射资源位置。要查询 `assets` 内的任意一个资源文件, 需按照以下步骤:

1. 打开 `indexes` 文件夹, 找到需要提取资源的 `<版本号>.json` 文件。其中的内容大致如下所示:

`.minecraft > assets > indexes > <版本号>.json`

```
1  {
2    "objects": {
3      "icons/icon_128x128.png": {
4        "hash": "b62ca8ec10d07e6bf5ac8dae0c8c1d2e6a1e3356",
5        "size": 9101
6      },
7      "icons/icon_16x16.png": {
8        "hash": "5ff04807c356f1beed0b86ccf659b44b9983e3fa",
9        "size": 781
10     },
11     "icons/icon_256x256.png": {
```

```

12     "hash": "8030dd9dc315c0381d52c4782ea36c6baf6e8135",
13     "size": 19642
14   },
15   "_comment": "后续还有很多其他资源"
16 }
17 }

```

2. 用编译软件的查询功能在 `<版本号>.json` 文件中查找所需资源，记录对应 `hash` 字段的值，此即为映射该资源的哈希值。
3. 打开 `objects` 文件夹，找到匹配的 `<哈希值前两位>` 文件夹，在此文件夹内找寻对应哈希值命名的文件，此即为需要找寻的资源。

例 1.6

在 `assets` 文件夹内找到 1.21.4 版本（哈希表版本号显示为 `19`）简体中文语言的资源文件。

解

在 `<19>.json` 文件中查询 `zh_cn`，可以找到一个键名为 `minecraft/lang/zh_cn.json` 的键值对：

```

"minecraft/lang/zh_cn.json": {
  "hash": "4674523c91196e0898c24a06531f94154111f2a3",
  "size": 459788
}

```

1.48

这时获取到哈希值 `4674523c91196e0898c24a06531f94154111f2a3`，其前两位是 `46`。然后打开文件夹 `objects > 46`，在其中找到名为 `4674523c91196e0898c24a06531f94154111f2a3` 的文件，此即为简体中文的语言文件。

📁 **backups**: 存放备份存档的文件夹。

📄 **<日期>_<时间>_<存档名称>.zip**: 一个备份存档。

📁 **bin**

📁 **<随机 ID>**

📄 **.dll** 或 **.so** 文件

📁 **crash-reports**: 存储游戏崩溃报告的文件夹。

📄 **crash-<日期>_<时间>-<逻辑端类型>.txt**: 一份崩溃报告（Crash Report）文件。

小提示

游戏可能会以各种原因而发生崩溃（Crash），读者可以从崩溃报告中查询崩溃原因。例如，以下是一份崩溃报告的开头部分内容：

```

.minecraft > crash-reports > crash-2024-02-08_21.25.56-server.txt
1  ---- Minecraft Crash Report ----
2  // I bet Cylons wouldn't have this problem.
3
4  Time: 2024-02-08 21:25:56
5  Description: Ticking entity

```

其中第二行是“诙谐的评论”，对崩溃报告的分析没有作用。`Description` 行是崩溃原因，此处的崩溃原因是 `Ticking entity`，这种崩溃通常意味着有实体发生了错误。后文通常是崩溃的具体原因。

鉴于崩溃原因多种多样，本教程无法介绍每一种崩溃原因及其解决办法，读者可以从社区获取各种崩溃原因的解决办法或者使用 AI 分析。

📁 **debug**: 存储函数调试结果的文件夹。

📄 **debug-trace-`<日期>`-`<时间>`.txt**: 一份调试结果。

小提示

命令 `/debug` 可用于函数的调试，并将调试的结果以 `.txt` 的文件格式存入 📁 `debug` 中。文件中的内容极为详细，可以以此观察函数的整个运行过程，并从中找到错误的地方。调试结果的具体内容如：

```
.minecraft > debug > debug-trace-2022-10-10_19.16.40.txt
1 [C] scoreboard players reset @s heart_of_life
2 [E] 未知的记分项'heart_of_life' [C] advancement revoke @s only plus:items/heart_of_life
3 [E] 无法撤销Mu_xian的进度plus:items/heart_of_life, 因为该玩家并未达成此进度 [C] effect clear @s absorption
4 [M] 已移除Mu_xian的伤害吸收效果
5 [R = 1] effect clear @s absorption
6 [C] scoreboard players add @s max_health 2
7 [M] 将Mu_xian的[max_health]增加了2 (现在是44)
8 [R = 44] scoreboard players add @s max_health 2
9 [C] function plus:game/attributes/max_health
10 [M] 已执行函数plus:game/attributes/max_health中的0条命令
11 [R = 0] function plus:game/attributes/max_health
12 [F] plus:game/attributes/max_health size=60
13 [C] execute as @a if score @s max_health matches 1 run attribute @s max_health base set 1 -> 0
14 [C] execute as @a if score @s max_health matches 2 run attribute @s max_health base set 2 -> 0
```

其中首行是函数的命名空间 ID。以 `[C]` 开头的内容为函数中的命令行；以 `[E]` 开头的内容指明了上一条命令行出现错误的地方；以 `[M]` 开头的内容说明上一条命令执行成功，并有 `[R=<值>]` 输出执行的结果。`[F]` 说明上一条命令引用了其他函数，并指出被引用的函数的大小。

📁 **libraries**: 按 Maven 仓库的标准目录结构组织和存储的第三方库。

📁 一个第三方库。

📁 **logs**: 存储日志文件的文件夹。

🇬🇧 **<日期>-<日志编号>.log.gz**: 压缩文件，可使用解压软件打开。

📄 **<日期>-<日志编号>.log**: 日志文件。

📄 **latest.log**: 最新一次游戏或当前正在进行的游戏所生成的日志文件。

小提示

日志文件会存储游戏运行全过程的各种反馈，包括加载错误时的反馈，这些文件对游戏调试很重要。文件内容大致如下所示：

```
.minecraft > log > 2025-03-07-1.log.gz > 2025-03-07-1.log
1 [19:06:06] [Render thread/INFO]: Using default channel type
2 [19:06:06] [Render thread/INFO]: Started serving on 10000
```

```

3 [19:06:06] [Render thread/INFO]: [System] [CHAT] 本地游戏已在端口[10000]上开启
4 [19:06:41] [User Authenticator #1/INFO]: UUID of player XVExodus is
  0caaf85c-27b0-4cf6-8f63-7278c55aa0e1
5 [19:06:42] [Server thread/INFO]: XVExodus[/172.20.10.5:53055] logged in with entity id
  252 at (-8.221424908762929, 0.0, 8.503485026934579)
6 [19:06:42] [Server thread/INFO]: XVExodus加入了游戏
7 [19:06:42] [Render thread/INFO]: [System] [CHAT] XVExodus加入了游戏
8 [19:09:58] [Server thread/WARN]: XVExodus moved too quickly!
  -7.169010753171733,-1.6414613841373864,7.738717431310306
9 [19:12:34] [Server thread/INFO]: [Mu_xian: 已将Mu_xian传送至XVExodus]
10 [19:13:54] [Server thread/INFO]: [Mu_xian: 已将Mu_xian传送至XVExodus]
11 [19:14:43] [Server thread/INFO]: [Mu_xian: 已将Mu_xian传送至XVExodus]
12 [19:16:11] [Server thread/INFO]: [XVExodus: 已将XVExodus传送至Mu_xian]
13 [19:16:37] [Render thread/INFO]: Loaded 610 advancements
14 [19:18:25] [Render thread/WARN]: Unable to play empty soundEvent:
  minecraft:entity.salmon.ambient
15 [19:18:32] [Render thread/INFO]: [System] [CHAT] 已设置重生点
16 [19:20:23] [Render thread/INFO]: Loaded 612 advancements
17 [19:22:03] [Server thread/WARN]: XVExodus moved too quickly!
  -4.311401208300595,-1.8757037979856364,17.86772802981045
18 [19:23:27] [Server thread/INFO]: [Mu_xian: 已将Mu_xian传送至XVExodus]
19 [19:23:27] [Render thread/INFO]: Loaded 612 advancements
20 [19:24:38] [Server thread/INFO]: [XVExodus: 已将XVExodus传送至Mu_xian]
21 [19:25:06] [Server thread/INFO]: [XVExodus: 已将XVExodus传送至Mu_xian]

```

- **resourcepacks**: 存储所有资源包的文件夹，其基本结构见 1.6，具体的制作方式将在《资源包》教程中给出。
- **saves**: 存储游戏中所有存档的文件夹，具体结构见 6.1。
 - └ **<存档名称>**: 一个存档。
- **screenshots**: 存储 **[F2]** 截屏图片的文件夹。
 - └ **<日期>_<时间>.png**: 一张截屏，名称可手动修改。
- **versions**: 存储游戏不同版本游戏资源的文件夹。
 - └ **<版本号>**: 一个游戏版本，可以是正式版，也可以是快照。
 - **<版本号>.jar**: 物理客户端文件，是存放该版本号游戏源代码的地方。可以用压缩软件打开这个文件。
 - **assets**: 存放该版本号原版资源包内容的文件夹，它决定了客户端游戏内容的外观。在制作资源包时可以参考这个文件夹的结构。不含在 **.minecraft > assets** 中存放的语言和声音文件。
 - **com**
 - **data**: 存放该版本号原版数据包内容的文件夹，它决定了可写注册表的内容，如进度、战利品表、配方、结构等。在制作数据包时可以参考这个文件夹的结构。

- **flightrecorder-config.jfc**: Java Flight Recorder 配置文件, 可用于 JFR 分析。JFR 分析, 即使用 Java Flight Recorder 分析数据和某些自定义事件。自定义事件包括:
 - `minecraft.ServerTickTime` — 采样事件。
 - `minecraft.ChunkGeneration` — 生成单个区块阶段所需的时间。
 - `minecraft.PacketRead` 或 `minecraft.PacketSent` — 网络流量。
 - `minecraft.WorldLoadFinishedEvent` — 初始化世界加载耗费的时间。
 在游戏中可使用命令 `/jfr` 进行 JFR 分析, 此命令用于开始或结束 JFR 分析, 分析结果以 JSON 的格式写入日志或 `debug` 文件夹。该命令所需权限等级为 4, 语法为:

jfr (start|stop)
1.49
- **META-INF**: `.jar` 文件的元数据。
 - **LICENSE**: 游戏许可协议。
 - **MANIFEST.MF**: 清单文件。
 - **MOJANGCS.RSA**: 用于验证 JAR 的文件。
 - **MOJANGCS.SF**: JAR 签名。
- **net**: 自 25w45a 起, Mojang 发布的未经混淆的客户端其源代码均存储于该文件夹内。其中的类文件均未被混淆, 可查看, 是制作 Mods 的重要依据。
 - **minecraft**
 - **<名称>.class**: 一个未混淆的 Java 类文件。
- **pack.png**: 原版资源包的图标。



图 1.6 原版资源包的图标 (pack.png)
- **versions.json**: 版本信息文件, 存储该版本的信息。
- **<版本号>.json**: 客户端清单文件。
- **webcache2**
- **allowed_symlinks.txt**: 信任符号链接列表文件。
- **command_history.txt**: 命令历史文件, 最多只能保留 50 条记录。
- **debug.stitched_items.png**
- **debug.stitched_terrain.png**
- **hotbar.nbt**: 存储在创造模式中保存的快捷栏信息的文件, 在创造模式中的快捷栏以 `[C] + <数字>` 存储, 然后以 `[X] + <数字>` 调用。可以用 NBT 编辑器打开这个文件。
- **launcher_cef_log.txt**
- **launcher_entitlements.json**
- **launcher_gamer_pics.json**
- **launcher_msa_credentials.json**
- **launcher_profiles.json**: 启动器档案文件。
- **launcher_quick_play.json**: 启动器快速进入游戏存档信息文件。
- **launcher_settings.json**: 启动器配置文件。

-  **launcher_skins.json**
-  **launcher_ui_state.json**
-  **options.txt**: 该文件存储了游戏中设定的选项，可以通过更改该文件中的内容以更改在游戏中的设置。此外一些在选项界面中不存在的设置也可以通过该文件更改。文件中内容如下所示:

```
.minecraft > options.txt
1 version:4189
2 ao:true
3 biomeBlendRadius:2
4 enableVsync:false
5 entityDistanceScaling:1.0
6 entityShadows:true
```
-  **output-client.log**
-  **output-server.log**
-  **realms_persistence.json**: 存储 Realms 数据的文件。
-  **servers.dat**: 存储玩家添加到服务器列表的多人游戏服务器的数据。
-  **textures_0.png**
-  **textures_1.png**
-  **textures_2.png**
-  **textures_3.png**
-  **textures_4.png**
-  **usercache.json**: 游戏为减少重复获取玩家档案信息所使用的缓存文件。

1.5 数据包

Minecraft 的命令系统虽然完善，但其功能十分有限。例如，命令没有办法直接指导游戏世界的生成；直接用命令模拟一些游戏机制也不够灵活。数据包可以看作是命令系统功能的延伸：它不仅为命令提供了程序化执行的环境，更开放了部分 API 以允许数据驱动内容。

数据包（Data pack）允许玩家在不修改游戏代码的前提下覆盖既有的或添加自定义的游戏内容。因此，**原版技术性开发从不添加任何不在可写注册表内的游戏内容，只会用各种手段模拟这些游戏内容**。数据包本质上是一个文件夹或压缩文件。一个数据包仅对特定的游戏世界有效，它被储存在 `.minecraft\saves\<存档名称>\datapacks` 中。数据包可以是文件夹，也可以是 `.zip` 类型的压缩文件。同一个  `datapacks` 文件夹内能存放多个数据包。

数据包有两种添加方式——

1. 手动添加
直接将数据包添加至 `.minecraft\saves\<存档名称>\datapacks`。
2. 创建世界时添加数据包

在创建新的世界界面，选择 **更多**，点击 **数据包** 选项，此时会进入选择数据包窗口，类似于资源包选项的窗口，可在“可用”一栏内选用数据包，只有“已选”一栏的数据包有效，且数据包的加载顺序可以在该栏中调换。点击 **打开包文件夹** 选项后游戏会弹出一个临时的文件夹，此时可以将数据包拖入其中。



图 1.7 选择数据包窗口

当一个存档中存在多个有效的已启用数据包时，游戏会根据数据包的顺序加载其内容，这里的“有效”是指数据包有合法的元数据且数据包内无任何语法错误。已启用数据包的加载顺序存储于 `level.dat` 中。在选择数据包窗口“已选”一栏的加载顺序表现为从下到上。

若这些数据包对同种资源进行定义，则**后加载的数据包会对先加载的数据包进行覆盖**，表明**越靠后加载的数据包其优先级越高**。可使用命令 `/datapack` 查询、修改、控制这些数据包的启用或禁用，`/datapack` 所需的权限等级为 2，以下是所有用法：

1. 启用指定数据包

`datapack enable <name>`

1.50

其中的参数： `<name>`（字符串 `brigadier:string`）^①——指定数据包的名称。必须使用单个词，可用引号括起整个字符串。可用字符有：

- 数字 `0123456789`；
- 大写字母 `ABCDEFGHIJKLMNOPQRSTUVWXYZ`；
- 小写字母 `abcdefghijklmnopqrstuvwxyz`；
- 下划线 `_`；
- 加号 `+`、减号 `-`；
- 点 `.`；
- 引号 `'`、`"`；
- 反斜杠 `\`。

如果用引号括起整个字符串，字符串内的同种引号与反斜杠前需要加上反斜杠 `\` 转义。

2. 禁用指定数据包

① 括号中内容为该参数的类型及该参数类型在注册表内的命名空间 ID，后续教程均如此。

```
datapack disable <name>
```

1.51

3. 列举所有数据包

```
datapack list [available|enabled]
```

1.52

其中的参数：[available|enabled]——可选，若设为 available 则列举所有可用数据包，无论是否启用；若设为 enabled，则仅列举已启用数据包。默认为 available。

4. 启用指定的数据包，并设置其优先级为最低或最高

```
datapack enable <name> (first|last)
```

1.53

其中的参数：(first|last)——设置 first 以将该数据包的加载位次设为**首位**，因此优先级设为**最低**；设置 last 以将该数据包的加载位次设为**末位**，因此优先级设为**最高**。

5. 启用指定的数据包，并调整其加载优先级居于另一个数据包

```
datapack enable <name> (before|after) <existing>
```

1.54

其中的参数：(before|after)——设置 before 以将该数据包的加载放于数据包 <existing> **之前 1 位**，因此优先级比数据包 <existing> **低 1 级**；设置 after 以将该数据包的加载放于数据包 <existing> **之后 1 位**，因此优先级比数据包 <existing> **高 1 级**。

<existing>（字符串 brigadier:string）——必须为一个存在并已启用的数据包的名称。可用字符与 <name> 一致。

6. 新建一个空数据包，并设置此数据包的描述，注意，被创建的数据包默认为禁用状态

```
datapack create <id> <description>
```

1.55

其中的参数：<id>（字符串 brigadier:string）——新建数据包的名称，可用字符与上述 <name> 参数一致。

<description>（文本组件 minecraft:component）——该数据包的描述，是为元数据 pack.mcmeta 内 `description` 的值。需要是文本组件，具体写法可参照第五章。

编写数据包是一个“修改——调试——再修改——再调试”的重复过程，在既有内容的基础上对数据包做出修改并保存后，游戏不会立即识别这些修改的内容，而是依旧在修改前数据包的基础上运行原先的内容。此时需要重新加载数据包。

每次玩家进入存档（或称之为启动服务端）后，游戏都会按照加载顺序加载数据包，无论是否为首次进入存档。如果新加载的数据包内含有无效数据，则使用先前版本的数据包。在游戏过程中可以用命令 `/reload` 重新加载数据包而不必退出游戏并重新进入存档，这种方式被称为**热重载**。`/reload` 需要的权限等级为 2，其语法中不需附带任何参数：

```
reload
```

1.56

但是，使用 `/reload` 和重启服务端加载数据包的加载行为不同。`/reload` 只能用于重新加载数据包标签、函数、进度、战利品表、物品修饰器、战利品表谓词和配方这些注册项，剩余的注册项无法使用 `/reload` 加载，必须通过重启服务端加载。

其中维度、世界生成这些控制世界生成方式的注册项，对于已经生成的区块或维度，也无法通过世界加载重新生成这些区块或维度。因此自定义世界生成模块需要在世界首次加载时就被应用，即需要在创建世界时添加数据包而非在世界运行过程中手动添加数据包。如果确需在在世界运行过程中修改世界生成，可在确保世界仅作调试使用的前提下删除存档中需要重新生成区块的 Anvil 文件或自定义维度文件夹使其重新加载。

小提示

删除 Anvil 文件或自定义维度文件夹是比较危险的行为，可能会删掉存档中一些重要数据，删除之前需慎重考虑！

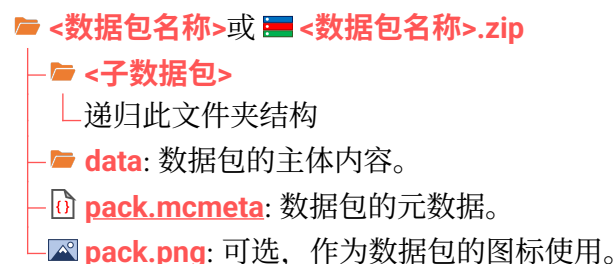
对于非数据包标签、函数、进度、战利品表、物品修饰器、战利品表谓词或配方的注册项，进入存档会出现**实验性设置 (Experimental settings)** 的警告，此时可点击创建备份并加载或我知道我在做什么！。但若这些注册项出现各种各样的错误（不一定是语法错误），则进入存档会出现**安全模式 (Safe mode)** 错误，可在官方启动器设置中打开“当《Minecraft: Java 版》启动时输出日志”一项以随时获得错误日志，或在 `.minecraft > debug` 文件夹中获取 `.txt` 输出日志以检查存在的错误。

数据包的编写是一个极为繁琐的过程，需要不断地调试、纠错，有时甚至要对其底层逻辑进行重构。在编写数据包之前，读者应提前做好规划，对其可行性进行初步的研究，还要考虑数据包运行过程中的流畅性、玩家游玩过程中的平衡性。编写过程合理使用文件层级，对文件适当分类，以免内容混乱，降低文件可读性。

原版数据包位于 `.minecraft > versions > <版本号> > <版本号>.jar > data`，是编写自定义数据包的重要依据，读者可参考之。

1.5.1 数据包的基本结构

一个数据包拥有以下的基本结构：



如果该数据包以压缩文件的形式存在，则 `<数据包名称>.zip` 和 `<子数据包>`、`assets`、`pack.mcmeta`、`pack.png` 这些文件（夹）之间不要插入其他层级的文件夹。

一、元数据

`pack.mcmeta` 是数据包的**元数据 (Metadata)**。所谓元数据，就是用于决定 `<数据包名称>` 或 `<数据包名称>.zip` 这个文件（夹）是否为一个数据包的基本数据。只有当元数据存在时，游戏才能识别数据包。

`pack.mcmeta` 使用 JSON 格式，其包含的内容如下所示：

📁 文件封装

📁 **pack**: 此数据包的基本信息。

”📁📁 **description**: 任意文本，使用文本组件格式，可用于对数据包的简单介绍。此段文本会出现在选项数据包中。使用 `/datapack list` 列举数据包时，将鼠标悬停于数据包名称上也会显示此文本。

📁📁 **max_format**: 数据包最高兼容的版本号。若使用 📁 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 📁 形式或在 📁 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 `0x7fffffff`。

📁📁 **min_format**: 数据包最低兼容的版本号。若使用 📁 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 📁 形式或在 📁 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 `0`。

📁 **pack_format**: 25w31a 以前用于指定数据包版本号的字段，现已弃用，可用于兼容旧版数据包。

📁📁📁 **supported_formats**: 25w31a 以前用于指定数据包版本号兼容范围的字段，现已弃用，可用于兼容旧版数据包。

若使用 📁 形式，则精确匹配，效果与 📁 **pack_format** 一致

若使用 📁 形式，则内部包含两个整数，第一个为最低兼容的版本号，第二个为最高兼容的版本号

若使用 📁 形式，则有以下字段：

📁 **max_inclusive**: 最高兼容的版本号。

📁 **min_inclusive**: 最低兼容的版本号。

📁 **features**: 可选，用于启用实验性内容，若指定该键，则数据包必须在创建世界时添加。

📁 **enabled**: 启用实验性内容数据包的列表。

” 一个实验性内容数据包的命名空间 ID，当前版本可用值有 `minecraft:trade_rebalance`（村民交易平衡性调整）、`minecraft:redstone_experiments`（红石实验性内容）和 `minecraft:minecart_improvements`（矿车改进）。

📁 **filter**: 可选，用于指定在数据包加载列表中优先级低于该包的数据包内要禁用的内容。

📁 **block**: 禁用内容列表。

📁 一项被禁用的内容。

” **namespace**: 要禁用的命名空间，若省略则禁用所有命名空间，可使用正则表达式。

” **path**: 要禁用的资源路径，若省略则禁用所有路径，可使用正则表达式。

📁 **overlays**: 可选，用于子数据包的识别。

📁 **entries**: 可用于子数据包的列表。

📁 一个子数据包。

” **directory**: 该子数据包相对于主数据包根目录的路径。允许使用的字符有：小写字母、`0123456789`、`_` 和 `-`。

📁📁 **max_format**: 该子数据包最高兼容的版本号。若使用 📁 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 📁 形式或在 📁 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 `0x7fffffff`。

[N][I] min_format: 该子数据包最低兼容的版本号。若使用 **[I]** 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 **[N]** 形式或在 **[I]** 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 **0**。

[N][I][O] formats: 25w31a 以前用于指定子数据包版本号兼容范围的字段，现已弃用，可用于兼容旧版数据包。

若使用 **[N]** 形式，则精确匹配

若使用 **[I]** 形式，则内部包含两个整数，第一个为最低兼容的版本号，第二个为最高兼容的版本号

若使用 **[O]** 形式，则有以下字段：

[N] max_inclusive: 最高兼容的版本号。

[N] min_inclusive: 最低兼容的版本号。

例如，下面是 1.21.11 版本的一个标准 **[O] pack.mcmeta** 文件：

```
pack.mcmeta
1 {
2   "pack": {
3     "description": "The default data for Minecraft",
4     "max_format": 94.1,
5     "min_format": 94.1,
6   }
7 }
```

例 1.7

尝试在 1.21.11 版本中禁用原版所有的进度。

解

原版所有进度位于路径 **[O] data > minecraft > advancement**，命名空间为 **minecraft**，只需在元数据中配置：

```
pack.mcmeta
1 {
2   "pack": {
3     "description": "The default data for Minecraft",
4     "max_format": 94.1,
5     "min_format": 94.1,
6   },
7   "filter": {
8     "block": [
9       {
10        "namespace": "minecraft",
11        "path": "advancement"
12      }
13    ]
14  }
15 }
```

二、数据包版本号

元数据中有一个很重要的参数：**数据包版本号 (Data pack format)**，这是一个用于区分不同版本数据包的参数。每当 Mojang 对数据包做出修改时，版本号都会发生变动。在 1.19.4 以前，数据包版本号一般一个大版本变更一次；自 1.19.4 起，由于 Mojang 对技术性开发的更新变得频繁，版本号一般每个快照变更一次。数据包应当使用其所在游戏版本的版本号，由于 Mojang 对数据包的改动可能是颠覆性的，版本号不对应可能会出现错误。

1.21.8 以前的版本号均为整数，例如，1.21.8 的数据包版本号为 81，25w31a 是 1.21.9 的快照，其引入了**次要版本号 (Minor versions)**的概念，原先的整数形式的版本号为**主要版本号 (Major versions)**，25w31a 是第一个使用此版本号格式的版本，是为 82.0。同一个主版本号内的数据包可以向下兼容，例如 83.1 的数据包可以兼容 83.0 的数据包。

下表罗列了所有主版本使用的数据包版本号，不包括快照版本。包含快照版本的数据包版本号参考附录 II.1 中的表 II.1。

表 1.8 数据包版本号

游戏版本	数据包版本号	游戏版本	数据包版本号
1.13 ~ 1.14.4	4	1.20.3 ~ 1.20.4	26
1.15 ~ 1.16.1	5	1.20.5 ~ 1.20.6	41
1.16.2 ~ 1.16.5	6	1.21 ~ 1.21.1	48
1.17 ~ 1.17.1	7	1.21.2 ~ 1.21.3	57
1.18 ~ 1.18.1	8	1.21.4	61
1.18.2	9	1.21.5	71
1.19 ~ 1.19.3	10	1.21.6	80
1.19.4	12	1.21.7 ~ 1.21.8	81
1.20 ~ 1.20.1	15	1.21.9 ~ 1.21.10	88.0
1.20.2	18	1.21.11	94.1

游戏允许编写者在元数据内指定数据包版本号的区间以使数据包兼容多个版本。但由于在不同版本中 `pack.mcmeta` 本身的格式也会发生变化，数据包版本号需要进行校验。不过，**这个校验仅作为“门槛”，数据包能否运行取决于其实际内容，而非元数据声明**。在 26.1 以前，校验失败会现实“已损坏或不兼容”；而在 26.1 以后，校验失败会直接认为元数据无效，从而不识别此数据包。

校验规则以 25w31a (1.21.9) 为分水岭实行“新旧双轨制”，以下分类讨论：

1. 如果数据包仅适用于 25w31a 之前，则元数据中：

必须使用的字段：`pack_format`

可以使用的字段：`supported_formats`。若使用，则此区间必须包含 `pack_format` 的值，且最大值不能低于 16，因为此字段是在 23w31a 引入的。

不能使用的字段：`max_format` 和 `min_format`

2. 如果数据包仅适用于 25w31a 及之后，则元数据中：

必须使用的字段：`max_format` 和 `min_format`

不能使用的字段：`pack_format` 和 `supported_formats`

3. 如果数据包同时适用于 25w31a 之前及之后，则必须同时指定 `N pack_format`、`N {} supported_formats`、`N max_format` 和 `N min_format`，且必须满足以下要求：
- (1) **区间验证：**
`N pack_format` 必须落在兼容区间内。
 - (2) **对最低版本号的验证：**
`N {} supported_formats` 的下限必须与 `N min_format` 相等。
 - (3) **对最高版本号的验证，以下两种方案二选一：**
 - ① `N {} supported_formats` 的上限与 `N max_format` 相等。
 - ② `N {} supported_formats` 的上限固定为 81，此时最高版本号由 `N max_format` 决定。

例 1.8

现需要编写一个适用于 1.20.5 至 1.21.11 的数据包，尝试编写其元数据。

解

查表 1.8，1.20.5 的版本号为 41，1.21.11 的版本号为 94.1。因为此数据包同时适用于 25w31a 之前及之后的版本，需要同时指定 `N pack_format`、`N {} supported_formats`、`N max_format` 和 `N min_format`。

首先，`N pack_format` 的值需要在 41 和 94.1 之间，此处直接写 41。其次，可将 `N {} supported_formats` 的下限调整为与 `N min_format` 一致，上限设为 81，`N max_format` 设为 94.1。故元数据可写为：

`pack.mcmeta`

```

1  {
2    "pack": {
3      "description": "元数据"
4      "max_format": [ 94, 1 ],
5      "min_format": [ 41, 0 ],
6      "pack_format": 41,
7      "supported_formats": [ 41, 81 ]
8    }
9  }
```

三、子数据包

数据包的基础功能有限，为了搭建一套复杂的体系，有时候会使用**前置数据包（Library datapack）**。这就像是在写程序时引用“第三方库”，可以极大地降低开发难度，还避免了“重复造轮子”。这些前置数据包也是数据包，可用子数据包的形式将它们添加到开发的数据包中。

子数据包会在当前主数据包的基础上添加内容，同时也会**覆盖**主数据包相同路径的文件。不过，仅在主数据包文件夹的子层级添加一个数据包并不会让主数据包识别到这个子数据包，应当在元数据的 `{} overlays` 中配置。配置方式见上文的数据格式。注意，由于 `"" directory` 字段允许包含的字符仅有小写字母、`0123456789`、`_` 和 `-`，那么有效子数据包的名称及相对路径也只能包含这些字符。

子数据包的 `{} pack.mcmeta` 不会被识别，其元数据都应在主数据的元数据中配置。因此子数据包不能嵌套子数据包。

例 1.9

一个版本号为 88.0 的数据包需要使用 `jigsaw_marker_v1.0` 这个前置数据包作为其子包，此前置数据包使用的数据包版本号也为 88.0，尝试配置子数据包。

解

首先，将数据包 `jigsaw_marker_v1.0` 移入主数据包，文件夹结构如下：



其次，在 `pack.mcmeta` 中做如下配置：

```
pack.mcmeta
1  {
2    "overlays": {
3      "entries": [
4        {
5          "directory": "jigsaw_marker_v1.0",
6          "max_format": [ 88, 0 ],
7          "min_format": [ 88, 0 ]
8        }
9      ]
10   },
11   "pack": {
12     "description": "数据包",
13     "max_format": [ 88, 0 ],
14     "min_format": [ 88, 0 ]
15   }
16 }
```

子数据包的版本号也需要进行校验，校验规则为：

1. 如果一个子数据包仅适用于 25w31a 之前，则子数据包所在的 `overlays` 项：

必须使用的字段： `formats`

不能使用的字段： `max_format` 和 `min_format`

2. 如果一个子数据包仅适用于 25w31a 及之后，则子数据包所在的 `overlays` 项：

必须使用的字段： `max_format` 和 `min_format`

关于 `formats`：若存在其他适用于 25w31a 之前的子数据包，则此字段必须指定；若其他子数据包均仅适用于 25w31a 及之后，则此字段不能使用。

3. 如果一个子数据包同时适用于 25w31a 之前及之后，则子数据包所在的 `overlays` 项必须同时指定 `formats`、`max_format` 和 `min_format`，且必须满足以下要求：

(1) **对最低版本号的验证：** `formats` 的下限必须与 `min_format` 相等。

(2) **对最高版本号的验证，以下两种方案二选一：**

① `formats` 的上限与 `max_format` 相等。

② `N` `I` `I` `formats` 的上限固定为 81，此时最高版本号由 `N` `max_format` 决定。

除了添加前置数据包外，子数据包的机制也允许数据包作者在不同的版本之间做兼容。当游戏版本在元数据内某一个子数据包声明的适用版本之内时，该子数据包会起效，并按照列表 `I` `overlays` 内子包从下到上的顺序覆盖主数据包的相应路径，从而将主数据包在当前游戏版本不可用的内容替换为子数据包适配此游戏版本的内容。

这是一种精细化配置，作者可以在主数据包内放通用逻辑，在子数据包内放随游戏版本变动的内容。通过元数据内的 `I` `overlays` 信息配置多个版本的兼容内容，并在恰当的版本启用子数据包以覆盖主数据包。如此作者可以只发布一个总的数据包，而不必为每个游戏版本分别发布数据包。

例 1.10

一个从 24w44a 适配到 26.1-Snapshot-6（含）的数据包需要包含以下的文件（夹），尝试为此数据包配置元数据。

主数据包

- `dime-25w42a-bef`: 该子包适用于 24w44a 到 25w41a 之间（含）的版本。
- `dime-25w42a-aft`: 该子包适用于 25w42a 到 26.1-Snapshot-6 之间（含）的版本。
- `left_click_listener-25w41a-aft`: 该子包适用于 25w41a 到 26.1-Snapshot-6 之间（含）的版本。
- `data`
- `pack.mcmeta`
- `pack.png`



首先整理主数据包和各子数据包的数据包版本号，各游戏版本的快照也计入在内：

表 1.9

数据包	适用版本	版本号区间
<code>dime-25w42a-bef</code>	24w44a ~ 25w41a	58 ~ 89.0
<code>dime-25w42a-aft</code>	25w42a ~ 26.1-Snapshot-6	90.0 ~ 99.0
<code>left_click_listener-25w41a-aft</code>	25w41a ~ 26.1-Snapshot-6	89.0 ~ 99.0
主数据包	24w44a ~ 26.1-Snapshot-6	58 ~ 99.0

存在其他适用于 25w31a 之前的子数据包，故这些子数据包都需要 `N` `I` `I` `formats` 字段。完整的元数据为：

```
pack.mcmeta
1 {
2   "pack": {
3     "description": "例题数据包",
4     "pack_format": 95,
5     "supported_formats": [58, 99],
6     "min_format": 58,
7     "max_format": 99
8   },
}
```

```

9     "overlays": {
10       "entries": [
11         {
12           "directory": "dime-25w42a-bef",
13           "formats": [58, 89],
14           "min_format": 58,
15           "max_format": [89, 0]
16         },
17         {
18           "directory": "dime-25w42a-aft",
19           "formats": [90, 99],
20           "min_format": [90, 0],
21           "max_format": [99, 0]
22         },
23         {
24           "directory": "left_click_listener-25w41a-aft",
25           "formats": [89, 99],
26           "min_format": [89, 0],
27           "max_format": [99, 0]
28         }
29       ]
30     }
31   }

```

四、data 文件夹

📁 **data** 文件夹是存储数据包主要内容的文件夹，下面展示了📁 **data** 文件夹的基本结构，这些文件（夹）就是表 1.2 所展示的可写注册表以及其他一些配置项的路径，它们不一定必须全部存在，游戏会根据指定的资源路径读取可写注册表中的内容，若相应的可写注册表需要存在，则必须有正确的资源路径和文件（夹）名称。

一个📁 **data** 文件夹中可以存在多个不同的命名空间，而命名空间 **minecraft** 下的内容会覆盖原版游戏内容。

在命名空间下的这些文件夹中，📁 **function** 内的文件使用 **.mcf** 格式，📁 **structure** 内的文件使用 **.nbt** 格式，除📁 **datapacks** 外其余文件夹内的文件一律使用 **.json** 格式，编写时务必使用正确的编译软件打开它们。此外，除了📁 **datapacks** 的文件夹内部都是可以自由指定资源路径的，那么在各游戏资源的命名空间 ID 中就可以使用这些资源路径。可参考例 1.1。

📁 **data**

- └─ 📁 **<命名空间>**
 - └─ 📁 **advancement**: 进度注册表
 - └─ 📁 **banner_pattern**: 旗帜图案注册表
 - └─ 📁 **cat_sound_variant**: 猫音效变种注册表
 - └─ 📁 **cat_variant**: 猫的变种注册表
 - └─ 📁 **chat_type**: 聊天类型注册表
 - └─ 📁 **chicken_sound_variant**: 鸡音效变种注册表

- **chicken_variant**: 鸡的变种注册表
- **cow_sound_variant**: 牛音效变种注册表
- **cow_variant**: 牛的变种注册表
- **damage_type**: 伤害类型注册表
- **datapacks**: 内置数据包, 均为功能数据包
- **dialog**: 对话框注册表
- **dimension**: 维度注册表
- **dimension_type**: 维度类型注册表
- **enchantment**: 魔咒注册表
- **enchantment_provider**: 魔咒提供器注册表
- **frog_variant**: 青蛙的变种注册表
- **function**: 函数
- **instrument**: 山羊角乐器注册表
- **item_modifier**: 物品修饰器注册表
- **jukebox_song**: 唱片机曲目注册表
- **loot_table**: 战利品表注册表
- **painting_variant**: 画的变种注册表
- **pig_sound_variant**: 猪音效变种注册表
- **pig_variant**: 猪的变种注册表
- **predicate**: 谓词注册表
- **recipe**: 配方注册表
- **structure**: 结构
- **tags**: 数据包标签
- **test_environment**: 测试环境注册表
- **test_instance**: 测试实例注册表
- **timeline**: 时间线注册表
- **trade_set**: 交易集注册表
- **trial_spawner**: 试炼刷怪笼配置注册表
- **trim_material**: 盔甲纹饰材料注册表
- **trim_pattern**: 盔甲纹饰图案注册表
- **villager_trade**: 村民交易注册表
- **wolf_sound_variant**: 狼音效变种注册表
- **wolf_variant**: 狼的变种注册表
- **world_clock**: 世界时钟注册表
- **worldgen**: 世界生成模块
 - **biome**: 生物群系注册表
 - **configured_carver**: 已配置的雕刻器注册表
 - **configured_feature**: 已配置的地物注册表
 - **density_function**: 密度函数注册表
 - **flat_level_generator_preset**: 超平坦世界生成预设注册表
 - **multi_noise_biome_source_parameter_list**: 多噪声参数列表注册表

- **noise**: 噪声注册表
- **noise_settings**: 噪声设置注册表
- **placed_feature**: 已放置的地物注册表
- **processor_list**: 处理器列表注册表
- **structure**: 已配置的结构地物注册表
- **structure_set**: 结构集注册表
- **template_pool**: 结构池注册表
- **world_preset**: 世界预设注册表
- **zombie_nautilus_variant**: 僵尸鹦鹉螺变种注册表

1.5.2 实验性内容 *

自 22w42a 起, Minecraft 部分更新内容会以内置数据包的形式加入游戏, 使玩家可以提前体验这些内容。这些内容被称为**实验性内容 (Experiments)**。在当前版本 (26.1), 可用的实验性内容有三项: 村民交易平衡性调整、红石实验性内容和矿车改进。

所有实验性内容都是**功能数据包 (Feature datapack)**的形式, 启用这些实验性内容的方式有两种: 一是在选择数据包窗口选择功能数据包; 二是创建新世界时点击实验性内容从而操控这些实验性内容的开关。



图 1.8 试验性内容窗口

数据包无法直接修改游戏代码, 但功能数据包似乎“注册”了新的游戏内容, 功能数据包是否有其独特的行为? 其实, 实验性内容分为硬编码内容和数据驱动内容, 其中的硬编码内容被称为特定组别的**功能元素 (Feature Element)**。**功能开关 (Feature Flag)**则用于启用或禁用这些功能元素。当一个功能数据包被启用时, 元数据中 `enabled` 字段启用, 相应的功能开关被打开, 其中的功能元素就能在游戏中正常运行。若一个功能数据包被关闭, 则相应的功能元素被过滤。

实验性内容除了可在新创建存档时启用或禁用外, 也可以通过修改 `level.dat` 中的 `enabled_features` 字段以在已创建的存档中启用或禁用。相应格式见 6.1 节的描述。

实验性内容中的数据驱动部分则交由数据包完成, 这些功能数据包作为子数据包存在, 存储于 `datapacks` 文件夹, 相应文件结构如下:



其中的 `<数据包名称>` 即为一个功能数据包，其结构与正常数据包无异，也需要有元数据。但这些数据包无法由自定义的数据包添加，仅由游戏内部提供，仅作参考。

1.5.3 数据包标签定义格式

小节 1.1.4 已提出了**数据包标签 (Tags in data packs)** 的概念，它是将游戏资源分类的一种办法。玩家不仅可以使用原版数据包既有的数据包标签，也可以新增或删改原有的标签。数据包标签模块在数据包内的文件结构如下：



每个注册表下的数据包标签只允许引用该注册表内的游戏资源。在数据包标签中所有可用的注册表如下表所示：

表 1.10 数据包标签可用注册表

资源类型	注册表 / 配置的路径	资源类型	注册表 / 配置的路径
旗帜图案	<code>banner_pattern</code>	画的变种	<code>painting_variant</code>
方块	<code>block</code>	兴趣点类型	<code>point_of_interest_type</code>
伤害类型	<code>damage_type</code>	药水效果	<code>potion</code>
魔咒	<code>enchantment</code>	时间线	<code>timeline</code>
实体类型	<code>entity_type</code>	村民交易	<code>villager_trade</code>
流体	<code>fluid</code>	自定义世界生成（生物群系）	<code>worldgen > biome</code>
函数	<code>function</code>	自定义世界生成（超平坦预设）	<code>worldgen > flat_level_generator_preset</code>
游戏世界	<code>game_event</code>	自定义世界生成（结构）	<code>worldgen > structure</code>
山羊角乐器	<code>instrument</code>	自定义世界生成（世界预设）	<code>worldgen > world_preset</code>
物品	<code>item</code>		

对于一个特定的数据包标签 `data > <命名空间> > tags > <注册名> > <标签>.json`，引用它的方式是带 `#` 号的命名空间 ID，其中 `<注册表>` 层级不书写：

#<命名空间>:<标签>

1.57

若命名空间不写，则默认使用 `minecraft` 内的数据包标签。`<注册表>` 层级下可以添加一定的路径。例如，`data\<命名空间>\tags\<注册名>\<路径>\<标签>.json` 的引用格式为

`#<命名空间>:<路径>/<标签>`

1.58

所有的数据包标签 `.json` 文件，无论其所属的注册表，一律有如下的格式：

❏ 文件封装

❏ **replace**: 指定此标签的引用是否覆盖较低优先级数据包中同命名空间内的同名标签，若设为 `true`，则忽略较低优先级数据包内的引用；若设为 `false`，则此标签内的引用作为对同名标签内引用内容的补充。默认为 `false`。

❏ **values**: 此标签引用的游戏资源，必须引用同类型的游戏资源。可以引用游戏资源本身，也可以引用其他的同类型数据包标签。

❏ **id**: 一个被引用游戏资源的命名空间 ID。

❏ **id**: 一个被引用的同类型数据包标签，需要带 `#` 号。

❏ 引用游戏资源的完整格式。

❏ **id**: 一个被引用游戏资源的命名空间 ID 或同类型数据包标签。

❏ **required**: 用 `false` 表示该条目是可选的，若该条目 **id** 所述内容不存在，则不会使标签加载失败。默认为 `true`。

例 1.11

原版存在一个名为 `#air` 的方块标签，有三种方块属于这个标签：空气、洞穴空气和虚空空气，试编写这个标签。

解

这个标签没有使用命名空间，默认命名空间为 `minecraft`。首先确定这个标签的文件路径：

```
data
├── minecraft
│   ├── tags
│   │   └── block
│   │       └── air.json
```

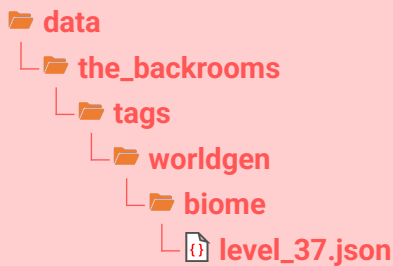
标签内容如下所示：

minecraft > tags > block > air.json

```
1 {
2   "values":[
3     "minecraft:air",
4     "minecraft:void_air",
5     "minecraft:cave_air"
6   ]
7 }
```


例 1.12

有一个生物群系标签如下所示：



- (1) 写出该标签的引用方式。
- (2) 同个数据包内已有如下的生物群系，尝试在该标签中引用这些生物群系。



解

- (1) `the_backrooms` 是命名空间，`tags` 是标签的路径，`worldgen` 和 `biome` 是标签内注册表的路径，因此该标签的引用方式为 `#the_backrooms:level_37`。
- (2) `the_backrooms` 是命名空间，`worldgen` 和 `biome` 是注册表的路径，因此这些生物群系的命名空间 ID 分别为 `the_backrooms:level_37/normal`、`the_backrooms:level_37/deep_water` 和 `the_backrooms:level_37/dark_zone`，现在在标签内引用它们：

data > the_backrooms > tags > worldgen > biome > level_37.json

```

1 {
2   "values":[
3     "the_backrooms:level_37/normal",
4     "the_backrooms:level_37/deep_water",
5     "the_backrooms:level_37/dark_zone"
6   ]
7 }
```

原版的一些数据包标签具有特殊的行为。例如，实体标签 `#arthropod` 引用的实体均被视为节肢生物，会受到节肢杀手魔咒的作用，如果往标签中添加新的实体，则新使用的实体也会被视为节肢生物。所有的原版数据包标签列举于附录 II.5。

1.6 资源包

为了搭配所制作的小游戏、冒险地图或原版模组，使得游戏的观感和体验感提高，作者通常会系统性地改变游戏的外观，例如方块的纹理、外形等。于是就需要使用资源包。

资源包 (Resource pack) 允许玩家在不修改源代码的情况下自定义纹理、模型、声音、语言等外观性资源，对客户端有效。资源包本质上是一个文件夹或压缩文件，被储存在 `.minecraft > resourcepacks` 中，同一个 `resourcepacks` 文件夹内能存放多个资源包。选项资源包窗口“可用”一栏仅罗列 `resourcepacks` 文件夹内的所有的有效资源包，可在这一栏选用资源包，只有位于“已选”一栏的资源包有效。点击打开包文件夹后可以手动添加资源包。



图 1.9 选择资源包窗口

在游戏中可以同时使用多个资源包，这些资源包按照“已选”一栏中从下到上的顺序依次加载，资源包的加载顺序可以在该栏中调换。和数据包类似，若这些资源包对同种资源的外观进行定义，则**后加载的资源包会对先加载的资源包进行覆盖，表明越靠后加载的资源包其优先级越高。**

资源包也可以以压缩包的形式存放在存档文件夹中，这时资源包作为**世界指定资源包 (World specific resources)** 使用，仅在当前存档起作用，且会使该资源包的优先级设为最高，并将已定义的资源外观覆盖选项资源包中已启用的资源包。有效的世界指定资源包必须以 `resources.zip` 为压缩文件名。

在服务器中，管理员可在 `server.properties` 中的 `resource-pack` 一项指定一个 `.zip` 文件的下载地址，从而将此 `.zip` 文件设为服务器的指定资源包。若启用，则游戏会强制将该资源包设为最顶层资源包且无法更改位置。

原版资源包位于 `.minecraft\versions\<版本号>\<版本号>.jar\assets`，是制作自定义资源包的重要依据，读者可参考之。

1.6.1 资源包的基本结构

一个资源包拥有以下的基本结构：

📁 <资源包名称>或 🇺🇸 <资源包名称>.zip

- 📁 <子资源包>
 - └ 递归此文件夹结构
- 📁 assets: 资源包的主体内容。
- 📄 pack.mcmeta: 资源包的元数据。
- 🖼️ pack.png: 可选，作为资源包的图标使用。

如果该资源包以压缩文件的形式存在，则 🇺🇸 <资源包名称>.zip 和 📁 <子数据包>、📁 assets、📄 pack.mcmeta、🖼️ pack.png 这些文件之间不要插入其他层级的文件夹。若该资源包为世界指定资源包，则名称一定为 🇺🇸 resources.zip。

资源包中 📁 assets 用于存放各种资源文件，📄 pack.mcmeta 作为资源包的元数据 (Metadata) 使用。和数据包一样，所谓元数据，就是用于决定 📁 <资源包名称> 或 🇺🇸 <资源包名称>.zip 这个文件 (夹) 是否为一个资源包，只有当元数据存在时，游戏才能识别资源包。

📄 pack.mcmeta 包含的内容如下所示：

📄 文件封装

- 📄 pack: 此资源包的基本信息。
 - ”📄📄 description: 任意文本，使用文本组件格式，可用于对资源包的简单介绍。此段文本会出现在选项资源包中。
 - 📄📄 max_format: 资源包最高兼容的版本号。若使用 📄 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 📄 形式或在 📄 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 0x7fffffff。
 - 📄📄 min_format: 资源包最低兼容的版本号。若使用 📄 形式，则内部包含两个整数，第一个为主要版本号，第二个为次要版本号。若使用 📄 形式或在 📄 形式内只填写一个数值，则视为只写主要版本号，次要版本号默认为次要版本号 0。
 - 📄 pack_format: 25w31a 以前用于指定资源包版本号的字段，现已弃用，可用于兼容旧版资源包。
 - 📄📄📄 supported_formats: 25w31a 以前用于指定资源包版本号兼容范围的字段，现已弃用，可用于兼容旧版资源包。
 - 若使用 📄 形式，则精确匹配，效果与 📄 pack_format 一致
 - 若使用 📄 形式，则内部包含两个整数，第一个为最低兼容的版本号，第二个为最高兼容的版本号
 - 若使用 📄 形式，则有以下字段：
 - 📄 max_inclusive: 最高兼容的版本号。
 - 📄 min_inclusive: 最低兼容的版本号。
- 📄 language: 可选，用于添加选项卡中的语言，可以添加多个语言。
 - 📄 <语言代码>: 一个新建的语言，键名按照 <语言>_<地区> 的格式，其与 📁 assets\minecraft\lang 中同名的 .json 文件相对应。
 - 📄 bidirectional: 布尔值，若为 true，则按照从右到左的格式显示。默认为 false。
 - ” name: 语言的名称。
 - ” reigon: 国家或地区的名称。
- 📄 filter: 可选，用于指定在资源包加载列表中优先级低于该包的资源包内要忽略的内容。

- ❏ **block**: 忽略内容列表。
 - ❏ 一项被忽略的内容。若此项为空则完全忽略所有优先级低的资源包。
 - ” **namespace**: 要忽略的命名空间, 若省略则忽略所有命名空间, 可使用正则表达式。
 - ” **path**: 要忽略的资源路径, 若省略则忽略所有路径, 可使用正则表达式。
- ❏ **overlays**: 可选, 用于子资源包的识别。
- ❏ **entries**: 可用子资源包的列表。
 - ❏ 一个子资源包。
 - ” **directory**: 该子资源包相对于主资源包根目录的路径。允许使用的字符有: 小写字母、`0123456789`、`_` 和 `-`。
 - N**❏ **max_format**: 该子资源包最高兼容的版本号。若使用❏形式, 则内部包含两个整数, 第一个为主要版本号, 第二个为次要版本号。若使用**N**形式或在❏形式内只填写一个数值, 则视为只写主要版本号, 次要版本号默认为次要版本号`0x7fffffff`。
 - N**❏ **min_format**: 该子资源包最低兼容的版本号。若使用❏形式, 则内部包含两个整数, 第一个为主要版本号, 第二个为次要版本号。若使用**N**形式或在❏形式内只填写一个数值, 则视为只写主要版本号, 次要版本号默认为次要版本号`0`。
 - N**❏❏ **formats**: 25w31a 以前用于指定子资源包版本号兼容范围的字段, 现已弃用, 可用于兼容旧版资源包。
 - 若使用**N**形式, 则精确匹配
 - 若使用❏形式, 则内部包含两个整数, 第一个为最低兼容的版本号, 第二个为最高兼容的版本号
 - 若使用❏形式, 则有以下字段:
 - N** **max_inclusive**: 最高兼容的版本号。
 - N** **min_inclusive**: 最低兼容的版本号。

例如, 下面是 1.21.11 版本的一个标准❏`pack.mcmeta` 文件:

```
pack.mcmeta
1 {
2   "pack": {
3     "description": "The default look and feel of Minecraft",
4     "max_format": 75.0,
5     "min_format": 75.0,
6   }
7 }
```

和数据包一样, **资源包版本号 (Resource pack format)** 是一个用于区分不同版本数据包的参数。每当 Mojang 对资源包做出修改时, 版本号都会发生变动。同样, 1.21.8 以前的版本号均为整数, 例如, 1.21.8 的数据包版本号为 64, 25w31a 引入了次要版本号, 是为 65.0。同一个主版本号内的资源包可以向下兼容, 例如 65.2 的资源包可以兼容 65.0 的资源包。

下表罗列了所有主版本使用的资源包版本号, 不包括快照版本。包含快照版本的资源包版本号参考附录 II.1 中的表 II.1。

表 1.11 资源包版本号

游戏版本	资源包版本号	游戏版本	资源包版本号
1.6.1 ~ 1.8.9	1	1.20.2	18

(续表)

游戏版本	资源包版本号	游戏版本	资源包版本号
1.9 ~ 1.10.2	2	1.20.3 ~ 1.20.4	22
1.11 ~ 1.12.2	3	1.20.5 ~ 1.20.6	32
1.13 ~ 1.14.4	4	1.21 ~ 1.21.1	34
1.15 ~ 1.16.1	5	1.21.2 ~ 1.21.3	42
1.16.2 ~ 1.16.5	6	1.21.4	46
1.17 ~ 1.17.1	7	1.21.5	55
1.18 ~ 1.18.2	8	1.21.6	63
1.19 ~ 1.19.2	9	1.21.7 ~ 1.21.8	64
1.19.3	11	1.21.9 ~ 1.21.10	69.0
1.19.4	12	1.21.11	75.0
1.20 ~ 1.20.1	15		

资源包的版本号同样具有校验规则，也以 25w31a (1.21.9) 为分水岭实行“新旧双轨制”，以下分类讨论：

1. 如果资源包仅适用于 25w31a 之前，则元数据中：

必须使用的字段：`N pack_format`

可以使用的字段：`N supported_formats`。若使用，则此区间必须包含 `N pack_format` 的值，且最大值不能低于 16，因为此字段是在 23w31a 引入的。

不能使用的字段：`N max_format` 和 `N min_format`

2. 如果资源包仅适用于 25w31a 及之后，则元数据中：

必须使用的字段：`N max_format` 和 `N min_format`

不能使用的字段：`N pack_format` 和 `N supported_formats`

3. 如果资源包同时适用于 25w31a 之前及之后，则必须同时指定 `N pack_format`、`N supported_formats`、`N max_format` 和 `N min_format`，且必须满足以下要求：

(1) **区间验证：**

`N pack_format` 必须落在兼容区间内。

(2) **对最低版本号的验证：**

`N supported_formats` 的下限必须与 `N min_format` 相等。

(3) **对最高版本号的验证，以下两种方案二选一：**

① `N supported_formats` 的上限与 `N max_format` 相等。

② `N supported_formats` 的上限固定为 64，此时最高版本号由 `N max_format` 决定。

和子数据包一样，子资源包的版本号也需要进行校验，校验规则与主资源包的校验规则类似，如下所示：

1. 如果一个子资源包仅适用于 25w31a 之前，则子资源包所在的 `overlays` 项：

必须使用的字段：`formats`

不能使用的字段：`N max_format` 和 `N min_format`

2. 如果一个子资源包仅适用于 25w31a 及之后，则子资源包所在的 `overlays` 项：

必须使用的字段： `N` `max_format` 和 `N` `min_format`

关于 `N` `I` `I` `formats`：若存在其他适用于 25w31a 之前的子资源包，则此字段必须指定；若其他子资源包均仅适用于 25w31a 及之后，则此字段不能使用。

3. 如果一个子资源包同时适用于 25w31a 之前及之后，则子资源包所在的 `I` `overlays` 项必须同时指定 `N` `I` `I` `formats`、`N` `max_format` 和 `N` `min_format`，且必须满足以下要求：

(1) **对最低版本号的验证：** `N` `I` `I` `formats` 的下限必须与 `N` `min_format` 相等。

(2) **对最高版本号的验证，以下两种方案二选一：**

① `N` `I` `I` `formats` 的上限与 `N` `max_format` 相等。

② `N` `I` `I` `formats` 的上限固定为 64，此时最高版本号由 `N` `max_format` 决定。

例 1.13

判断以下的资源包元数据是否符合版本号的校验要求。

```
pack.mcmeta
1  {
2    "pack": {
3      "pack_format": 55,
4      "supported_formats": {
5        "min_inclusive": 55,
6        "max_inclusive": 64
7      },
8      "description": {
9        "translate": "森罗物语: 装饰"
10     },
11     "min_format": [ 55, 0 ],
12     "max_format": [ 1000, 0 ]
13   }
14 }
```

解

`N` `pack_format`、`N` `I` `I` `supported_formats`、`N` `max_format` 和 `N` `min_format` 四个字段同时存在，说明此资源包同时适用于 25w31a 之前及之后。

首先进行区间验证：`N` `pack_format` 的值在兼容区间内。

其次对最低版本号进行验证：`N` `I` `I` `supported_formats` 的下限与 `N` `min_format` 相等。

最后对最高版本号进行验证，`N` `I` `I` `supported_formats` 的上限与 `N` `max_format` 不相等。

再检查，`N` `I` `I` `supported_formats` 的上限为 64，`N` `max_format` 是一个大于 64 的值。

故此资源包的版本号编写正确。

下面展示了 `assets` 文件夹的基本结构，这些文件（夹）不一定必须全部存在，游戏会根据指定的资源路径读取资源包中的内容，因此若相应的资源文件（夹）需要存在，则必须有正确的资源路径和文件（夹）名称。

assets

└─ <命名空间>

└─ atlases: 纹理图集

- **blockstates**: 方块状态映射
- **equipment**: 装备模型
- **font**: 字体
- **items**: 物品模型映射
- **lang**: 语言
- **models**: 烘焙模型
- **particles**: 粒子纹理定义
- **post_effect**: 后处理管线
- **sounds**: 声音
- **shaders**: 着色器
- **texts**: 文本
- **texture**: 纹理
- **waypoint_style**: 路径点样式
- **gpu_warnlist.json**: GPU 警告列表
- **regional_compliancies.json**: 地区合规性警告
- **sounds.json**: 声音事件定义文件

1.6.2 GPU 警告列表 *

资源包负责游戏的画面渲染。部分计算机显卡太旧、驱动版本不匹配，或者 GPU 属于某些已知会造成游戏崩溃的型号，因此资源包内存在 `gpu_warnlist.json` 这个用于自检硬件兼容性的配置文件。其格式如下所示：

文件封装

- **renderer**: 需要显示渲染器警告的渲染器名称（显卡型号）。
 - 一个渲染器名称，使用正则表达式。
- **version**: 需要显示渲染器版本警告的渲染器版本（通常为显卡驱动的版本号）。
 - 一个渲染器版本，使用正则表达式。
- **vendor**: 需要显示渲染器厂商警告的渲染器生产厂商。
 - 一个渲染器厂商，使用正则表达式。

例如，原版资源包的 `gpu_warnlist.json` 文件内容如下：

```
assets > minecraft > gpu_warnlist.json
1  {
2    "renderer" : [],
3    "version" : [
4      "\\bMetal\\b"
5    ],
6    "vendor" : []
7  }
```

这份配置专门针对 macOS 用户。当渲染器版本信息中包含独立的 Metal 单词时，会触发警告。因为 macOS 使用苹果系统独特的图形接口 Metal，而不使用 OpenGL。

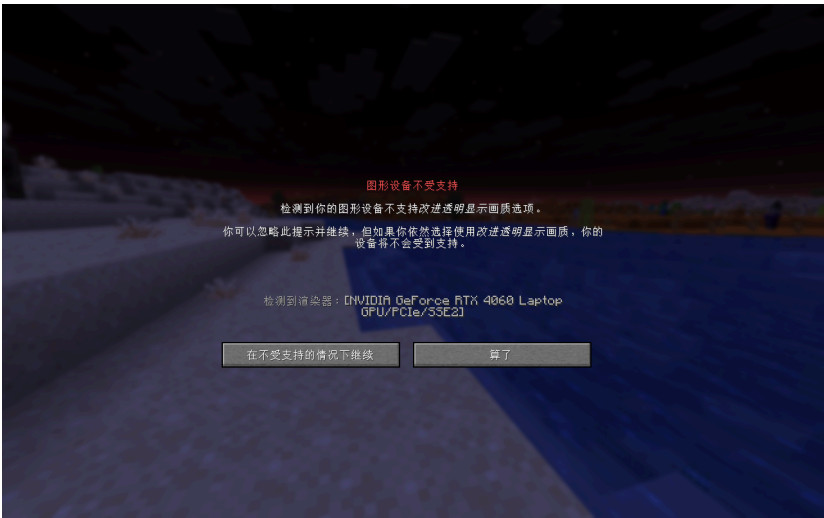


图 1.10 GPU 警告页面

警告页面会在玩家开启游戏极佳画质时出现。不过，这个页面以及 GPU 警告列表的配置都只是警告机制，并不是禁止硬件被匹配到的计算机运行 Minecraft。游戏具体的运行情况取决于硬件本身。例如，以下的配置文件可以使得持有 RTX 4060 显卡的计算机显示警告页面，显示的页面如图 1.10 所示。

```
assets > minecraft > gpu_warnlist.json
1 {
2   "renderer": [
3     ".*RTX 4060.*"
4   ],
5   "version": [],
6   "vendor": []
7 }
```

1.6.3 地区合规性警告 *

部分国家或地区针对游戏颁布了一定的法律法规，资源包内 `regional_compliances.json` 可以相应地设置游戏在运行一段时间后出现的弹窗警告，其格式如下所示：

- 文件封装
 - <地区代码>: 键名为 ISO 3166-1 三位字母地区代码，游戏会针对该系统地区进行弹窗。
 - 一项弹窗。
 - `delay`: 第一次弹窗时游戏的运行时间，单位为分钟，默认值为 0。
 - `period`: 弹窗周期，单位为分钟。
 - `title`: 弹窗标题，需要是一个翻译标识符，详见小节 5.2.2。
 - `message`: 弹窗的具体信息，需要是一个翻译标识符，详见小节 5.2.2。

原版资源包内的 `regional_compliances.json` 内容如下：

```
assets > minecraft > gpu_warnlist.json
1 {
2   "KOR" : [
3     {
```

```
4     "delay": 1440,  
5     "period": 60,  
6     "title": "compliance.playtime.greaterThan24Hours",  
7     "message": "compliance.playtime.message"  
8   },  
9   {  
10    "period": 60,  
11    "title": "compliance.playtime.hours",  
12    "message": "compliance.playtime.message"  
13  }  
14 ]  
15 }
```

它只设置了针对韩国的弹窗警告：每 1 小时提醒一次，连续游戏时间超过 24 小时也会提醒。

1.7 游戏机制

虽然技术性开发是能够调控游戏运行方式的手段，但开发成果还是不免受到游戏机制的制约。在时间上受到游戏循环驱动的影响，以游戏刻为单位计算内容；在空间上受到区块加载的影响，绝大多数操作都只能在允许运算的区块中进行。本节旨在介绍游戏加载、运行、更新的一些基本游戏机制。

1.7.1 端

Minecraft 的架构是**客户端-服务端模型**，顾名思义，Minecraft 使用**客户端（Client）**和**服务端（Server）**来运作自身。这两个**端（Sides）**之间的通信是由**封包（Packet）**实现的。在网络工程中，这个概念一般译为“数据包”，而 Minecraft 中另有一个叫 Datapack（数据包）的概念，故 Packet 在 Minecraft 技术性开发领域会特地译为“封包”。

然而，仅通过客户端和服务端理解 Minecraft 的运作是远远不够的，因为 Minecraft 的架构还包括**物理端（Physical sides）**和**逻辑端（Logical sides）**，并且物理端和逻辑端分别具有各自的客户端和服务端。

一、物理客户端

物理客户端（Physical client）是指下载游戏版本得到的 `<version>.jar` 文件，它的默认文件路径为 `.minecraft\<版本号>\<version>.jar`。物理客户端包含了游戏的全部内容，也包含了内置的客户端和服务端，即**逻辑客户端（Logical client）**和**逻辑服务端（Logical server）**，其中逻辑服务端又称**内置服务器（Integrated server，或译为集成服务端）**。内置服务器会受到客户端的影响。

逻辑客户端负责接收来自玩家的输入、处理资源包、渲染游戏画面，并将数据输送给逻辑服务端处理；逻辑服务端负责处理由客户端发送的数据，运行游戏逻辑。例如，当玩家在游戏中移动时，客户端会根据玩家输入的移动方向渲染玩家此时的游戏画面，同时又将玩家移动的信息通过封包发送给逻辑服务端，逻辑服务端计算玩家的坐标、玩家周围是否存在任何的碰撞箱阻止玩家

移动，将计算结果通过封包返还给逻辑客户端，渲染玩家移动的游戏画面。客户端的渲染会与服务端产生不一致的情况，例如标记是一种仅存在于服务端的实体，在客户端上并不会渲染标记，参见 6.8。

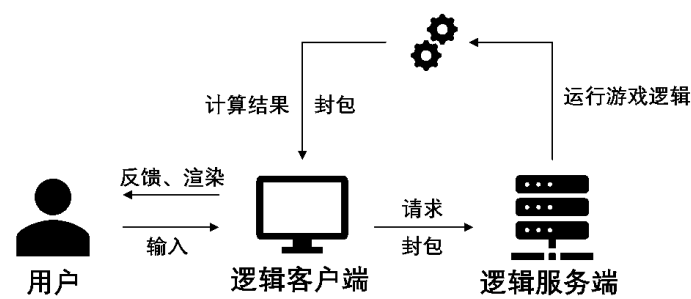


图 1.11 逻辑客户端和逻辑服务端的运行流程

即使是进行单人游戏，Minecraft 依旧会在玩家进入本地世界时创建一个内置服务器，在本地世界关闭时内置服务器即被关闭。这个内置服务器可以开放至局域网，从而将单人游戏开放为局域网联机的多人游戏。此时内置服务器拥有一个地址，其格式为

<IPv4 地址>:<端口>

1.59

局域网联机的 IPv4 地址可由 CMD 的 `ipconfig` 命令查询。端口是一个数值，可以自由指定，范围为 0 至 65535（含两端）。除了通过暂停游戏的对局域网开放选项外，玩家还可以通过命令 `/publish` 开放内置服务器，该命令所需权限等级为 4，且仅能在单人游戏中使用，其语法为：

`publish [<allowCommands>] [<gamemode>] [<port>]`

1.60

其中的参数：

- `<allowCommands>`（布尔值 `brigadier:bool`）——可选，指定是否启用命令，默认为否。
- `<gamemode>`（游戏模式 `minecraft:gamemode`）——指定新玩家进入游戏的游戏模式，可用值有 `survival`（生存模式）、`creative`（创造模式）、`adventure`（冒险模式）和 `spectator`（旁观模式）。若不指定，则使用该游戏世界的默认游戏模式。
- `<port>`（整数 `brigadier:integer`）——指定端口，必须为介于 0 和 65535 之间（含）的值，若不指定，则随机选择大于 1024 的端口。

例 1.14

使用命令 `/publish` 开放当前本地世界，要求关闭命令、设置玩家游戏模式为生存模式、端口指定为 12345。

解 命令为

`publish false survival 12345`

1.61

二、物理服务端

除了使用局域网联机进行多人游戏，Minecraft 提供了另一种进行多人游戏的方法，即**物理服务端 (Physical server)**。物理服务端只包含一个逻辑服务端，并不包含逻辑客户端。这意味着物理服务端只能负责服务端的任务，而不能使用户参与游戏；但同时也意味着若服主不在游戏中，服务器也不会关闭；此外，物理服务端在运行过程中只能加载一个游戏世界，切换其他游戏世界需要重启服务器。

物理服务端内的逻辑服务端又可被称为**专用服务器 (Dedicated server，或译为独立服务端)**，该逻辑服务端包含配置文件 `server.properties`，用于存储服务器的所有设置。专用服务器不会受到连接的逻辑客户端的影响。同局域网联机一样，专用服务器也拥有一个地址，其格式与语法 1.59 所述一致。

表 1.12 各种情况使用的客户端和服务端

	单人游戏	局域网多人游戏	专用服务器多人游戏
客户端	逻辑客户端	玩家各自的逻辑客户端	玩家各自的逻辑客户端
服务端	内置服务器	内置服务器	专用服务器

1.7.2 游戏刻

由于游戏不可能时时刻刻都进行计算，正常情况下，游戏以一定的频率循环驱动，即每隔一定的时间进行一次计算，计算完毕后游戏会进行休眠，此时游戏不作任何计算直到下一游戏刻。一个循环周期被称为一个**刻 (Tick)**，或**游戏刻 (Game tick，简称 gt)**。

一、刻率和帧率

每秒游戏刻的数量数由**每秒刻数 (Ticks per second，简称 TPS)** 这个指标显示；此外还有一个指标与每秒游戏刻数相关，即**每刻毫秒数 (Milliseconds per tick，简称 MSPT)**，它反映的是游戏刻计算的平均时间。TPS 是一个可变量，它可以由命令 `/tick` 修改，不做修改的默认值为 20。也就是说，正常情况下每秒有 20gt，或者称最大 TPS 频率为 20。MSPT 可以由 `F3`（调试屏幕）查看，这个统计量名称为 `ms ticks`。正常情况下 MSPT 不会大于 50，且只有当 MSPT 值不大于 50 时才能保证 TPS 维持在 20。

MSPT 与 TPS 的数量关系可表示为

$$\text{MSPT} \times \text{TPS} \leq 1000$$

(1.1)

受限于游戏中的计算量及计算机的性能，若计算量过大，MSPT 增大，则 TPS 会相应地减小，造成**掉刻**。TPS 无法维持在最大频率时，可由下式计算出实际的 TPS：

$$\text{TPS} = \frac{1000}{\text{MSPT}}$$

(1.2)

如果按照默认的每秒 20gt 的频率渲染画面，难免会产生肉眼可见的不连续画面。因此客户端渲染游戏画面时，并不是完全按照刻率渲染，而是在刻之间**补帧**以形成平滑画面。用于描述渲染频率的指标为**帧率 (Frame per second，简称 FPS)**，它反应的是客户端的每秒渲染的帧数。帧率受到客户端渲染计算量、计算机性能的影响，可以通过**最大帧率**选项控制最高 FPS。

当客户端渲染计算量较大时，FPS 会下降，造成**掉帧**。因此分析客户端画面卡顿时，可以考虑的若干可能性有：渲染计算量较大，或是游戏刻计算量较大造成渲染补帧无法形成平滑画面。若遇到画面较为流畅、但游戏内容卡顿——如实体不移动、放置破坏方块相应时间较长——则说明客户端渲染计算正常而游戏刻计算量大。

游戏的流畅程度是影响玩家游戏体验的关键因素，**无论是搭建红石电路还是制作数据包，均需要综合考虑成品对 TPS 和 FPS 的影响。**

二、命令/tick 的用法

命令 `/tick` 可用于控制游戏刻运行，该命令所需权限等级为 3。以下是所有用法。

1. 查询当前游戏刻频率，并返回性能数据。语法为：

```
tick query
```

1.62

2. 定义游戏刻频率，语法为：

```
tick rate <rate>
```

1.63

其中的参数：`<rate>`（浮点数 `brigadier:float`）——需要设置的游戏刻频率。设置后，最大 TPS 频率即为这个参数设置的值。

例 1.15

将游戏刻频率设为 40。

解

所需命令为

```
tick rate 40
```

1.64

负载正常时，MPST 应不大于 $1000 \div 40 = 25$ 。

3. 冻结游戏刻，语法为：

```
tick freeze
```

1.65

4. 步进特定数量的游戏刻，语法为：

```
tick step [<time>]
```

1.66

该语法仅能在游戏刻已冻结的情况下使用。步进结束后，游戏刻会继续冻结。

其中的参数：`[<time>]`（时间 `minecraft:time`）——步进时间长度。格式为：

`<单精度浮点数>[<单位>]`

单位可以为：`t`（游戏刻），`s`（秒）或 `d`（游戏日，1 游戏日固定为 24000 游戏刻），若不写单位，则默认单位为游戏刻。时间参数会在单位转化成游戏刻后取最近的整数。例如，`.2d` 会换算为 $24000 \times 2 = 48000$ gt。若游戏刻频率为 40，则 `.5s` 会换算为 $40 \times 0.5 = 20$ gt。

例 1.16

使游戏刻步进 10 秒。

解

所需命令为

```
tick step 10s
```

1.67

5. 停止正在进行的步进，并冻结游戏刻。语法为：

```
tick step stop
```

1.68

6. 取消冻结游戏刻，语法为：

```
tick unfreeze
```

1.69

7. 忽略游戏刻频率的限制持续进行更新，语法为：

```
tick sprint [<time>]
```

1.70

该语法用于在指定的时间 [<time>] 内尽可能快地运行游戏，结束后恢复先前的游戏刻速率并返回性能信息。参数 [<time>] 的用法与语法 1.66 中所述的一致。

8. 停止正在进行的忽略游戏刻频率进行的更新，并恢复先前的游戏刻频率。语法为：

```
tick sprint stop
```

1.71

三、游戏刻计算流程 *

Minecraft 的游戏计算内容繁多，在同一个线程中的计算不可能同步完成，各模块的计算在同一游戏刻内一定有先后顺序，这种先后顺序被称为**微时序（Microtiming，或称微观延迟）**。在一个游戏刻内，服务端的计算流程顺序如下所示^①：

1. 检查游戏是否被暂停，若已暂停，则自动保存游戏。若此时内置服务器正在主持局域网联机，则统计世界打开时间。
2. 已暂停游戏若恢复运行，则对所有玩家强制时间同步。
3. 运行服务端逻辑。内容顺序如下：
 - (1) 更新游戏刻计数器。
 - (2) 若此时正在使用 `/tick step` 步进游戏刻，则计算剩余需步进游戏刻。
 - (3) 关闭与客户端的网络自动发送队列的刷新。
 - (4) 如果游戏世界被重新加载（如使用 `/reload`），则调用 `#minecraft:load` 中的函数，调用顺序与列表 `[ ] value` 中的函数顺序一致。一个函数被调用时按 `.mcfunction` 文件内的命令顺序依次执行命令。
 - (5) *调用一次 `#minecraft:tick` 中的函数，顺序与 (4) 中所述一致。
 - (6) 遍历所有维度，遍历顺序为：主世界、下界、末地、有先后顺序的自定义维度。遍历到某个维度时，按以下流程计算：
 - ① 每隔 20 gt 对玩家同步一次该维度的时间。

^① 带*的项目表示在游戏刻冻结时忽略执行。

- ② 运行维度游戏刻逻辑, 若计算出现异常, 则游戏崩溃。游戏刻逻辑按以下流程计算:
- A. 更新世界边界。
 - B. *计算天气循环、更新降雨和雷暴计时器。
 - C. 计算日夜更替, 若玩家入睡情况满足跳过当前时间至下一次日出, 则将时间调整至下一次日出。若此时正在降雨, 则重置天气循环。
 - D. 更新内部光照等级乘数。
 - E. *更新时间。
 - F. *在非调试维度执行方块计划刻。
 - G. *在非调试维度执行流体计划刻。
 - H. *更新袭击事件。
 - I. 计算区块数据, 按以下流程计算:
 - a. *更新计算标签。
 - b. 计算需要执行区块刻的区块, 并打乱区块刻执行顺序。
 - c. *计算生物生成。
 - d. *按区块顺序执行区块刻。
 - e. *执行计划周期生成。
 - f. 更新各个玩家追踪的区块。
 - g. 更新兴趣点数据。
 - h. 卸载不需要的区块。
 - J. *计算方块事件。
 - K. 如果维度内有玩家或强加载区块, 则重置限制超时。
 - L. 如果闲置超时小于 300 gt, 则运行实体和方块计算逻辑, 按以下流程计算:
 - a. *计算末影龙战斗数据。
 - b. 遍历所有实体, 进行实体计算。
 - c. *计算方块实体。
 - M. 加载区块内未加载的实体, 卸载不需要的实体。
- (7) 检查客户端连接。
- (8) 每 600gt 向所有玩家发送更新延迟网络包。
- (9) 若存在服务器 GUI, 则更新服务器 GUI。
- (10) 对所有玩家发送正在等待发送的区块网络包, 并打开与玩家客户端的网络自动发送队列刷新。
- (11) 若与上一次自动保存已有 100 gt, 则尝试自动保存。
- (12) 运行处理队列中的事件。
4. 在客户端更新渲染距离和模拟距离。

1.7.3 区块加载

一个 Minecraft 世界在水平方向上的长宽均超过了六千万格, 如果一次性将所有内容全部加载出来, 则会消耗非常多的内存。为使游戏运行过程中不占用过多的内存, 游戏仅会加载部分区域供玩家游玩, 这些加载和数据存储的单位被称为**区块 (Chunk)**, 每一个区块均是 $16 \times 384 \times 16$ 的立方体, 一个区块又可以沿高度分割成 24 个 $16 \times 16 \times 16$ 大小的**区段 (Chunk section)**, 或称**子区块 (Sub chunk)**。在必要的情况下, 游戏会**卸载**一些区块, 并在需要的时候**加载**区块。

已卸载的区块不会处理游戏的任何事件，这其中也包括了实体的生成、活动，红石电路、命令的运行等等。若长距离执行命令，区块的加载是需要着重考虑的一方面。

一、加载等级和计算等级

游戏使用**加载等级（Load level）**和**计算等级**^①来确定一个区块的加载情况。

1. 加载等级

区块的加载等级范围可表示为 0 ~ 45（含）的整数，且等级越高，区块内加载的内容就越多。加载等级可划分为如下表所示的不同加载等级类型：

表 1.13 加载等级类型

类型	加载等级	描述
实体计算	≤ 31	可以计算实体、方块实体和计划刻，方块修改可以被追踪。
方块计算	32	不可计算但可以追踪实体，可以计算方块实体和计划刻，可以追踪方块修改。
完全加载	33	不可计算但可以追踪实体，不可以计算方块实体和计划刻，不可以追踪方块修改。
不可访问	34	大于该加载等级时，实体和方块修改都不可以追踪，方块实体和计划刻不可以计算，但可以进行初始化光照计算。
	35	可进行地形雕刻。
	36	可填充生物群系。
	37 ~ 44	结构允许生成。
卸载	45	区块已被卸载

2. 计算等级

类似于加载等级，区块的加载等级范围可表示为 0 ~ 33（含）的整数。且等级越高，区块内加载的内容就越多。区块内具体可以加载何内容由加载等级和计算综合决定。下表列举了不同加载程度的区块^②：

表 1.14 区块加载类型

区块类型	加载等级	计算等级	每游戏刻的具体行为
强加载区块	≤ 31	≤ 31	可以计算实体、方块实体和计划刻，任何命令都能正常执行。
弱加载区块	32	32	可以计算方块实体和计划刻，实体可以被命令追踪和记录，但无法计算实体。
加载边界区块	33	32	实体可以被命令追踪和记录，但不计算实体、方块实体和计划刻。
不可访问	≥ 34	33	不计算实体、方块实体和计划刻，任何命令都不可执行。

二、加载标签

一个区块加载等级和计算等级的具体数值可以由**加载标签（Load ticket）**决定。一个加载标签有**基础等级**、**标签类型**、**存活时间**和**持久化**四个属性，基础等级指一个区块被赋予某种加载标签时该区块的加载等级和计算等级（两者不一定相等），存活时间指该类加载标签能够持续的时长。以下是注册表内所有类型的加载标签：

① 无英文原文。

② 表中一种类型的区块必须同时满足该行加载等级和计算等级的要求。

1. 玩家标签

玩家标签在注册表内分为**玩家加载标签 (Player loading ticket)**和**玩家计算标签 (Player simulation ticket)**。玩家所在的区块会被赋予这玩家标签，其中加载等级由玩家加载标签赋予，计算等级由玩家计算标签赋予，此时该区块的加载等级主要取决于**渲染距离 (Render distance)**，计算等级主要取决于**模拟距离 (Simulation distance)**，渲染距离和模拟距离均可以由选项设置。渲染距离主要决定游戏世界的渲染距离，即渲染区块的数量，对FPS的影响程度较高。通常来说渲染区域以玩家所在区块为中心，在平面上为正方形，其边长为

$$a = 2d_r + 1 \tag{1.3}$$

式中： a —— 渲染区域边长。
 d_r —— 在单人游戏中为渲染距离，原版的渲染距离必须为介于 2 和 32 之间（含）的整数。
在多人游戏中为 `server.properties` 中 `view-distance` 的值。

对于上述正方形区域内的每一个区块，其加载等级均为 31。

模拟距离主要决定实体更新、计算方块计划刻、流体计划刻，这些计算主要由服务端负责，因此相对于渲染距离而言，它对TPS的影响更大。玩家所在区块的计算等级为

$$L_s = \max\{0, 31 - d_s\} \tag{1.4}$$

式中： L_s —— 计算等级。
 d_s —— 模拟距离，原版的模拟距离必须为介于 5 和 32 之间（含）的整数。

2. 传送门标签

当有实体在下界传送门或未地传送门中且正在被传送至另一个维度时，游戏将会为目的维度的目标区块赋予传送门标签，加载等级和计算等级均为 30，存活时间为 300 gt。

3. 末影龙标签

玩家与末影龙发生战斗时会在末地的 [0,0] 区块产生末影龙标签，加载等级和计算等级均为 24，存活时间不定，直到末影龙被杀死或与末影龙战斗的玩家数量清零时才撤销该标签。

4. 强制加载标签

命令 `/forceload` 所指定区块的加载等级和计算等级为 31，并被添加该标签，这是持久性标签，除非使用命令 `/forceunload` 移除标签。参见节 2.4.2。

5. 末影珍珠标签

当玩家掷出末影珍珠后，该末影珍珠所在区块每 39 gt 会被赋予该标签，加载等级和计算等级均为 31，存活时间 40 gt。

6. 临时标签

由游戏代码决定而创建，通常用于计算生物的 AI 和生成，加载等级一般大于等于 32，不设置计算等级。存活时间仅为 1 gt。

整理上述信息可得下表：

表 1.15 加载标签

标签类型	注册名称	基础等级		存活时间	持久化
		加载等级	计算等级		
玩家加载标签	<code>player_loading</code>	31	无	永久	否
玩家计算标签	<code>player_simulation</code>	无	$\max\{0, 31 - d_s\}$		

(续表)

标签类型	注册名称	基础等级		存活时间	持久化
		加载等级	计算等级		
传送门标签	portal	30		300 gt	是
末影龙标签	dragon	24		永久	否
强制加载标签	forced	31		永久	是
末影珍珠标签	ender_pearl	31		40 gt	否
临时标签	unknown	≥ 32	无	40 gt	否

三、等级传播

拥有一定加载等级和计算等级的区块可以向周围的 8 个区块传播加载等级和计算等级，每次传播时等级增加 1。加载等级的最大值为 45，等级为 45 的区块会直接从内存中卸载。计算等级的最大值为 33，此时若继续向外传播则等级会维持在 33。

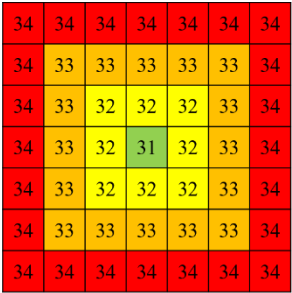


图 1.12 等级的传播

根据这种传播规则，假设有一个区块 A 被赋予了加载标签，已知其加载等级或计算等级，在周围没有其他区块具有加载标签的情况下，区块等级的传播使用**切比雪夫距离（Chebyshev distance）**来计算。则区块 B 的等级为

$$L_{z,B} = \begin{cases} L_{z,A} + d_{\infty}(A,B) , & L_{z,A} + d_{\infty}(A,B) \leq 44 \\ \text{不存在} & , \text{otherwise} \end{cases} \tag{1.5}$$

$$L_{s,B} = \min\{33, L_{s,A} + d_{\infty}(A,B)\} \tag{1.6}$$

式中： $L_{z,A}$ 、 $L_{z,B}$ 、 $L_{s,A}$ 、 $L_{s,B}$ ——下标“z”表示加载等级，“s”表示计算等级，逗号后的字母表示该等级所属的区块。

x_A 、 x_B 、 z_A 、 z_B ——区块 A 、 B 分别在 x 、 z 方向上的区块坐标。

$d_{\infty}(A,B)$ ——区块 A 、 B 之间的切比雪夫距离， $d_{\infty}(A,B) = \max\{|x_B - x_A|, |z_B - z_A|\}$

若一个区块接受到多个传播等级，则选取等级最低者作为自己的加载（计算）等级。假设一定区域内存在被赋予加载标签的区块 1、2、3、……则该区域内任意区块的加载（计算）等级可表示为

$$L_z = \begin{cases} \min_{i \geq 1} \{L_{z,i} + d_\infty\}, \min_{i \geq 1} \{L_{z,i} + d_\infty\} \leq 44 \\ \text{不存在}, \text{ otherwise} \end{cases} \tag{1.7}$$

$$L_s = \begin{cases} \min_{i \geq 1} \{L_{s,i} + d_\infty\}, \min_{i \geq 1} \{L_{s,i} + d_\infty\} \leq 44 \\ \text{不存在}, \text{ otherwise} \end{cases} \tag{1.8}$$

式中： $x、z$ ——被计算区块的区块坐标。
 $x_i、z_i$ ——区块 i 的区块坐标， $i = 1, 2, 3 \cdots$ 。
 d_∞ ——被计算区块到区块 i 的切比雪夫距离， $d_\infty = \max\{|x - x_i|, |z - z_i|\}$ 。

例 1.17

图 1.13 展示了一个区域的区块，玩家所在的区块为 P ，此时模拟距离为 5，渲染距离为 4，判断在区块 A 内能否正常执行命令。

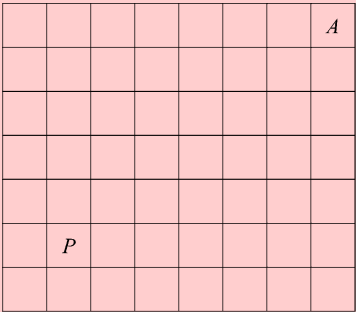
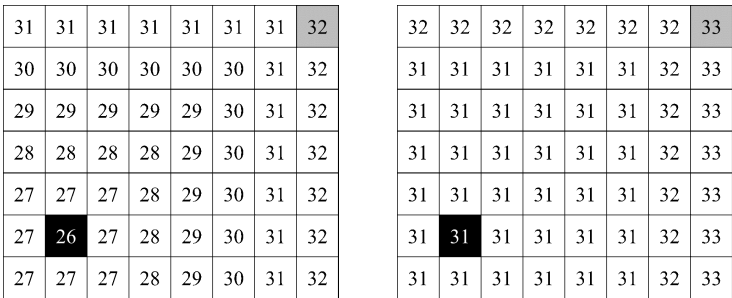


图 1.13

解

显然区块 P 被赋予了玩家标签，已知模拟距离为 5，由式 (1.4) 可得区块 P 的计算等级为 26。已知渲染距离为 4，由式 (1.3) 得以区块 P 为中心 9×9 区块的加载等级均为 31。由等级的传播，该区域的等级如图 1.14 所示。



(a) 计算等级 (b) 加载等级

图 1.14

则区块 A 的计算等级为 32，加载等级为 33，属于加载边界区块，可以使用命令追踪该区块内的实体。

例 1.18

一玩家所在区块的区块坐标为 $[5, 12]$ ，试通过调整渲染距离和模拟距离使区块 $[17, -5]$ 为强加载区块。

解

遇到这种两个区块之间相隔较远不能通过画图直接得到结果的，可先计算两个区块之间的切比雪夫距离 $d_{\infty} = \max\{|17 - 5|, |-5 - 12|\} = 17$ ，要使区块 $[17, -5]$ 为强加载区块，则该区块的加载和计算等级必须均小于等于 31，所以渲染距离必须至少为 17。设模拟距离为 d_s ，则玩家当前区块的计算等级为 $\max\{0, 31 - d_s\}$ ，切比雪夫距离为 17 的区块的计算等级 $\max\{0, 31 - d_s\} + 17 \leq 31$ ，得 $d_s \geq 17$ 。所以渲染距离和模拟距离都应大于 17。

四、闲置超时

若玩家离开某一维度的时间超过 300 gt，除非该维度中有其他玩家或有使用 `/forceload` 强制加载的区块，否则该维度会无视区块加载等级而停止方块实体、实体等有关的运算，这种限制被称为**闲置超时 (Idle timeout)**。玩家进入或离开维度时均会重置闲置超时。

1.7.4 区块刻 随机刻 计划刻 红石刻

游戏刻的计算过程中还有一些特殊的计算模式。

一、区块刻

游戏计算区块数据时，以区块为单位进行一次计算，该计算被称为**区块刻 (Chunk tick)**。区块刻按以下流程计算：

1. 按随机顺序遍历所有满足区块刻条件、加载标签类型为实体计算、水平方向上区块中心距玩家 128 格以内的区块。
 - (1) 增加区块时间。
 - (2) 如果当前为雷暴，执行雷暴相关逻辑。
 - (3) 在世界边界内执行周期生成。
2. 遍历所有满足区块刻条件的区块。
 - (1) 如果正在降雨，执行雨雪相关逻辑。
 - (2) 执行随机刻。

二、随机刻

接受到区块刻的区块会随机挑选区块内的一些方块并赋予**随机刻 (Random tick)**，接收到随机刻的方块会进行方块更新，如农作物的生长、藤蔓的蔓延、草方块的蔓延等。随机刻的赋予规则是：每一游戏刻在所有区段内随机挑选若干方块，游戏规则 `random_tick_speed` 可用于指定随机挑选方块的数量（默认为 3），每次挑选的方块可以为同一个。

假设 `random_tick_speed` 的值为 m ，则在一个游戏刻内区段内某方块被选中的概率为

$$\begin{aligned}
 p_0 &= 1 - C_0^m \left(\frac{1}{16^3} \right)^0 \left(1 - \frac{1}{16^3} \right)^m \\
 &= 1 - \left(1 - \frac{1}{16^3} \right)^m
 \end{aligned} \tag{1.9}$$

显然该方块在 t gt 内获取随机刻是独立的重复伯努利试验, 服从参数为 p_0 的几何分布, 则第 t gt 得到随机刻的概率为

$$P(X = t) = (1 - p_0)^{t-1} p_0 \quad (1.10)$$

则方块接受到随机刻的平均间隔为

$$E(X) = \sum_{t=1}^{+\infty} X_t P_t = \frac{1}{p_0} = \frac{1}{1 - \left(1 - \frac{1}{16^3}\right)^m} \quad (1.11)$$

上式结果的单位为游戏刻。若将 `random_tick_speed` 的默认值 3 代入, 且此时游戏刻率为 20, 可以得到默认的随机刻的平均间隔时间 1365.67 gt, 合 68.28 秒。若 `random_tick_speed` 的值为零, 有 $\lim_{x \rightarrow 0} E(X) = +\infty$, 即永远不会接受到随机刻。在设计方块更新的速率时可参考上述公式。不过, 上述公式是在概率论及数理统计的范围内讨论的, 并不意味着随机刻平均间隔时间一定是该值, 极端情况下可能会出现 1 gt 或超过 1000000 gt 的间隔, 只能说接受到随机刻这一事件大致服从几何分布。总体上来说, `random_tick_speed` 的值越大, 方块更新的频率就越高。

三、计划刻 *

一些方块除了会被动接受随机刻, 有时还会主动请求在未来某一游戏刻更新方块, 这种更新方式被称为**计划刻 (Schedule tick)**, 如水的流动、红石中继器的信号变更等。计划刻分为**方块计划刻**和**流体计划刻**, 分别控制普通方块和液体的计划更新。方块计划刻会根据优先级按顺序依次执行, 优先级一般为非正数, 且优先级越小, 执行时间越早。流体计划刻则没有优先级。一个游戏刻内最多执行 65536 个计划刻。超出限制数量的计划刻将延后至下一游戏刻处理。

四、红石刻

一般而言, 大部分红石元件的工作时间以 2 gt 为基本单位, 可令 2 gt 为 1 个**红石刻 (Redstone tick, 简称 rt)** 以方便计算电路的延迟。在 TPS 等于 20 的情况下, 1 rt 等效于 0.1 秒。不过, 红石刻不是真实存在的游戏机制, 是仅在红石电路或使用命令方块的命令系统中用于描述延迟的基本单位。

例 1.19

创建一个 10 秒的延迟至少需要多少红石中继器?

解

10 秒钟的延迟即 100 rt, 一个红石中继器最多可提供 4 rt 的延迟, 至少需要 $100 \div 4 = 25$ 个红石中继器。

1.7.5 方块

方块 (Block) 是构成 Minecraft 的基本单位。一般而言方块是 $1 \times 1 \times 1$ 大小的实物, 部分方块可能有特殊的大小和形状。

空气和**液体**是两类特殊的方块。其中空气包括普通空气 (命名空间 ID `minecraft:air`)、洞穴空气 (命名空间 ID `minecraft:cave_air`) 和虚空空气 (命名空间 ID `minecraft:void_air`) 三种。洞穴空

气是生成世界时创建洞穴生成的空气，虚空空气是生成于建造区域以外的空气。这三类空气共同填满了任何未被其他方块占用的空间，因此使用命令移除其他方块是通过在这些方块的位置上放置空气从而实现的。此外，物品形式的空气也广泛存在于物品栏中未被其他物品填充的槽位，使用命令 `/item` 移除物品栏中的物品也是通过在该槽位放置空气从而实现的。

液体是可以自由流动的方块，目前游戏中只有水和熔岩两种液体。液体会扩散，具有深度，该性质用于控制液体最大可扩散的距离。

一、方块状态

特别地、当指定部分方块时，这些方块很可能拥有变种，比如门的开关状态、小麦的成熟度等，这些变种便是**方块属性（Block property）**。不同方块属性的集合被称为**方块状态（Block states）**，相应地，液体的属性集合是**流体状态（Fluid states）**。方块状态在命名空间 ID 的基础上进一步定义了一个方块的模型、行为，方块状态是方块本身拥有的性质，是硬编码的。附录 II.2 列举了所有方块及其可用的方块状态。

以 Minecraft 中的门为例，如果给门的开关状态分别配置一个 ID，那门的一个 ID 就会被拆分成两个。不仅如此，门还有不同的朝向、上半扇和下半扇、门轴的位置、是否被激活这些变种，不同的变种组合在一起的所有结果一共有 64 种。

可见，如果将这些方块的每一个变种都用单独的命名空间 ID 来表示则可能会占用更多的内存，也会让原本清晰的命名空间 ID 变得难以书写、阅读，而使用数字 ID 附带 Damage 值的做法在扁平化后已被淘汰了。为此，在指定方块状态的时候，不直接使用命名空间 ID，而是在命名空间 ID 后添加类似键值对的表示方式，格式为：

```
<命名空间>:<ID>[<属性>=<值>,<属性>=<值>,...]
```

1.72

注意：

1. 括号 `{ }` 要紧贴命名空间 ID，不能出现空格。
2. 不同的键值对之间用英文逗号 `,` 隔开，最后一个键值对后面不要加逗号。
3. 属性必须是这个方块所拥有的，门没有年龄这个属性，因此无法对门设置年龄这种方块状态。属性可以使用 `Tab` 键补全。
4. 属性的值必须按照 Minecraft 要求的参数格式填写，比如布尔值、方向或是整型。每个属性都有其需要的参数类型，这些参数类型不尽相同，有些需要布尔值、有些需要方向值。比如，门的 `open` 属性需要布尔值 `true` 或 `false`，像 `east` 这样的方向值是无效的。

例如，一个开启的、朝向为东的铁门可写成如下的形式：

```
minecraft:iron_door[open=true,face=east]
```

1.73

不定义任何方块状态时，系统会选择这个方块的默认方块状态，铁门的默认方块状态为朝向北、关闭。

调试棒（Debug stick） 可用于快速更改一个方块的方块状态，对方块点击（默认为 `鼠标左键`）可以切换更改的方块状态种类，对方块使用（默认为 `鼠标右键`）可以切换此方块状态的值。

二、方块实体

方块实体（Block entity） 是一个很有趣的概念，它将一般被认为相对静态的方块和相对动态的实体结合起来。这样做的意义在于——在方块状态规定的有穷集合的基础上，使方块能够容

纳更多数据，并使得方块数据便于编辑、修改。方块实体可以每游戏刻都进行计算，从而提供更好的渲染动画，但同时也可能使得计算量超过游戏刻计算的负载。

例如，告示牌是典型的既有方块实体又有多个方块状态的方块，其方块状态 `Rotation` 决定告示牌为何种朝向（告示牌的朝向只有 16 种），从而调用相应的模型使告示牌显示出需要的朝向。而告示牌又是一种可以显示文本的方块，显然几个模型无法承载大量各种式样的文本，因此使用方块实体让告示牌能够容纳更多的数据。

和方块状态一样，只有部分方块拥有方块实体。方块实体的数据使用 NBT 格式，详见节 6.4。

三、方块更新 *

受限于计算机性能，游戏无法每游戏刻都对方块进行计算。只有当方块被放置、破坏、修改，或方块状态产生变化时，该方块会通知其毗邻方块（即上、下、东、南、西、北六个面的邻接方块）进行相应，这种游戏机制被称为**方块更新（Block update）**。使用非 `strict` 模式的命令放置、移除方块时也会产生方块更新。Minecraft 有三种类型的方块更新，即 PP 更新、NC 更新和比较器更新。

方块更新一般依照以下的顺序依次计算：调用被替代方块状态的破坏行为 → 调用替代方块状态的放置行为 → 进行 NC 更新 → 进行比较器更新 → 进行 PP 更新。

方块更新会向外传播，在执行更新的过程中可能在毗邻方块产生新的更新，一直到所有可用的更新都执行完毕，但是在更新无法完全清除的情况下可能会造成游戏崩溃。例如在只有一层沙子的超平坦世界中破坏任意沙子，则方块更新传播会持续进行，并且计算更新的方块数量越来越多，最终会不可避免地造成游戏崩溃。服务端配置文件 `server.properties` 的 `max-chained-neighbor-updates` 一项可用于设置最大的连锁更新数量，超过此值的新增更新将会被忽略。

当一个方块发生变化时，即产生 PP 更新。对于一个方块上的六个毗邻方块，依次沿 x 、 z 、 y 轴的方向，各方向上先检查负轴方向上的方块，再检查正轴方向上的方块，即按照西、东、北、南、下、上的顺序传播 PP 更新。PP 更新是广泛存在的一种更新类型，包括但不限于附着性方块的掉落、连接性方块的连接判断、重力方块的掉落检测等。

NC 更新是一种在红石元件中更常见的更新类型，除了方块放置、修改或移除外，方块实体也可能产生 NC 更新。与 PP 更新不同，除红石线外，NC 更新的传播次序是西、东、下、上、北、南；对于红石线则为更新传入方向的后、前、左、右、下、上方。

比较器更新只适用于在放置后可发出模拟信号的方块。

1.7.6 实体

实体（Entity） 是一个动态的对象，包括**玩家、生物、交通运输工具**（所有种类的船和所有种类的矿车）、物品、物品展示框、画、盔甲架、经验球、所有的弹射物、激活的 TNT、下落的方块、漂浮的鱼饵、闪电、栓绳结、末影水晶、尖牙和标记等。**实体每游戏刻都会被计算，是技术性开发的主要研究对象。又由于其计算量较大，当已加载区域的实体数量过多，则容易引起掉刻，因此在开发过程中涉及实体计算的部分需要仔细斟酌。**

判定箱（Hitbox） 是规定的方块和实体的边界，用于计算碰撞和选取。所有实体的判定箱都是长方形，无论该实体的外形如何。末影龙比较特殊，它的判定箱是由多个判定箱组合而成的。同方块判定箱一样，实体判定箱也是硬编码的，无法通过命令或数据包修改，但可以通过修改实体属性对其进行放缩。实体判定箱有以下几种类型：**边界箱（Boundary box）** 是计算实体碰撞、

交互事件的区界，可使用快捷键 **F3 + B** 查看；**视平线（Eye level）** 显示为红色，用于判定窒息和溺水伤害。

Java 版原版所有可用的实体可分为若干类别，这些实体的命名空间均为 `minecraft`。下面列举了所有可用的实体：

- 1. 玩家
- 2. 生物

表 1.16 所有可用生物及其 ID

ID	名称	ID	名称	ID	名称
<code>allay</code>	悦灵	<code>goat</code>	山羊	<code>skeleton</code>	骷髅
<code>armadillo</code>	犛狍	<code>guardian</code>	守卫者	<code>skeleton_horse</code>	骷髅马
<code>armor_stand</code>	盔甲架	<code>happy_ghast</code>	快乐恶魂	<code>slime</code>	史莱姆
<code>axolotl</code>	美西螈	<code>hoglin</code>	疣猪兽	<code>sniffer</code>	嗅探兽
<code>bat</code>	蝙蝠	<code>horse</code>	马	<code>snow_golem</code>	雪傀儡
<code>bee</code>	蜜蜂	<code>husk</code>	尸壳	<code>spider</code>	蜘蛛
<code>blaze</code>	烈焰人	<code>illusioner</code>	幻术师	<code>squid</code>	鱿鱼
<code>breeze</code>	旋风人	<code>iron_golem</code>	铁傀儡	<code>stray</code>	流浪者
<code>camel</code>	骆驼	<code>llama</code>	羊驼	<code>strider</code>	炽足兽
<code>cat</code>	猫	<code>magma_cube</code>	岩浆怪	<code>tadpole</code>	蝌蚪
<code>cave_spider</code>	洞穴蜘蛛	<code>mannequin</code>	玩家模型	<code>trade_llama</code>	行商羊驼
<code>chicken</code>	鸡	<code>mooshroom</code>	哞菇	<code>tropical_fish</code>	热带鱼
<code>cod</code>	鳕鱼	<code>mule</code>	骡	<code>turtle</code>	海龟
<code>copper_golem</code>	铜傀儡	<code>nautilus</code>	鹦鹉螺	<code>vex</code>	恼鬼
<code>cow</code>	牛	<code>ocelot</code>	豹猫	<code>villager</code>	村民
<code>creaking</code>	嘎吱	<code>panda</code>	熊猫	<code>vindicator</code>	卫道士
<code>creeper</code>	苦力怕	<code>parrot</code>	鹦鹉	<code>wandering_trader</code>	流浪商人
<code>dolphin</code>	海豚	<code>phantom</code>	幻翼	<code>warden</code>	监守者
<code>donkey</code>	驴	<code>pig</code>	猪	<code>witch</code>	女巫
<code>drowned</code>	溺尸	<code>piglin</code>	猪灵	<code>wither</code>	凋灵
<code>elder_guardian</code>	远古守卫者	<code>piglin_brute</code>	猪灵蛮兵	<code>wither_skeleton</code>	凋灵骷髅
<code>ender_dragon</code>	末影龙	<code>pillager</code>	掠夺者	<code>wolf</code>	狼
<code>enderman</code>	末影人	<code>polar_bear</code>	北极熊	<code>zombie</code>	僵尸
<code>endermite</code>	末影螨	<code>pufferfish</code>	河豚	<code>zombie_horse</code>	僵尸马
<code>evoker</code>	唤魔者	<code>rabbit</code>	兔子	<code>zombie_nautilus</code>	僵尸鹦鹉螺
<code>fox</code>	狐狸	<code>ravager</code>	劫掠兽	<code>zombified_piglin</code>	僵尸猪灵
<code>frog</code>	青蛙	<code>salmon</code>	鲑鱼	<code>zombie_villager</code>	僵尸村民
<code>ghast</code>	恶魂	<code>sheep</code>	绵羊	<code>zoglin</code>	僵尸疣猪兽
<code>giant</code>	巨人	<code>shulker</code>	潜影贝		

(续表)

ID	名称	ID	名称	ID	名称
glow_squid	发光鱿鱼	silverfish	蠢虫		

3. 弹射物

表 1.17 所有可用弹射物及其 ID

ID	名称	ID	名称	ID	名称
arrow	箭	fireball	火球	small_fireball	小火球
breeze_wind_charge	旋风人风弹	firework_rocket	烟花火箭	snowball	雪球
dragon_fireball	末影龙火球	fishing_bobber	浮漂	spectral_arrow	光灵箭
egg	掷出的鸡蛋	llama_spit	羊驼唾沫	trident	三叉戟
ender_pearl	掷出的末影珍珠	potion	药水	wind_charge	风弹
experience_bottle	掷出的附魔之瓶	shulker_bullet	潜影弹	wither_skull	凋灵之首

4. 交通工具

表 1.18 所有可用交通工具及其 ID

ID	名称	ID	名称	ID	名称
boat	船	chest_minecart	运输矿车	hopper_minecart	漏斗矿车
chest_boat	运输船	command_block_minecart	命令方块矿车	spawner_minecart	刷怪笼矿车
minecart	矿车	furnace_minecart	动力矿车	tnt_minecart	TNT 矿车

5. 可悬挂实体

这一类实体有物品展示框 `item_display`、荧光物品展示框 `glow_item_frame`、画 `painting`。

6. 技术类实体

这一类实体有标记 `marker`、展示实体和交互实体 `interaction`。其中展示实体分为方块展示实体 `block_display`、物品展示实体 `item_display` 和文本展示实体 `text_display`。

7. 物品 `item`，即掉落物形式的物品，它是一种实体。

8. 下落的方块 `falling_block`

9. 被激活的 TNT `tnt`

10. 末地水晶 `end_crystal`

11. 区域效果云 `area_effect_cloud`

12. 唤魔者尖牙 `evoker_fangs`

13. 经验球 `experience_orb`

14. 末影之眼 `eye_of_ender`

15. 烟花火箭 `firework_rocket`

16. 栓神结 `leash_knot`

17. 闪电束 `lightning_bolt`

1.7.7 难度 游戏模式 游戏规则

难度、游戏模式和游戏规则是基本的游戏设置。

一、难度

难度 (Difficulty) 是控制游戏难易程度的选项。游戏中一共有四种难度，难易程度由低到高依次为**和平**、**简单**、**普通**和**困难**。玩家可以在选项中调整游戏难度，也可以通过命令 `/difficulty` 来调整，该命令所需权限等级为 2，语法为：

```
difficulty [easy|hard|normal|peaceful]
```

1.74

其中的参数： `[easy|hard|normal|peaceful]` —— 游戏难度，依次为简单、普通、困难、和平，若不指定该参数则视作查询当前游戏难度。

二、游戏模式

游戏模式 (Game mode) 是玩家进行游戏的方式，Minecraft 一共有四种游戏模式：**生存模式**、**创造模式**、**冒险模式**和**旁观模式**，**注意极限模式是一种世界设置而不是一种游戏模式**。

每一个游戏世界都有其默认的游戏模式，使得新进入世界（服务器）的玩家会使用该默认游戏模式。在创建新世界页面选择的游戏模式是该世界的默认游戏模式，极限模式比较特殊，它使用的默认游戏模式为生存模式。默认游戏模式可以在游戏过程中通过命令 `defaultgamemode` 修改，该命令所需权限等级为 2，语法为：

```
defaultgamemode <mode>
```

1.75

其中的参数： `<mode>`（游戏模式 `minecraft:gamemode`）—— 必须为下列值其中一者：`survival`（生存模式）、`creative`（创造模式）、`adventure`（冒险模式）、`spectator`（旁观模式）。

玩家自身的游戏模式不一定必须是世界默认游戏模式，可以由命令 `/gamemode` 修改，该命令所需权限等级为 2，语法为：

```
gamemode <mode> [<targets>]
```

1.76

其中的参数： `<mode>`（游戏模式 `minecraft:gamemode`）—— 与语法 1.75 一致。

`[<targets>]`（实体 `minecraft:entity`）—— 可选，指定需要更改游戏模式的玩家，必须为玩家名称、UUID 或目标选择器，且目标选择器必须指定玩家。如不指定则更改命令执行者的游戏模式。

当玩家的游戏模式是旁观模式时，他可以手动旁观另一个实体，也可以通过命令 `/spectate` 自动旁观。被旁观的实体不一定必须为玩家。`/spectate` 需要的权限等级为 2，语法为：

```
spectate [<target>] [<player>]
```

1.77

其中的参数： `<target>`（实体 `minecraft:entity`）—— 可选，若填写，则需要是玩家名称、UUID 或目标选择器，必须仅指定一个实体。

`<player>`（实体 `minecraft:entity`）——可选，若填写，则需要是玩家名称、UUID 或目标选择器，其中目标选择器必须指定单个玩家。

此处有必要强调：`<target>` 是被旁观的实体，`<player>` 是执行旁观行为的玩家。若 `<targets>`、`<player>` 两个参数均不填写，只使用命令 `/spectate`，则取消旁观实体；若 `<targets>` 填写而 `<player>` 不使用，则默认执行旁观行为的玩家是命令执行者自身。

当玩家不是旁观模式时，对该玩家执行 `/spectate` 会执行失败。这是 `/spectate` 常见的执行失败原因，读者可在自己设计的程序中检查此类原因。

三、游戏规则

游戏规则（Game rule） 是控制游戏玩法的一种手段。Minecraft 拥有很多种游戏规则，自 25w44a 起，游戏规则变为了注册项，被移动到了注册表中，现使用命名空间 ID 映射这些游戏规则。不同的游戏规则可以设置它们各自的值，不是所有的游戏规则都适用布尔值，一些游戏规则会使用整数，这些整数的可用范围为有符号的 32 位整数，即 $-2147483648 \sim 2147483647$ （含）。每个游戏规则都会有一个默认值，玩家没有指定它们的值时，便使用这些默认值。



图 1.15 游戏规则页面

在创建游戏世界时，可以通过 `更多` → `游戏规则` 页面修改。自 26.1-snapshot-3 起，也可以在游戏过程中点击 `世界选项...` → `编辑游戏规则` 手动修改。命令 `/gamerule` 也可用于更改游戏模式，其所需权限等级为 2，语法为：

```
gamerule <rulename> [<value>]
```

1.78

其中的参数：`<rulename>` ——必须为有效的游戏规则。
`[<value>]` ——可选，必须为该游戏规则的可用值，如不指定则查询该游戏规则的当前值。

所有游戏规则及其可用值如表 1.19 所示，表中所有游戏规则均省略其命名空间前缀 `minecraft`。

表 1.19 游戏规则表

游戏规则	说明	可用值	默认值
<code>advance_time</code>	游戏内时间是否流逝。	布尔值	<code>true</code>
<code>advance_weather</code>	天气是否更替。	布尔值	<code>true</code>
<code>allow_entering_nether_using_portals</code>	是否允许实体通过下界传送门进入下界。	布尔值	<code>true</code>
<code>block_drops</code>	方块被破坏时是否掉落物品和经验球。	布尔值	<code>true</code>
<code>block_explosion_drop_decay</code>	在床或重生锚的爆炸中，方块是否有概率不会掉落战利品。	布尔值	<code>true</code>
<code>command_block_output</code>	是否广播命令方块的输出。	布尔值	<code>true</code>
<code>command_blocks_work</code>	是否启用命令方块。	布尔值	<code>true</code>
<code>drowning_damage</code>	是否有溺水伤害。	布尔值	<code>true</code>
<code>elytra_movement_check</code>	是否启用鞘翅移动检测。	布尔值	<code>true</code>
<code>ender_pearls_vanish_on_death</code>	玩家投掷的末影珍珠是否在玩家死亡时消失。	布尔值	<code>true</code>
<code>entity_drops</code>	非生物实体是否掉落战利品。	布尔值	<code>true</code>
<code>fall_damage</code>	是否有掉落伤害。	布尔值	<code>true</code>
<code>fire_damage</code>	是否有火焰伤害。	布尔值	<code>true</code>
<code>fire_spread_radius_around_player</code>	以玩家为中心一定范围内火焰可以更新（包括火焰蔓延、熄灭及自然生成）的半径。若为 <code>0</code> ，则禁用火焰更新；若为 <code>-1</code> ，则无视玩家位置，所有位置都会进行火焰更新。	整数	<code>128</code>
<code>forgive_dead_players</code>	是否宽恕死亡玩家。若为 <code>true</code> ，则愤怒的中立型生物将在其目标玩家于附近死亡后息怒。	布尔值	<code>true</code>
<code>freeze_damage</code>	是否有冰冻伤害。	布尔值	<code>true</code>
<code>global_sound_events</code>	特定游戏事件发生时，声音是否可在所有地方听见。	布尔值	<code>true</code>
<code>immediate_respawn</code>	玩家死亡时是否立即重生。	布尔值	<code>false</code>
<code>keep_inventory</code>	死亡后是否保留物品栏物品和经验。	布尔值	<code>false</code>
<code>lava_source_conversion</code>	是否允许流动熔岩转化成熔岩源。	布尔值	<code>false</code>
<code>limited_crafting</code>	是否需要配方。若启用，玩家只能使用已解锁的配方合成。	布尔值	<code>false</code>
<code>locator_bar</code>	是否启用玩家定位条。	布尔值	<code>true</code>
<code>log_admin_commands</code>	是否在日志中记录管理员使用的命令。	布尔值	<code>true</code>
<code>max_block_modifications</code>	一条命令（如 <code>/fill</code> 和 <code>/clone</code> ）修改方块的数量限制。	整数	<code>32768</code>
<code>max_command_forks</code>	命令能够执行的最大分支数量。	整数	<code>65536</code>
<code>max_command_sequence_length</code>	命令连锁执行数量限制，应用于命令方块链和函数。	整数	<code>65536</code>
<code>max_entity_cramming</code>	实体挤压数量上限。	整数	<code>24</code>
<code>max_minecart_speed</code>	此游戏规则仅在启用“矿车改进”后可用。控制矿车最大运行速度，单位为格 / 秒。上限为 1000。	整数	<code>8</code>
<code>max_snow_accumulation_height</code>	下雪时地面的雪能够堆积的最大层数。	整数	<code>1</code>

(续表)

游戏规则	说明	可用值	默认值
<code>mob_drops</code>	生物在死亡时是否掉落物品和经验球。	布尔值	<code>true</code>
<code>mob_explosion_drop_decay</code>	在生物爆炸中，方块是否有概率不会掉落战利品。	布尔值	<code>true</code>
<code>mob_griefing</code>	是否允许生物破坏性行为。生物破坏性行为包括：生物放置、修改或破坏方块的行为、生物捡起物品、村民繁殖、幻魔者更改绵羊颜色等。	布尔值	<code>true</code>
<code>natural_health_regeneration</code>	生命值是否自然恢复。	布尔值	<code>true</code>
<code>player_movement_check</code>	是否启用玩家移动检测。	布尔值	<code>true</code>
<code>players_nether_portal_creative_delay</code>	创造模式的玩家需要在下界传送门内等待多少游戏刻才能进入另一维度。	整数	<code>0</code>
<code>players_nether_portal_default_delay</code>	非创造模式的玩家需要在下界传送门内等待多少游戏刻才能进入另一维度。	整数	<code>80</code>
<code>players_sleeping_percentage</code>	入睡比例：跳过夜晚所需的入睡玩家占比。设为 <code>0</code> 时仅需 1 个玩家入睡即可跳过夜晚。设为大于 <code>100</code> 的值会使玩家无法跳过夜晚。	整数	<code>100</code>
<code>projectiles_can_break_blocks</code>	弹射物能否破坏紫颂花、滴水石锥或饰纹陶罐。	布尔值	<code>true</code>
<code>pvp</code>	是否启用 PVP。	布尔值	<code>true</code>
<code>raids</code>	是否启用袭击。	布尔值	<code>true</code>
<code>random_tick_speed</code>	一个游戏刻内随机刻挑选方块的数量。	整数	<code>3</code>
<code>reduced_debug_info</code>	是否简化调试信息、限制调试屏幕的内容。	布尔值	<code>false</code>
<code>respawn_radius</code>	重生点半径。	整数	<code>10</code>
<code>send_command_feedback</code>	是否发送命令反馈。	布尔值	<code>true</code>
<code>show_advancement_messages</code>	是否在聊天栏中通知进度的达成。	布尔值	<code>true</code>
<code>show_death_messages</code>	是否显示死亡信息。	布尔值	<code>true</code>
<code>spawn_mobs</code>	是否自然生成生物，一些实体可能有其特定的规则，如刷怪笼、命令 <code>/summon</code> 生成的生物不受此影响。	布尔值	<code>true</code>
<code>spawn_monsters</code>	是否自然生成敌对生物。	布尔值	<code>true</code>
<code>spawn_patrols</code>	是否生成灾厄巡逻队。	布尔值	<code>true</code>
<code>spawn_phantoms</code>	是否生成幻翼。	布尔值	<code>true</code>
<code>spawn_wandering_traders</code>	是否生成流浪商人。	布尔值	<code>true</code>
<code>spawn_wardens</code>	是否生成监守者。	布尔值	<code>true</code>
<code>spawner_blocks_work</code>	刷怪笼和试炼刷怪笼是否运作。	布尔值	<code>true</code>
<code>spectators_generate_chunks</code>	是否允许旁观模式的玩家生成区块。	布尔值	<code>true</code>
<code>spread_vines</code>	藤蔓是否蔓延。	布尔值	<code>true</code>
<code>tnt_explodes</code>	TNT 是否可被激活并爆炸	布尔值	<code>true</code>
<code>tnt_explosion_drop_decay</code>	在 TNT 爆炸中，方块是否有概率不会掉落战利品。	布尔值	<code>false</code>

(续表)

游戏规则	说明	可用值	默认值
<code>universal_anger</code>	是否启用无差别愤怒，若值为 <code>true</code> ，愤怒的中立型生物将攻击附近所有的玩家，而不同于激怒它们的玩家。	布尔值	<code>false</code>
<code>water_source_conversion</code>	是否允许流动水转化成水源。	布尔值	<code>true</code>

1.8 服务器管理

专用服务器是在 Minecraft 中实现多人游戏的一种手段。玩家们可以连接服务器游玩各种小游戏，体验 SMP、PVP 或各种自定义多人游戏地图，极大地提高了 Minecraft 的可玩性。篇幅有限，本教程并不提供服务器的架设方法，仅提供服务器配置以及能够在服务器上使用的命令的解释，供服务器管理人员参考。

1.8.1 `server.properties` *

 `server.properties`，即**服务端配置文件**，文件中一个配置属性占据一行，每一行的格式为：

<属性>=<值>	1.79
----------	------

例如：

<code>gamemode=survival</code>	1.80
--------------------------------	------

<code>enable-command-block=false</code>	1.81
---	------

下表列举了所有可用的属性：

表 1.20  `server.properties` 可用属性表

属性	值类型	默认值	描述
<code>accepts-transfers</code>	布尔值	<code>false</code>	服务器是否接受以 Transfers 封包作为登录请求的传入连接。若为 <code>true</code> ，则在其他服务器中使用命令 <code>/transfer</code> 能将玩家转移至该服务器。
<code>allow-flight</code>	布尔值	<code>false</code>	是否允许玩家在安装添加飞行功能的 Mods 前提下在生存模式下飞行。
<code>broadcast-console-to-ops</code>	布尔值	<code>true</code>	是否向所有在线管理员发送命令执行输出。
<code>broadcast-rcon-to-ops</code>	布尔值	<code>true</code>	是否向所有在线管理员发送 RCON 命令执行输出。

(续表)

属性	值类型	默认值	描述
<code>difficulty</code>	字符串	<code>easy</code>	游戏的难度, 可用值: <code>peaceful</code> (和平)、 <code>easy</code> (简单)、 <code>normal</code> (普通)、 <code>hard</code> (困难)。
<code>enable-jmx-monitoring</code>	布尔值	<code>false</code>	是否暴露一个具有对象名 <code>net.minecraft.server:type=Server</code> 的 MBean 和两个属性 <code>averageTickTime</code> 和 <code>tickTimes</code> 用于暴露以毫秒为单位的 tick 时间。
<code>enable-query</code>	布尔值	<code>false</code>	是否允许使用 GameSpy4 协议的服务器监听器。
<code>enable-rcon</code>	布尔值	<code>false</code>	是否允许远程访问服务器控制台 (RCON)。
<code>enable-status</code>	布尔值	<code>true</code>	是否使服务器在服务器列表中显示为在线。
<code>enforce-secure-profile</code>	布尔值	<code>true</code>	是否要求进入服务器的玩家必须具有 Mojang 签名的公钥。
<code>enforce-whitelist</code>	布尔值	<code>false</code>	是否强制启用白名单。
<code>entity-broadcast-range-percentage</code>	整数	<code>100</code>	控制实体需要距离玩家有多近才会将封包发送给客户端。值越高, 则实体可以在更远的地方被渲染。可用值为 <code>10 ~ 1000</code> (含)。
<code>force-gamemode</code>	布尔值	<code>false</code>	是否强制玩家加入时为默认游戏模式。
<code>function-permission-level</code>	整数	<code>2</code>	设置函数的权限等级, 可用值为 <code>1 ~ 4</code> (含)。
<code>gamemode</code>	字符串	<code>survival</code>	设置默认游戏模式, 可用值: <code>survival</code> (生存模式)、 <code>creative</code> (创造模式)、 <code>adventure</code> (冒险模式)、 <code>spectator</code> (旁观模式)。
<code>generate-structures</code>	布尔值	<code>true</code>	是否在未生成的区块中生成结构。
<code>generator-settings</code>	字符串	<code>{}</code>	定义自定义世界的生成。
<code>hardcore</code>	布尔值	<code>false</code>	是否使用极限模式进行游戏。
<code>hide-online-players</code>	布尔值	<code>false</code>	是否在响应客户端状态请求时不返回在线玩家列表。值为 <code>true</code> 时不返回。
<code>initial-disabled-packs</code>	字符串	无	设置在创建世界过程中需要禁用的数据包名称, 存在多个数据包时用逗号分隔。
<code>initial-enabled-packs</code>	字符串	<code>vanilla</code>	设置在创建世界过程中需要启用的数据包名称, 存在多个数据包时用逗号分隔。
<code>level-name</code>	字符串	<code>world</code>	设置该游戏世界的自定义名称及文件夹名。
<code>level-seed</code>	字符串	无	该游戏世界的种子。

(续表)

属性	值类型	默认值	描述
<code>level-type</code>	字符串	<code>minecraft:normal</code>	定义使用的世界预设 ID, 以命名空间 ID 表示, 冒号 <code>:</code> 前需要加 <code>\</code> 以转义。原版世界预设可省略命名空间。
<code>log-ips</code>	布尔值	<code>true</code>	定义是否当新玩家加入游戏时在服务器日志中记录其 IP 地址。
<code>management-server-allowed-origins</code>	字符串	无	在使用浏览器 WebSocket API 的情况下, 服务端管理协议允许连入的来源, 以 <code>,</code> 分割, 且不允许有前后空格。
<code>management-server-enabled</code>	布尔值	<code>false</code>	是否启用 Minecraft 服务器管理协议。
<code>management-server-host</code>	字符串	<code>localhost</code>	服务端管理协议绑定的主机名。
<code>management-server-port</code>	整数	<code>0</code>	服务端管理协议监听的窗口, 若为 <code>0</code> , 则绑定当前未被绑定的可绑定窗口。
<code>management-server-secret</code>	字符串	随机	用于认证的令牌, 必须为一个长度为 40 且仅包含大写字母、小写字母和数字。
<code>management-server-tls-enabled</code>	布尔值	<code>true</code>	是否启用 TLS。
<code>management-server-tls-keystore</code>	字符串	无	保存服务端用于 TLS 的私钥和证书的 KeyStore。
<code>management-server-tls-keystore-password</code>	字符串	无	KeyStore 的密码。
<code>max-chained-neighbor-updates</code>	整数	<code>1000000</code>	限制连锁 NC 更新的数量, 超过此数量的连锁 NC 更新会被跳过。
<code>max-players</code>	整数	<code>20</code>	设置服务器同时能容纳的最大玩家数量。可用值 <code>0 ~ 2147483647</code> (含)。
<code>max-tick-time</code>	整数	<code>60000</code>	设置每个 tick 花费的最大毫秒数。可用值 <code>0 ~ 2⁶³ - 1</code> (含)。
<code>max-world-size</code>	整数	<code>29999984</code>	设置世界边界最大可用半径。可用值 <code>1 ~ 29999984</code> (含)。
<code>motd</code>	字符串	<code>A Minecraft Server</code>	玩家客户端的多人游戏服务器列表中显示的服务器信息。
<code>network-compression-threshold</code>	整数	<code>256</code>	若值为 n , 则允许 $n - 1$ 字节的封包正常发送, 如果封包为 n 字节或更大时会进行压缩。 <code>-1</code> 代表完全禁用封包压缩, <code>0</code> 代表压缩全部封包。
<code>online-mode</code>	布尔值	<code>true</code>	是否让服务器对比 Minecraft 账户数据库验证登录信息, 为 <code>true</code> 时只允许正版玩家进入。
<code>op-permission-level</code>	整数	<code>4</code>	设定使用 <code>/op</code> 命令时管理员的权限等级, 可用值 <code>1 ~ 4</code> (含)。
<code>pause-when-empty-seconds</code>	整数	<code>60</code>	服务器在没有玩家在线后多少秒暂停。

(续表)

属性	值类型	默认值	描述
<code>player-idle-timeout</code>	整数	<code>0</code>	设置玩家可空闲不被提出服务器的最大时间（单位为分钟），为 <code>0</code> 时不踢出。
<code>prevent-proxy-connections</code>	布尔值	<code>false</code>	是否允许玩家使用虚拟专用网络或代理。
<code>query.port</code>	整数	<code>25565</code>	设置监听服务器的端口号，可用值 <code>1</code> ~ <code>65534</code> （含）。
<code>rate-limit</code>	整数	<code>0</code>	设置玩家被踢出服务器前可发送的封包数量，设为 <code>0</code> 表示不发送。
<code>rcon.password</code>	字符串	无	RCON 远程访问的密码。
<code>rcon.port</code>	整数	<code>25575</code>	RCON 远程访问的端口号，可用值 <code>1</code> ~ <code>65534</code> （含）。
<code>region-file-compression</code>	字符串	<code>deflate</code>	设置区域文件压缩算法，可用值： <code>deflate</code> （Deflate 算法）、 <code>lz4</code> （LZ4 算法）、 <code>none</code> （不压缩）。
<code>require-resource-pack</code>	布尔值	<code>false</code>	是否对玩家强制启用服务器资源包。
<code>resource-pack</code>	字符串	无	可选，指向一个资源包的 URI，玩家可选择是否使用该资源包。值中的 <code>:</code> 和 <code>/</code> 前必须有 <code>\</code> 作为转义。
<code>resource-pack-id</code>	UUID	无	可选， <code>resource-pack</code> 指定的资源包的 UUID。
<code>resource-pack-prompt</code>	字符串	无	可选，使用 <code>require-resource-pack</code> 时在资源包提示界面显示自定义信息。
<code>resource-pack-sha1</code>	字符串	无	资源包的 SHA-1 值，必须为小写十六进制。
<code>server-ip</code>	字符串	无	将服务器与特定 IP 绑定。
<code>server-port</code>	整数	<code>25565</code>	服务器监听端口号，可用值 <code>1</code> ~ <code>65534</code> （含）。
<code>simulation-distance</code>	整数	<code>10</code>	模拟距离，可用值 <code>3</code> ~ <code>32</code> （含）。
<code>spawn-protection</code>	整数	<code>16</code>	设该值为 x ，则以出生点所在方块为中心边长为 $2x + 1$ 的正方形区域内存在出生点保护。若值为 <code>0</code> 则禁用出生点保护。
<code>status-heartbeat-interval</code>	整数	<code>0</code>	控制管理服务器向已连接的客户端发送心跳通知的间隔，单位为秒。若为 <code>0</code> ，则禁用此功能。
<code>sync-chunk-writes</code>	布尔值	<code>true</code>	是否使区块文件以同步模式写入。
<code>text-filtering-config</code>	字符串	无	服务器中需要被屏蔽的文本。
<code>text-filtering-version</code>	整数	<code>0</code>	服务器中需要被屏蔽文本格式的版本。
<code>use-native-transport</code>	布尔值	<code>true</code>	是否使用针对 Linux 平台的封包收发优化。

(续表)

属性	值类型	默认值	描述
<code>view-distance</code>	整数	<code>10</code>	渲染距离，可用值 <code>3</code> ~ <code>32</code> (含)。
<code>white-list</code>	布尔值	<code>false</code>	是否启用白名单。

1.8.2 仅在多人游戏可用命令

本小节讲述的一系列命令是对服务器管理有用的一类命令，仅能在多人游戏中使用。由于它们的权限等级均大于 2，因此在命令方块上无法运行这些命令。如果 `server.properties` 中的 `function-permission-level` 没有设为足够的权限等级，那么数据包函数也是不能执行这些命令的。

一、用于封禁玩家与设置黑名单的命令

1. 命令 `/ban`

`ban <targets> [reason]`

1.82

该命令用于封禁特定的玩家，并将其加入黑名单。加入黑名单的玩家将不被允许进入服务器。该命令所需权限等级为 3。

其中的参数：`<targets>` (游戏档案 `minecraft:game_profile`) ——需要是玩家名称或 UUID，无论玩家是否在线。成功后，拥有该名称的所有玩家都无法进入该服务器。

`[reason]` (文本 `minecraft:message`) ——可选，是一个贪婪词组，接受含空格的字符串。用于表示封禁的理由。可记录于服务器日志中。

2. 命令 `/ban-ip`

`ban-ip <target> [reason]`

1.83

该命令主要是针对 IP 地址的封禁，并将此 IP 地址列入黑名单，但也可以支持对玩家名称的封禁 (不能是 UUID)，成功后所有由被封 IP 进入服务器的玩家都不被允许。该命令所需权限等级为 3。

其中的参数：`<target>` (字符串 `brigadier:string`) ——需要被封禁的 IP 地址或玩家名称。

3. 命令 `/banlist`

`banlist (ips|players)`

1.84

该命令可供查询被封禁的 IP 地址或玩家，所需权限等级为 3。

其中的参数：`(ips|players)` ——在 `ips` 和 `players` 中任选其一，表示需要查询的内容为处在黑名单中的 IP 地址或玩家。

二、设置玩家解封的命令

1. 命令 `/pardon`

```
pardon <targets>
```

1.85

该命令用于解封特定的玩家，并将该玩家从黑名单中移除。该命令所需权限等级为 3。

2. 命令 `/pardon-ip`

```
pardon-ip <target>
```

1.86

该命令用于解封特定的 IP 地址，并将该 IP 地址从黑名单中移除。该命令所需权限等级为 3。

三、白名单

黑名单制度通常用于管理开放类型的服务器，并对个别不遵守服务器规则的玩家实施单独的封禁，其他玩家依旧可以自由进出服务器。而白名单制度常见于一些私人服务器，仅针对特定的玩家开放，这样可以最大限度地防止外来人员对服务器造成破坏。命令 `/whitelist` 是专门针对白名单的命令，该命令所需权限等级为 3，它有以下几种语法：

1. 添加玩家至白名单

```
whitelist add <targets>
```

1.87

2. 查询在白名单上的玩家

```
whitelist list
```

1.88

3. 关闭白名单

```
whitelist off
```

1.89

4. 开启白名单

```
whitelist on
```

1.90

5. 重新加载白名单

```
whitelist reload
```

1.91

6. 将玩家从白名单上移除

```
whitelist remove <targets>
```

1.92

四、管理员命令

1. 命令 `/op`

用于给予玩家管理员权限。该命令所需权限等级为 3，语法为：

```
op <targets>
```

1.93

此管理员权限等级由 `server.properties` 中的 `op-permission-level` 决定，详见表 1.20。

2. 命令 `/deop`

剥夺玩家的管理员权限。该命令所需权限等级为 3，语法为：

```
deop <targets>
```

1.94

五、用于设置玩家在服务器中最长挂机时间的命令 `/setidletimeout`

该命令所需权限等级为 3，语法为：

```
setidletimeout <minutes>
```

1.95

其中的参数：`<minutes>`（整型 `brigadier:integer`）——单位是分钟不是而是游戏刻，可用范围 `[0, 214748364]`。

六、服务器保存相关的命令

以下命令所需权限等级均为 4。

1. 命令 `/save-all`：将服务器手动保存至硬盘。

```
save-all [flush]
```

1.96

其中的参数：`[flush]`——可选字面量，若填入该字面量，则服务器会立即保存所有的区块数据，并造成服务器临时冻结。

2. 命令 `/save-off`：关闭自动保存服务器，无附加参数。

```
save-off
```

1.97

3. 命令 `/save-on`：开启自动保存服务器，无附加参数。

```
save-on
```

1.98

七、用于关闭服务器的命令 `/stop`

该命令所需权限等级均为 4，语法为：

```
stop
```

1.99

八、将玩家从一个服务器转移至另一服务器的命令 `/transfer`

此命令仅会发出请求，实际能否转移成功取决于目的服务器 `server.properties` 中的 `accepts-transfers` 配置，详见表 1.20。 `/transfer` 的语法为：

```
transfer <hostname> [<port>] [<players>]
```

1.100

其中的参数：`<hostname>`（字符串 `brigadier:string`）——目的服务器的主机名。
`[<port>]`（整型 `brigadier:integer`）——可选，目的服务器端口号，若不指定则为 `25565`。
`[<players>]`（实体 `minecraft:entity`）——可选，被转移的玩家，必须为玩家名、目标选择器或 UUID。若不指定则默认为命令执行者。

九、命令 `/perf`

它用于记录服务器游戏刻执行时长和占用的堆内存大小等性能指标，并将结果保存于游戏文件 `.minecraft\debug\profiling\<时间戳>.zip`。命令 `perf start` 即开始长达 10 秒的性能分析，`perf stop` 可以在 10 秒之前结束性能分析。此命令需要权限等级 4。

```
perf (start|stop)
```

1.101

由于它仅在专用服务器上使用，在单人游戏中可以用 `F3 + L` 代替它的性能分析功能。

1.8.3 其他与服务器相关的命令

一、命令 `/kick`

将玩家踢出服务器，注意只是踢出，如果不加封禁，该玩家还可以继续加入服务器。事实上，该命令不仅在多人游戏中可用，在单人游戏中同样可用。该命令所需权限等级为 3，语法为：

```
kick <targets> [reason]
```

1.102

其中的参数：`<targets>`（实体 `minecraft:entity`）——指定提出服务器的玩家，必须为玩家名、目标选择器或 UUID。

`[reason]`（文本 `minecraft:message`）——可选，格式与语法 1.82、1.83 一致。

二、命令 `/list`

该命令用于列出服务器中的所有玩家，在单人游戏中同样可用，所需权限等级为 0，语法为：

```
list [uuids]
```

1.103

其中的参数：`[uuids]`——可选，如果加了会让玩家的名称和 UUID 一起显示。

第一章思考题与习题

- 判断下列说法是否正确。
 - 所有的仅适用于多人游戏的命令都需要大于 2 的权限等级。
 - 在聊天栏中可输入的命令字符数小于命令方块的最大字符数。
 - 在创建世界时关闭作弊意味着所有命令在这个存档中均不可用。
 - 生存模式的玩家在开启作弊的存档中无法打开命令方块，也无法使用命令。
- 列举出所有在不允许作弊的单人游戏中可使用的命令。
- 列举出所有无法在命令方块中运行的命令。
- 查阅相关资料，填写下列游戏资源的命名空间 ID：
例：石头——`minecraft:stone`

羊毛、旗帜、唱片、红砂岩台阶、火把、雪块、红石中继器、煤炭、岩浆块、树叶、海晶石、末地石砖。

5. 查阅相关资料，填写下列方块的所有属性以及各自的可用值。

例：铁砧——方块状态 `facing`，可用值 `north`、`south`、`east`、`west`。

织布机、树叶、漏斗、讲台、砂轮、南瓜灯、原木、下界疣、梯子、虞美人、红石粉、雪、水、唱片机、可可果、灯笼、树苗、甘蔗、织布机、干草块、玻璃板、堆肥桶、紫颂花。

6. 在 TPS 为 20 的情况下，将下列时间单位进行换算（不考虑命令中的取整）：

2 s = _____ d 18 t = _____ s 0.25 d = _____ s 0.8 d = _____ t
1.2 s = _____ t 360 t = _____ d

7. 解释下列名词：

命令、命令历史、权限等级、命令执行者、命名空间、方块状态、数据值、扁平化、MSPT、强加载区块、物理服务端。

8. 在游戏刻频率为 40 的情况下，由调试屏幕查阅得到 MSPT 值为 20，则此时 TPS 值为何？若 MSPT 值为 100，则 TPS 值又为何？

9. 能进行方块更新的区块可以正常运行命令这一说法是否正确？反之，可以正常运行命令的区块是否一定能进行方块更新？

10. 游戏中发生掉帧的原因可能有哪些？

11. 假如一个玩家在某服务器中发现画面很流畅，而实体完全不移动，可能的原因是什么？

12. 判断下列命名空间 ID 的解析结果。

- (1) `minecraft`
- (2) `custom`
- (3) `minecraft:custom`
- (4) `custom:minecraft`
- (5) `minecraft:custom/custom`
- (6) `custom:minecraft/custom`
- (7) `minecraft/custom:custom`
- (8) `minecraft:custom:custom`

13. *尝试列举命令 `/tick` 在解析中出现的所有节点。

14. 已知一个玩家位于区块 [17,39]，渲染距离为 17，模拟距离为 15，则区块 [10,20] 的加载等级和计算等级分别为多少？

15. 当渲染距离为 16 时，以玩家为中心的强加载区块个数为____，弱加载区块个数为____。

16. 将以下树状形式的数据写为 JSON。



17. 若一个字符串类型的 JSON 字段 `\" text` 需要的值分别如下所示，写出各自对应的字段。

- (1) 分节符“\”的作用很大

- (2) `\\\"Hello World!\\\"`
- (3) JSON形式的文本组件为`{\"text\":\"\\\"Hello World!\\\"\"}`
- (4) 在SNBT中, 反斜杠\直接使用反斜杠\转义即可, 即\\

18. *什么是 `.txt` 文件? `.minecraft` 文件夹中有哪些文件是以 `.txt` 的格式存在的?
19. *任意列举 10 个需要使用 JSON 格式的文件。
20. *如何在 `assets` 文件夹中寻找村民悠闲时的声音文件?
21. *若一份完整的 `debug` 文件内容如下所示, 则该函数中能够成功执行的命令有多少条?

```
debug-trace-2024-10-19_15.19.03.txt
1 the_backrooms:developer/temp
2 [F] the_backrooms:developer/temp size=15
3 [C] tag @s remove evidence_1
4 [E] 对象没有这个标签 [M] 对象没有这个标签
5 [C] tag @s remove gaming
6 [E] 对象没有这个标签 [M] 对象没有这个标签
7 [C] data remove storage the_backrooms:player data
8 [E] 无变化, 所指定的属性已有这些值 [M] 无变化, 所指定的属性已有这些值
9 [C] execute in the_frontrooms:main store result score #forceload bug_test run forceload add -1 -1 0 0
10 [M] 已将the_frontrooms:main中的[-1, -1]至[0, 0]间的4个区块标记为强制加载
11 [R = 4] execute in the_frontrooms:main store result score #forceload bug_test run forceload add -1 -1 0 0
12 [C] tellraw @a {\"score\":{\"name\":\"#forceload\",\"objective\":\"bug_test\"}} -> 1
13 [C] execute in the_frontrooms:main if dimension the_frontrooms:main run say 该命令在前厅内执行 -> 1
14 [C] execute in the_frontrooms:main positioned 0 0 0 store result score #decorations bug_test if entity @e[tag=decorations]
15 [E] 测试失败 [M] 测试失败
16 [C] tellraw @a {\"score\":{\"name\":\"#decorations\",\"objective\":\"bug_test\"}} -> 1
17 [C] execute in the_frontrooms:main positioned 0 0 0 store result score #decorations bug_test run kill @e[tag=decorations]
18 [E] 未找到实体 [M] 未找到实体
19 [C] tellraw @a {\"score\":{\"name\":\"#decorations\",\"objective\":\"bug_test\"}} -> 1
20 [C] execute in the_frontrooms:main positioned 0 0 0 store result score #decorations bug_test if entity @e[type=painting]
21 [E] 测试失败 [M] 测试失败
22 [C] tellraw @a {\"score\":{\"name\":\"#paintings\",\"objective\":\"bug_test\"}} -> 1
23 [C] execute in the_frontrooms:main positioned 0 0 0 store result score #paintings bug_test run kill @e[type=painting]
24 [E] 未找到实体 [M] 未找到实体
25 [C] tellraw @a {\"score\":{\"name\":\"#paintings\",\"objective\":\"bug_test\"}} -> 1
26 [C] execute in the_frontrooms:main run forceload remove all
27 [M] 已解除标记the_frontrooms:main内所有的强制加载区块
28 [R = 0] execute in the_frontrooms:main run forceload remove all
```

22. 解释下列名词: 数据包、实验性设置、安全模式错误、元数据、数据包标签。
23. 编写一个数据包的元数据, 使得此数据包能支持从 45 到 100.0 的数据版本。
24. 编写一个资源包的元数据, 使得此资源包能支持从 20 到 80.0 的数据版本。

25. 列举出在数据包内所有不使用 `.json` 格式的数据项类型。
26. 尝试覆盖原版数据包中定义的 `#base_stone_overworld` 方块标签, 使其仅引用石头 (`stone`)、深板岩 (`deepslate`) 两种方块。

第二章 坐标

Minecraft 的游戏世界是三维的。在编写数据包的时候，有时需要确定实例所需的位置参数。这样的参数被称为**坐标 (Coordinate)**。本章将详细介绍各种坐标参数以及这些参数在命令上的应用。

2.1 坐标系与坐标

Minecraft 使用的空间直角坐标系是右手坐标系。在这种空间直角坐标系中， x 轴和 z 轴所反映的是水平方向上的位置， y 轴所反映的是垂直方向上的位置。其中， x 轴的正方向指向正东，而 z 轴的正方向指向正南。

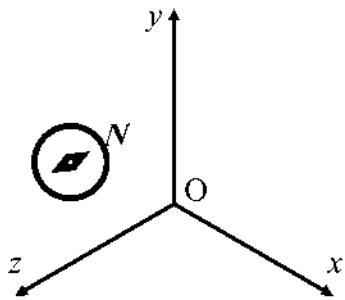


图 2.1 适用于 Minecraft 的空间直角坐标系

在命令参数中，可以用三个分量来表示某一点的位置，这是一个有序的实数三元组：

<x> <y> <z>

2.1

比如，用数学方法表示的点 $(0, 4, 4)$ 在一些命令参数中直接表示为 `0 4 4`。

例 2.1

空间直角坐标系中有两个点 $A(124.5, 76, -64.29)$ 、 $B(-10.003, 80, -33.33)$ ，则 B 点在水平位置上位于 A 点的什么方向？

解

B 点 x 坐标位于 A 点 x 坐标的负方向，因此 B 点在东西方向上位于 A 点的西方； B 点 z 坐标位于 A 点 z 坐标的正方向，因此 B 点在东西方向上位于 A 点的南方。综上所述， B 点位于 A 点的西南方向。

虽然不同的坐标表示方式基本一致，但不同的命令作用的对象不同，其坐标参数类型也不完全一致，下文将梳理命令系统使用的所有种类的坐标参数。

2.1.1 坐标的命令参数类型

命令使用 4 种与坐标相关的参数类型：方块坐标 `minecraft:block_pos`、三维坐标 `minecraft:vec3`、平面方块坐标 `minecraft:column_pos` 和二维坐标 `minecraft:vec2`。它们的关系和应用场景可以很清晰地列于下表：

表 2.1 Minecraft 中的坐标参数

	一般应用于方块	一般应用于非方块的游戏内容
三维	<code>minecraft:block_pos</code>	<code>minecraft:vec3</code>
二维	<code>minecraft:column_pos</code>	<code>minecraft:vec2</code>

一、方块坐标

坐标表示的是一个没有体积的点，而方块是有体积的，因此需要给方块规定一个基准点，用这个基准点的坐标来表示该对象的坐标。

Minecraft 规定：大部分方块的长、宽和高均为 1 米，体积为 1 立方米。用坐标系表示这些位置时，默认了方块的边长和坐标系的单位长度在数值上相等，这意味着坐标系的基本单位为米，或称为“格”。

一个方块使用其**西北下角**的点作为它的**方块坐标 (Block position)**。若一个方块的西北下角顶点坐标为 (x, y, z) ，则该方块的方块坐标记为 (x, y, z) ，而这个方块位于 (x, y, z) 和 $(x + 1, y + 1, z + 1)$ 这两个坐标围成的立体几何图形之间。

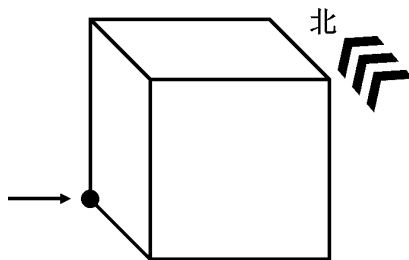


图 2.2 用方块这个方向的顶点来表示方块坐标

由于方块的角总是位于整数坐标点，作为命令参数 `minecraft:block_pos` 的方块坐标一定是由三个整数构成的有序三元组。

例 2.2

如图所示，方块坐标为 `0 0 0` 的方块为哪一个？

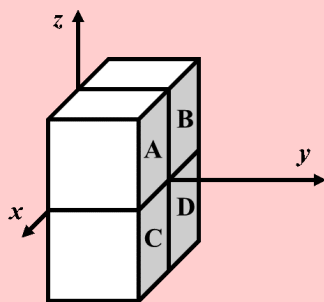


图 2.3

解

方块坐标严格按照西北下角顶点来计算，而正西、正北分别是 x 轴和 z 轴的负方向，因此用方块坐标指示的方块位置，永远位于实际坐标的东南方向，且在垂直方向上位于上方，在空间直角坐标系中的反映即为 x 、 y 、 z 三个坐标轴的正方向。因此方块坐标为 `0 0 0` 的方块位于第一卦限，即方块 A。

二、三维坐标

三维坐标 (Three-dimensional coordinates) 是精确表示一个位置的坐标参数，命令参数类型为 `minecraft:vec3`，用于表示坐标位置的三个元素均为双精度浮点数。三维坐标一般应用于实体，它也可能会在粒子生成和声音播放的时候被使用。例如，这是一个合法的三维坐标：

5.0 56.0 17.0

2.2

这个坐标带有小数点，因为三维坐标的三个参数均是双精度浮点数。但是，这并不意味着三维坐标只能使用浮点数。也可以在三维坐标中使用整数形式，如：

5 56 17

2.3

注意，上述这两个坐标描述的位置并不是一致的。在实际操作中，可以发现参数 `5 56 17` 指定的坐标实际上是 $(5.5, 56.0, 17.5)$ ，这个现象在 `/tp`、`/summon` 等命令使用的坐标参数中都可以观察到。如图，可以观察到 x 坐标和 z 坐标都发生了“偏移”，与实际坐标有所出入，而 y 坐标不受影响。

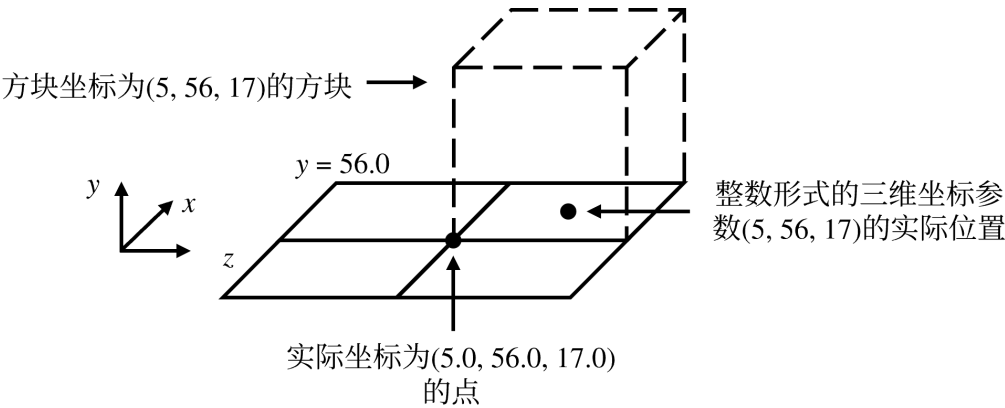


图 2.4 整数坐标发生的“偏移”

这些位置的偏移都位于相对方块两条对边的中心线上，这是因为三维坐标使用了**中心校准 (Center correct)**，即使用整数形式的三维坐标，当其某一个坐标参数为 n ($n \in \mathbb{Z}$) 时，其实际坐标为 $n - 0.5$ ，这样可以与方块的位置相适应。注意**中心校准仅适用于 x 坐标和 z 坐标。 y 坐标严格使用实际坐标。**

注意这里不使用“三维坐标根据方块坐标位于方块中心”的说法，是因为三维坐标的三个参数中整数和浮点数形式可以混用，并且使用小数形式的参数严格遵循实际坐标，整数形式的参数则使用中心校准。比如，位于 `5 56 17.0` 的玩家实际位于 $(5.5, 56, 17.0)$ 。

三、平面方块坐标

故名思义，平面方块坐标 `minecraft:column_pos` 就是二维的方块坐标，以西北角的二维坐标作为一个方块纵列的平面坐标，两个元素均为整数。

四、二维坐标

即只由 x 坐标和 z 坐标构成的**二维坐标 (Two-dimensional coordinates)**。二维坐标的命令参数类型为 `minecraft:vec2`，两个元素均为双精度浮点数。二维坐标若为整数，则也使用中心校准。

2.1.2 相对坐标 局部坐标

一、相对坐标

世界坐标是以空间直角坐标系为基准的、固定的坐标体系，每一个位置都有其固定的坐标。在表示这些坐标的时候，有时候需要确定“相对位置”，即抛开固有的以原点为基准的坐标系，使用“相对偏移量”来表达一个位置相对于另一个位置的坐标，即下文所要介绍的**相对坐标 (Relative world coordinates)**。与之相对的固定空间直角坐标系坐标被称为**绝对坐标 (Absolute world coordinates)**。

在相对坐标系中，必须要确定一个原点，这个原点通常是命令执行位置。如果命令由玩家执行，则原点为玩家所在的位置；如果命令由命令方块执行，则原点为该命令方块所在的位置。相对坐标用波浪号 $\sim$ 和相对偏移量表示，即

$\sim[\langle dx \rangle] \sim[\langle dy \rangle] \sim[\langle dz \rangle]$

2.4

若相对偏移量 $[\langle dx \rangle]$ 、 $[\langle dy \rangle]$ 、 $[\langle dz \rangle]$ 不填写，则偏移量为 0。偏移量可以为负数。这就相当于建立了一个以命令执行位置为原点的空间直角坐标系，一个世界中可以存在多个不同的相对坐标系，只要这些坐标都有特定的执行位置。例如，相对于该点正东面的 3 格距离可以表示为 $\sim 3 \sim \sim$ 。

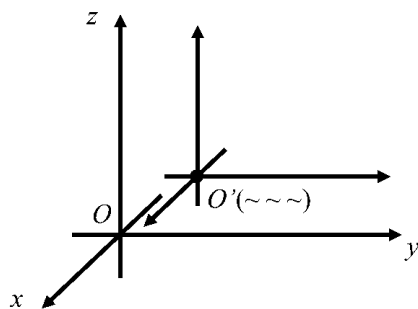


图 2.5 在绝对坐标系中建立的相对坐标系

例 2.3

一个位于点 $(-24, 55, 10)$ 的命令方块，其相对坐标 $\sim 12 \sim -3 \sim -5$ 所指的方块坐标为_____。

解

相对坐标规定命令方块所在的位置即为原点，在将相对坐标转换为绝对坐标时，只需要在绝对坐标的基础上做相应的加减，这个题中的方块坐标为 $(-24 + 12, 55 - 3, 10 - 5)$ ，计算可得 $(-12, 52, 5)$ 。

相对坐标可以与绝对坐标混合使用。在不使用波浪号的坐标参数中，计算绝对坐标。比如， $\sim 10 \ 10 \ \sim 10$ 会根据锚点变换 x 坐标和 z 坐标，但 y 坐标被固定为 10。

二、局部坐标

除了相对坐标外，命令系统还有一种更加灵活的坐标，即**局部坐标 (Local coordinates)**。局部坐标也用于表示相对偏移量，由脱字符 $\^$ 和相对偏移量的格式来表示：

`^[<dx>] ^[<dy>] ^[<dz>]`

2.5

与相对坐标不同的是，局部坐标脱离了绝对坐标系规定的方向，它以命令执行朝向作为基准，由朝向角度参数决定，可以是任意的。如图 2.6，使用局部坐标相当于建立了一个坐标轴方向任意的坐标系，但是坐标轴之间的正交关系不变。若执行朝向发生变动，这个坐标系也随之转动。

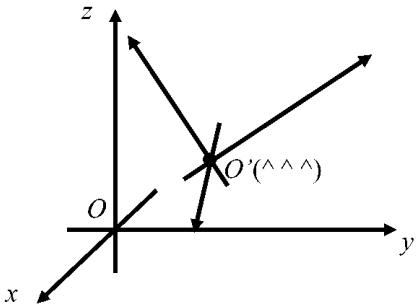


图 2.6 局部坐标系

局部坐标系规定：**命令执行朝向被视为 z 轴的正方向，若命令执行者为命令方块，则局部坐标系与相对坐标系无异**，因此 x 轴正方向位于命令执行位置的左边，y 轴正方向位于命令执行位置的上方。比如，执行位置右边 3 米距离的局部坐标为 `^-3 ^ ^`。**由于局部坐标不使用绝对坐标系规定的坐标轴方向，因此同一坐标参数中局部坐标不能与绝对坐标和相对坐标混用。**

局部坐标会被执行锚点影响。当锚点是脚部时，**相对坐标和局部坐标的原点都在执行位置（脚部）**。而当锚点是眼部时，**相对坐标的原点在执行位置，也就是脚部；局部坐标的原点在眼部**。

以下举一个例子以进一步说明：

`execute as @a at @s anchored eyes run setblock ~ ~ ~ stone`

2.6

这条命令将执行锚点设为玩家的眼部，而执行位置仍然为玩家碰撞箱底部中心点，这个位置被相对坐标 `~ ~ ~` 使用，因此石头会放置在玩家下半身所在方块。

`execute as @a at @s anchored eyes run setblock ^ ^ ^ stone`

2.7

同样是将执行锚点设为玩家的眼部，但 `^ ^ ^` 使用执行锚点，因此石头会放置在玩家上半身所在方块。

2.1.3 调试屏幕的坐标

使用 `F3` 打开调试屏幕时，屏幕上会显示和坐标有关的一些信息。首先是屏幕中央的准星会由十字形变成一个简约的正交分解三维坐标轴。规定红线代表 x 轴正方向，绿线代表 y 轴正方向，蓝线代表 z 轴正方向。这个图形会随着玩家的朝向改变而旋转，可以很容易地通过该图形中辨别方向。

使用 `F3` + `F6` 打开调试选项页面，其中的 `player_position`、`player_section_position`、`looking_at_block`、`looking_at_fluid` 等都会显示坐标信息。`player_position` 中一共有四行用于显示世界坐标的信息，如图 2.7 所示。在框内信息中，第一行表示的是玩家当前的位置，其中 y 坐标表示的是玩家脚底的 y 坐标。第二行表示的是玩家脚部的方块坐标。第三行显示的是区块信息，第

四行是玩家的朝向信息，指明了玩家大致的方向朝向（东南西北）以及具体的朝向角度参数（偏航角/俯仰角）。`player_section_position` 显示的是玩家在区块内的相对坐标。

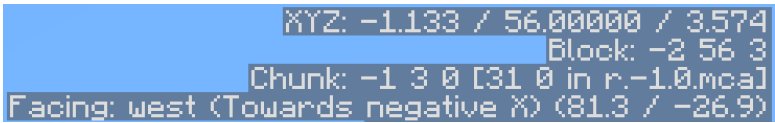


图 2.7 调试屏幕中的玩家坐标

如图 2.8，当 `looking_at_block` 开启时，若玩家指向一个方块，调试屏幕会显示玩家指向方块的方块坐标及其方块状态与其所属的数据包标签；当 `looking_at_fluid` 开启时，会显示指向液体的信息。

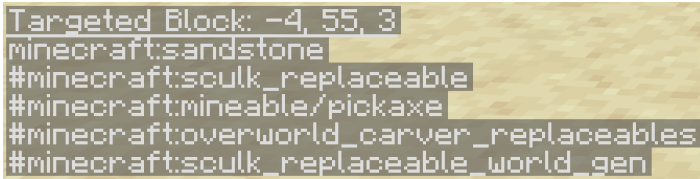


图 2.8 调试屏幕中的方块坐标信息

2.2 朝向

上一节对局部坐标的描述使用了“朝向”这一概念。通常地、为了在空间直角坐标系中实体的朝向，可以使用两个旋转角度来表示：即水平方向上的**偏航角（Yaw）**和竖直方向上的**俯仰角（Pitch）**。其中偏航角表示实体使用这两个参数可表示三维空间中实体所有朝向，因为原版的 Minecraft 并未使用翻滚角。在命令中，所有的朝向均采用角度制，参数不写度数符号，一般来说命令参数类型为 `minecraft:rotation`，其中含有两个参数，分开表示成

<yaw> <pitch>

2.8

2.2.1 偏航角

偏航角定义了实体绕绝对坐标 y 轴旋转的角度，因此它也可以被称为绕 y 轴旋转角度。它是一个实体的水平朝向与绝对坐标 z 轴的夹角 γ ，其中 $\gamma \in [-180, 180)$ 。并且规定： γ 随顺时针方向增大， z 轴正方向 $\gamma = 0$ 。

γ 的符号可以按如下方法判断：对水平面内的向量进行正交分解，若有同向平行于 x 轴正方向或与 x 轴重合的分向量，则 $\gamma < 0$ ；反向平行于 x 轴则 $\gamma > 0$ 。用方位的语言描述：水平朝向偏向东，则 $\gamma < 0$ ；偏向西则 $\gamma > 0$ 。

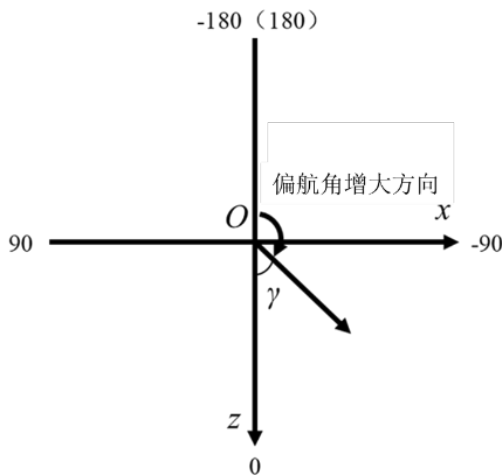


图 2.9 偏航角

注意在以上描述中， γ 的值都被限定在 $[-180, 180)$ 内。这个范围一般被视为偏航角的可用值（调试屏幕显示的范围），但是超出这个范围的参数仍然可以被识别。识别的规则是：**给一个偏航角参数加上或减去 360，则新的偏航角参数与原来的偏航角参数等效**，最终将任意偏航角换算至 $[-180, 180)$ 的范围内。例如，`180` 会被识别为 `-180`，即正北方向；`270` 会被识别为 `-90`；`580` 会被识别为 `-140`。

例 2.4

要求设置某实体的朝向为北偏东 75° ，则偏航角参数应为_____。

解 如图 2.10 所示，根据偏航角的旋转角度特性，北偏东 75° 的方向参数就是在正北方向参数上加 75，即 $-180 + 75 = -105$ 。偏航角参数为 `-105`。本题没有规定参数范围，因此填 `255`、`-465` 之类的答案也算正确。

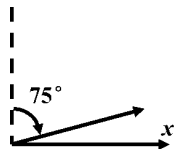


图 2.10

类似于相对坐标，游戏允许使用 `~` 和相对偏移量表示相对偏航角，格式为：

~[<yaw>]

2.9

表示在原来偏航角的基础上增加一定角度值后形成的偏航角。例如，若初始偏航角参数为 `30`，`~15` 则表示在 `30` 的基础上增加 15，即 `45`。

2.2.2 俯仰角

俯仰角定义了实体绕局部坐标 x 轴旋转的角度，因此它也可以被称为绕 x 轴旋转角度。它是一个实体的水平朝向与局部坐标 x 轴的夹角 θ ，其中 $\theta \in [-90, 90]$ 。并且规定： θ 随俯视方向增大，与 yOz 平面平行时 $\theta = 0$ 。

θ 的符号可以按如下方法判断：对局部坐标 yOz 面内的向量进行正交分解，若与 y 轴正方向重合，则 $\theta < 0$ ；与 y 轴反向则 $\theta > 0$ 。用方位的语言描述：仰视时 $\theta < 0$ ；俯视则 $\theta > 0$ 。

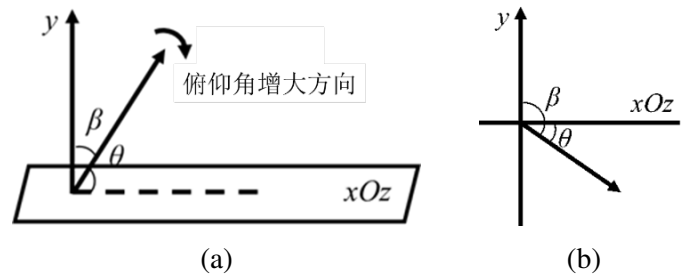


图 2.11 俯仰角

在以上描述中， θ 的值都被限定在 $[-90, 90]$ 内。这个范围一般被视为俯仰角的可用值（调试屏幕显示的范围），但是超出这个范围的参数仍然可以被识别。识别的规则是：**给一个俯仰角参数加上或减去 360，则新的俯仰角参数与原来的俯仰角参数等效**。最终将任意俯仰角换算至的范围 $[-90, 90]$ 内。例如，`370` 会被识别为 `10`。但是显然任意俯仰角的范围局限于 $[-90 + 360n, 90 + 360n]$ ， $n \in \mathbb{Z}$ ，诸如 `-180` 这样的参数不在此范围内，因此该参数是无效的。

和相对偏航角一样，游戏也允许使用 `~` 和相对偏移量表示相对俯仰角，格式为：

`~[<pitch>]`

2.10

表示在原来俯仰角的基础上增加一定角度值后形成的俯仰角。由偏航角、俯仰角组成的朝向参数允许绝对朝向和相对朝向混用。

2.2.3 命令/rotate

命令 `/rotate` 用于修改实体的朝向，以下是所有用法：

- 1. 使实体旋转至特定的朝向

`rotate <target> <rotation>`

2.11

其中的参数：`<targets>`（实体 `minecraft:entity`）—— 需要被旋转的实体。
`<rotation>`（朝向 `minecraft:rotation`）—— 特定的朝向，含有两个参数，依次为偏航角、俯仰角。允许使用波浪号 `~` 以表示相对朝向。

例 2.5

将最近的玩家修改为水平面向正南方。

解

水平面向正南方的偏航角为 `0.0`，俯仰角为 `0.0`，因此命令为

`rotate @p 0.0 0.0`

2.12

- 2. 使实体朝向特定的坐标

`rotate <target> facing <facingLocation>`

2.13

其中的参数：`<facingLocation>`（三维坐标 `minecraft:vec3`）——指定朝向的坐标，必须为精确的三维坐标，整数坐标会被中心校准。

执行锚点会对该命令的执行结果造成影响，因为它会改变命令上下文。若该命令的执行者锚点为 `feet`，则以朝向位置对脚底的方向作为视线方向（如图 2.12 (a)），因此如果朝向位置的 y 坐标与指定实体脚底 y 坐标相等，理论上该指定实体的视线方向应是水平的；若该命令的执行者锚点为 `eyes`（如图 2.12 (b)），则以朝向位置对眼睛的方向作为视线方向。若该命令单独使用，则默认锚点为 `feet`，如果需要将锚点设为 `eyes`，则可以用 `/execute` 命令修改：

```
execute anchored eyes run rotate @s facing 0 70 0
```

2.14

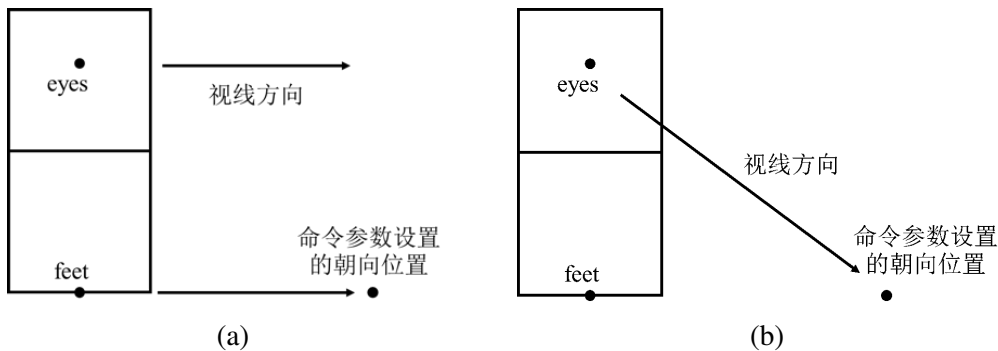


图 2.12 锚点与朝向位置的关系对视线方向的影响

3. 使实体朝向特定的实体

```
rotate <target> facing entity <facingEntity> [<facingAnchor>]
```

2.15

其中的参数：`<facingEntity>`（实体 `minecraft:entity`）——指定朝向的目标实体
`[<facingAnchor>]`（实体锚点 `minecraft:entity_anchor`）——指定朝向目标实体的锚点，可用值：
`feet`（默认）朝向目标实体的脚部。
`eyes`（默认）朝向目标实体的眼部。

朝向的准则遵循图 2.12 所示的规律，视线方向会严格按照锚点之间的位置关系。例如，单独使用如下的命令：

```
rotate A facing entity B eyes
```

2.16

这时指定实体 A 的锚点为 `feet`，而朝向为实体 B 的锚点 `eyes`，此时朝向实体的上方而不是实体 B 的眼睛部位，如图 2.13 所示。

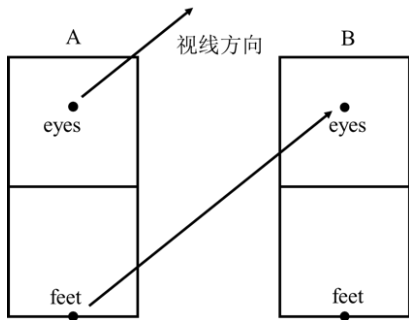


图 2.13 `/tp` 命令锚点例子

只有当实体 A 的锚点为 `eyes` 时, 才会朝向实体 B 的眼睛部位, 这时需要使用命令 `/execute`。

2.2.4 朝向与局部坐标的关系 *

对于空间内的任意向量 α , 在计算其朝向时, 可以先将其正交分解为两个向量: 即如图 2.14 所示的平行于水平面的向量 $\alpha_{\parallel}$ 和垂直于水平面的向量 $\alpha_{\perp}$ 。偏航角是分向量 $\alpha_{\parallel}$ 与 z 轴的夹角 (注意符号), 俯仰角是 α 与其水平分向量的夹角 (注意符号)。运用方向角公式可以求出两个向量 $\alpha_1 = (x_1, y_1, z_1)$ 、 $\alpha_2 = (x_2, y_2, z_2)$ 之间的夹角 γ :

$$\cos \gamma = \frac{\alpha_1 \alpha_2}{\|\alpha_1\| \|\alpha_2\|} = \frac{x_1 x_2 + y_1 y_2 + z_1 z_2}{\sqrt{x_1^2 + y_1^2 + z_1^2} \sqrt{x_2^2 + y_2^2 + z_2^2}} \quad (2.1)$$

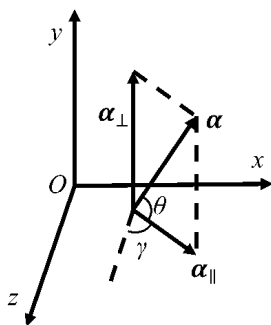


图 2.14 任意朝向

据此可以求出两个朝向参数。计算偏航角的时候需要构造同向平行于 z 轴的单位向量。下面举一例以说明之。

例 2.6

有一锚点为 $A(12, 100, -170)$ 的实体, 其朝向为坐标 $B(14, 102, -172)$ 的位置, 求该实体的朝向角度 (取值范围 $[-180, 180)$, 取三位有效数字)。

解

首先进行向量的分解, 先得出向量 $\overline{AB} = (2, 2, -2)$ 在水平面上的分向量, 如图 2.15 所示。

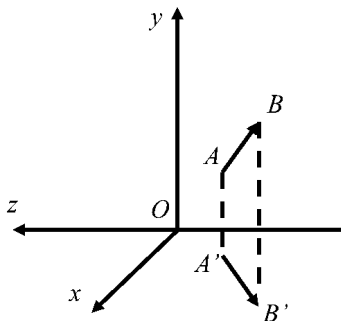


图 2.15 水平投影

水平分向量 $\overline{A'B'} = (2, 0, -2)$, 此时只需要计算它与 z 轴的方向角, 取同向平行于 z 轴的单位向量 $e = (0, 0, 1)$, 应用方向角余弦公式可得

$$\cos \gamma = \frac{-2}{\sqrt{2^2 + (-2)^2} \times 1} = -\frac{\sqrt{2}}{2} \quad (2.2)$$

因为方向向量的指向偏东，故偏航角为 -135° 。

计算俯仰角则只需求 $\overrightarrow{AB}$ 与 $\overrightarrow{A'B'} = (2, 0, -2)$ 的夹角，则

$$\cos \theta = \frac{2 \times 2 + (-2) \times (-2)}{\sqrt{2^2 + 2^2 + (-2)^2} \times \sqrt{2^2 + (-2)^2}} = \frac{\sqrt{6}}{3} \quad (2.3)$$

又 $\overrightarrow{A'B'} = (2, 0, -2)$ 的竖直分量与 y 轴正方向平行，故 $\theta < 0$ ，则俯仰角取近似值 -35.3° 。

引入偏航角、俯仰角两个朝向参数后，局部坐标指向的具体位置便可以准确计算出来了。设偏航角为 γ ，俯仰角 θ ，则局部坐标系内某向量 α 实际上也跟随朝向角度增量发生转动得到向量 $\alpha' = (x', y', z')$ ，这里称为等效的相对坐标。图 2.16 (b) 是这两个向量在 xOz 平面上的投影，图 2.16 (c) 是它们在 yOz 平面上的投影。

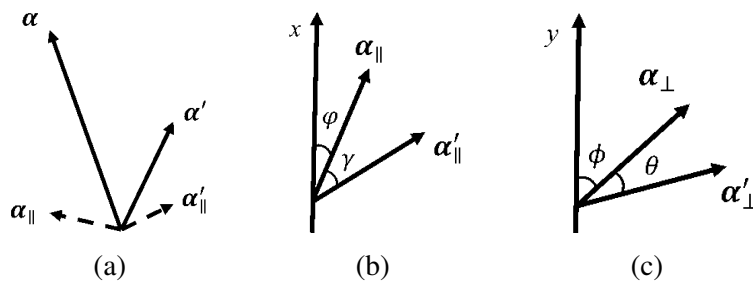


图 2.16 两个向量的投影结果

先分析在 xOz 平面上的投影，实际上水平方向的旋转即绕 y 轴旋转，此时 y 坐标不变。假设向量 $\alpha_{\parallel}$ 与 x 轴的夹角为 φ ，令 $\alpha_{\parallel}$ 的模为 l ，则有

$$\begin{cases} x = l \cos \varphi \\ y = y_0 \\ z = l \sin \varphi \end{cases} \quad (2.4)$$

同理可得向量 $\alpha'_{\parallel}$ 的坐标关系

$$\begin{cases} x'_{\parallel} = l \cos(\varphi + \gamma) = l \cos \varphi \cos \gamma - l \sin \varphi \sin \gamma \\ y'_{\parallel} = y_0 \\ z'_{\parallel} = l \sin(\varphi + \gamma) = l \sin \varphi \cos \gamma + l \cos \varphi \sin \gamma \end{cases} \quad (2.5)$$

式 (2.4) 代入得

$$\begin{cases} x'_{\parallel} = x \cos \varphi - z \sin \varphi \\ y'_{\parallel} = y \\ z'_{\parallel} = x \sin \varphi + z \cos \varphi \end{cases} \quad (2.6)$$

矩阵是一种用于求解线性方程组和实现线性变换的数学工具，可以将式 (2.6) 写成矩阵形式

$$\begin{pmatrix} x'_{\parallel} \\ y'_{\parallel} \\ z'_{\parallel} \end{pmatrix} = \begin{pmatrix} \cos \gamma & 0 & -\sin \gamma \\ 0 & 1 & 0 \\ \sin \gamma & 0 & \cos \gamma \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} \quad (2.7)$$

其中令矩阵

$$\mathbf{R}_y(\gamma) = \begin{pmatrix} \cos \gamma & 0 & -\sin \gamma \\ 0 & 1 & 0 \\ \sin \gamma & 0 & \cos \gamma \end{pmatrix} \quad (2.8)$$

即得到向量绕 y 轴旋转的矩阵^①，这个矩阵与偏航角相关。

同理可以分析 yOz 上的投影，此时向量绕 x 轴旋转，得到旋转矩阵

$$\mathbf{R}_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix} \quad (2.9)$$

这个矩阵与俯仰角相关。绕 z 轴旋转的情况这里不需要使用，因为 Minecraft 暂时还不存在翻滚角。等效的相对坐标计算可以直接使用这两个矩阵，即

$$\alpha' = \mathbf{R}_y(\gamma)\mathbf{R}_x(\theta)\alpha \quad (2.10)$$

用向量形式表示的实际绝对坐标 β 即等效的相对坐标与命令执行位置 (x_0, y_0, z_0) 的和

$$\beta = \alpha' + (x_0, y_0, z_0)^T \quad (2.11)$$

下面举一个计算的例子。

例 2.7

有一个位于点 $(30, 50, 10)$ 的命令执行者，执行位置为该点，其偏航角和俯仰角分别为 45° 和 45° ，求其局部坐标 $^{\wedge}10 \ ^{\wedge}10$ 的绝对坐标。

解

由题意得 $\gamma = 45^\circ$ ， $\theta = 45^\circ$ ， $\alpha = (10, 0, 10)^T$ 。由式 (2.10)，此坐标对于命令执行者的相对坐标为

$$\begin{aligned} \alpha' &= \mathbf{R}_y(\gamma)\mathbf{R}_x(\theta)\alpha \\ &= \begin{pmatrix} \cos 45^\circ & 0 & -\sin 45^\circ \\ 0 & 1 & 0 \\ \sin 45^\circ & 0 & \cos 45^\circ \end{pmatrix} \begin{pmatrix} \cos 45^\circ & 0 & -\sin 45^\circ \\ 0 & 1 & 0 \\ \sin 45^\circ & 0 & \cos 45^\circ \end{pmatrix} \begin{pmatrix} 10 \\ 0 \\ 10 \end{pmatrix} \\ &= \begin{pmatrix} 5\sqrt{2} - 5 \\ -5\sqrt{2} \\ 5\sqrt{2} + 5 \end{pmatrix} \end{aligned} \quad (2.12)$$

则绝对坐标为

^① 这个矩阵与数学上通用的旋转矩阵不同，数学上向量旋转以逆时针为正方向，Minecraft 的朝向角度以顺时针为正方向。

$$\boldsymbol{\beta} = \boldsymbol{\alpha}' + (30, 50, 10)^T = \begin{pmatrix} 5\sqrt{2} + 25 \\ -5\sqrt{2} + 50 \\ 5\sqrt{2} + 15 \end{pmatrix} \approx \begin{pmatrix} 32.07 \\ 42.93 \\ 22.07 \end{pmatrix} \quad (2.13)$$

2.3 坐标参数在命令中的应用

本节讲述一些使用坐标参数的命令语法。

2.3.1 使用方块坐标的命令

一些命令使用的坐标参数均为方块坐标 `minecraft:block_pos`，这部分命令包括用于放置方块的 `/setblock`、`/fill`、`/clone` 和用于设置出生点的 `/spawnpoint`、`/setworldspawn`。

一、命令 `/setblock`

此命令用于更改单个方块。这里的“单个方块”意为长宽高均小于等于 1 米的方块，如石头一类的标准方块，楼梯、台阶、栅栏、玻璃板一类的“不完整方块”。若遇到长度或高度大于 1 米的方块则无法使用单个 `/setblock` 命令以放置完整的方块，如床、门等。此命令所需权限等级为 2，语法为：

```
setblock <pos> <block> [destroy|keep|replace|strict]
```

2.17

其中的参数：<pos>（方块坐标 `minecraft:block_pos`）——需要放置方块的坐标。

<block>（方块状态 `minecraft:block_state`）——参数格式为 `命名空间ID[方块状态]{NBT}`，表示被放置的方块，其中方块状态和方块实体 NBT 是可选的。

[destroy|keep|replace|strict] ——选填，四种有效值的效果如下：

`destroy` ——原方块会像被不带有精准采集或时运的工具破坏那样掉落其掉落物，并且会播放方块被破坏的声音。例如，将玻璃方块更改为其他方块时不会掉落任何掉落物，更改草方块时会掉落泥土，以此类推。

`keep` ——只有空气方块会被更改，非空气方块将被保留。注意：如果使用 `/setblock` 去更改非空气方块，则命令执行失败。

`replace` ——完全更改原方块，原方块不会掉落其掉落物、没有被破坏的声音。如果不指定

参数 `[destroy|keep|replace|strict]`，命令默认使用这种更改方式。

`strict` ——使用这种模式时更改方块不会触发方块更新。其他模式均会触发方块更新。

例 2.8

在 (0,0,0) 放置一个石头。

解 命令为

```
setblock 0 0 0 minecraft:stone
```

2.18

例 2.9

在 (0,0,0) 放置一个浮空的沙子。

解 如果使用非 `strict` 模式放置，则命令会触发方块更新，进一步造成沙子坠落而无法浮空，因此需使用 `strict` 模式：

```
setblock 0 0 0 minecraft:sand strict
```

2.19

二、命令 `/fill`

该命令用指定的方块填充一个长方体区域的全部或部分，所需权限等级为 2。以下是它的一种语法，这条语法仅会更改填充区域内的非空气方块，与 `/setblock` 中的 `keep` 模式完全相同。

```
fill <from> <to> <block> keep
```

2.20

其中的参数：`<from>`、`<to>`（方块坐标 `minecraft:block_pos`）——如图 2.17 所示，两个对角方块可以决定空间内的一个长方体，一般将这个长方体占据的空间称为**源区域（Source region）**。在 `/fill` 命令中，决定源区域的两个方块坐标不必刻意强调其某个坐标轴上坐标参数的大小。

`<block>`（方块状态 `minecraft:block_state`）——与语法 2.17 一致。

`/fill` 的另一种语法如下：

```
fill <from> <to> <block> [replace <filter>] [destroy|hollow|outline|strict]
```

2.21

其中的参数：`[<filter>]`（方块谓词 `minecraft:block_predicate`）——可选，用于指定需要被替换的方块种类；若不指定，则所有方块都会被替换，注意此

时字面量 `replace` 也需要省略。对于方块谓词这种参数，可以用命名空间 ID 指定某一种方块，也可以用数据包标签指定某一类方块，同时允许指定方块状态或方块实体 NBT。具体格式为：

```
<命名空间ID>[<方块状态>=<值>,...]
{<方块实体数据>}
```

其中 `[<方块状态>=<值>,...]` 和 `{<方块实体数据>}` 在不需要时可以省略。检查方块时只会检查此参数指定的方块状态是否匹配。

`[destroy|hollow|outline|strict]` ——可选，方块处理方式。注意无论是哪种处理方式，其更改的方块都只能是 `[<filter>]` 指定的方块，除非不指定被替换的方块。`destroy` 与 `strict` 的用法和效果与 `/setblock` 中的用法完全相同，参考语法 2.17。其他的可选参数有其特殊的用处：

`hollow` ——它的效果相当于一个空心的、外部只有一层方块包裹的长方体。源区域的外层将被替换为指定的方块，内部将被替换为空气。

`outline` ——与 `hollow` 类似，`outline` 也会生成有一层方块包裹的长方体，但是内部的方块则不会发生变化。这样的效果可以理解为为内部的结构套一层方块外框。

一条 `/fill` 命令的源区域的方块数量上限由游戏规则 `max_block_modifications` 决定，默认为 32768，可由命令 `/gamerule` 修改。若源区域体积超过此上限则命令执行失败，无论实际上修改的方块数量。

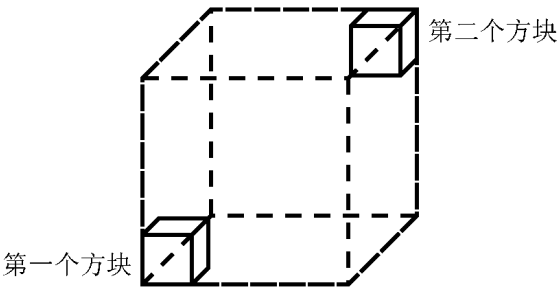


图 2.17 `/fill` 命令图解

例如，以下两条命令均能够被识别，且二者表示的源区域完全相同。

```
fill 12 24 17 13 18 16 stone
```

2.22

```
fill 13 18 16 12 24 17 stone
```

2.23

设第一个方块坐标为 (x_1, y_1, z_1) ，第二个方块坐标为 (x_2, y_2, z_2) ，则源区域的方块总数

$$S = (|x_1 - x_2| + 1)(|y_1 - y_2| + 1)(|z_1 - z_2| + 1) \tag{2.14}$$

例 2.10

判断下列命令是否超过了默认的 `max_block_modifications` 规定的限制。

(1) 其中源区域均为空气方块：

```
fill ~ ~ ~ ~24 ~-22 ~17 stone
```

2.24

(2) 其中源区域含有 600 个非空气方块：

```
fill 23 179 66 5 223 104 stone keep
```

2.25

(3) 其中源区域均为空气方块：

```
fill ~3 ~ ~9 ~36 ~-25 ~-49 stone outline
```

2.26

解

- (1) 直接代入式 (2.14) 得 $S = 25 \times 23 \times 18 = 10350$ ，小于默认的 `32768` 上限。
- (2) 代入公式得源区域方块总数 $S = 19 \times 45 \times 39 = 33345$ ，虽然源区域有 600 个非空气方块，实际更改的方块数量为 $33345 - 600 = 32745 < 32768$ ，但源区域的方块总数已超限。
- (3) 代入公式， $S = 34 \times 26 \times 41 = 36244 > 32768$ ，超限。

三、命令 `/clone`

此命令用于将某一个区域的方块复制到另一个区域。对于被复制的区域一般称其为源区域，而 `/clone` 的目的位置称为目标区域 (Destination region)。允许跨维度复制区域。它所需权限等级为 2，语法为：


```
clone [from <sourceDimension>] <begin> <end> [to <targetDimension>]
<destination> [strict] [replace|masked] [force|move|normal]
```

2.27

或

```
clone [from <sourceDimension>] <begin> <end> [to <targetDimension>]
<destination> [strict] filtered <filter> [force|move|normal]
```

2.28

其中的参数：[from <sourceDimension>] ——区域复制来源维度，选填，若需指定，字面量 **from** 必不可少。参数 <sourceDimension> 的类型为 `minecraft:dimension`，必须为维度的命名空间 ID。不指定则默认使用命令执行维度。

<begin>、<end>（方块坐标 `minecraft:block_pos`）——决定源区域的两个方块坐标。

[to <targetDimension>] ——区域复制目标维度，选填，若需指定，字面量 **to** 必不可少。参数 <sourceDimension> 的类型为 `minecraft:dimension`，必须为维度的命名空间 ID。不指定则默认使用命令执行维度。

<destination>（方块坐标 `minecraft:block_pos`）——目的位置被定义为目标区域西北下角的方块，复制到目标区域的结构在每个坐标轴上均位于该方块坐标的正方向，如图 2.18 所示。

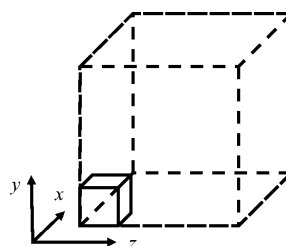


图 2.18 `/clone` 命令目的位置方块在目标区域中的位置

[strict] ——可选，若使用该参数则不会触发方块更新。

对于语法 2.27 而言，

其中的参数：[replace|masked] ——可选，两者各自的作用为：

replace ——强制复制所有方块，即用源区域的方块完全替换目标区域的方块。为方块处理方式的默认值。

masked ——仅复制非空气方块，即只有目标区域中源区域非空气方块对应的位置会被替换，源区域空气方块对应的位置则保持不变。

[force|move|normal] ——可选，它们各自的作用是：

force ——无论源区域和目标区域是否有重叠，一律强制复制。若有重叠的部分，则该部分目标区域会覆盖源区域。

move ——将源区域“移动”到目标区域，即复制过后源区域会被替换为空气。

normal ——默认的复制方式，在该复制方式下，源区域和目标区域不能有重叠的部分。

语法 2.28 是 `/clone` 命令的过滤模式，其中的参数：`<filter>`（方块谓词 `minecraft:block_predicate`）——在源区域中选定需要被复制的方块 ID，只有指定的方块才会被复制到目标区域中。

例 2.11

将对于命令执行位置而言 `~ ~ ~` 和 `~2 ~1 ~3` 之间区域的所有草方块按其在区域内的位置复制到 `~ ~10 ~`。

解

显然需要使用 `/clone` 的过滤模式，命令为：

```
clone ~ ~ ~ ~2 ~1 ~3 ~ ~10 ~ filtered minecraft:grass_block
```

2.29

四、命令 `/spawnpoint`

这条命令用于设置玩家的出生点，需要的权限等级为 2，其语法为：

```
spawnpoint [<targets>] [<pos>] [<angle>]
```

2.30

其中的参数：`[<targets>]`（实体 `minecraft:entity`）——如果不填指定的玩家，则默认作用于命令执行者。如果命令执行者为命令方块，则必须填写指定的玩家。允许使用目标选择器指定玩家。

`[<pos>]`（方块坐标 `minecraft:block_pos`）——三个参数必须为整数，因此无法在设置玩家出生点时使用精确的实际坐标，所以玩家的出生点只能被设置在一个方块底部的中心点。若不填写坐标参数则会将玩家出生点设置在命令执行者的当前位置。

`[<angle>]`（角度 `minecraft:angle`）——指定玩家出生时的偏航角，只有偏航角一个参数，不可指定俯仰角。

读者也可以使用这条命令设置在任意维度的出生点。但是在末地使用 `/spawnpoint` 需谨慎，因为末地传送门会将玩家传送回出生点，无论该出生点所在的维度，因此很有可能会将玩家困在末地，不过玩家依旧可以使用 `/tp` 传送至其他维度的玩家或使用 `/execute` 子命令 `in` 直接传送至指定维度。

五、命令 `/setworldspawn`

用于设置世界出生点，当命令执行成功后出生点区块会变为该出生点所在区块，也会影响区块刻的赋予。注意：该命令不能用于设置除主世界外其他维度的世界出生点，需要的权限等级为 2，语法为：

```
setworldspawn [<pos>] [<angle>]
```

2.31

2.3.2 使用三维坐标的命令

命令 `/tp` 和 `/teleport` 使用的坐标参数均为三维坐标 `minecraft:vec3`，一般使用浮点数，允许使用整数以进行中心校准。这两条命令可以互通，语法也完全相同。出于习惯以及在实际操作中为了方便输入命令，通常在语法格式介绍中使用更为简便的 `tp` 字符。`/tp` 的主要作用为传送实体至指定的位置或指定其朝向，所需权限等级为 2，以下是它的所有用法。

1. 将命令执行者直接传送至指定的目标实体，并与目标实体保持相同的朝向，语法为：

```
tp <destination>
```

2.32

其中的参数： `<destination>`（实体 `minecraft:entity`）——指定要传送至的实体，可以是玩家名称、UUID 或目标选择器，但只允许选择一个实体。

2. 将命令执行者传送至指定的坐标，朝向保持不变，语法：

```
tp <location>
```

2.33

其中的参数： `<location>`（三维坐标 `minecraft:vec3`）——可以使用整数坐标以进行中心校准。

3. 将指定的实体传送至其他目标实体，并与目标实体保持相同的朝向，语法为：

```
tp <targets> <destination>
```

2.34

其中的参数： `<targets>`（实体 `minecraft:entity`）——被传送的实体。

`<destination>`（实体 `minecraft:entity`）——同语法 2.32 一致。

例如，将玩家 A 传送至玩家 B（不要混淆成将玩家 B 传送至玩家 A）：

```
tp A B
```

2.35

4. 将指定的实体传送至指定的坐标，并允许修改被传送者的朝向角度，语法为：

```
tp <targets> <location> [<rotation>]
```

2.36

其中的参数： `<rotation>`（朝向 `minecraft:rotation`）——选填，偏航角在前，俯仰角在后。如果不使用该参数，则会保持指定实体原本的朝向。

5. 将指定的目标实体传送至指定的坐标位置，并指定其朝向的坐标，语法为：

```
tp <targets> <location> facing <facingLocation>
```

2.37

其中的参数： `<facingLocation>`（三维坐标 `minecraft:vec3`）——决定目标实体朝向的坐标。

6. 将指定实体传送至指定的坐标位置，并指定其朝向的实体，语法为：

```
tp <targets> <location> facing entity <facingEntity> [<facingAnchor>]
```

2.38

其中的参数： `<facingEntity>`（实体 `minecraft:entity`）——指定要朝向的实体。

`<facingAnchor>`（实体锚点 `minecraft:entity_anchor`）——可选，指定朝向实体的何种锚点，可以为 `eyes` 或 `feet`，默认为 `feet`。

2.3.3 使用二维坐标的命令

`/spreadplayers` 和 `/worldborder` 使用的坐标参数均为二维坐标 `minecraft:vec2`。

一、命令 `/spreadplayers`

这条命令用于分散多个实体、将实体传送到任意位置，所需权限等级为 2，语法为：

```
spreadplayers <center> <spreadDistance> <maxRange> [under <maxHeight>]
<respectTeams> <targets>
```

2.39

其中的参数：`<center>`（二维坐标 `minecraft:vec2`）——只使用 x 坐标和 z 坐标，两个元素均为双精度浮点数，指定传送区域的中心点。

`<spreadDistance>`（浮点数 `brigadier:float`）——目标实体之间的最小间距，最小值 `0.0`。

`<maxRange>`（浮点数 `brigadier:float`）——指传送区域的边界与中心点在 x 轴和 z 轴上的距离，最小值应为 `1.0`。因此传送区域为一个正方形而不是圆形。

`[under <maxHeight>]`——可选，包含两个参数：`under` 是字面量，无需更改写法，`<maxHeight>` 参数类型为整型 `brigadier:integer`，参数之间的空格不要缺失。如果指定了实体传送的最大高度，则在最大范围内一定要保证该最大高度下有足够的空间，否则命令执行失败。

`<respectTeams>`（布尔值 `brigadier:bool`）——若为 `true`，则同队伍的所有成员会传送到同一个位置。

`<targets>`（实体 `minecraft:entity`）——需要被传送的实体，可以为多个玩家名称、UUID 或目标选择器，玩家名称之间应该用空格隔开。也可以指定除玩家外的实体。

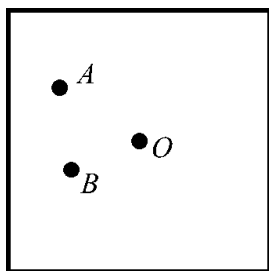


图 2.19 命令 `/spreadplayers` 图解

如图 2.19， O 点为传送区域的中心点，最大范围决定了 O 点离正方形边界的距离，分散间距决定了实体 A 和实体 B 之间的最短距离。在实际的命令编写中还需要考虑的问题是分散间距与最大范围的关系，如果最大范围无法满足分散间距的需求，则命令执行失败。在这条命令中，两个实体的间距不能大于传送区域的边长，否则便会判定最大范围无法满足分散间距的需求。根据正方形的几何性质来看，当目标实体数量不大于 4 个时，分散间距必须小于最大范围的两倍。

下面为该命令的具体使用举例：

例 2.12

使用 `/spreadplayers` 按要求写出下列命令：

- (1) 将两个不属于同一队伍的玩家 `Waterman12345`、`Mu_xian` 传送至随机位置，其中中心点为命令执行者所在位置，最大范围为 12，两个玩家间距不能小于 7；
- (2) 将所有实体随机传送，要求：中心点为 (0,0)，最大范围 100，传送的最大高度为 100，同队的所有实体必须传送在一起，队伍与队伍之间最小间距为 12。

解

- (1) 命令为

```
spreadplayers ~ ~ 7 12 false Waterman12345 Mu_xian
```

2.40

- (2) 命令为

```
spreadplayers 0 0 12 100 under 100 true @e
```

2.41

二、命令 `/worldborder`

虽然 Minecraft 的游戏世界巨大无比，但其仍然是有可玩游戏区域界限的，自定义世界边界则是这个界限的一种保障。从宏观上来看，世界边界是一个巨大的正方形边框。世界边界的中心决定了世界边界的位置，而正方形边界的边长则决定了世界边界的大小，两种参数均可自定义。

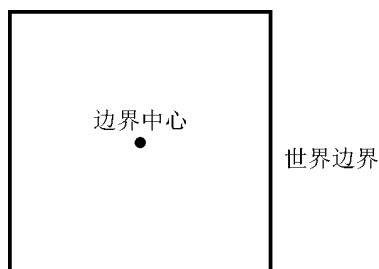


图 2.20 宏观的世界边界及边界中心

边界中心 (World border's center) 使用二维坐标。由于二维坐标缺少 y 坐标，因此它不能用于指定方块坐标，所有的二维坐标无论其是否为整数均指定精确的实际 xOz 坐标。玩家可以自定义一些带有小数的二维坐标世界中心，并且世界边界会严格按照边界中心确定其位置。默认的边界中心为 (0,0)。边界中心与出生点区块和玩家出生点无关，`/setworldspawn` 和 `/spawnpoint` 无法指定边界中心。

边界的大小一般指**世界边界直径 (World border diameter)**，即图 2.20 中正方形的**边长**。该参数可以为小数并且世界边界会严格按照该参数确定其大小。世界边界直径可以接受的最大值为 `59999968`，最小值为 `1`，因此边界中心距世界边界的最大值为 `29999984`，最小值则为 `0.5`。

1. 设置世界边界直径增量，语法为：

```
worldborder add <distance> [<time>]
```

2.42

其中的参数：`<distance>`（双精度浮点数 `brigadier:double`）——即世界边界直径。在原来的世界边界直径基础上增加或减少指定的距离，距离参数可以为负。

若该参数为正，则世界边界直径增加，若为负，则减小。

[<time>] (整型 `brigadier:integer`) —— 选填，用于指定世界边界直径发生变化所需的时间，注意**该参数类型实际上是整型而非真正的时间参数，且单位为秒而非游戏刻**。若不指定则默认为 0，即世界边界直径会瞬间发生变化。

2. 设置世界边界直径大小，语法为：

`worldborder set <distance> [<time>]`

2.43

其中的参数：<distance> (双精度浮点数 `brigadier:double`) —— 直接指定世界边界直径的大小。

例 2.13

有中心位于 (0, 0)、原直径为 300 的世界边界，要使该世界边界在一定时间内直径变化至 50。且由于地图制作的要求，一堵边界墙的移动速度需要约等于玩家正常步行的速度 (4.317m/s)。写出符合要求的命令，各参数需要精确到个位。

解 世界边界的变化可用 `/worldborder add` 和 `/worldborder set` 指定，如图 2.21 所示：

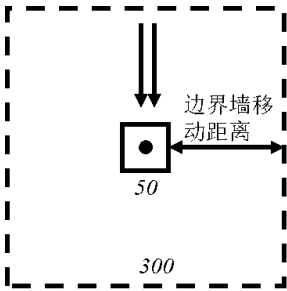


图 2.21

先探讨 `/worldborder add` 命令所需的参数：由正方形的性质，可以很快求得边界墙移动距离 $\Delta d = \frac{300 - 50}{2} = 125$ ，该结果即为距离参数。又该世界边界直径缩小，所以距离参数应为负，即 -125。同时要求边界墙的移动速度需要等于玩家正常步行的速度，故时间参数为 $t = \frac{\Delta d}{v} = \frac{125}{4.317} \approx 29$ 。所以可用的命令可以是

`/worldborder add -125 29`

2.44

在 `/worldborder set` 的情况下，时间参数的计算仍然需要使用边界墙移动距离 Δd ，但是 Δd 本身不会出现在命令参数中。所以可用的命令也可以是

`worldborder set 50 29`

2.45

3. 设置边界中心的位置，语法为：

worldborder center <pos>

2.46

其中的参数：`<pos>`（二维坐标 `minecraft:vec2`）——世界中心，两个元素均为双精度浮点数。

4. 设置玩家越过世界边界后作用于玩家的伤害，语法为：

worldborder damage buffer <distance>

2.47

worldborder damage amount <damagePerBlock>

2.48

其中的参数：`<distance>`（双精度浮点数 `brigadier:double`）——边界墙的外围有一层缓冲区，在这层缓冲区中的玩家可免受世界边界的伤害，此参数指定该缓冲区的宽度。该参数为浮点数，必须为非负数，若指定为 `0`，则边界外围没有缓冲区，玩家一旦越过边界墙即受到伤害。

`<damagePerBlock>`（浮点数 `brigadier:float`）——缓冲区的外围是一层层的伤害区，每一层伤害区宽度为 1 格方块，此参数用于决定每一层伤害区每秒钟对玩家造成的伤害，其计算方法为：

$$D_n = n \cdot d$$

(2.15)

式中： D_n ——第 n 层伤害区每秒造成伤害。
 n ——层数。
 d ——`<damagePerBlock>` 的值。

例如，`/worldborder damage amount 0.1` 会让玩家在第 5 层伤害区受到每秒 0.5 的伤害值。每方格伤害值必须为浮点数和非负数，其默认值为 `0.2`。

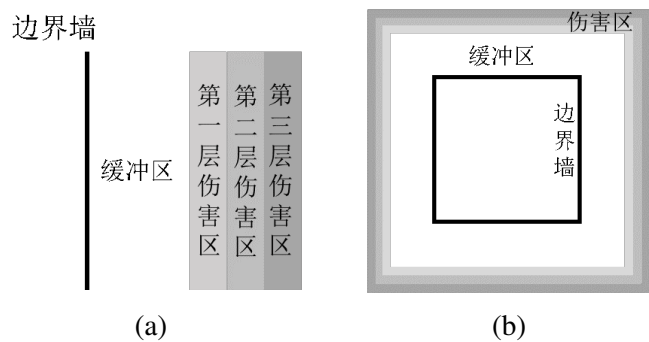


图 2.22 世界边界作用于玩家的伤害区域

5. 返回当前世界边界的直径，语法为：

worldborder get

2.49

6. 设置世界边界的警告

世界边界的警告可用 `distance` 和 `time` 两种模式来指定。若指定：玩家距离边界墙一定距离 d 时产生警告，适用 `distance` 模式；若指定：某位置在 t 秒后即将被边界墙越过，若玩家进入该位置即产生警告，则适用 `time` 模式。语法分别为：

小提示

当玩家靠近边界墙时，玩家的屏幕会变红，但若图像设置为流畅，则屏幕不会变红。这种视觉效果在冒险地图中被广泛使用，且通常用于表现紧张或恐惧的情景。

`worldborder warning distance <distance>`

2.50

`worldborder warning time <time>`

2.51

其中的参数：`<distance>`（整型 `brigadier:integer`）——最小值可以为 `0`，默认值为 `5`。
`<time>`（整型 `brigadier:integer`）——最小值可以为 `0`，默认值为 `15`。

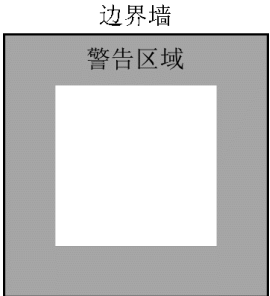


图 2.23 边界墙与警告区域（灰色）的位置关系

2.4 区块

Minecraft 游戏世界的加载单位被称为**区块**，一个区块的水平横截面为 16×16 的方格。在高度未扩展之前，一个区块高 256 格方块，覆盖方块 y 坐标 $0 \sim 255$ 的高度。1.18 游戏高度扩展之后，一个区块高达 384 格方块，覆盖方块 y 坐标 $-64 \sim 319$ 的高度。

2.4.1 区块坐标和区段坐标

快捷键 `F3 + G` 可以显示区块边界。区块的边缘总是与方块边缘相贴合，当 x 坐标或 z 坐标能被 16 整除时该位置为区块的边界，因此两个区块的接触面方程可以表示为 $x = 16n$ 或 $z = 16n$ ($n \in \mathbb{Z}$)。因此在 xOz 平面内，一个区块任意顶点的二维坐标可以表示为 $(16n_1, 16n_2)$ ($n_1, n_2 \in \mathbb{Z}$)。

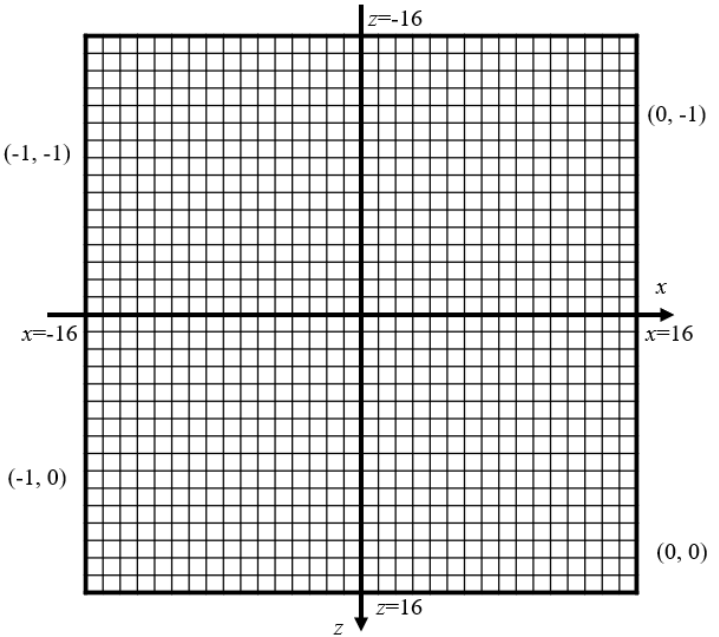


图 2.24 水平面上 (0, 0) 附近的区块

由于 Minecraft 一个世界中区块数量众多，因此有必要对这些区块进行一定处理以方便了解它们大致的位置。表述一个方块的位置使用的是直角坐标系，它对每个方块都规定了其坐标。那么能否使用类似坐标的手段对区块进行描述呢？

对于一个方块而言，其西北下角的顶点是其方块坐标；对于一个区块而言，同样可以使用其西北角的棱作为其区块坐标。在 xOz 平面内，若一个区块截面的西北角顶点坐标为 $(16n_1, 16n_2)$ ($n_1、n_2 \in \mathbb{Z}$)，则该区块的区块坐标可用方括号表示为 $[n_1, n_2]$ ($n_1、n_2 \in \mathbb{Z}$)。如图 2.24 所示，若要表示由 $x = 16$ 、 $z = 16$ 、 x 轴和 z 轴围成区块的区块坐标，其西北角的点为坐标轴原点，则区块坐标可以表示为 $[0, 0]$ 。

对于一个区块内的区段，它相比区块而言多了 y 轴方向的度量，并且当 y 坐标能被 16 整除时该位置为区段顶部或底部的边界。游戏使用区段的西北下角顶点作为它的区段坐标：若一个区段的西北下角顶点为 $(16n_1, 16n_2, 16n_3)$ ($n_1、n_2、n_3 \in \mathbb{Z}$)，则该区段的区段坐标可表示为 $[n_1, n_2, n_3]$ ($n_1、n_2、n_3 \in \mathbb{Z}$)。这对于 y 坐标小于 0 的空间同样适用。

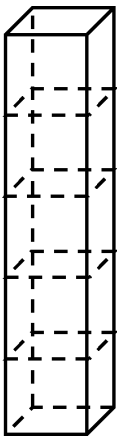


图 2.25 一个区块被分成若干个区段

当描述一个方块在该区块内的位置时，就需要使用到区段坐标。并且需要在该区段的西北下角建立一个空间直角坐标系，用这个坐标系描述方块在区段内的相对位置。因此在这个坐标系

内，所有坐标参数均为大于等于 0 且小于等于 15 的整数，西北下角的方块在区段内位置可表示为 (0,0,0)。

对于一个方块坐标为 (a,b,c) ($a、b、c \in \mathbb{Z}$) 的方块，可以直接使用 floor 函数^①求得该方块所在区段的区段坐标为：

$$\left[\left\lfloor \frac{a}{16} \right\rfloor, \left\lfloor \frac{b}{16} \right\rfloor, \left\lfloor \frac{c}{16} \right\rfloor \right] \quad (2.16)$$

该数据显示于调试屏幕 `Chunk` 一行。因此该方块所在区块为

$$\left[\left\lfloor \frac{a}{16} \right\rfloor, \left\lfloor \frac{c}{16} \right\rfloor \right] \quad (2.17)$$

该区段西北下角方块的方块坐标可由区段坐标转化：

$$\left(16 \left\lfloor \frac{a}{16} \right\rfloor, 16 \left\lfloor \frac{b}{16} \right\rfloor, 16 \left\lfloor \frac{c}{16} \right\rfloor \right) \quad (2.18)$$

而西北下角的方块在区段内位置表示为 (0,0,0)，由坐标与方向的关系知结构西北下角的所有坐标参数在结构内最小，而该方块的方块坐标为 (a,b,c) ，则该方块在区块中的位置可表示为

$$\left(a - 16 \left\lfloor \frac{a}{16} \right\rfloor, b - 16 \left\lfloor \frac{b}{16} \right\rfloor, c - 16 \left\lfloor \frac{c}{16} \right\rfloor \right) \quad (2.19)$$

对于任何方块坐标，都可以通过代入上式计算其所在的区块，并求得该方块在区块中的位置。如果是玩家的坐标，也可以以此计算调试屏幕中 `player_section_position` 的数据。

2.4.2 命令/forceload

在 18w31a 之前，根据预传送加载标签的原理，一般使用 `/spreadplayers` 来加载所需要的区块，但是这种加载方式非常地不稳定。于 18w31a 加入的命令 `/chunk`（后改名为 `/forceload`）很好地解决了这个问题。

`/forceload` 用于指定强制加载的区块，并把这些区块的加载等级和计算等级指定为 31。它需要的权限等级为 2，所有用法如下：

1. 指定或解除强制加载的区块，语法为：

```
forceload (add|remove) <from> [<to>]
```

2.52

其中的参数：`<from>`、`[<to>]`（平面方块坐标 `minecraft:column_pos`）——两个参数均只接受整数形式的二维化方块坐标，元素顺序为 `<x> <z>`，允许使用相对坐标，不接受局部坐标。`[<to>]` 参数可选填，

^① floor 函数即向下取整函数，取不大于该实数的最大整数。数学语言一般记作 $\lfloor x \rfloor$ ，例如 $\lfloor 3.14 \rfloor = 3$ ， $\lfloor -1.5 \rfloor = -2$ ， $\lfloor 1 \rfloor = 1$ 。

若不填则仅操作 `<from>` 坐标所在区块。注意,该语法中所用的二维坐标非区块坐标,使用命令的时候需要手动计算确定方块所在的区块。

2. 在一个维度中解除所有区块的强制加载,语法为:

```
forceload remove all
```

2.53

3. 查询区块是否为强制加载区块,语法为:

```
forceload query [<pos>]
```

2.54

其中的参数: `[<pos>]` (平面方块坐标 `minecraft:column_pos`) ——指定要查询的方块所在的区块,若不指定则返回强制加载的所有区块,返回值会使用区块坐标。

例 2.14

在一个初始情况下没有强制加载区块的世界中使用命令 `/forceload add 45 -32 87 10`。

- (1) 求强制加载的区块个数;
- (2) 写出 `/forceload query` 的返回内容。

解

首先根据起始、终止两个参数的二维方块坐标确定两个指定点所在的区块。由上文推导的结果可得到:二维方块坐标 (a, c) 所在的区块为 $\left[\left\lfloor \frac{a}{16} \right\rfloor, \left\lfloor \frac{c}{16} \right\rfloor\right]$ 。对于起始点 $(45, -32)$, 代入可得起始区块 $[2, -2]$, 同理可得终止区块 $[5, 0]$, 因此强制加载的区块共有 $4 \times 3 = 12$ 个。

第 (2) 小题所问即写出所有强制加载的区块,一共有 12 个区块: $[2, -2]$ 、 $[2, -1]$ 、 $[2, 0]$ 、 $[3, -2]$ 、 $[3, -1]$ 、 $[3, 0]$ 、 $[4, -2]$ 、 $[4, -1]$ 、 $[4, 0]$ 、 $[5, -2]$ 、 $[5, -1]$ 、 $[5, 0]$ 。

第二章思考题与习题

1. 一个命令执行位置为 $(-27, 32, 102)$, 其相对坐标 `~29 ~-3 ~23` 所表示的绝对坐标为_____。
2. 一个命令执行位置为 $(457, 23, -1283)$, 则用于表示绝对坐标 $(321, 21, -1234)$ 的相对坐标为_____。
3. 一个命令方块位于 $(-90, 122, -18)$, 其局部坐标 `^-18 ^19 ^77` 所表示的绝对坐标为_____。
4. 某玩家的水平朝向为南偏西 24° , 24° 的偏航角参数为_____。
5. 已知某玩家的锚点为 $(220, 17, 26)$, 其偏航角为 `135`, 俯仰角为 `-45`, 求 (取小数点后一位):
 - (1) 相对坐标 `~23 17 ~-9` 表示的绝对坐标;

- (2) 局部坐标 $^3 \ ^7 \ ^{-6}$ 表示的绝对坐标。
- 一锚点位于 (3, 7, 9) 的实体朝向坐标 (6, 8, 7), 求该实体的朝向, 并判断在调试页面中显示的大概朝向信息。(取小数点后一位)
 - 某玩家的锚点为 (33, 56, -10), 其偏航角和俯仰角分别为 30 和 45, 若要用局部坐标表示绝对坐标为 (12, 40, -12) 的点, 则求局部坐标。(取小数点后一位)
 - 命令 `/fill ~-1 ~7 ~3 ~19 ~6 ~-5 stone replace` 源区域的方块数量为
 - 已知玩家 Sky 位于 (1.5, 57.0, -36.6), 一个命令执行者使用

```
tp @s ~ ~ ~ facing entity Sky feet
```

2.55

则该命令执行者实际朝向的坐标为_____。

- 设置玩家出生点为 (0, 64, 0), 且出生时平视、面朝正东方的命令为_____。
- 在下列命令中, 判断其是否有错误, 若有则指出其错误。
 - `spreadplayers ~ ~ 13 5 under 90 false @a`
 - `setblock 30 73 -45 quartz_block replace`
 - `tp 0 70 0 66.5 -45`
 - `worldborder warning distance 1.5`
 - `clone 12 15 4 15 17 -1 9 17 -6`
- 现用如下命令自定义一个世界边界:

```
worldborder center 0 0
```

2.56

```
worldborder set 50
```

2.57

```
worldborder damage buffer 5
```

2.58

```
worldborder damage amount 0.5
```

2.59

某生命值为20的玩家位于 (0, 35), 由于该位置处于边界之外, 因此其以疾跑的速度 (5.612m/s) 奔向安全区域。

- 判断该玩家能否在生命值归零之前跑到安全区域;
- 在上述条件下, 又以如下命令定义世界边界的移动: `/worldborder add -30 10`, 玩家的疾跑与世界边界发生变化同时开始, 判断该玩家能否在生命值归零之前跑到安全区域。
- 区块 [4, 5] 和区块 [3, 5] 的接触面方程为_____。
- 命令 `/forceload add 1247 -359` 所指定的强加载区块为_____。
- 求下列方块坐标所在的区块 (区段), 并求出这些坐标在区段中所处的位置。
 - (127, 93, -29)
 - (-9825, 2, 193)
 - (83, 48, -2148)
 - (-293, 212, -249)
- 在一个均为空气方块的区域中, 若一个位于区块 [2, -3] 的命令执行者使用命令:

```
setblock x 0 0 command_block
```

2.60

其中 x 为未知数, 此时渲染距离为 10。

- (1) 若命令方块能够被正常放置, 求 x 的最大值和最小值。
- (2) 若放置的命令方块能被红石激活且执行命令, 求 x 的最大值和最小值。
17. 在初始情况下没有强加载区块的区域中依次执行以下命令:

```
forceload add 12 17 124 156
```

2.61

```
forceload remove -13 14 100 110
```

2.62

求最终指定的强加载区块数量。

第三章 UUID 与目标选择器

一些命令，诸如 `/tp`、`/spreadplayers` 等，不可避免地会与实体产生互动。这些命令基本上都会使用有如 `minecraft:entity`、`minecraft:game_profile` 的参数类型。这些参数类型的表示方式是：如果要选择特定玩家，则可以直接指定**玩家的名称**，选择其他实体的情况则可能需要使用其他的参数，这便是下文所讲的 UUID 和目标选择器。

3.1 UUID

实体种类繁多，不同的实体可能拥有不同的名称，存储有不同的 NBT 数据，行为、基础属性也不尽相同。为了区分这些实体，现在给每个实体单独派发一张“身份证”，每张“身份证”都标有一个号码，这些号码彼此之间各不相同。这些用于区分不同对象的数字串即**通用唯一识别码 (Universally Unique Identifier, 简称 UUID)**。

UUID 具有以下性质：

1. 唯一性

每个 UUID 对应一个对象，在不使用修改器的情况下，不存在共用同一个 UUID 的多个实体。因此可以通过指定一个 UUID 以选择唯一的对象，此举使得目标的选择变得精准、无差错。游戏内部存在随机数生成器以保证每次生成的 UUID 都是不同的。

2. 大容量

UUID 为 128 位数。因此 UUID 支持二进制下从 -2^{127} 到 $2^{127} - 1$ 的所有整数值。

UUID 有以下几种表示方式：

1. 有连字符的十六进制 (Hyphenated hexadecimal)

使用 32 位长的 16 进制表示，拆分成五个部分，相邻部分之间用连字符隔开，格式为

```
<8 位>-<4 位>-<4 位>-<4 位>-<12 位>
```

3.1

例如，一个开启的、朝向为东的铁门可写成如下的形式：

```
0a324608-a033-42f4-b753-c34de1ad3154
```

3.2

其中每一部分都单独作为一个数字处理，允许存在先导零，也可以不写先导零。**UUID 作为单独的参数使用时，通常使用这种写法。**

例 3.1

已知一个玩家的 UUID 为 654c6848-d79d-4e07-bbf4-0a88e65a57ef，尝试将命令执行者传送至该玩家。

解

根据 `/tp` 的语法，命令为

```
tp 654c6848-d79d-4e07-bbf4-0a88e65a57ef
```

3.3

在聊天栏中输入命令时，若遇到参数类型 `minecraft:entity`，且此时执行者的视线正好指向另一个实体的碰撞箱，则自动补全会以有连字符的十六进制的形式显示指向实体的 UUID。

2. 十六进制 (Hexadecimal)

使用 32 位长的 16 进制表示但不使用连字符。其中的每个零都不能被省略，例如

```
00000001000100010001000000000001
```

3.4

3. 整型数组 (Int array)

在 NBT 中使用，使用 4 个 10 位的十进制数表示，例如

[I;1699506248,-677556729,-1141634424,-430286865]

3.5

3.2 目标选择器

3.2.1 目标选择器变量

通过玩家名称或 UUID 选择目标，每次只能选择一个。若需要批量选择或附加特定的限制条件，则这两种方法就无能为力了。而穷举玩家名称或 UUID 明显不是一种有效的办法。命令系统提供了一种高效选择目标实体的参数，即**目标选择器（Target selectors）**。

目标选择器共有 6 种变量：`@p`、`@r`、`@a`、`@e`、`@s`、`@n`。下面将逐个介绍这 6 种变量的功能：

1. `@p`
- 用于选择距离命令执行者**附近的玩家**，如果有多个玩家与命令执行者的距离相同，则会选择最后进入服务器的玩家。该变量也可用于选择死亡的玩家。
- 例如，如果一个命令执行者位于 (0, 70, 0)，有两个玩家分别位于 (13, 70, 0) 和 (-14, 0, 0)，显然位于 (13, 70, 0) 的玩家距离命令执行者更近一些，因此 `@p` 会选择该玩家。
- 如果命令执行者为玩家，则直接使用 `@p` 会指向身为命令执行者的玩家本身。比如，在聊天栏内输入命令

tp @p <x> <y> <z>

3.6

会将玩家自身传送至指定的坐标。

2. `@r`
- 用于选择**随机的玩家**，也可用于选择死亡的玩家。
3. `@a`
- 用于选择**所有的玩家**，死亡的玩家也会被包括在内，已卸载区块内的实体不会被选择。
4. `@e`
- 在已加载的区块中选择**所有存活的实体**，已卸载区块内的实体不会被选择。
5. `@s`
- 用于选择**当前实体，即命令执行者本身**，无论命令执行者是否存活。若在命令方块或服务器控制台中引用该变量，则不会选择任何的实体。
6. `@n`
- 与 `@p` 类似，但用于选择距离命令执行者**附近的实体**。
- 下表总结了这些目标选择器变量的功能：

表 3.1 目标选择器变量特性表

目标选择器变量	作用实体	关键词	能否选择死亡的实体
<code>@p</code>	仅玩家	附近	能

(续表)

目标选择器变量	作用实体	关键词	能否选择死亡的实体
@r	仅玩家	随机	能
@a	仅玩家	所有	能
@e	所有实体	所有	否
@s	所有实体	当前	能
@n	所有实体	附近	否

3.2.2 目标选择器参数

目标选择器的六种变量仅提供了最基础的实体选择功能，可以在此基础上为这些变量添加一些限定条件以进一步选择需要的实体。这些限定条件被称为**目标选择器参数 (Target selector arguments)**。要添加这些参数，只需要在目标选择器变量后方添加方括号，在方括号内输入参数名及其值，参数与参数之间用逗号隔开，方括号与目标选择器变量之间不能有空格，语法为：

<目标选择器变量>[参数名=值,参数名=值]

3.7

示例：

@p[team=A]

3.8

@e[type=creeper,limit=1]

3.9

当目标选择器含有多个参数时，所有的参数都会被计算，这有点类似于“与”逻辑运算，这也意味着可以在同一个目标选择器中设置多个限定条件。但在同一个目标选择器中，大部分情况下的参数名不能出现两次，否则该选择器将无法被解析。错误示例：

@e[type=sheep,type=cow]

3.10

由于不存在既为绵羊又为牛的实体，因此该选择器不会被解析。

参数的值可以为数字、浮点数范围或字符串，当值为字符串时，可以使用 **=!** 进行反选，即选择参数不为该值的实体，语法为：

<目标选择器变量>[参数=!值,参数=!值]

3.11

当一个参数定义为反选时，该参数在同一个目标选择器中允许出现两次。其中的逻辑关系可以通过举例说明：无法选择既为鸡又为牛的生物，但可以选择既不为鸡又不为牛的生物，这时候用于选择实体类型的参数就可以在同一个目标选择器中出现两次。

目标选择器的参数种类繁多，本教程将这些参数分为若干个类别，下面是每一类参数的具体介绍：

一、原点参数

原点使用一组双精度浮点数作为参数的值，共包括了 `x`、`y`、`z` 三个参数，用于在世界中定义一个位置。参数的值可以为小数，原点的坐标不适用中心校准，所有原点坐标一律使用其实际坐标。用法举例：

```
[x=0,y=57,z=0]
```

3.12

这个选择器参数定义了精确的原点 (0.0, 57.0, 0.0)。原点参数是可选的，三个参数均可选填。当某个坐标的参数未定义值时，则在未指定的坐标轴上使用命令执行位置。例如，定义下列参数：

```
[x=0,z=0]
```

3.13

由于参数 `y` 未定义，则使用命令执行位置的 `y` 坐标作为参数 `y` 的值。若三个参数均未定义值，则完全使用命令执行位置。

一般而言，原点参数需要配合距离、体积、排序或数量参数使用，**不配合这些参数单独使用原点参数是无效的**。

二、距离参数

距离参数的参数名为 `distance`，值可以为浮点数或浮点数范围，但必须为非负数，其用于选择与位置锚点有特定距离的实体，使用**欧几里得距离 (Euclidean distance)**。若指定了原点（球心），与该点之间有特定距离的点的集合为一个球面，因此距离参数也可以被理解为通过球形空间选择实体。概括地说，距离参数一共有以下四种用法：

1. 球面选择

若定义位置锚点参数分别为 x_1 、 y_1 、 z_1 ，距离参数为 d ，设实体坐标为 (x_2, y_2, z_2) ，当且仅当满足关系式 $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$ ，该实体才会被选择。语法为：

```
[distance=<值>]
```

3.14

该用法要求实体与锚点有特定的距离。在空间上看，选择的区域仅为一层球面，处于球内的实体不会被选择。

2. 球体选择

若定义位置锚点参数分别为 x_1 、 y_1 、 z_1 ，距离参数为 d ，设实体坐标为 (x_2, y_2, z_2) ，实体被选择时需要满足关系式 $d \leq \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$ 。只需定义上限临界值，语法为：

```
[distance=..<值>]
```

3.15

由于被选择的实体必须小于或等于 `distance` 定义的值，因此被选择的区域在空间中形成一个实心的球体。

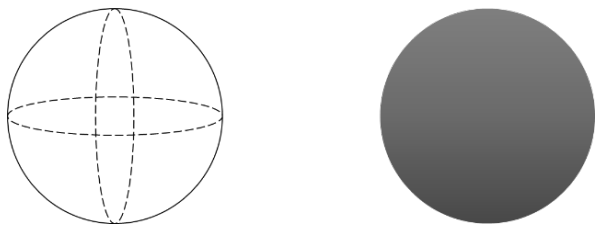


图 3.1 距离参数定义的球面（左）和球体（右）

3. 球体反选

若定义位置锚点参数分别为 x_1 、 y_1 、 z_1 ，距离参数为 d ，设实体坐标为 (x_2, y_2, z_2) ，实体被选择时需要满足关系式 $d \geq \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$ 。只需定义下限临界值，语法为：

[distance=<值>..]

3.16

在这种情况下被选择的实体必须大于 `distance` 定义的值，在欧式空间中的位置锚点周围挖去一个球体，剩下的无限大区域就是该情况选择的范围。读者可以理解为“球体反选”，即选择不属于该球体的区域。但是球体反选在游戏中受制于区块的加载，选择的区域并不是无穷大的。

4. 球壳选择

球壳选择定义了两个参数，一个为上限 d_{MAX} ，另一个为下限 d_{min} ，即选择距离锚点不超过 d_{MAX} 又不小于 d_{min} 的实体。现定义位置锚点参数分别为 x_1 、 y_1 、 z_1 ，设实体坐标为 (x_2, y_2, z_2) ，则被选择的实体需满足 $d_{min} \leq \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \leq d_{MAX}$ 。需要定义两个临界值，语法为：

[distance=<下限>..<上限>]

3.17

在空间内，选择的区域为一个有厚度的球壳（注意与没有厚度的球面区分）。



图 3.2 球壳（左）和球壳沿某一方向的剖面（右）

三、体积参数

体积参数用于在长方体区域内选择实体，具体方法为：在原点的基础上沿三个坐标轴延伸一定的距离，组成的区域为一个长方体。体积参数共分为三个参数：`dx`、`dy` 和 `dz`，三个参数均可以为负数。当参数为正数时，选择的区域会在其对应的坐标轴正方向上延伸特定的长度；若为负数则往负方向延伸。语法为：

[dx=<值>,dy=<值>,dz=<值>]

3.18

设三个体积参数值分别为 dx 、 dy 、 dz ，理论上长方体区域的三条边长分别为 $|dx|$ 、 $|dy|$ 、 $|dz|$ ；但根据 MC-123441，实际上三条边长分别为 $|dx| + 1$ 、 $|dy| + 1$ 、 $|dz| + 1$ ，该特性至今仍然存在游戏中。这意味着若三个参数都设为 0，目标选择器依旧会创建一个 $1 \times 1 \times 1$ 大小的选区。

由于 `dx`、`dy` 和 `dz` 可以为负数，不能认为选区是以原点为基础往正方向延伸 $|dx| + 1$ 、 $|dy| + 1$ 和 $|dz| + 1$ 长度，而是取以 (x, y, z) 和 $(x + dx, y + dy, z + dz)$ 为顶点的长方体区域的西北下角顶点（坐标值最小点）为基础，往正方向延伸 $|dx| + 1$ 、 $|dy| + 1$ 和 $|dz| + 1$ 长度。因此，选区的坐标值最小点为 $(\min\{x, x + dx\}, \min\{y, y + dy\}, \min\{z, z + dz\})$ ，坐标值最大点为 $(\max\{x, x + dx\} + 1, \max\{y, y + dy\} + 1, \max\{z, z + dz\} + 1)$ 。

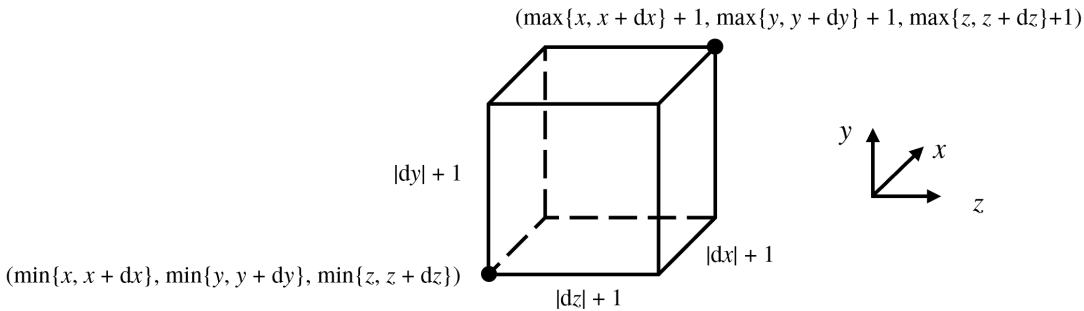


图 3.3 体积参数的选区

不同于其他的位置参数，当实体的碰撞箱与选取有重叠时，该实体就会被选择。

例 3.2

- 写出符合下列要求的目标选择器。
- (1) 距离命令执行者 10 格以内的所有实体；
 - (2) 距离点 (72, 76, 155) 15 格开外又不超过 32 格的所有玩家；
 - (3) 由 $(-13, 40, 32)$ 和 $(10, 55, 67)$ 决定的长方体区域内的随即玩家。

解 (1) 至 (3) 题所需的选择器如下所示，其中第 (3) 小题答案不唯一：

@e[distance=..10]

3.19

@a[x=72,y=76,z=155,distance=15..32]

3.20

@r[x=-13,y=40,z=32,dx=22,dy=14,dz=34]

3.21

四、水平旋转参数

水平旋转的参数名为 `y_rotation`。当选择特定偏航角的实体时，有如下语法：

[y_rotation=<值>]

3.22

同样地、水平旋转参数地值也可以是浮点数范围，当定义了两个临界值(含)时，语法可以为：

[y_rotation=<最小偏航角>..<最大偏航角>]

3.23

该语法用于选择偏航角介于定义的最小偏航角和最大偏航角（含）之间的实体。举例：

[y_rotation=-10..10]

3.24

该参数用于选择偏航角介于 -10° 和 10° 之间的实体。在以右为 z 轴正方向、以上为 x 轴正方向建立的平面坐标系中，偏航角的范围可以表示为（阴影部分）：

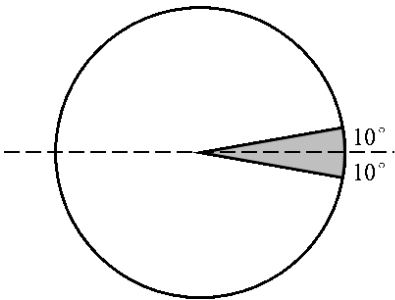


图 3.4

给一个偏航角参数加上或减去 360，则新的偏航角参数与原来的偏航角参数等效。对于目标选择器中超出 $[-180, 180)$ 范围的参数，可以对其加上或减去 360 换算到合适的区间，例如下面的目标选择器参数选择的范围与 3.24 一致：

[y_rotation=-10..370]

3.25

[y_rotation=-190..10]

3.26

[y_rotation=-190..730]

3.27

对于下面的参数：

[y_rotation=10..190]

3.28

将 190° 换算为 -170° 后，结果为

[y_rotation=10..-170]

3.29

发现最大的偏航角参数反而比最小的偏航角小。不妨对这个范围取反，选择偏航角位于 $10^\circ..-170^\circ$ 的实体相当于选择偏航角不位于 $-170^\circ..10^\circ$ 的实体，即对图 3.5 (a) 的范围取反，得到图 3.5 (b) 所示的范围。

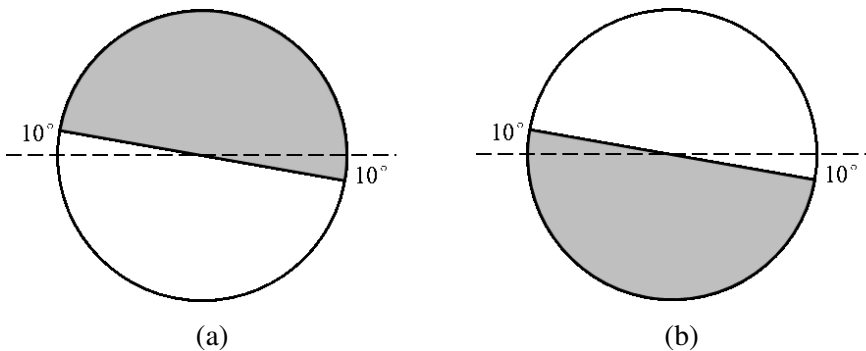


图 3.5

综上所述，对于任意的水平旋转参数，先将所有超出 $[-180, 180)$ 的角度换算到该值域内，若换算后存在下限比上限大的浮点数范围，则交换上下限，对交换上下限后的范围进行取反，最终得到实际的偏航角角度范围。

五、竖直旋转参数

竖直旋转的参数名为 `x_rotation`，用于选则有特定俯仰角的实体。与水平旋转有所不同的是，**竖直旋转的参数被严格限定在 $[-90, 90]$** 。当选择特定俯仰角的实体时，有如下语法：

```
[x_rotation=<值>]
```

3.30

同样地、竖直旋转参数的值也可以是浮点数范围，当定义了两个临界值(含)时，语法可以为：

```
[y_rotation=<最小俯仰角>..<>最大俯仰角>]
```

3.31

举例：在以上为 y 轴正方向建立的坐标系中，参数

```
[x_rotation=-45..45]
```

3.32

表示的范围如图 3.6 所示。

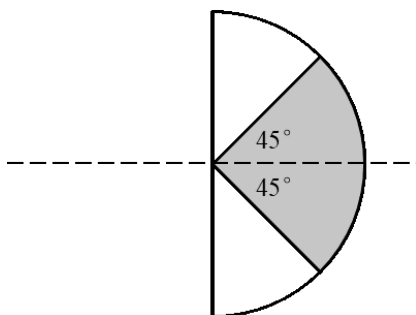


图 3.6

如果存在下面的参数：

```
[x_rotation=45..-45]
```

3.33

显然最大的俯仰角比最小的俯仰角小，同水平旋转参数一样，不妨对这个范围取反，选择俯仰角位于 `45..-45` 之间的实体即选择俯仰角不位于 $-45..45$ 之间的实体，而俯仰角位于 `-45..45` 的范围已经由图 3.6 给出，只需对这个范围取反即可，如图 3.7 所示。

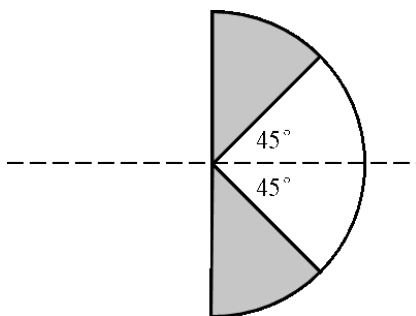


图 3.7

综上所述，**对于任意的水平旋转参数，若存在下限比上限大的浮点数范围，则交换上下限，对交换上下限后的范围进行取反**，最终得到实际的俯仰角角度范围。

六、记分板标签参数

记分板标签是一种将实体分类的方法，由节 7.1 的命令为实体赋予记分板标签后，就可以在目标选择器中使用记分板标签参数选择这些被分类后的实体了。记分板标签参数名为 `tag`，是少有的可以在非反选的情况下于同一个目标选择器中多次出现的参数，这是由于同一个实体可以被赋予多个不同的记分板标签。语法为：

```
[tag=<标签名>]
```

3.34

```
[tag=<标签名>,tag=<标签名>]
```

3.35

其中的标签名均为字符串，因此也可以通过 `!=` 对实体进行反选，即选择没有定义该记分板标签的实体，语法为：

```
[tag!=<标签名>]
```

3.36

例如，要选择一个实体，而该实体带有记分板标签 `A` 和 `B`，则这两个目标选择器参数都能选中该实体：

```
[tag=A]
```

3.37

```
[tag=B]
```

3.38

若游戏内存在其他只带有记分板标签 `A` 或 `B` 的实体，而只需选择同时带这两个记分板标签名实体时，可以进一步将参数写成：

```
[tag=A,tag=B]
```

3.39

带有记分板标签 `A`、`B` 两者之一的实体不会被选择。若游戏内存在其他同时带有记分板标签 `A`、`B` 和 `C` 的实体，则该实体满足既有记分板标签 `A`、又有记分板标签 `B` 的限定条件，则参数 3.39 也会选择到该实体。若只需选择仅带有 `A` 和 `B` 两个记分板标签的实体，就需要把参数写成：

```
[tag=A,tag=B,tag!=C]
```

3.40

即选择带有记分板标签 `A` 和 `B` 而不带有记分板标签 `C` 的实体。

特别地、当记分板标签参数的值为空时，会选择所有没有记分板标签的实体：

```
[tag=]
```

3.41

若对其进行反选，则会选择所有拥有记分板标签的实体：

```
[tag!=]
```

3.42

七、队伍参数

同记分板标签一样，队伍也是记分板的重要组成部分。一个玩家最多只能存在于一个队伍中，因此同一个目标选择器中队伍参数最多只能出现一次。队伍参数的参数名为 `team`，语法：

```
[team=<队伍名>]
```

3.43

选择不属于某队伍的实体：

```
[team!=<队伍名>]
```

3.44

当队伍参数的值为空时，会选择所有不属于任何队伍的实体：

```
[team=]
```

3.45

若进行反选，则会选择所有属于任意队伍的实体：

```
[team=!]
```

3.46

八、记分项参数

记分项参数的作用为：当实体在某一个或多个记分项上的分数同时满足定义的分数范围时，该实体就会被选择。记分项参数支持同时定义多个记分项的选择范围，语法格式较一般的目标选择器参数有所不同，即在参数值的位置上加花括号，在花括号中定义各记分项的分数范围，记分项之间用逗号隔开：

```
[scores={<记分项>=<值>}]
```

3.47

```
[scores={<记分项>=<值>,<记分项>=<值>}]
```

3.48

定义的记分项分数可以为特定的值或整数范围，例如选择在记分项 [A] 的分数介于 12 至 15 之间、在记分项 [B] 的分数为 10 的实体：

```
[scores={A=12..15,B=10}]
```

3.49

九、名称参数

名称参数用于选择有特定名称的实体，语法为：

```
[name=<名称>]
```

3.50

此参数接受反选，即选择不是该名称的实体：

```
[name!=<名称>]
```

3.51

名称必须为字符串，不能包含文本组件。若名称中含有空格，则需在字符串两端添加引号，如：

```
[name="A B"]
```

3.52

如果名称中含有双引号 "，则应在名称的双引号前使用反斜杠 \ 对双引号进行转义。

十、实体类型参数

实体类型参数用于选择特定类型的实体，语法为：

```
[type=<实体类型>]
```

3.53

接受反选，即选择不为该类型的实体：

```
[type=!<实体类型>]
```

3.54

实体类型的值必须为一个有效的命名空间 ID 或数据包标签。当定义命名空间 ID 时，位于命名空间 `minecraft` 内的实体 ID 可以省略命名空间前缀。如果值需要是数据包实体标签，示例有：

```
[type=#raiders]
```

3.55

这个参数用于选择位于 `#raiders` 标签下的实体，这类实体包括唤魔者、卫道士、掠夺者、女巫等，`#raiders` 为默认命名空间 `minecraft` 下的标签，因此这里省略了 `minecraft` 的命名空间前缀。读者也可以在数据包中自定义标签，这些自定义标签也可用于选择实体。

要特别之处的是，目标选择器是一个非常占用游戏运算的组件，因为游戏需要根据选择器中的参数一一遍历实体。尤其是当制作的冒险地图、数据包中目标选择器数量过多时，如无特别要求，应尽量在选择器中使用 `type` 参数，这样游戏就可以仅遍历特定类型的实体，从而减少运算量。

十一、排序参数

在计算机领域中，**排序算法 (Sorting algorithm)** 通常指将一些数据排列成为一组有序序列的过程。在 MC-CMD 中也可以通过这种思路将实体排序，并依照序列处理实体。目标选择器变量便提供了排序算法，例如，变量 `@p` 用于选择命令执行者附近的实体，则游戏会将所有实体按照与命令执行者间的距离进行排序，然后选取距命令执行者更近的玩家。

目标选择器一共提供了四种不同的排序方式，它们分别是 `nearest`、`furthest`、`random`、`arbitrary`。语法为：

```
[sort=(nearest|furthest|random|arbitrary)]
```

3.56

这四种排序方式的作用分别为：`nearest` —— 将实体由近及远排序，即优先选择最近的实体，是变量 `@p` 和 `@n` 的默认排序方式。

`furthest` —— 将实体由远及近排序，即优先选择最远的实体。

`random` —— 将实体随机排序，是变量 `@r` 的默认排序方式。

`arbitrary` —— 将实体按生成时间由先到后排序，即优先选择先生成的实体，是变量 `@a` 和 `@e` 的默认排序方式。

对实体进行排序后，目标选择器会根据排序得到的结果对实体进行一个筛选。这时候可以结合下文所述的数量参数来一次性选择特定数量符合要求的实体。

十二、数量参数

数量参数的语法为：

limit=<值>

3.57

数量参数的值只能为正整数，不接受浮点数范围。事实上，这个数量参数充当了“最大选择数量”的角色，当场上所有符合要求的实体数小于该值时，则所有符合要求的实体都会被选择；反之则会在所有符合要求的实体中选择数量为该值的实体。

目标选择器变量 `@p` 和 `@r` 的数量参数默认为 1，即分别选择距命令执行者最近和随机的一个实体。可以定义其他数量参数的值来扩大可选的实体数量，例如：`@p[limit=3]` 可用于选择距命令执行者最近的三个玩家。`@r[limit=5]` 可用于选择随机的五个玩家。

变量 `@a` 和 `@e` 没有默认的数量参数值，因为它们是分别用于选择所有玩家和所有实体的变量。这时候使用数量参数会限制选择的实体数量。

小提示

变量 `@p` 实际上与 `@a[limit=1,sort=nearest]` 等效。同样，变量 `@r` 与 `@e[limit=1,sort=nearest]` 等效。考虑命令的可读性、直观性，一般使用前者，这样做既省去了不必要的累赘，又适当压缩了命令的长度。

例 3.3

如图 3.8 所示，空间中有实体 A~K 和命令执行位置 O。其中●代表玩家，■代表非玩家实体。

图 3.8

(1) 写出下列目标选择器所选择的实体：

`@a`、`@a[limit=3,sort=nearest]`、`@e[limit=4,sort=furthest]`

(2) 写出可用于选择 D、G、J 三个实体的目标选择器。

解

对于在空间内分布的实体，一般先对其进行排序，主要按照其与命令执行位置之间的距离、实体类型进行排序。按照离命令执行位置由近及远的排序结果如下表所示：

表 3.2 排序结果

序号	1	2	3	4	5	6	7	8	9	10	11
实体	E	D	F	I	G	J	C	H	A	B	K
距离	$2\sqrt{2}$	4	5	$\sqrt{34}$	$3\sqrt{5}$	$\sqrt{65}$	$6\sqrt{2}$	$\sqrt{74}$	$\sqrt{130}$	$\sqrt{145}$	$9\sqrt{2}$
是否为玩家	是	否	是	是	否	否	是	否	是	否	是

(1) 由上表可以很清晰地分析出排序的结果。`@a` 用于选择所有的玩家，即 A、C、E、F、I、K；`@a[limit=3,sort=nearest]` 用于选择距命令执行者最近的三个玩家，值得注意的是，这

里的命令执行者是一个玩家，故命令执行者本身也会被选择，由此可以通过上表找出序号靠前的两个玩家即可，即 E、F，故该目标选择器选择的实体为 O、E、K；

`@e[limit=4,sort=furthest]` 用于选择距命令执行者最远的四个实体，即 K、B、A、H。

- (2) 首先，D、G、J 三个实体均为非玩家，因此可以使用 `type` 参数来限定选择的实体类型，选择不是玩家的实体，则可以使用反选。其次，观察表 3.2 不难发现，D、G、J 是距离命令执行者最近的三个非玩家实体，故可以把目标选择器写成：

```
@e[type=!player,limit=3,sort=nearest]
```

3.58

十三、游戏模式参数

游戏模式参数可用于选择处于该游戏模式的玩家，但也**仅能够用于选择玩家类型的实体**：

```
[gamemode=<游戏模式>]
```

3.59

选择不处于该游戏模式的玩家：

```
[gamemode=!<游戏模式>]
```

3.60

游戏模式参数只接受 `adventure`（冒险模式）、`creative`（创造模式）、`spectator`（旁观模式）和 `survival`（生存模式）四个值，这些值不能用数字代替。

十四、经验等级参数

经验等级参数通过定义玩家的经验等级以过滤玩家，仅能够用于选择玩家类型的实体，语法为：

```
[level=<值>]
```

3.61

其中的值可以为整型或范围。若值为范围形式，则上限和下限必须为整数。

十五、进度参数

进度参数通过玩家的进度来筛选玩家，**仅能够用于选择玩家类型的实体**。参数名为 `advancements`，值为用花括号包括的键值对^①，语法为：

```
[advancements={<键>=<值>}]
```

3.62

其中 `<键>` 为所指定进度的 ID，`<值>` 必须为布尔值或用花括号包括的键值对，若为布尔值，则目标选择器用于筛选 `true`（是）`false`（否）取得该进度的玩家。例如，选择取得进度  石器时代 `story/mine_stone` 的玩家的目標选择器参数可以为

```
[advancements={story/mine_stone=true}]
```

3.63

若 `<值>` 的位置填充了用花括号包括的键值对，则语法又为：

^① 这里指诸如 `<键>=<值>` 形式的字符串，即有对应关系的键值。

```
[advancements={<键>={<键>=<值>}}]
```

3.64

在键值对中嵌套键值对的意义为：**根据进度 JSON 的格式，玩家取得一定进度一定是通过满足这个进度的某些准则达成的。**比如进度  整装上阵 `story/obtain_armor` 的准则之一为装备过铁头盔 `iron_helmet`，可以选择通过装备铁头盔以取得进度整装上阵的玩家：

```
[advancements={story/obtain_armor={iron_helmet=true}}]
```

3.65

其中被选择的玩家当前不必正在装备铁头盔。这些准则的具体情况可以查阅数据包内的进度定义文件，读者也可以使用自定义的进度及其准则。

十六、NBT 参数

NBT 参数用于选择有指定 NBT 的实体，有关 NBT 的内容参考第四章及第六章，语法为：

```
[nbt=<NBT>]
```

3.66

在填写 NBT 时，需要加上花括号，括号内严格按照 NBT 的格式与层级关系填写。此处的 NBT 是一个测试 NBT 标签，用于对比实体的 NBT 数据，具体的对比规则见节 4.2。举例，选择所有手持石头的玩家：

```
@a[nbt={SelectedItem:{id:"minecraft:stone"}}]
```

3.67

此参数也可以使用反选以选择 NBT 不匹配的实体：

```
[nbt=!<NBT>]
```

3.68

然而，由于实体 NBT 的计算本身是耗费较大的项目，加上目标选择器本身的高消耗，因此不建议在目标选择器中定义 NBT 参数，应尽量改用第八章讲述的 `if data` 或 `if items` 子命令。或将实体数据存储于其他低消耗的媒介再进行测试比对。

十七、谓词参数

选择匹配指定战利品表谓词的目标，有关谓词的内容参考《数据包》教程，语法为：

```
[predicate=<命名空间 ID>]
```

3.69

其中 `<命名空间ID>` 为指定谓词的命名空间 ID，谓词文件的路径一定要填写正确。不能使用内联形式。若需要选择不匹配指定谓词的目标，则语法为：

```
[predicate=!<命名空间 ID>]
```

3.70

同样，在目标选择器中直接指定谓词也是耗费较大算量的做法，为了优化命令的执行，应尽量改用第八章讲述的 `if predicate` 子命令。

第三章思考题与习题

根据下面的要求，分别编写目标选择器：

- 1. 所有处于创造模式的玩家；
- 2. 任意 3 个玩家；
- 3. 距离 (3,2,4) 不超过 10 格又不小于 5 格、且距离该点最近的一个玩家；
- 4. 当前实体；
- 5. 既没有记分板标签 A 又没有记分板标签 B 的所有实体；
- 6. Johnny 卫道士；
- 7. 在队伍 `frank` 中、`[ai]` 分数小于 0 的所有玩家；
- 8. 随机的一个实体；
- 9. 随机选择一个玩家，要求水平朝向范围和竖直朝向范围分别如下图所示：

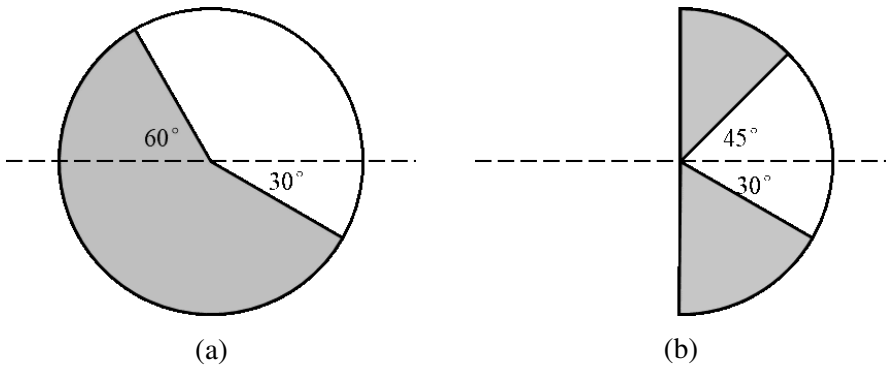


图 3.9

- 10. 由 (117, 83, -95) 和 (120, 90, -67) 决定的长方体区域内的所有标记；

如图 3.10 所示，★为命令执行者（盔甲架）且位于同心圆圆心，●均为盔甲架，同心圆每往外一层，半径就增加 4 格，最里面的圆半径为 4，据此选择第 11 ~ 13 题所需的实体。

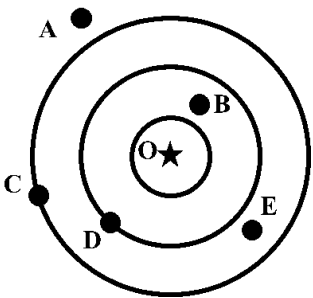


图 3.10

- 11. A、C、E；
- 12. B；
- 13. A、D。

14. 过获取黑石 `blackstone` 以取得进度  石器时代 `story/mine_stone` 的所有玩家。

第四章 NBT 格式

一个面包、一块石砖、一只绵羊……这些游戏内容本质上是许多游戏数据构成的集合，Minecraft 的游戏数据主要由这种格式存储——NBT。

4.1 概述

Minecraft 存在一种用于存储数据的格式，即**二进制命名标签 (Named Binary Tags)**，简称**NBT**，它是一种树状的数据存储格式，是由 Mojang Studio 首创的、极具 Minecraft 游戏特色的一种数据存储格式。NBT 存储的内容比较精细化，主要包含方块、实体、物品等数据。

4.1.1 SNBT 的概念

Minecraft 中大部分具体游戏资源的数据都是以 NBT 文件的格式存储在游戏文件夹中的，这些具体的内容可能是——某一个方块、某一个区块、区块内某一个实体等。这些内容使用统一的 NBT 格式存储数据，鉴于其可读性较低，且在命令中无法直接处理这样的数据。因此在命令中使用一种便于编写的 NBT 格式，即**字符串化的二进制命名标签 (Stringified NBT)**，简称**SNBT**。当命令中的标签解析成功时，SNBT 就会被转换为 NBT 以存储数据。

SNBT 的基本格式为键值对。写法为

```
<标签名>:<值>
```

4.1

举例：

```
tag:123
```

4.2

SNBT 的标签名允许包含字母 A ~ Z、a ~ z、数字 0 ~ 9、下划线 `_`、引号 `"`、冒号 `:`、空格、反斜杠 `\`、汉字等，对大小写敏感，比如，`tag` 和 `Tag` 是两个不同的标签名。

一般而言，标签名只使用英文字母、数字和下划线的组合，因为这种书写方式直接且无需添加额外的用于标识的字符。对于由多个单词组成的标签名，其命名方法一般有三种——**大驼峰命名法 (Upper camel case)**，即所有单词首字母均大写，如 `CanDestory`；**小驼峰命名法 (Lower camel case)**，即除首个单词外的所有单词首字母均大写，如 `maxUses`；**蛇形命名法 (Snake case)**，即所有字母均小写、单词之间用下划线隔开，如 `map_scale_direction`。近年来游戏新数据在标签名上一般采用蛇形命名法，且原有的数据标签名也逐步改为了蛇形命名法，为便于统一管理游戏数据，建议读者自己进行开发时，尽量使用蛇形命名法定义标签。

若标签名带有引号、冒号、空格、汉字等字符，则标签名需要被一对引号包裹，如 `"minecraft:item_model"`。可以使用单引号 `'` 或双引号 `"`。在必要的时候，标签名中的字符还需被转义。当标签名中只有双引号 `"` 时，可用单引号包裹标签名，如 `'tag'`；当标签名中只有单引号 `'` 时，可用双引号包裹标签名，如 `"tag"`；当标签名同时含有单引号和双引号时，可任意使用单引号或双引号包裹标签名，但标签名中对应种类的引号需要被转义，如 `"'Tag'"`。空格在标签名中也是可以接受的：`"a b"`，汉字同样如此：`"标签名"`。反斜杠不需要被引号包裹：`tag\`。

4.1.2 数据类型与数据树

在 SNBT 中，对于每一个诸如 `<标签名>:<值>` 这样形式的内容，称其为一个单独的**标签 (Tag)**。一个标签由三部分组成：一是**标签类型**，它用于决定该标签需要使用什么类型的数据；二是**标签**

名，它用于区分不同的标签；三是**该标签存储的数据**，对于不同类型的标签，其需要的数据也不尽相同。NBT 本身一共存在 13 种数据类型，在 SNBT 中一共可以使用 13 种标签类型，其中包含一种较为特殊的标签类型。本教程在介绍 SNBT 的语法时，采用与 Minecraft Wiki 一致的结构化树状图。下面分类介绍所有的数据类型：

一、整型类数据

1. 字节型

字节型 (Byte) 占据 1 个字节，存储容量为 -128 ~ 127，且数值必须为整数。在填写字节型数据时，可以在数值后面加一个字母 ，大小写均可，格式为：

```
<标签名>:<值>b
```

4.3

示例：

```
Difficulty:2b
```

4.4

结构化表示为

 **Difficulty:** 

这个字母  被称为数据的后缀，后缀用于决定该数据为何种类型的数据，同时也有助于将 SNBT 转换为 NBT。字母  决定了这个数据为字节型数据。不过后缀是可选的，因为 NBT 的数据类型是自动更正的。然而笔者还是强烈建议不要省略后缀以避免小概率解析不成功的情况。

2. 布尔值

NBT 本身并没有**布尔值 (Bool)** 这种数据类型，因此使用字节型数据的  来表示布尔值中的“假”，用非  的数据来表示布尔值中的“真”。但出于习惯，一般用  来表示“真”。在 SNBT 中可以直接使用  和 ，它们分别能转换为 NBT 格式的 code  和 。事实上，在 SNBT 中直接使用字节形式的  和  也是可接受的。示例：

```
NoAI:true
```

4.5

或

```
NoAI:1b
```

4.6

结构化表示为

 **NoAI:** 

3. 短整型

短整型 (Short) 占据 2 个字节，存储容量为 -32768 ~ 32767，且值必须为整数。该数据类型需要的后缀为 ，大小写均可。示例：

```
Fire:10s
```

4.7

结构化表示为

 **Fire:** 

① 本教程使用这些图标表示各 SNBT 数据类型。

4. **I** 整型

整型 (Int) 是非常常见的一种数据类型，被用于存储大量的标签数据。它占据 4 个字节，存储容量为 $-2^{31} \sim 2^{31} - 1$ （即 $-2147483648 \sim 2147483647$ ），值必须为整数。该数据类型需要的后缀为 **i**，大小写均可，也可以不写这个后缀。示例：

Age:20

4.8

结构化表示为

I Age: 20

若在填写其他数据类型时没有加后缀，则实际上填写的数据类型默认为整型，因此需要经过更正才能被识别。如果数据不在整型可用范围内，则需使用 **L** 长整型，并添加 **L** 长整型的类型后缀。

5. **L** 长整型

长整型 (Long) 可用于存储绝对值很大的数据，其占据 8 个字节，存储容量为 $-2^{63} \sim 2^{63} - 1$ （即 $-9223372036854775808 \sim 9223372036854775807$ ），值同样需要为整数。长整型数据的后缀为 **l**（注意不是字母 i 的大写），大小写均可。示例：

DeathLootTableSeed:72148557953084471

4.9

上述四种数据类型均为整数形式，可以在类型后缀处添加 **s** 或 **u** 分别代表有符号或无符号的数据，如果为无符号数据，则表示负数时需要在数值前添加 **-**。格式为

<值><符号><类型后缀>

4.10

例如，**129sb** 即为有符号的字节型数据，它等同于 **-127b**、**-127sb**，但 **129ub** 作为无符号字节型数据超出了其可用范围，故无效。**如果对整型数据指定有无符号，则整形数据的后缀 i 不能缺省。**例如 **123u** 的写法是错误的，必须要写成 **123ui**；**123s** 会被识别为短整型数据，正确的有符号整型数据应写为 **123si**。

这些数据类型还可以使用二进制和十六进制，若为二进制，则需要在值前添加 **0b** 前缀；若为十六进制，则需要在值前添加 **0x** 前缀。例如，**0b1001** 即为整型的值 **9**，**0xcad** 即为整型的值 **3245**。值前面不加任何前缀时，为十进制数据。

进制前缀还可以与类型后缀结合使用，例如 **0b1100b** 表示二进制的值 **12b**，**0xabl** 表示十六进制的长整型数据 **171l**，**0xaub** 表示无符号的十六进制字节型数据 **10b**，**0b11ss** 表示有符号的二进制短整型数据 **3s**。**如果使用十六进制字节型数据，则符号类型后缀 s 或 u 不能缺省，这是因为字节型类型后缀 b 也是有效的十六进制数字。**例如，**0xab** 会被识别为整型数据 **171**，字节型数据的写法应为 **0xasb** 或 **0xaub**。

没有符号类型后缀 **s** 或 **u** 时，默认十进制数有符号，而二进制数和十六进制数无符号。无论值有没有符号，数字的前面一定不能带有前导零，如 **1** 不能写成 **01**、**-1** 不能写成 **-01**，否则该数据就会失效，这是因为带有 **0** 前缀的数据一般表示八进制数，SNBT 规避了这种写法以防止冲突。

此外，数字之间允许有下划线 **_** 分隔，但下划线不能位于数值的开头或结尾。

二、浮点数类数据

6. **F** 单精度浮点数

单精度浮点数 (Float) 采用 IEEE754 标准, 占据 4 个字节, 长度为 32 位。它用于存储一个实型数据, 数据可以带符号, 如 `1.2`、`-0.5` 等。单精度浮点数使用字母 `f` 作为它的数据后缀。

7. **D** 双精度浮点数

双精度浮点数 (Double) 同样采用 IEEE754 标准。它占据 8 个字节, 长度为 64 位, 能用于存储精度比单精度浮点数更高的实型数据。双精度浮点数使用字母 `d` 作为它的数据后缀。

浮点数小数点后面的位数不定, 这取决于其所存储的数值的整数位数, 但是双精度浮点数存储的数据一般带有更多的小数点位数。浮点数可以写成整数形式或小数形式, 如 `2f`、`1.5f`、`10d` 等。虽然小数点不是必要的, 但还是建议为数值为整数的浮点数加上小数点, 可以精确到小数点后一位, 比如, 将 `1f` 写成 `1.0f` 的形式。不同于整型, 浮点数的整数部分可以带有前导零, `1.0f` 和 `01.0f` 均可以识别。

带小数点的浮点数分为整数和小数两个部分, 这两个部分都是可以省略的。若省略整数部分, 则默认在整数部分补上 0, 如 `.4f` 会被识别为 `0.4f`; 若省略小数部分, 则默认会在小数部分补上 0, 如 `1.f` 会被识别为 `1.0f`。

浮点数还可以用科学计数法表示。数学上科学计数法一般写作 $m \times 10^n$, 在浮点数中, 科学计数法就可以写作 `<m>e<n>`, 例如 1.2×10^{-2} 就可以写作 `1.2e-2`。其中, m 被称为尾数, 必须是带符号的浮点数, 可以为整数也可以为小数; 而 n 被称为指数, 它只能是带符号的整数。此外, 尾数和指数均必须有数值, 不能为空。例如, `4.5e4`、`.2e-3`、`4.e+2`、`03e0` (允许带有前导零、指数可以为 0) 均为正确的科学计数法表示; 而 `e9` (尾数缺失)、`-3e` (指数缺失)、`.2e-.2` (指数不能为浮点数) 均为错误的科学计数法表示。

三、字符串类数据

8. **"** 字符串

字符串 (String) 为若干任意字符按特定顺序的排列, 允许包含中文、标点符号、特殊字符等。字符串最多允许存储 32767 个字节的字符。字符串必须被一对双引号或单引号定义。

字符串存在一些特殊字符的转义序列:

(1) 单引号 `'`、双引号 `"`

若使用双引号定义字符串, 则字符串中所有的双引号都需要用反斜杠进行转义; 若使用单引号定义字符串, 则字符串中所有的单引号都需要用反斜杠进行转义。如果字符串中含有双引号, 则可以用单引号来定义字符串而不必为字符串中的引号加上反斜杠, 反之亦然。举例:

```
Name:"\" is a quotation mark"
```

4.11

```
Name:'" is a quotation mark'
```

4.12

结构化表示为

```
"Name: \" is a quotation mark"
```

出于可读性要求、社区标准及  VSCode 中  DHP 扩展要求的书写规范, 在字符串中一般需尽量规避转义行为, 因此 4.11 的写法不符合可读性要求, 应采用 4.12 的写法。若字符串中既有双引号又有单引号, 则转义无法规避, 此时正常使用反斜杠即可。

(2) 反斜杠 `\`

反斜杠 \ 直接使用反斜杠 \ 转义即可，即 \\。

(3) Unicode 字符

字符串可以接受 Unicode 字符，使用十六进制表示。由以下方式对 Unicode 字符进行转义：

- ① 如果 Unicode 码位于两位数以内，则使用 \x，如 \x3e 对应的字符为 >。
- ② 如果 Unicode 码位于四位数以内，则使用 \u，如 \u4e0a 对应的字符为 上。
- ③ 也可以使用八位数的 Unicode 码，此时转义写法为 \U，如 \U00004e0a 对应的字符为 上。
- ④ 如果一个 Unicode 字符有名称，则可以使用名称指定之，格式为 \N{名称}。如字符 ☑ 的名称为 Ballot Box with Bold Check，则可以使用 \N{Ballot Box with Bold Check} 转义之。

(4) 退格符 \b

(5) 空格 \s

(6) 水平指标符 \t

(7) 换行符 \n

(8) 换页符 \f

(9) 回车 \r

四、数组类数据

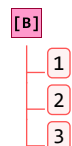
9. [B] 字节型数组

字节型数组 (Byte array) 将若干个**有序**字节型数据整合到一起。数组需要用方括号将所有数据包括起来，并在开头标上 [B] 以定义该数组为字节型数组。[B] 后面跟随若干个字节型数据，数据与数据之间用逗号 (一定为英文逗号) 隔开。数组末尾的数据后面允许添加且最多只能添加一个逗号，其他数组类数据相同。格式为：

```
[B;<字节型数据 1>,<字节型数据 2>,<字节型数据 3>,...]
```

4.13

数组内数据的顺序很重要，例如，[B;1b,2b,3b] 与 [B;3b,2b,1b] 是两个不同的标签数据。其中前者的结构化表示为



数组内的数据会假定与使用与定义一致的数据类型，如 [B;1,2,3] 会被识别为 [B;1b,2b,3b]。

10. [I] 整型数组

整型数组 (Int array) 是若干个整型数据构成的**有序列表**。其写法与上面字节型数组的写法类似，但数组开头为 [I] 以定义该数组为整型数组。格式为：

```
[I;<整型数据 1>,<整型数据 2>,<整型数据 3>,...]
```

4.14

11. [L] 长整型数组

长整型数组 (Long array) 是若干个长整型数据构成的**有序列表**。其写法与上面字节型、整型数组的写法类似，但数组开头为 [L] 以定义该数组为长整型数组。格式为：

```
[L;<长整型数据 1>,<长整型数据 2>,<长整型数据 3>,...]
```

4.15

数组可以接受可用范围比该数组的定义更小的值，例如 `[L;1b,2,3l]` 会被识别为 `[L;1l,2l,3l]`。短整型数组虽然还未使用，但允许在整型数组和长整型数组中使用短整型的数据，如 `[L;1s,2s,3s]` 会被识别为 `[L;1l,2l,3l]`。

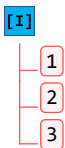
12. **L** 列表

列表 (List) 是若干个任意类型的数据构成的**有序列表**，NBT 格式的列表内的数据类型需一致，但 SNBT 可接受类型不一致的异构列表，存储为 NBT 时会将不同的数据类型转换为相同的数据类型，例如列表内同时存在字节型数据和复合标签时，会将字节型数据按列表内位置套在另一个复合标签内再进行存储；从 NBT 读取为 SNBT 时，相同的数据类型并不会逆向转换为不同的数据类型。列表的开头不需要加任何的内容以表明它是哪种类型的数组，格式为：

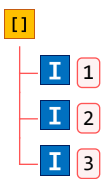
[<数据 1>,<数据 2>,<数据 3>,...]

4.16

若列表内的数据类型为字节型、整型或长整型，它们并不能视为 `[b]` 字节型数组、`[i]` 整型数组或 `[l]` 长整型数组。例如，`[I;1,2,3]` 与 `[1,2,3]` 是两个完全不同的标签数据，前者为 `[i]` 整型数组，后者为 `[L]` 列表。其中前者的结构化表示为



后者的结构化表示为



五、复合标签与数据树

13. **C** 复合标签

复合标签 (Compound) 使得标签和标签的嵌套成为可能，其基本格式为

<标签名>:{子标签}

4.17

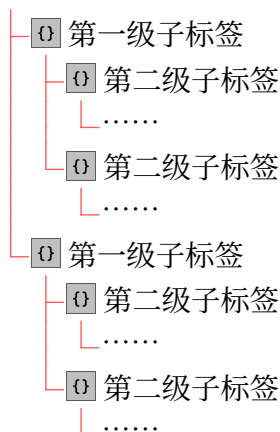
现在对这个格式进行进一步的解释：标签名和冒号为一个标签写法的组成部分，值的部分为一个花括号。一般称这一整个标签为**父标签 (Parent tag)**，花括号内的内容被称为**子标签 (Children tag)**。子标签允许存在多个不同的标签，这些子标签之间使用逗号分割，最后一个子标签后面允许添加且最多只能添加一个 `,`。每一个标签都是父标签的子标签，于是复合标签的基本格式又可以写成如下的形式：

<父标签名>:{<子标签名 1>:<值>,<子标签名 2>:<值>,...}

4.18

花括号内的子标签也可以成为下一级标签的父标签，于是又可以在子标签内嵌套子标签。将这些子标签分层命名为第一级子标签、第二级子标签……经过层层嵌套，最终可以得到如下所示的树形结构，这便是**数据树 (Data tree)** 的基本结构。

C 父标签



然而，父标签和子标签的界限并不是明确的，它们只有相对的关系：一个父标签可能是上一级标签的子标签，一个子标签也可能是下一级标签的父标签，这就好比一棵树上分叉的树枝。不过，即使一棵树的枝干分叉再复杂、树枝的层级再多，它终究是有树干和根部的，这样的道理在数据树上仍成立。对于一个特定的游戏资源，如一个具有方块实体的方块、一个实体，游戏使用一棵数据树存储它所有的信息，在这棵数据树内存在一个标签，由这个标签衍生出所有的子标签，再经过层层嵌套、衍生，最终形成一棵数据树，这棵数据树就可以用于存储数据。对于这样的标签，可以给予其形象的名称：**根标签（Root tag）**，以将其比作一棵数据树的“根”。

在了解数据树的基本结构后，现在取数据树上一个小部分（可以不包含根标签）进行分析。假设一个标签 **Me** 衍生出来的数据树如下所示：



相应地、它的 SNBT 格式为^①：

```
Me:{
  Name:"Mu_xian",
  Game:"Minecraft",
  Country:{
    Name:"中国",
    Area:"Asia",
    Language:"汉语"
  }
}
```

4.19

可以看到，标签 **Me** 一共有三个子标签，它们分别是 **Name**、 **Game** 和 **Country**，存储的数据类型分别为字符串、字符串和复合标签。可以理解为，这三个子标签分属父标签 **Me**

^① 为方便阅读，本教程部分 SNBT 会分行表示，在客户端内编写命令的时候不要使用回车键。

的三个不同的属性，在查阅这些数据时，首先需要保证每种属性所对应标签的唯一性，不能让数据出现冲突、矛盾的地方。规定：**同一级子标签内不能存在标签名相同的标签。**

但是不难发现，标签  **Country** 作为一个复合标签，其三个子标签中也有一个名为  **Name** 的标签，这是不是与上述规定有所冲突呢？答案是否定的。上述规定有一个前提——同一级子标签内，标签  **Country** 的子标签  **Name** 与  **Me** 的子标签  **Name** 不属于同一级子标签，因此标签名可以相同。**对于一个标签而言，其子标签的子标签不是它的子标签。**

标签  **Me** 的三个子标签，它们的数据类型也不尽相同。这是因为复合标签是对父标签多个不同属性的内容的描述，与同样可以存储多个数据的数组（包括列表）相比，复合标签中允许存在不同数据类型的标签；而对列表而言，即使在 SNBT 中写为异构列表，存储为 NBT 时所有数据类型必须一致。复合标签与数组的另一个区别是，复合标签内的所有子标签是不论次序的，标签的先后顺序不会影响到数据的处理或存储。而数组内数据的先后顺序很重要，数据顺序的改动会影响到数据存储的位置。

在数据树中，复合标签与数组是可以相互嵌套的，这意味着复合标签中可以有数组，而数组内的数据类型也可以是复合标签。下面给出了  **<玩家>.dat** 数据树的一部分：



它的 SNBT 格式为：

```
Player:{
  playerGameType:0,
  Inventory:[
    {count:1,Slot:0b,id:"minecraft:diamond_sword"},
    {count:45,Slot:1b,id:"minecraft:bread"}
  ]
}
```

4.20

可以看到标签  **Player** 的一个子标签  **Inventory** 为一个列表，列表中的数据类型为  复合标签，可以称它为复合标签的列表。 列表和  复合标签能相互嵌套形成非常复杂的数据树结构，但嵌套深度不能超过 512。

14. 结束（End）

这种数据类型仅用于标记复合标签的结束，无存储容量。SNBT 不使用这种数据类型。

4.1.3 SNBT 操作

特别地、SNBT 可以接受一些未经处理的数据，从而讲它们转换为 NBT 可以接受的值，这就是 SNBT 的语法糖——**SNBT 操作 (SNBT Operations)**，大小写均可，格式为：

```
<操作名>(<参数>)
```

4.21

将其写在 SNBT 的值中：

```
<标签名>:<操作名>(<参数>)
```

4.22

现在一共有两种可用的 SNBT 操作：

1. 将参数转换为  布尔值，格式为

```
bool(<arg>)
```

4.23

其中 `<arg>` 必须为布尔值或数字。若输入布尔值，则直接使用该值；若输入数字，则将非 `0` 的数据转换为 `true`，`0` 转换为 `false`；若输入的不是布尔值或数字，则转换失败。例如：

- (1) `bold:bool(true)` → `bold:true`
- (2) `NoAI:bool(0)` → `NoAI:false`
- (3) `Invulnerable:bool(5)` → `Invulnerable:true`
- (4) `italic:bool("italic")` → 转换失败

2. 将有连字符的十六进制形式的 UUID 转换为  整型数组，格式为

```
uuid(<str>)
```

4.24

其中 `<str>` 必须是有连字符的十六进制形式的 UUID。例如：

```
UUID:uuid("8890812a-c393-41e0-a9aa-4b93aa46927f")
→ UUID:[I;-2003795670,-1013759520,-1448457325,-1438215553])
```

4.1.4 SNBT 转换为 NBT *

对于一个输入的 SNBT，游戏需要将其转换为 NBT 格式以使用。在转换的时候，游戏会对输入的 SNBT 做一定处理以适应目标程序对象的数据格式。转换行为可总结为以下四点：

1. 不能被程序对象使用的直接丢弃

若属性在程序对象中不存在（未被使用）或不可写，则输入的 SNBT 会被直接丢弃。此过程不会产生错误，但也不会保留数据。“不存在”的情况有如：实体的数据有一个字段  `Invulnerable`，如果错误地把这个字段写为了  `invulnerable`，此字段实体未使用，所以输入的  `invulnerable` 会被丢弃。“不可写”的情况有如：方块实体的 `id` 不可被修改，传入的 `id` 也会被忽略。

2. 编码层级的错误会发生报错

如果一个字段本身存在，但其编码格式违反硬性约束，则会产生报错，并使命令执行失败。例如，一些字段需要使用文本组件作为其值，如果输入的文本组件有错误，则整个输入都会产生错误。以下命令会产生报错，即使传入的数据在 SNBT 的层面上是一个空复合标签：

```
give @s ladder[minecraft:custom_name={}]
```

4.25

3. 数据能转换就转换

如果输入的 SNBT 与期望的数据不符，若存在明确的转换规则，则会按照转换规则纠正输入数据：

- (1) 若期望的值是一个命名空间 ID，而输入的值是一个省略命名空间前缀的字符串，则会为其添加默认的命名空间 `minecraft`。如：
输入 `id: "stone"` → 转换为 `id: "minecraft:stone"`
- (2) 若期望的值是 `f` 布尔值，而输入的值是 `B` 字节型、`S` 短整型、`I` 整型、`L` 长整型、`F` 单精度浮点数或 `D` 双精度浮点数，则向下取整转换为字节型，非 `0b` 的值被视为 `1b`。
- (3) 若期望的值是 `B` 字节型、`S` 短整型、`I` 整型、`L` 长整型、`F` 单精度浮点数或 `D` 双精度浮点数，而输入的值与目标类型不符，则自动转换为目标类型。若目标的类型是 `B` 字节型、`S` 短整型、`I` 整型或 `L` 长整型，而输入的值是 `F` 单精度浮点数或 `D` 双精度浮点数，则会先向下取整再进行转换。

4. 不能转换的数据就归零或置空

如果输入的 SNBT 与期望的数据不符且无法转换，则：

- (1) 若期望的值是一个 `f` 布尔值，而输入的值是 `l` 列表、`B` 字节型数组、`I` 整型数组、`L` 长整型数组、`"` 字符串或 `{}` 复合标签，则强制使用 `0b`。
- (2) 若期望的值是 `B` 字节型、`S` 短整型、`I` 整型、`L` 长整型、`F` 单精度浮点数或 `D` 双精度浮点数，而输入的值是 `l` 列表、`B` 字节型数组、`I` 整型数组、`L` 长整型数组、`"` 字符串或 `{}` 复合标签，则强制赋值为 `0`，具体数据类型取决于期望的类型。
- (3) 若期望的值是 `"` 字符串，而输入的值不是字符串，则强制使用空字符串 `""`。
- (4) 若期望的值是 `l` 列表、`B` 字节型数组、`I` 整型数组或 `L` 长整型数组，而输入的值不是对应的类型，则强制使用空列表 `[]` 或空数组。
- (5) 若期望的值是 `{}` 复合标签，而输入的值不是复合标签，则强制使用空复合标签 `{}`。

4.1.5 二进制格式 *

NBT 文件大部分遵循马库斯·阿列克谢·佩尔松（Notch）的原始标准，是 GZip 格式的压缩文件，但仍有小部分未被压缩。在被压缩的 NBT 文件中，一定使用一个复合标签用于文件的封装，也就是说，这个未命名的复合标签便是根标签。

每个 NBT 文件都使用 **二进制格式**，其内容是由一个个字节组成的，可读性较低。将每个字节用十六进制表示，如：

01 00 05 63 6f 75 61 74 01

4.26

对于这样的格式，有一套特定的方法将其转换为可读性高的 SNBT。对于所有的标签，它们首先需要 一个字节存储各自的数据类型，因此每种数据类型都有一个 ID 作为它们的标识。各数据类型的 ID 以及各 ID 的二进制格式如表 4.1 所示。

表 4.1 数据类型及其二进制格式

ID	数据类型	图标	二进制格式
0	结束	-	00
1	字节型	B	01

(续表)

ID	数据类型	图标	二进制格式
2	短整型		02
3	整型		03
4	长整型		04
5	单精度浮点数		05
6	双精度浮点数		06
7	字节型数组		07
8	字符串		08
9	列表		09
10	复合标签		0a
11	整型数组		0b
12	长整型数组		0c

1. 结束

对于结束类型的数据，它只有一个字节的长度，且这个字节固定为 00，它一定会出现在复合标签的末尾。

2. 字节型、短整型、整型和长整型

对于这四种类型的数据，每一个标签都由四部分组成：第 1 个字节标识该标签的类型。第 2、3 字节标识该标签之标签名长度，必须为无符号整数，两个字节能存储的最大数值为 65535，因此一个标签的标签名最多不能超过 65535 个字符。根据第 2、3 字节定义的标签名长度，接下来若干字节用于存储该标签的标签名，名称中每个 ASCII 字符占据一个字节，第 2、3 字节定义的值有多大，则这部分的字节数量就为多少。最后若干字节是该标签的负载，负载包括了该标签的值。对于 B 字节型，此部分的字节数为 1；对于 S 短整型则为 2；对于 I 整型则为 4，对于 L 长整型则为 8。四种数据类型的负载均包含有符号的值。

例如，数据 4.26 的第 1 位为 01，这一位定义了数据类型，为 B 字节型，第 2、3 位为 00 05，它定义了标签名的长度，说明该标签名有五个字节。往后数 5 位，第 4~8 位 63 6f 75 61 74 是标签名的字符，根据 ASCII 码，63 代表 c、6f 代表 o、75 代表 u、61 代表 n、74 代表 t，因此标签名为 count。最后一位 01 是负载，定义了该标签的值 1。综上所述，该标签为

count: 1b

4.27

3. 浮点数

对于单精度浮点数和双精度浮点数，它们的字节构成与上述整型数据类似，唯一不同之处在于浮点数的负载字节遵循 IEEE 754-2008 标准。对于 F 单精度浮点数，负载字节长 4 位；对于 D 双精度浮点数，负载字节长 8 位。

例如，对于如下的数据：

05 00 06 48 65 61 6c 74 68 40 90 00 00

4.28

由 05 00 06 48 65 61 6c 74 68 知该标签类型为 **F** 单精度浮点数, 标签名为 Health, 40 90 00 00 为负载, 需将其转化为单精度浮点数。先将 40 90 00 00 转化为二进制

```
01000000 10010000 00000000 00000000
```

4.29

接下来按 1 位符号位、8 位指数位、23 位尾数位分割这个数据:

```
0 10000001 00100000000000000000000
```

4.30

最高位 0 表示该值为正数, 10000001 是指数部分, 指数可计算得 $2^8 + 2^0 - 127 = 129 - 127 = 2$, 00100000000000000000000 是尾数部分, 可计算得 $1 + 2^{-3} = 1.125$, 浮点数为 $(-1)^0 \times 2^2 \times 1.125 = 4.5$ 。因此该标签为

```
Health: 4.5f
```

4.31

4. 字符串

["] 字符串的二进制格式为: 1 位类型标识、2 位标签名长度、若干位标签名字符、2 位值长度、若干位负载。例如, 对于如下的数据:

```
08 00 02 49 64 00 0e 6d 69 6e 65 63 72 61 66 74 3a 73 74 6f 6e 65
```

4.32

08 表示该标签为 **["]** 字符串类型, 00 02 69 64 表示标签名为 id, 00 0e 表示值有 15 个字符, 后面的 6d 69 6e 65 63 72 61 66 74 3a 73 74 6f 6e 65 表示值为 minecraft:stone。故该标签为

```
id: "minecraft:stone"
```

4.33

5. 字节型数组、整型数组、长整型数组

对于这三类数组, 二进制格式为: 1 位类型标识、2 位标签名长度、若干位标签名字符、4 位有符号整数表示数组的长度。若数组的长度为 n , 则最后使用 $n \times s$ 字节表示负载, 其中 s 的值对于 **[B]** 字节型数组而言为 1, 对于 **[I]** 整型数组是 4, 对于 **[L]** 长整型数组是 8。例如:

```
11 00 04 55 55 49 44 00 00 00 04 4d 90 3b 0b 2b 4b 98 b3 0f 8e 7d 8e f2 95 15 c3
```

4.34

其中 11 表示该标签为 **[I]** 整型数组, 00 04 表示该标签的标签名有 4 个字符, 55 55 49 44 表示该标签的标签名为 UUID, 00 00 00 04 表示该数组有 4 个元素, 4d 90 3b 0b 2b 4b 98 b3 0f 8e 7d 8e f2 95 15 c3 存储了这 4 个元素的值, 由于每个值均为整型, 因此每 4 个字节为 1 个有符号整数, 换算结果为 188452941、-1281864917、-1904374257、-1021995534, 综上所述, 该标签为

```
UUID: [I; 188452941, -1281864917, -1904374257, -1021995534]
```

4.35

6. 列表

[L] 列表的二进制格式在 1 位类型标识、2 位标签名长度和若干位标签名字符后, 又使用了 1 字节用于标识列表内元素的数据类型, 其值仍按表 4.1 使用。然后是 4 位有符号整数表示数组的长度以及和长度相符的若干位负载。例如:

```
09 00 08 52 6f 74 61 74 69 6f 6e 05 00 00 00 02 42 b4 00 00 00 00 00 00
```

4.36

09 代表该标签为 **I** 列表，00 08 52 6f 74 61 74 69 6f 6e 是该标签的标签名 Rotation，05 说明该列表内元素均为单精度浮点数，00 00 00 02 是列表长度，42 b4 00 00 和 00 00 00 00 分别为列表内的元素 90、0，故标签为

Rotation: [90f, 0f]

4.37

7. 复合标签

一个 **0** 复合标签使用 1 字节标识数据类型，2 字节标识标签名长度，若干字节表示标签名。紧随其后使用若干字节表示其子标签，各子标签的格式与上文所述完全一致，但复合标签末尾一定存在一个 00 字节。例如：

0a 00 0d 62 6c 65 6e 64 69 6e 67 5f 64 61 74 61 03 00 0b 6d 61 78 5f 73 65 63 74 69 6f 6e 00 00 00 20 03 00 0b 6d 69 6e 5f 73 65 63 74 69 6f 6e ff ff ff fc 00

4.38

其中 0a 标识了 **0** 复合标签类型，00 0d 62 6c 65 6e 64 69 6e 67 5f 64 61 74 61 是标签名长度和标签名，为 13 个字符的 blending_data，03 是 blending_data 第一个子标签的数据类型，是为 **I** 整型。接下来的 00 0b 6d 61 78 5f 73 65 63 74 69 6f 6e 是第一个子标签的标签名 max_section，00 00 00 04 是这个标签的值 20。随后的 03 是 blending_data 第二个子标签的数据类型，是为 **I** 整型。第二个子标签的标签名可解读为 max_section，值为 -4。末尾的字节 00 是结束类型。故该标签为

blending_data: {max_section: 20, min_section: -4}

4.39

4.2 测试 NBT 标签

对于一段已有的 NBT 数据，有时会有需要检测它是否满足一定要求，检测方法是提供一段 SNBT 用于对比，这样的 SNBT 被称为**测试 NBT 标签 (Tseting NBT Tags)**。测试 NBT 标签主要在目标选择器的 NBT 参数、custom_data 数据组件谓词和实体谓词中使用。本节将以目标选择器 NBT 参数为主描述测试 NBT 标签的匹配方法。

一、对普通标签的匹配

满足这一类匹配要求的标签类型为除了 **0** 复合标签和 **I** 列表外的其他所有类型，**8** 字节型数组、**7** 整型数组和 **9** 长整型数组均位于此列。对于这些标签，提供的测试 NBT 标签和接受对比的目标 NBT 必须在名称、标签类型和值上完全一致。

比如，对于一个目标 NBT：

bold: true

4.40

能够与之匹配的测试 NBT 标签为 bold: true 或 bold: 1b，添加符号后缀也未尝不可，如 bold: 1ub。

如果一个目标 NBT 的值是命名空间 ID：

id: "minecraft:stone"

4.41

能够与之匹配的测试 NBT 标签必须为完整的 `id: "minecraft:stone"`，省略命名空间前缀的写法 `id: "stone"` 无法匹配。**这是因为测试 NBT 标签不会像 SNBT 转换为 NBT 那样自动添加命名空间前缀，它只会检查标签本身是否匹配，`"stone"` 和 `"minecraft:stone"` 对它而言是不一样的值。**

小提示

写入 NBT 和检查 NBT 是两个完全不同的操作。由节 4.1.4，写入 NBT 时游戏会自动添加默认的命名空间前缀；检查 NBT 的时候则不会，所以需要写成完整的命名空间 ID。

如果一个目标 NBT 的值是 `[B]` 字节型数组、`[I]` 整型数组或 `[L]` 长整型数组，则数组内容必须完全一致才能匹配。比如：

UUID: [I; 1, 2, 3, 4]

4.42

1. 能匹配的测试 NBT 标签： `UUID: [I; 1, 2, 3, 4]` ✓
2. 不能匹配的测试 NBT 标签：
- (1) 缺失元素 `UUID: [I; 1, 2, 3]` ✗

(2) 元素顺序调换 `UUID: [I; 4, 3, 2, 1]` ✗

(3) 更改数据类型 `UUID: [B; 1, 2, 3, 4]` ✗

(4) 写成 `[I]` 列表 `UUID: [1, 2, 3, 4]` ✗

二、对复合标签的匹配

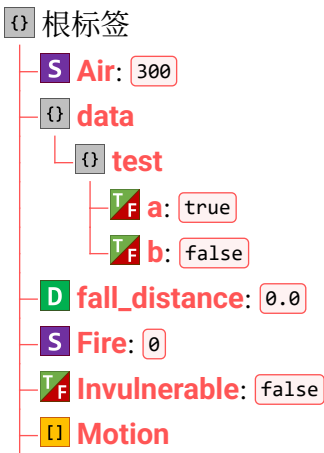
复合标签的匹配规则是：只要目标 NBT 存在测试 NBT 标签指定的标签，就匹配成功，无论复合标签内是否存在其他的标签。如果测试 NBT 标签是一个空标签 `{}`，则只要目标 NBT 是一个复合标签，就匹配成功。

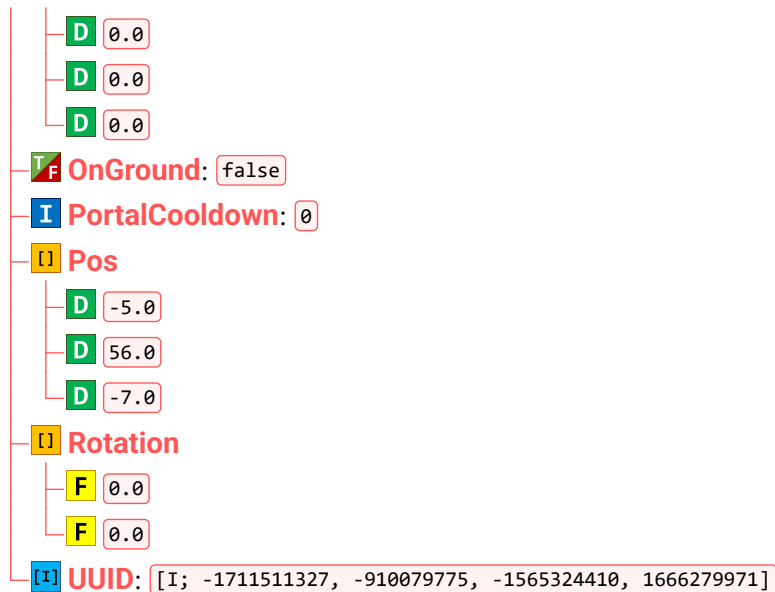
例如，一个标记拥有如下的数据：

{Motion: [0.0d, 0.0d, 0.0d], data: {test: {a: 1b, b: 0b}}, Pos: [-5.0d, 56.0d, -7.0d], Fire: 0s, Invulnerable: 0b, fall_distance: 0.0d, Air: 300s, OnGround: 0b, PortalCooldown: 0, UUID: [I; -1711511327, -910079775, -1565324410, 1666279971], Rotation: [0.0f, 0.0f]}

4.43

结构化表示为





- 能匹配的测试 NBT 标签（以下全部写成目标选择器）有：
 - 空复合标签，因为根标签也是一个复合标签 `@e[nbt={}]` ✓
 - 任意匹配的子标签 `@e[nbt={Air:300s}]` ✓
 - 任意匹配的子标签的子标签 `@e[nbt={data:{test:{a:true}}}]` ✓
 - 如果子标签为复合标签，空复合标签也可以匹配 `@e[nbt={data:{}}]` ✓
 - 对于目标选择器，也可以用反选 `@e[nbt=!{Air:100s}]` ✓
- 不能匹配的情况：
 - 子标签的值不匹配 `@e[nbt={Air:100s}]` ✗
 - 子标签是数组，但数组不匹配 `@e[nbt={UUID:[I;-1711511327]}]` ✗
 - 不存在的字段 `@e[nbt={SelectedItem:{}}]` ✗

三、对列表的匹配

列表的匹配规则是：只要目标列表中存在测试 NBT 标签指定的元素，就匹配成功，且列表匹配不考虑元素顺序。但是**空列表只能匹配空列表，无法匹配有元素的列表**。

依旧以数据 4.43 为例，其中有一个标签 `Pos: [-5.0d, 56.0d, -7.0d]`。

- 匹配的目标选择器有：
 - 完全一致 `@e[nbt={Pos: [-5.0d, 56.0d, -7.0d]}]` ✓
 - 只匹配部分元素 `@e[nbt={Pos: [-5.0d]}]` ✓
 - 调换元素顺序 `@e[nbt={Pos: [-7.0d, 56.0d, -5.0d]}]` ✓
 - 调换元素顺序并省略部分元素 `@e[nbt={Pos: [-7.0d, -5.0d]}]` ✓
- 不能匹配的情况：
 - 空列表 `@e[nbt={Pos: []}]` ✗
 - 不存在的元素 `@e[nbt={Pos: [80.0d]}]` ✗

4.3 NBT 路径

有时，为了访问所要寻找的 NBT 数据，需要在数据树上确定该数据的地址，即形成该数据的路径。这个概念为 **NBT 路径 (NBT path)**，它用于通过特定的有序遍历路径指向指定的标签。基本思路为：在数据树中，从根标签起通过层层标签最终索引得到指定的标签。数据树中每一层级的标签为一个**节点 (Node)**。NBT 路径的基本语法为：

<节点>.<节点>.....<节点>

4.44

例如，路径

a.b.c

4.45

就指向标签 **b** 的子标签 **c**，其中标签 **b** 又为 **a** 的子标签。其中，**a**、**b**、**c** 为三个不同的节点，分别位于三级不同的子标签，由节点拼凑得到最终的地址。

4.3.1 节点

由于标签有不同的数据类型，相应地，节点也有几种不同的类型。具体划分后，节点一共有六种基本类型；按照指向的内容分类，则一共可以将节点分为两大类。

一、指向标签的节点类型

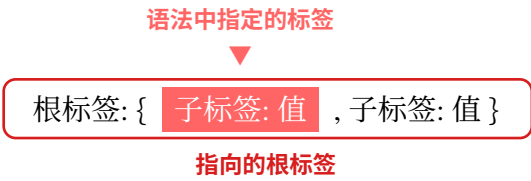
这类节点指向**一个完整的标签**。在命令 `/data get` 中使用这些节点时，返回的是标签的值。

1. 根复合标签 (Root compound tag)

语法：

{标签}

4.46



标签处应为一个测试 NBT 标签，它的写法在节 4.2 中已讲过，需要匹配的标签必须为根标签的子标签，也可以为空。若标签处为空或指定的数据与标签的实际数据匹配时，则指向根标签；若不匹配则不指向任何标签。例如，标签处为一只绵羊的某一个标签 `Brain`，这个标签为绵羊标签的子标签，则指向的是绵羊这个根标签。

用法示例：

(1) 该节点指向该复合标签本身：

{}

4.47

(2) 该节点会先判断根标签的子标签  `main` 的值是否为 `true`，若是则指向根标签；若否则不指向任何内容：

`{main:true}`

4.48

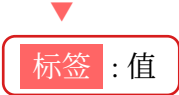
2. 某名称的标签 (Named tag)

语法：

标签名

4.49

语法中指定的标签名



指向的标签

这种节点会直接指向拥有该标签名的标签，无论该标签的数据类型。如果标签名中带有单引号 `'`、双引号 `"` 和空格，则这个节点需要被引号包裹，带有汉字和反斜杠则不需要。

用法举例：若有一个根标签的子标签名为 `a`，则下面的节点指向该标签：

`a`

4.50

3. 某名称的复合标签 (Named compound tag)

语法：

标签名{子标签}

4.51

语法中指定的标签



指向的标签

用法与根复合标签类似，`子标签` 是一个测试 NBT 标签，可选填，指向不必为根标签的标签，且指向的标签必须为复合标签。注意，`标签名` 和 `{子标签}` 之间不能添加冒号。用法举例：

(1) 已知有一个标签 `main:{a:true,b:false}`，则：

① 该节点指向  `main` 标签：

`main{}`

4.52

② 该节点会先判断子标签  `a` 的值是否为 `true`，显然结果为是，因此也指向标签  `main`：

`main{a:true}`

4.53

(2) 已知有一个标签 `NoAI:true`，则该节点无法指向任何标签，因为  `NoAI` 不是复合标签：

`NoAI{}`

4.54

二、指向列表或数组中元素的节点类型

这类节点指向一个列表或数组中的一个或多个元素。由于一个列表或数组中的元素实际上是上一级标签的值，则这类节点实际指向标签的值而不是一个完整的标签。

1. 当前列表或数组中的某个元素（Element of named list or array tag）

语法：

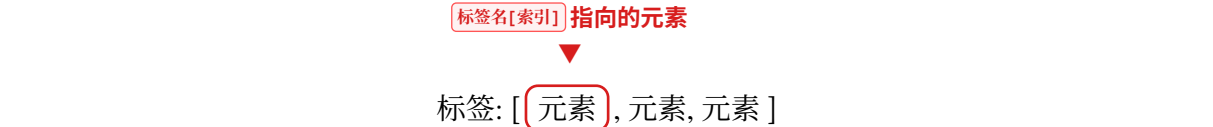
标签名[索引]

4.55

或

标签名.[索引]

4.56



用于指向数组类标签中某个元素，索引 指名为 标签名 的列表或数组中的第几个元素，需要是一个整数值。

设索引的值为 i ，若 i 为非负数，则指向该列表或数组的第 $i + 1$ 个元素：所以 0 指向列表或数组的第一个元素、1 指向第二个元素，以此类推。若 i 为负数，假设列表或数组的长度（即其包含的元素个数）为 n ，则指向该列表或数组的第 $i + n$ 个元素：所以 -1 表示该列表或数组的倒数第一个元素、-2 表示倒数第二个元素，以此类推。

表 4.2 长度为 n 的列表及数组的索引值

元素的位数	1	2	3	...	$n - 1$	n
非负索引值	0	1	2	...	$n - 2$	$n - 1$
负数索引值	$-n$	$1 - n$	$2 - n$	...	-2	-1

索引指向的地址不能超出列表或数组的长度，否则节点就不指向任何标签。

用法举例：假设有一个标签 [B] a:[B;1,0,0,1]，则

- (1) a[0] 指向第一个元素，值为 1b；
- (2) a[3] 指向第四个元素，值为 1b；
- (3) a[-2] 指向倒数第二个（即第三个）元素，值为 0b；
- (4) a[4] 指向的地址超出了数组的长度，故无效。

2. 当前列表或数组中的所有元素（All elements of named list or array tag）

语法：

标签名[]

4.57

或

标签名.[]

4.58



指向具有该标签名的列表或数组中的所有元素，此标签必须为数组类标签。如果该列表或数组有多个元素，则指向的是多个元素。

用法举例：假设有一个字节型数组 a:[B;1,0,0,1]，则节点 a[] 指向 1b、0b、0b、1b 这四个元素。

3. 当前列表或数组中的复合标签元素 (Compound elements of named list tag)

语法:

标签名[{{标签}}]

4.59

或

标签名.{{标签}}

4.60

标签名[{{标签}}] 指向的元素

标签: [{{ 标签: 值 }}, { 标签: 值 }]

{{标签}} 处为选填内容, 使用测试 NBT 标签, 可以为空。为空时, 则会指向具有该标签名的列表或数组中所有为 {{}} 的复合标签; 不为空时, 会先检查标签的数据是否匹配, 若匹配则指向具有该标签名的列表或数组中所有与之匹配的复合标签, 若不匹配则不指向任何内容。

注意, 具有该标签名的标签必须为一个复合标签的列表, 也就是列表中的元素必须为复合标签, 否则不指向任何内容。

用法举例:

(1) 假设有一个复合标签列表 a:[{{b:1}},{{c:2}}], 则:

① a[{}] 不指向任何元素。

② a[{{b:1}}] 与列表中的 {{b:1}} 匹配, 故指向 {{b:1}} 这个复合标签。

③ a[{{b:2}}] 与列表中的元素都不匹配, 故不指向任何内容。

(2) 假设有一个字节型数组 a:[B;1,0,0,1], 则 a[{}] 不指向任何内容, 因为标签 [B] a 不是复合标签的列表。

特别地、当列表内的元素均为子列表时, 依旧可以使用这种节点的变形, 形如

标签名[索引或为空][索引或为空]...[索引或为一个标签]

4.61

或

标签名.[索引或为空].[索引或为空]....[索引或为一个标签]

4.62

其中不同层级列表之间的点 . 可以任意省略。如此该节点指向列表的子列表的所有元素, 可以嵌套多层, 由外向内分别为第 1 层、第 2 层……设指向元素的列表位于 n 层, 则节点语法中的方括号就有 n 个。

除最后一个方括号中的内容外, 中间这些方括号中可以为一个索引值, 指向规则与命名列表或数组标签的元素相同, 若为空则指向所有元素。最后一个方括号中的内容可以为一个索引值或标签, 其中标签的用法与命名列表标签的复合元素相同, 不作赘述。

用法举例:

(1) a[0][0] 指向列表 a 的第一个子列表的第一个元素。

(2) a[0][{{b:1}}] 指向列表 a 的第一个子列表的元素 {{b:1}}, 当且仅当该子列表中存在复合标签 {{b:1}}。

小提示

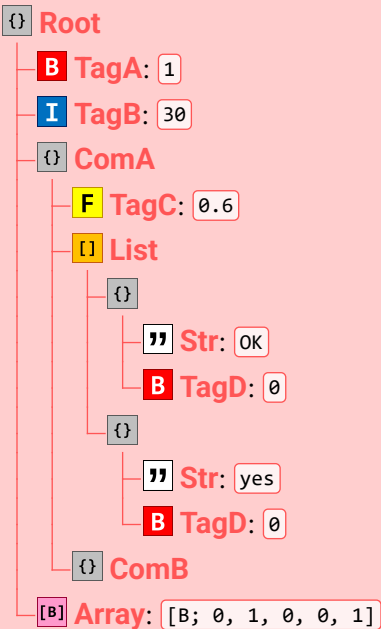
总结: 在基本节点类型中, 花括号中的内容一般为限定条件(检查数据是否匹配), 方括号中的内容一般为数组的索引。

4.3.2 路径

路径是由若干个节点组成的，在编写路径时，一定要注意**从根标签开始编写**。

例 4.1

有一棵数据树如下所示，其中 `{}` `Root` 为根标签。



- (1) 写出标签 `{}` `Root` 的值。
- (2) 分别表示下列地址：
 - ① `[B]` `Array` 的第四个元素；
 - ② `{}` `ComB`；
 - ③ `[I]` `List` 中的所有元素；
 - ④ 所有名为 `'` `Str` 的标签；
 - ⑤ 名为 `Str` 且值为 `yes` 的标签。

解

- (1) 根标签 `{}` `Root` 为一个复合标签，里面有 `B` `TagA`、`I` `TagB`、`{}` `ComA` 和 `[B]` `Array` 四个子标签，其中 `{}` `ComA` 又为一个复合标签，含有二级子标签 `F` `TagC`、`[I]` `List` 和 `{}` `ComB`。
`[I]` `List` 是一个复合标签的列表，内部含有两个复合标签。因此 `{}` `Root` 标签的值如下所示。

```
{
  TagA:1b,
  TagB:30,
  ComA:{
    TagC:0.6f,
    List:[
      {

```

4.63

```

        Str:"OK",
        TagD:0b
    },
    {
        Str:"yes",
        TagD:1b
    }
],
ComB:{}
},
Array:[B;0,1,0,0,1]
}

```

- (2) ① **[B]** **Array** 是根标签 **{}** **Root** 的子标签，对于根标签的子标签，其首先占据一个节点。其次，**[B]** **Array** 是一个长度为 5 的字节型数组，对于其第四个元素，可以写成 **Array[3]**，也可以认为它是倒数第二个元素，因此也可以写成 **Array[-2]**。
- ② **{}** **ComB** 是一个复合标签，它是 **{}** **ComA** 的子标签，但不是根标签的子标签，**{}** **ComA** 才是根标签的子标签，因此 **{}** **ComA** 占据一个节点，**{}** **ComB** 占据第二个节点。这里有两种基本的写法：一种是 **ComA.ComB**；另一种是 **ComA{}.ComB**。后者的第一个节点指向 **{}** **ComA** 这个标签，由于 **{}** **ComA** 是个复合标签，因此命名标签和命名复合标签两种基本节点类型都可以使用。当然，给花括号中添加限定条件也是可以的，本题中，**ComA{ComB:{}}.ComB** 路径也是正确的，同样正确的写法还有 **ComA{TagC:0.6f}.ComB**。
- ③ **[]** **List** 为一个列表，要指向列表中的所有元素，则使用命名列表或数组标签的所有元素这种节点类型，于是该节点写为 **List[]**，而 **[]** **List** 是 **{}** **ComA** 的子标签，因此路径为 **ComA.List[]**。
- ④ **[]** **Str** 是复合标签列表 **[]** **List** 中的子标签，从数据树上看，它似乎比 **[]** **List** 低了两个等级。实际上它是包含它本身的复合列表的子标签，而列表中的所有复合标签都是同等级的，列表的存在不增加嵌套等级，列表只负责将这些复合标签统整到一起。因此这个路径只需要三个节点，即 **ComA.List[].Str**。
- ⑤ 名为 **Str** 且值为 **yes** 的标签存在于复合标签中，该复合标签是列表 **[]** **List** 的第二个元素。因此对于第二个节点，可以使用命名列表或数组标签的元素类型，将节点写作 **List[1]**；或者使用命名列表标签的复合元素类型，这时候添加限定条件以指向列表的第二个元素，如 **List[{Str:"yes"}]**。所以最终路径为 **ComA.List[1].Str** 或 **ComA.List[{Str:"yes"}].Str**。

4.4 命令/data 的语法

一些 NBT 数据是可以通过命令修改的。在进行修改之前，需要先了解 NBT 路径以明确需要修改的数据所处的位置，这便是上一节所讲述的内容。能够对 NBT 数据进行操作的命令有若干

条, 其中命令 `/data` 是其中应用范围最广的、操作性最强的, 也是最基础的一条命令, 本节将主要介绍命令 `/data` 的用法。

命令 `/data` 只能用于获取或修改**方块实体**、**实体**和**命令存储**的 NBT 数据, 但是**无法修改玩家的任何数据**。`/data` 一共有四条子命令, 它们分别是 `get`、`merge`、`modify` 和 `remove`, 这四条子命令组成了最基本的 `/data` 语法结构。`/data` 所需的权限等级为 2。

4.4.1 `get` 子命令

`/data get` 用于获取方块实体、实体或命令存储的 NBT 数据, 语法为:

```
data get (block <targetPos>|entity <target>|storage <target>) [<path>]
[<scale>]
```

4.64

其中的参数: `block <targetPos>` ——需要修改数据的方块实体, 首先需要明确的就是该方块实体所在方块的方块坐标, 其中 `<targetPos>` 的参数类型为方块坐标 `minecraft:block_pos`。

`entity <target>` ——一共有三种选择实体的方法: 即目标选择器、玩家名称或 UUID, 只能指定一个实体。`<target>` 的参数类型为实体 `minecraft:entity`。

`storage <target>` ——以命令存储的命名空间 ID 指定需要修改的命令存储内容, `<target>` 的参数类型为命名空间 ID `minecraft:resource_location`。

`[<path>]` (NBT 路径 `minecraft:nbt_path`) ——可选, 需要获取的数据的 NBT 路径, 若不指定则使用根标签。

`[<scale>]` (双精度浮点数 `brigadier:double`) ——可选, 将返回的值进行缩放的倍率。缩放操作为: 先将 NBT 值向下取整, 再乘以指定的缩放倍率。

若路径指向的是一个完整的标签, 则返回的该标签的值; 若路径指向的是列表或数组中的元素, 则返回指向的元素。如不符合下列要求, 则 `/data get` 命令将无法执行:

1. 获取的 NBT 路径必须存在。
2. 返回的标签或数值必须少于两个, 注意, 不是指复合标签或数组中的标签必须少于两个, 一个复合标签或一个数组被视为上一级标签的值。因此必须谨慎使用诸如命名列表或数组标签的所有元素这样的节点, 因为这类节点通常指向的标签或数值大于一个。假设有一个字节型数组 `a:[B;1,0,0,1]`, 由于路径 `a[]` 指向 `1b`、`0b`、`0b`、`1b` 这四个数值, 因此不能在 `/data get` 中设置路径 `a[]`, 但是路径 `a` 指向整个标签 `[B] a`, 则返回的内容为标签 `a` 的值, 即 `[B;1,0,0,1]`。
3. 若指定了倍率参数, 则返回的内容必须为一个数值。

例 4.2

对于例 4.1 所示的数据树, 假设该数据树是位于坐标 (0, 56, 0) 的方块的方块实体数据。写出命令 `/data get block 0 56 0 ComA.List[1].TagD 2` 返回的内容。



路径 `ComA.List[1].TagD` 指向标签 `ComA` 的子标签 `List` 中第二个复合标签内名为 `TagD` 的标签，即指向标签 `TagD:1b`，于是返回该标签的值，即 `1b`。而命令的结尾又添加了一个倍率参数 `2`，即将 `1b` 乘以 2，因此返回的内容为数值 `2`。

4.4.2 remove 子命令

`/data remove` 用于移除指定方块实体、实体或命令存储的 NBT 数据，语法为：

```
data remove (block <targetPos>|entity <target>|storage <target>) <path>
```

4.65

其中的参数：`<targetPos>`、`<target>` ——与语法 4.64 一致。

`<path>` (NBT 路径 `minecraft:nbt_path`) ——必填，需要移除的数据的 NBT 路径，不能指向根标签。

4.4.3 merge 子命令

`/data merge` 使方块实体、实体或命令存储的 NBT 数据与指定 NBT 发生合并，语法为：

```
data merge (block <targetPos>|entity <target>|storage <target>) <nbt>
```

4.66

其中的参数：`<nbt>` (NBT 复合标签 `minecraft:nbt_compound_tag`) ——需要指定的复合标签。与原本的根标签值进行合并，若合并后的值与原本的值相同则命令执行失败。

在填写 `<nbt>` 时，需要用花括号将标签括起以表示它是根标签的值。一般来说根标签为复合标签，其子标签不止一个。现在令 `A`、`B`、`C` 为一个根标签的子标签，则该根标签的值为：

```
{A,B,C}
```

4.67

现在使用命令

```
data merge ... {B'}
```

4.68

其中标签 `B'` 与 `B` 标签名相同，值不同。子命令 `merge` 使 `{B'}` 与 `{A,B,C}` 发生合并，覆盖掉原本的标签 `B`，而标签 `A` 和 `C` 不发生改变，所以修改后的根标签值为

```
{A,B',C}
```

4.69

4.4.4 modify 子命令

`/data modify` 用于在原本方块实体、实体或命令存储的 NBT 数据基础上以标签为单位进行修改，修改后的值必须与原本的值不同，部分语法为：

```
data modify (block <targetPos>|entity <target>|storage <target>)
<targetPath> (append|insert <index>|merge|pretend|set) ...
```

4.70

其中的参数：`<targetPath>`（NBT 路径 `minecraft:nbt_path`）——需要修改的标签，需要是合法的 NBT 路径。

`(append|insert <index>|merge|pretend|set)`——规定 NBT 的修改方式，有五种不同的方式。若使用 `insert`，则需要指定一个 `<index>`（整型 `brigadier:integer`）值。

下表为这五种修改方式的作用形式：

表 4.3 `/data` 子命令 `modify` 的可用修改方式

修改方式	作用标签类型	效果	示例
<code>append</code>	列表或数组	在列表或数组的末尾插入一个元素。	将一个值 <code>E</code> 以 <code>append</code> 的形式插入列表 <code>[A,B,C,D]</code> ，则结果为 <code>[A,B,C,D,E]</code> 。
<code>insert <索引></code>	列表或数组	在列表或数组的指定位置插入一个元素。设索引值为 i ，则元素会被插入到列表或数组的第 i 个位置。索引值可以为负以表示倒数第 i 个位置。列表或数组中原先第 i 个位置及之后的元素均向后移一位。	对于列表 <code>[A,B,C,D]</code> ，若以 <code>insert 1</code> 的语法将 <code>E</code> 插入其中，则结果为 <code>[A,E,B,C,D]</code> 。
<code>merge</code>	复合标签	将语法中的 NBT 与指定的复合标签进行合并。与 <code>merge</code> 子命令不同的是， <code>merge</code> 子命令作用于整个根标签，这里的 <code>merge</code> 修改方式作用于根标签下的子复合标签或多级子复合标签。	-
<code>pretend</code>	列表或数组	在列表或数组的首位插入一个元素。	将一个值 <code>E</code> 以 <code>pretend</code> 的形式插入列表 <code>[A,B,C,D]</code> ，则结果为 <code>[E,A,B,C,D]</code> 。
<code>set</code>	任意类型的标签	将指定的标签替换为新的值。在该修改方式中，NBT 路径不能指向根标签。	将标签 <code>A:1b</code> 的值以 <code>set</code> 的方式替换为 <code>0b</code> ，则结果为 <code>A:0b</code> 。

命令 `/data modify` 还需要确定值的来源。来源可以是其他标签的值，也可以自己指定的一个值。

1. 若来源为其他标签的值，则语法 4.70 省略号部分的语法为：

```
from (block <sourcePos>|entity <source>|storage <source>) [<sourcePath>]
```

4.71

其中的参数：`<sourcePos>`（方块坐标 `minecraft:block_pos`）——数据来源方块实体的方块坐标。

`<source>`（`entity` 模式，实体 `minecraft:entity`）——数据来源实体，可以为玩家名称、UUID 或目标选择器，但只能指定一个实体。

- <source> (storage 模式, 命名空间 ID minecraft:resource_location —数据来源命令存储, 需要是该命令存储的命名空间 ID。
- [<sourcePath>] (NBT 路径 minecraft:nbt_path) —可选, 来源数据的 NBT 路径, 若不指定则为根标签。
2. 若来源的值需要为其他方块实体、实体或命令存储中的字符串, 则可以使用下面的语法, 字符串可以切片处理:

string (block <sourcePos>|entity <source>|storage <source>) [<sourcePath>]
[<start>] [<end>]

4.72

其中的参数: [<start>] (整型 brigadier:integer) —索引值, 开始截取的字符位置。从 0 开始计, 可以使用负数索引值。

[<end>] (整型 brigadier:integer) —索引值, 终止截取的字符位置。从 0 开始计, 可以使用负数索引值。终止索引位置的字符不会被截取。若 [<start>] 值为 n , [<end>] 值为 m , 则截取第 n 个至第 $m - 1$ 个字符会被截取 (包括第 n 个和第 $m - 1$ 个字符)。

字符串切片

	字符	...	字符	字符	字符	...	字符	字符	...
字符的位数	1	...	n	$n + 1$	$n + 2$	...	m	$m + 1$	...
字符的索引值	0	...	$n - 1$	n	$n + 1$	...	$m - 1$	m	...

例 4.3

命令存储 custom:main 中 `id:"minecraft:iron_ingot"` 的值是一个命名空间 ID, 试去除它的命名空间前缀、只保留 ID, 将结果存入 `name` 标签。

解 显然需要使用字符串切片, 需要去除的部分为 `minecraft:`, 其后的 `i` 索引值为 `10`, 从此处一直截取到最后一个字符, 命令为

data modify storage custom:main name set string storage custom:main id
10 -1

4.73

3. 若来源的值由玩家自己指定, 则语法为:

value <value>

4.74

其中的参数: <value> (NBT 值 minecraft:nbt_tag) —需要为 SNBT 的格式值, 必须符合需要修改的标签所需的数据类型。

例 4.4

在例 4.2 所述的情景中, 编写一条命令使之在列表 `List` 第 1 和第 2 个复合标签之间插入一个新的复合标签, 这个新的复合标签与列表 `List` 第 1 个复合标签相同。

解

首先需要写出需要修改的标签的路径，即指向标签 `List`，由此写出：

```
data modify block 0 56 0 ComA.List
```

4.75

对列表型标签进行修改时，考虑在 `modify` 子命令中使用 `append`、`insert` 或 `prepend` 这三种修改方式。题目要求在第 1 和第 2 个复合标签之间插入一个新的复合标签，因此使用 `insert`。根据题意，新的复合标签会成为列表的第 2 个元素，因此可以写出：

```
data modify block 0 56 0 ComA.List insert 2
```

4.76

最后要指明标签的值来源为列表 `List` 的第 1 个复合标签。列表 `List` 的第 1 个复合标签属于位于 (0, 56, 0) 的方块实体的数据树。其路径为 `ComA.List[0]`。于是可以写出整条命令：

```
data modify block 0 56 0 ComA.List insert 2 from block 0 56 0 ComA.List[0]
```

4.77

4.4.5 命令/data 的语法树

命令 `/data` 语法成分复杂，嵌套层级较多，为此笔者制作了如图 4.1 所示的完整语法树作为参考。其中虚线表示的部分为可选内容。

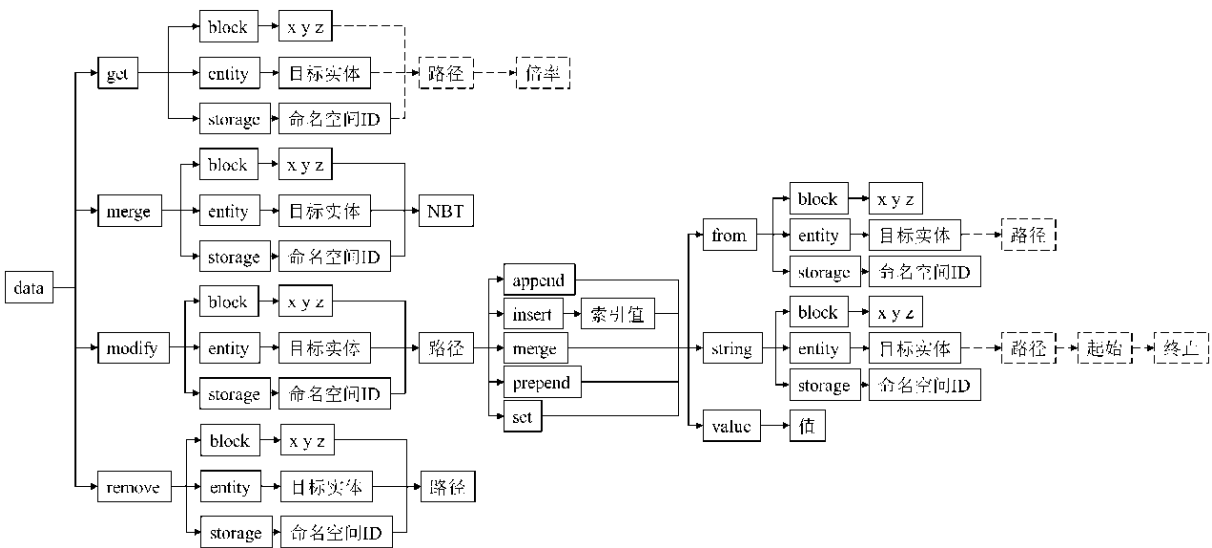


图 4.1 命令 `/data` 的语法树

4.5 NBT 与 JSON

4.5.1 NBT 和 JSON 格式的转换

NBT 和 JSON 格式结构类似，但还有很多不同之处，在一些情况下，游戏必须对这两种格式进行相互转换以满足计算的需要，即使这两种格式的转换可能会造成数据丢失。

1. NBT 转换为 JSON *

表 4.4 NBT 转换为 JSON

NBT 数据类型	转换后的 JSON 数据类型
B S I L F D	N 数值
"	" 字符串
{	{ 对象
[B I L F D	[数组

2. JSON 转换为 NBT

表 4.5 JSON 转换为 NBT

JSON 数据类型	转换后的 NBT 数据类型
T 布尔值	B
N 数值	若位于字节型的取值范围内，则转换为 B ； 否则，若位于短整型的取值范围内，则转换为 S ； 否则，若位于整型的取值范围内，则转换为 I ； 否则，若位于长整型的取值范围内，则转换为 L ； 否则，若其能精确存储为一个单精度浮点数，则转换为 F ； 若不为上述任意一者，则转换为 D 。
" 字符串	"
[数组	若 JSON 数组内元素的数据类型不同，则无法转换为 NBT； 若 JSON 数组内元素被转换为 B ，则将数组转换为 [B] ； 若 JSON 数组内元素被转换为 I ，则将数组转换为 [I] ； 若 JSON 数组内元素被转换为 L ，则将数组转换为 [L] ； 若不为上述任意一者，则转换为 [U]
{ 对象	{
Null	不转换

有时候，开发过程中可能需要开发者手动转换格式。例如，数据包的战利品表、谓词和物品修饰器一般使用 JSON 格式定义，对于一些未在数据包中定义的战利品表、谓词和物品修饰器，可以用内联的方式在直接命令中定义，这时需要将战利品表、谓词和物品修饰器以 SNBT 的形式写在命令中。以下是内联定义的一个例子：

例 4.5

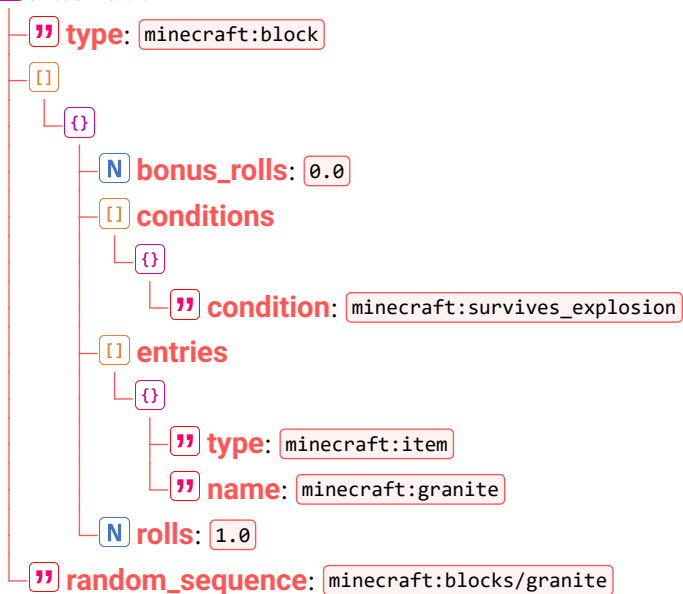
将以下的战利品表 `.json` 文件以内联的形式写入命令。

```
data > minecraft > loot_table > blocks > granite.json
1  {
2    "type": "minecraft:block",
3    "pools": [
4      {
5        "bonus_rolls": 0.0,
6        "conditions": [
7          {
8            "condition": "minecraft:survives_explosion"
9          }
10       ],
11       "entries": [
12         {
13           "type": "minecraft:item",
14           "name": "minecraft:granite"
15         }
16       ],
17       "rolls": 1.0
18     }
19   ],
20   "random_sequence": "minecraft:blocks/granite"
21 }
```

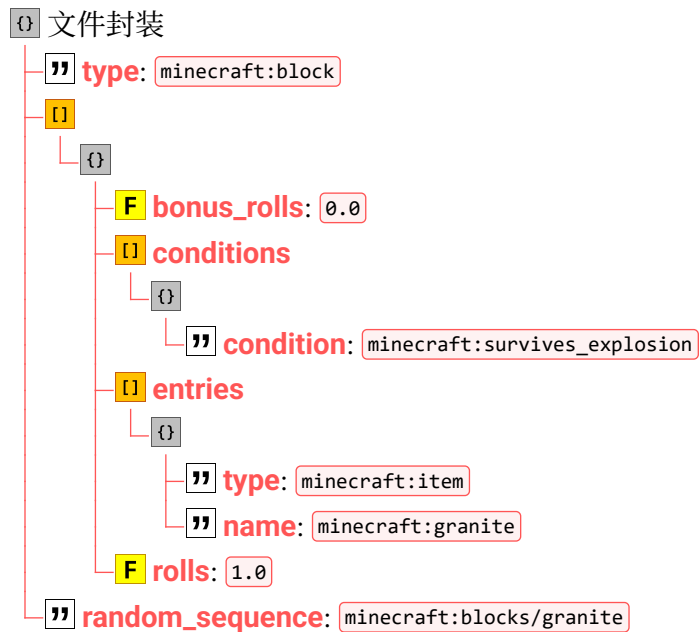
解

先以树状图列出该文件的 JSON 数据：

文件封装



内联形式，即将 JSON 格式手动转换为 SNBT 格式，此时参考表 4.5 的转换方式，可得到相应的 SNBT 树形图：



因此内联形式为

```

{
  type: "minecraft:block",
  pools: [
    {
      bonus_rolls: 0.0f,
      conditions: [
        {
          condition: "minecraft:survives_explosion"
        }
      ],
      entries: [
        {
          type: "minecraft:item",
          name: "minecraft:granite"
        }
      ],
      rolls: 1.0f
    }
  ],
  random_sequence: "minecraft:blocks/granite"
}

```

4.78

4.5.2 SNBT 和 JSON 的嵌套

一、在 SNBT 中使用的 JSON

部分情况下，SNBT 值需要为一段完整的 JSON，典型例子就是在 25w02a 之前的版本中在 SNBT 中使用的文本组件。SNBT 与 JSON 本身不兼容，在 SNBT 中使用 JSON 时，一般值类型

为字符串。根据字符串数据类型的定义方法，双引号和单引号均可用于定义字符串。若字符串中有单引号或双引号中的某一种引号，且用于定义字符串的引号与字符串中存在的引号类型相同，则必须为字符串中的引号添加转义字符。而大部分的 JSON 都是存在双引号的，根据社区规范，需尽量减少转义字符的使用，**定义 SNBT 值类型时一般使用单引号**，对于以下的 JSON 片段：

```
{
  "text": "Hello World!",
  "color": "red"
}
```

它作为 SNBT 的值时，写法可以为

```
Name: '{"text": "Hello World!", "color": "red"}'
```

4.79

使用转义字符的写法虽然符合语法，但不符合社区规范：

```
Name: '{"text\\":\\"Hello World!\\",\\"color\\":\\"red\\"}'
```

4.80

在 SNBT 值内部的 JSON 中所有需要被转义的字符前都需要添加反斜杠，如 `\` 和 SNBT 字符串定义使用的引号，无论这些字符在原本的 JSON 中是否起到实际作用。遇到嵌套层级较深的情况，可以**由内向外书写，一层层添加反斜杠**，并适当规避转义。例如，对于以下的 JSON 片段：

```
{
  "text": "\"Hello World!\"",
  "color": "red"
}
```

在 JSON 中它的写法为

```
{"text": "\"Hello World!\"", "color": "red"}
```

4.81

当这个 JSON 片段的值作为 SNBT 的值时，应在每个需要被转义的字符前添加反斜杠。但由于其中只有双引号，因此可以用单引号定义字符串以规避反斜杠：

```
Name: '{"text": "\"Hello World!\"", "color": "red"}'
```

4.82

例 4.6

将以下的 JSON 片段写为 SNBT 字段 `raw` 的值：

```
{"text": "A\nB", "bold": true}
```

4.83

解

JSON 中只有双引号，故用单引号定义 SNBT 字符串，并在 `\` 前加反斜杠，如：

```
raw: '{"text": "A\\nB", "bold": true}'
```

4.84

例 4.7

将以下的 JSON 片段完整地写为 SNBT 字段 `Name` 的值。

```
{}
```



解

先将树状图转换为 JSON:

```
[
  {
    "text": "\\Hello World!\\\"
  },
  {
    "text": "\\\"Hello World!\\\"\\\"\"
  }
]
```

4.85

注意到其中只有双引号，所以用单引号定义 SNBT 字符串，然后在所有的 `"` 前加反斜杠，因此 `{ "text": "\\Hello World!\\\"` 为

```
Name: '[{"text": "\\\"Hello World!\\\"\\\""}, {"text": "\\\"Hello World!\\\"\\\""}]'
```

4.86

二、在 JSON 中使用的 SNBT

一些 JSON 需要的值为 SNBT，这些 JSON 的数据类型基本上都是字符串，拥有定义字符串的双引号 `"`。这些 SNBT 中出现的**每一个** `"`、`\` 均需要被转义。

例 4.8

对于由例 4.7 得到的 SNBT 字段，在其外侧套一层复合标签后写入 JSON 字段 `{ "nbt": ... }`。

解

只需将代码 4.86 中每一个 `"` 和 `\` 前添加一个 `\` 即可，结果为

```
"nbt": "{Name: '[{\"text\": \"\\\"Hello World!\\\"\\\"\"}, {\"text\": \"\\\"Hello World!\\\"\\\"\"}]'}"
```

4.87

第四章思考题与习题

1. 判断下列 NBT 数据值的写法是否正确。

- (1) `13us`
- (2) `128b`
- (3) `0.e+03`

(4) `.2d`

(5) `[I;000000001,000000001,000000001,000000001]`

2. 根据如下所示的 SNBT，回答下列问题：

```
Player:{
  Inventory:[
    {count:1,Slot:0b,id:"minecraft:diamond_sword"},
    {count:45,Slot:1b,id:"minecraft:bread"}
  ]
}
```

4.88

(1) 标签 `count:1` 是否是标签 `Player` 的子标签？为什么？

(2) 指出标签 `Inventory` 的数据类型。

3. 解释下列名词：节点、根复合标签、某名称的复合标签、索引、NBT 路径。

4. 假设一个根标签 `orange_banner` 的值如下所示：

```
{x:5,y:56,z:4,id:"minecraft:banner",patterns:
 [{pattern:"diagonal_left",color:"magenta"}]}
```

4.89

(1) 画出标签 `orange_banner` 的树状图；

(2) 写出下列路径指向的内容：

① `x`;

② `patterns[0]`;

③ `id`;

④ `patterns[0][0]`

5. 有一个数据的树状图如下所示，将它写成 SNBT 的形式。



6. 根据第 5. 题的树状图，试写出指向下列内容的路径：

(1) 整个根标签；

(2) `Records` 的第二个元素；

(3) `pos`。

7. 对于例 4.1 所示的数据树, 假设该数据树是位于坐标 (0,56,0) 的方块实体的数据。使用命令以进行如下的操作:
- (1) 获取标签 **I** TagB 的数据;
 - (2) 删除 **L** List 的第二个元素;
 - (3) 将标签 **F** TagC 的值改为 1.2f;
 - (4) 将列表 **L** List 第一个复合标签的值复制到标签 **ComB**;
 - (5) 在数组 **B** Array 的第 2 个和第 3 个元素之间插入一个新的元素, 要求来源为列表 **L** List 第一个复合标签中标签 **B** TagD 的值。
8. 编写命令使例 4.1 所示的整棵数据树写入命令存储 test:root。
9. 将第一章思考题与习题第 16. 题的 JSON 改写为内联 SNBT 形式。
10. 将第一章思考题与习题第 17. 题得到的各 JSON 字段分别完整地写作 SNBT 字段 **json** 的值。
11. 将代码 4.82 所示的 SNBT 作为一个字符串类型的 JSON 字段 **nbt** 需要的值, 写出对应的 JSON 字段。

第五章 文本组件

Minecraft 中有各式各样的文本，它们有不同的内容、不同的样式，有些文本甚至能够与玩家产生交互。本章将介绍这些文本使用的格式——文本组件。

5.1 概述

文本组件 (Text component)，旧称**原始 JSON 文本 (Raw JSON text)**，是用于向玩家发送、显示富文本的一种格式。文本组件可以使用 SNBT 和 JSON 两种格式书写：在命令及 SNBT 中使用的文本组件应写为 SNBT 格式，在 `.json` 文件中使用的文本组件应写为 JSON 格式。

在 25w02a 以前的版本中，文本组件统一使用严格的 JSON 格式。对于一些接受文本组件作为值的 SNBT 字段，文本组件必须作为整个字符串 `" "` 写入 SNBT 字段，在 SNBT 内嵌套 JSON 的写法在小节 4.5.2 中已有说明，这样的嵌套需要注意引号的配对和转义字符的使用，在书写和维护的过程中很容易有疏忽。自 25w02a 起，文本组件可以从相应字段的节点开始直接存储为数据树的一部分，不再需要考虑两种格式的兼容性问题，此举更利于文本组件的编辑。

文本组件一共由以下三部分组成：

1. 文本组件类型 (Text component content)

定义文本类型、来源和解析规则，内容可以为空。

2. 文本组件样式 (Text component style)

定义文本的样式、字体、交互事件等信息，若不定义，则文本使用其载体的默认样式，不同载体的默认样式不同。交互事件默认不存在。

3. 子组件 (Text component children)

定义依附于当前文本的文本组件，子组件会继承父组件的样式。

文本组件可以接受的  SNBT 数据类型为 `" "` **字符串**、`[]` **列表**和 `{ }` **复合标签**，对应的 `{ }` JSON 数据类型则为 `" "` **字符串**、`[]` **数组**和 `{ }` **对象**，**其他的数据类型一概不接受**。

当文本组件使用 `" "` 字符串形式时，该组件被当作纯文本处理，不能添加任何样式。例如，组件 5.1 是一个 `" "` 字符串形式的文本组件，输入后返回的文本为 `Hello World!`（字符串的引号不会被返回）：

```
"Hello World!"
```

5.1

当使用 `[]` 列表/`[]` 数组时，其中的元素可以是任意形式的文本组件，包括 `" "` 字符串、`[]` 列表/`[]` 数组和 `{ }` 复合标签/`{ }` 对象，同一 `[]` 列表/`[]` 数组内元素的类型可以不同。例如，组件 5.2 返回的内容是 `[{"text": "A"}, {"text": "B"}, {"text": "C"}]`：

```
["A", "B", "C"]
```

5.2

`[]` 列表/`[]` 数组内的任一元素不能为文本组件不接受的数据形式。例如，组件 5.3 无法被解析，因为 `[]` 列表/`[]` 数组内的所有元素均为 `I` 整型/`N` 数值：

```
[1, 2, 3]
```

5.3

组件 5.4 也无法被解析，因为 `[]` 列表/`[]` 数组有元素为 `I` 整型/`N` 数值，即使数组内已有能被解析的 `" "` 字符串类型的数据：

[1,"2",3]

5.4

当使用  复合标签/ 对象时，允许在其中指定文本组件类型和文本组件样式，数据格式为^①：

-  文本组件
- 文本组件类型

— 文本组件样式

其中文本组件类型是必填项目，文本组件样式可选。如果不需要使用特别的组件类型和定义特别的样式，为了节省命令长度，一般使用字符串形式的文本组件而不使用  复合标签/ 对象。在游戏解析过程中，**无样式和子组件的纯文本内容会被序列化成  字符串形式的文本组件**。例如，输入的文本组件为：

{text:"Hello World!"}

5.5

再次提取这个数据，就会发现它已被序列化成：

"Hello World!"

5.6

使用文本组件编写的文本在样式上会很丰富。使用文本组件的物件主要有命令  `/tellraw`、命令  `/title`、告示牌、成书、文本展示实体、对话框等，由于告示牌、成书、文本展示实体是存档格式的一部分，因此将在后面第六章中详细介绍。下面先介绍两条命令的语法。

1. 命令 `/tellraw`

该命令用于在聊天栏中对指定的目标显示文本，所需的参数等级为 2，语法为：

tellraw <targets> <messages>

5.7

其中的参数： `<targets>`（实体  `minecraft:entity`）——指定文本接受者，可以为玩家名称、UUID 或目标选择器，但必须指定玩家。

 `<messages>`（文本组件  `minecraft:component`）——要显示的富文本，使用文本组件，可以在其中使用换行符  `\n`。

例 5.1

向所有玩家的聊天栏显示文本  `Hello World!`。

 解 命令为

tellraw @a "Hello World!"

5.8

2. 命令 `/title`

该命令用于在玩家的屏幕上显示一行大号的字体，共有三个位置可共显示。标题文本的大小取决于页面大小设置，过长的标题**不会**自动换行，只会溢出屏幕。它需要的权限等级为 2，以下是所有用法：

^① 这里同时提供了 SNBT 和 JSON 的格式，后续教程皆如此。

(1) 对指定的玩家在特定的位置文本，语法为：

```
title <targets> (title|subtitle|actionbar) <title>
```

5.9

其中的参数：(title|subtitle|actionbar)——显示位置，分别为 title（主标题（Screen title））、subtitle（副标题（Subtitle））和 actionbar（动作栏（Action bar）），这三个显示位置如图 5.1 所示。其中副标题只有当主标题存在时才会显示。动作栏是显示玩家一些操作的位置，如使用音乐唱片、睡觉、骑乘实体等，这些操作信息会覆盖用命令 /title 设置的文本。

<title>（文本组件 minecraft:component）——要显示的富文本，使用文本组件，**不可以其中使用换行符 \n**。



图 5.1 各标题显示的位置

(2) 对指定的玩家设置主标题和副标题渐入、保持和渐出的时间，语法为：

```
title <targets> times <fadeIn> <stay> <fadeOut>
```

5.10

其中的参数：<fadeIn>、<stay>、<fadeOut>（时间 minecraft:time）——分别为标题的渐入、保持和渐出的时间，均使用带单位的时间参数，不写单位则默认为游戏刻，其默认时长分别为 10gt、70gt 和 20gt。注意，**该命令不能用于设置动作栏的渐入、保持和渐出的时间，动作栏默认在其显示后 3 秒左右淡出。**

(3) 对指定的玩家，将其标题的渐入、保持、渐出时间参数恢复为默认值，语法为：

```
title <targets> reset
```

5.11

(4) 对指定的玩家移除其正在显示的所有标题，语法为：

title <targets> clear

5.12

例 5.2

对所有玩家显示主标题 `Level 0`，副标题 `前厅`。

解

命令为

title @a title "Level 0"

5.13

title @a subtitle "前厅"

5.14

例 5.3

对所有玩家而言，标题渐入、保持和渐出的时间已被修改，若需要将其改回默认值，则需要的命令为何？

解

由于不存在直接重置标题显示时间的命令，因此需要使用语法 5.10 手动修改，命令可以为

title @a times 10 70 20

5.15

5.2 文本组件类型

文本组件一共有七种可用的组件类型：**纯文本**、**翻译文本**、**记分板分数**、**实体名称**、**按键绑定**、**NBT 值**和**精灵图**，这部分数据的结构为

- ❏

❏

文本组件
- ❏❏ type: 可选，标识此对象使用的组件类型，它使用组件类型的 ID 作为它需要的值，因此有效值有 `text`（纯文本）、`translatable`（翻译文本）、`score`（记分板分数）、`selector`（实体名称）、`keybind`（按键绑定）和 `nbt`（NBT 值）。

对于指定的 `❏❏ type` 使用的相应额外字段。

对于一个指定的 `❏❏ type`，同一个 `❏` 复合标签/`❏` 对象内必须存在相应额外字段，如表 5.1 所示。

表 5.1 组件类型

优先级	❏❏ type 的值	相应的额外字段	组件类型
1	<code>text</code>	<code>❏❏ text</code>	纯文本
2	<code>translatable</code>	<code>❏❏ translate</code>	翻译文本

(续表)

优先级	type 的值	相应的额外字段	组件类型
3	keybind	keybind	按键绑定
4	score	score	记分板分数
5	selector	selector	实体名称
6	nbt	nbt	NBT 值
7	object	object	精灵图

例如，若指定

```
type:"text"
```

5.16

这说明组件标识了纯文本，那么在同一个 复合标签/ 对象内就需要使用字段 text 以说明纯文本的具体内容，如：

```
{type:"text",text:"Alpha"}
```

5.17

然而，字段 type 在文本组件中并不是必须的，使用 type 只是为了提高组件解析、检查错误的速度。如果 type 不指定值、或是 type 的值不属于上述有效值中的任何一者、亦或是 type 标识的组件类型在同一个 复合标签/ 对象内不存在相应的额外字段，则组件类型会依次检查对象内的如下键：text、translate、score、selector、keybind、nbt，最终使用第一个检查得到的有效键作为该组件使用的组件类型。

例如，对于以下的文本组件：

```
{text:"A"}
```

5.18

它没有使用 type 而是直接指定了 text，那么这个组件识别的组件类型为纯文本。而对于下面的文本组件：

```
{type:"score",translate:"addServer.add"}
```

5.19

指定的 type 为 score，而使用的键却为 translate，此时识别的结果为翻译文本而不是记分板分数。

如果 type 不指定值且 复合标签/ 对象内存在多种组件类型，则按照表 5.1 所示的优先级，选择表中最靠上的一种组件类型。组件类型所属键出现在 复合标签/ 对象中的先后顺序不会影响组件类型的优先级。例如，下面的文本组件

```
{
  keybind:"key.left",
  score:{objective:"A",name:""},
  selector:"@p"
}
```

5.20

出现了三种组件类型：按键、记分板分数、实体名称，根据表 5.1，记分板分数的优先级最高，故返回记分板分数。此外，如果一个 `§` 复合标签/ `§` 对象中出现了多个同种类型的文本时，则会优先使用 `§` 复合标签/ `§` 对象中最靠后的文本类型字段。例如，

```
{text:"1",text:"2",text:"3"}
```

5.21

返回的文本为 `3`

5.2.1 纯文本组件

显而易见，**纯文本组件 (Plain text)** 用于直接输出一段固定的文本。格式为：

- `§` `§` 文本组件
 - `§§` `type`: `text`
 - `§§` `text`: 具体的文本内容。

例 5.4

在所有玩家的主标题显示纯文本 `Hello World!`。

解 如果使用 `§§` `text`，则命令为

```
title @a title {type:"text",text:"Hello world!"}
```

5.22

然而，节 5.1 指出，对于命令中没有任何样式的纯文本 `§` 复合标签/ `§` 对象，游戏总是会将它序列化成 `§§` 字符串，即

```
title @a title "Hello world!"
```

5.23

这样的写法也是有效的，并且因为它输入的字符少，通常更推荐如 5.23 这样的写法。

纯文本内允许包含各种转义序列，如 Unicode、换行符 `\n`。例如下面的命令：

```
tellraw @a {text:"\u2605\nCiallo~"}
```

5.24

它会在聊天栏中返回  `Ciallo~`。

如果还需在其中使用引号和转义字符，则参考节 1.3.2 和 4.1.2 说明的转义方式。

5.2.2 翻译文本组件

翻译文本组件 (Translated text, 或译本地化文本组件) 通过翻译标识符返回一段已翻译的文本，文本显示的语言为客户端当前的语言。这种组件类型的作用在于，可以让一段文本转换为不同玩家所需要的语言。例如，翻译标识符 `commands.forceLoad.added.failure` 的简体中文 `zh_cn` 显示为“没有被标记为强制加载的区块”，而在使用美式英语 `en_us` 的客户端中，这段文本就被显示为“No chunks were marked for force loading”。

翻译标识符（Translation identifier，本地化键名）存在于资源包语言文件中，用于表示一段已翻译的文本，一个翻译标识符大致按照以下的格式书写：

<内容>.<内容>.<内容>....

5.25

它们被存储在 `assets\minecraft\lang\xx_xx.json` 中，每一个翻译标识符后面都有一个值，这些值便是在当前语言中的翻译文本。在 `en_us.json` 中，部分翻译标识符如下所示：

```
assets > minecraft > lang > en_us.json
1 {
2   "commands.clone.success": "Successfully cloned %s blocks",
3   "commands.debug.started": "Started tick profiling"
4 }
```

简体中文语言文件 `zh_cn.json` 相应的部分如下所示：

```
assets > minecraft > lang > zh_cn.json
1 {
2   "commands.clone.success": "已成功复制%s个方块",
3   "commands.debug.started": "已开始刻分析"
4 }
```

如果自制的资源包需要支持多语言，应该在资源包 `lang` 路径下添加语言文件，所有命名空间均可用，不仅限于 `minecraft`。一般而言至少应支持简体中文和美式英语：

```
assets
├── <命名空间>
│   └── lang
│       ├── en_us.json
│       └── zh_cn.json
```

翻译标识符及其翻译的内容可完全自定义。翻译内容允许各种转义序列，如 Unicode 码、换行符 `\n` 等。

在翻译文本文本组件中，决定组件类型的键为 `type` `translate`，数据格式如下：

`type` 文本组件

`type`: `translatable`

`translate`: 翻译标识符。若输入正确，则会返回和客户端当前语言一致的已翻译文本，即返回在相应语言 `.json` 文件中该翻译标识符的值。也可以不使用资源包，直接填入允许带译文变量的一段文本，此时文本内的译文变量有效。

`fallback`: 若相应的语言文件中没有需要的翻译标识符，则显示 `en_us.json` 中的内容，即美式英语的文本。若在 `en_us.json` 中也没有找到该翻译标识符，则可以使用这个字段以输出替代的文本，它需要的值为字符串，因此不能添加格式，但可以使用格式化代码。

`with`: 一些翻译标识符的值中含有 `%s` 的字样，这些被称为标识符中的**译文变量（英文原文为 Slots，槽位）**。若不定义此字段，则这些译文变量的位置会显示为无文本。可以使用这个字段对这些译文遍历进行自定义。

一个文本组件，允许使用字符串、数组、对象等组件。

对于翻译标识符值中的译文变量，参数列表 `with` 给它们分配参数的规则如下：

- 对于出现的第 n 个 `%s`，其会被分配参数列表中的第 n 个参数。
- 对于 `%n$s`，其会被分配参数列表中的第 n 个参数。

下面的例子解释了这种分配规则：

例 5.5

已知翻译标识符 `translation.test.complex` 对应字段在 `zh_cn.json` 为

"translation.test.complex": "前缀, %s%2\$s 然后是 %s 和 %1\$s 最后是 %s 还有 %1\$s! "

5.26

写出文本组件

{translate:"translation.test.complex",with:["1","2",
{text:"3",bold:true},"4"]}

5.27

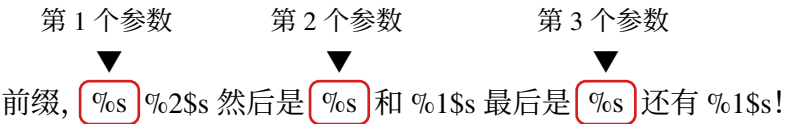
返回的文本。

解 参数列表中一共有 4 个参数，它们分别是：

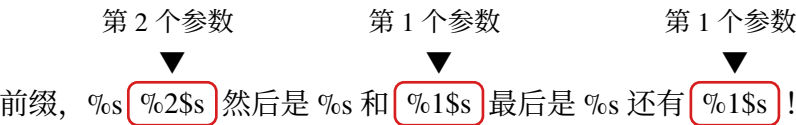
表 5.2 参数列表

第 1 个参数	第 2 个参数	第 3 个参数	第 4 个参数
1	2	3	4

对于所有的 `%s`，给它们按顺序分别分配参数列表中的参数：



对于 `%n$s`，给它们强制分配参数列表中第 n 个参数：



于是返回的内容为 前缀, 12 然后是 2 和 1 最后是 3 还有 1!。第 4 个参数虽然存在于列表中，但并没有使用。

若 `fallback` 也没有指定值，则返回翻译标识符本身。例如，`custom.1` 是原版资源包中任何一个语言文件中都没有的翻译标识符，那么在不定义 `fallback` 时，文本组件

{translate:"custom.1"}

5.28

返回的内容为 `custom.1`。读者可以通过在资源包中添加这些自定义的翻译标识符，并为这些翻译标识符编写相应的值。

例 5.6

一张冒险地图需要配合资源包使用, 玩家进入游戏时地图会向玩家展示主标题, 若玩家启用了正确的资源包, 则主标题文本为 `资源包已启用`; 若玩家未使用资源包, 则主标题文本为 `请安装正确的资源包!`。试实现这个效果。

解

要使客户端在装载资源包前后显示不同的文本, 不妨使用翻译文本组件。当资源包启用的时候, 按照翻译标识符显示对应的文本; 当资源包未启用的时候, 翻译标识符不存在, 从而显示 `fallback` 中的内容。

首先在资源包中定义一个 `zh_cn.json` 语言文件, 在其中任意定义一个翻译标识符:

```
assets > tutorial > lang > zh_cn.json
1 {
2   "resourcePack.loaded": "资源包已启用"
3 }
```

其次在文本组件中写:

```
{translate:"resourcePack.loaded",fallback:"请安装正确的资源包!"}
```

5.29

接下来只需要在主标题中显示:

```
title @a title {translate:"resourcePack.loaded",fallback:"请安装正确的
资源包!"}
```

5.30

例 5.7

显示文本 `<F>胜利, <B>失败! <F>获得<x>金币, <B>获得<y>金币`, 其中 `<A>` 的值为 `kyifyuy`, `<B>` 的值为 `planet00shaper`, `<x>` 的值为 `78`, `<y>` 的值为 `13`。

解

不妨可以用翻译文本组件的译文变量将参数传入至文本, 此处不用资源包, 直接将带译文变量的文本填入 `translate`:

```
{translate:"%1$s 胜利, %2$s 失败! %1$s 获得 %3$s 金币, %2$s 获得 %4$s 金
币",with:["kyifyuy","planet00shaper","78","13"]}
```

5.31

因为带有重复的参数, 因此译文变量写成了 `%n$s` 的形式。

不过, 在实际的应用情境中, 本题的所有变量都会是动态的, `<A>`、`<B>` 很有可能需要实体名称组件, `<x>`、`<y>` 很有可能需要记分板分数组件。这些组件类型的写法见后文的说明。

5.2.3 按键绑定组件

按键绑定组件 (Keybind) 用于返回可设置键位的动作当前设置的键位, 返回的内容会随着玩家的键位设置而改变, 数据结构为

文本组件

type: keybind

keybind: 需要返回的键位，值为**键位标识符（Keybind identifier）**。所有可用的键位标识符如表 5.3 所示：

表 5.3 可用键位标识符

键位标识符	控制	默认键位
key.advancements	进度	L
key.attack	攻击/摧毁	鼠标左键
key.back	向后移动	S
key.chat	打开聊天栏	T
key.command	输入命令	/
key.debug.clearChat	清除聊天信息	F3 + H
key.debug.copyLocation	按维度复制当前坐标	F3 + C
key.debug.copyRecreateCommand	复制指向的方块数据	F3 + I
key.debug.crash	按住 10 秒强制触发崩溃	F3 + C
key.debug.debugOptions	调试选项	F3 + F6
key.debug.dumpDynamicTextures	保存动态纹理	F3 + S
key.debug.dumpVersion	显示版本信息	F3 + V
key.debug.focusPause	控制失去焦点时游戏暂停	F3 + P
key.debug.fpsCharts	FPS 图表	F3 + 2
key.debug.modifier	调试功能组合用键	F3
key.debug.networkCharts	网络图表	F3 + 3
key.debug.overlay	显示调试屏幕	F3
key.debug.profiling	开始/停止性能分析	F3 + L
key.debug.profilingChart	分析图表	F3 + 1
key.debug.reloadChunk	重新加载区块	F3 + A
key.debug.reloadResourcePacks	重新加载资源包	F3 + T
key.debug.showAdvancedTooltips	显示物品高级提示框	F3 + H
key.debug.showChunkBorders	显示区块边界	F3 + G
key.debug.showHitboxes	显示判定箱	F3 + B
key.debug.spectate	在上一个游戏模式和旁观模式之间切换	F3 + N
key.debug.switchGameMode	使用游戏模式切换器	F3 + F4
key.drop	丢弃所选物品	Q
key.forward	向前移动	W
key.fullscreen	全屏显示切换	F11
key.hotbar.1	快捷栏 1 ~ 9	1
key.hotbar.2		2

(续表)

键位标识符	控制	默认键位
key.hotbar.3		3
key.hotbar.4		4
key.hotbar.5		5
key.hotbar.6		6
key.hotbar.7		7
key.hotbar.8		8
key.hotbar.9		9
key.inventory	开启/关闭物品栏	E
key.jump	跳跃	Space
key.left	向左移动	A
key.loadToolbarActivator	创造模式加载物品工具栏	X
key.pickItem	选取方块	鼠标中键
key.playerlist	玩家列表	Tab
key.quickActions	对话框快捷操作	G
key.right	向右移动	D
key.saveToolbarActivator	创造模式保存物品工具栏	C
key.screenshot	截图	F2
key.smoothCamera	切换电影视角	无
key.sneak	潜行	左 Shift
key.socialInteractions	打开社交屏幕	P
key.spectatorHotbar	旁观模式呼出快捷栏	鼠标中键
key.spectatorOutlines	旁观模式高亮玩家	无
key.sprint	疾跑	左 Ctrl
key.swapOffhand	交换手中的物品	F
key.toggleGui	隐藏/显示 HUD	F1
key.togglePerspective	切换视角	F5
key.toggleSpectatorShaderEffects	切换旁观者着色器效果	F4
key.use	使用物品/放置方块	鼠标右键

例 5.8

试编写一段翻译文本组件，大意为按使用键（默认为 鼠标右键）使用物品。

解

首先配置语言文件，此处配置了简体中文和美式英文：

```
assets > tutorial > lang > zh_cn.json
1 {
2   "item.use": "按下以%s使用"
3 }
```

```
assets > tutorial > lang > en_us.json
```

```
1 {
2   "item.use": "Press %s to use"
3 }
```

然后在翻译文本组件的 `with` 中定义传入的参数，显然需要传入按键绑定组件：

```
{keybind:"key.use"}
```

5.32

组合起来得到完整的文本组件：

```
{translate:"item.use",with:[{keybind:"key.use"}]}
```

5.33

5.2.4 记分板分数组件

记分板分数组件 (Scoreboard value) 这种组件类型用于返回指定分数持有者在指定记分项上的分数。若该分数持有者在指定记分项上没有分数，则不会返回任何内容。这种组件类型需要被解析。数据格式为：

文本组件

type: `score`

score: 需要显示的记分板分数。

name: 可以是一个实体名称，也可以是一个目标选择器。但是一个记分板分数文本组件只能解析一个分数持有者在一个指定记分项上的分数，因此目标选择器必须将选择的目标数量限定为一个，仅使用 `@a`、`@e` 这样的目标选择器变量是不可接受的。除了实体名称和目标选择器外，`name` 还可以接受 `*` 作为它的值。若值为 `*`，则会返回观察者（这里指观察这段文本的实体）自己在指定变量上的分数，这样可以使不同的玩家分别观察到他们自己的分数。

objective: 指定的记分项。

例 5.9

已知玩家 `Mu_xian` 在记分项 `[test]` 上的分数为 1，`IIIIIfrit` 在记分项 `[test]` 上的分数为 2。编写命令在这些玩家各自的客户端内返回各自的分数。

解

命令为

```
tellraw @a {score:{name:"*",objective:"test"}}
```

5.34

在 `Mu_xian` 的客户端中返回的文本为 `1`，在 `IIIIIfrit` 的客户端中返回的文本为 `2`。

5.2.5 实体名称组件

实体名称组件 (Entity names) 用于返回所有被选中实体的名称，需要被解析，可返回多个实体。数据格式为：

  文本组件

type: `selector`

selector: 返回的实体，需要的值为目标选择器。

separator: 可选，表示显示时分割各实体名称的文本，值可以是文本组件可接受的任意数据类型。默认值为 `{"text": ",", "color": "gray"}`，显示的是灰色的分隔符 。

例 5.10

在聊天栏中显示所有玩家的名字，名字之间用  分隔。

解

命令为

```
tellraw @a {selector:"@a",separator:"|"} 5.35
```

5.2.6 NBT 组件

NBT 组件 (NBT values) 这种组件类型用于返回**方块实体**、**实体**或**命令存储**的指定 NBT 值，需要被解析，数据格式为

  文本组件

type: `nbt`

source: NBT 值的来源，可用值 `block`（方块实体）、`entity`（实体）和 `storage`（命令存储）。若该字段不存在，则会根据 `nbt` 自动确定来源。

block: 获取方块实体数据的方块坐标，可以为绝对坐标、相对坐标或局部坐标，坐标值之间用空格分隔。

entity: 获取实体数据的实体，需要为目标选择器，可以选择多个实体。

storage: 获取命令存储数据的命令存储的命名空间 ID。

nbt: 所返回 NBT 值的路径。

interpret: 是否将从 NBT 获取的值当作文本组件解析，若解析失败则什么内容都不会返回。默认值为 `false`

plain: 默认情况下返回的 SNBT 会有语法高亮，此字段为 `true` 时只会以纯文本的形式显示返回内容。默认值为 `false`。不能与 `interpret` 同时为 `true`。

separator: 可选，表示显示时分割各 NBT 值的文本，值可以是文本组件可接受的任意数据类型。默认值为 `" , "`，显示的是白色的分隔符 。

在一个同级文本组件  复合标签/  对象中，NBT 数据值组件类型只能从   `block`、
 `entity` 和   `storage` 中选择一个，且这三个字段必须存在任何一者。若在同一个对象中出现多个，则它们按照如下顺序的优先级决定最终选择的键：  `block`、  `entity`、  `storage`。

例 5.11

获取方块坐标为 (0,0,0) 之方块实体  `Items` 标签的值。

 文本组件为

```
{source:"block",nbt:"Items",block:"0 0 0"} 5.36
```

例 5.12

返回所有实体  `Rotation` 标签的值。

 文本组件为

```
{source:"entity",nbt:"Rotation",entity:"@e"} 5.37
```

例 5.13

返回存储 `test:a` 中  `test` 标签的值。

 文本组件为

```
{source:"storage",nbt:"test",storage:"test:a"} 5.38
```

5.2.7 精灵图组件

精灵图组件 (Object) 用于显示当前客户端的精灵图，可显示纹理图集精灵图和玩家皮肤精灵图。显示的精灵图会被转换为字体基准的 8 × 8 像素大小。

一、纹理图集精灵图

格式为

-   文本组件
 -   `type`: `object`
 -   `object`: `atlas`
 -   `atlas`: 使用的纹理图集，默认为 `blocks`。
 -   `sprite`: 精灵图在纹理图集命名空间 ID。

纹理图集的概念将在《资源包》教程中给出，以下直接以例题说明此组件的用法。

例 5.14

用文本组件显示苹果的精灵图 。

解

苹果是物品，使用 `items` 这个纹理图集，相应片段如下：

assets > minecraft > atlases > items.json

```
1 {
2   "sources": [
3     {
4       "type": "minecraft:directory",
5       "prefix": "item/",
6       "source": "item"
7     }
8   ]
9 }
```

资源包内苹果纹理的地址为  `assets\minecraft\textures\item\apple.png`，它在纹理图集内的命名空间 ID 为 `minecraft:apple`。故文本组件应写为

```
{object:"atlas",atlas:"items",sprite:"minecraft:apple"}
```

5.39

例 5.15

尝试显示 。

解

 是 GUI 图集所属的精灵图，此纹理在资源包内的位置为

 `assets\minecraft\textures\gui\sprites\hud\heart\full.png`

GUI 纹理图集的内容如下所示：

assets > minecraft > atlases > gui.json

```
1 {
2   "sources": [
3     {
4       "type": "minecraft:directory",
5       "prefix": "",
6       "source": "gui/sprites"
7     },
8     {
9       "type": "minecraft:directory",
10      "prefix": "mob_effect/",
11      "source": "mob_effect"
12    }
13  ]
14 }
```

故这个精灵图在纹理图集内的命名空间 ID 为 `minecraft:hud/heart/full`，完整的文本组件为

```
{object:"atlas",atlas:"gui",sprite:"minecraft:hud/heart/full"}
```

5.40

二、玩家皮肤精灵图

实际上，这种精灵图显示的是玩家的头而非完整的玩家模型或玩家皮肤，格式为

 文本组件

type: `object`

object: `player`

hat: 是否渲染皮肤的帽子，默认为 `true`。

player: 要显示的玩家皮肤。有 `字符串` 和 `复合标签/对象` 两种格式。

当使用 `字符串` 形式时，需要为玩家名称，格式要求与 `复合标签/对象` 形式中的 `name` 一致。

当使用 `复合标签/对象` 形式时，具有以下字段：

id: 玩家的 UUID。

name: 玩家名称，不能超过 16 个字符。若此项不使用，则按 `id` 字段确定玩家。

properties: 玩家游戏档案。

若使用 `列表/数组` 形式，则可用带签名的游戏档案，并具有以下字段：

一项游戏档案属性。

name: 该属性的名称。

value: 该属性的值，是 Base64 编码的 JSON 数据。

Signature: 该属性的签名。

若使用 `复合标签/对象` 形式，则具有以下字段：

<游戏档案属性名称>: 一个游戏档案属性。

该属性的值，是 Base64 编码的 JSON 数据。

以下字段均可选，若填写了则会在上述玩家档案数据的基础上进行修改，用于定制玩家皮肤。其中的纹理均可用资源包指定，对客户端有效。

cape: 披风的纹理，使用命名空间 ID，地址从 `assets\<命名空间>\textures` 开始计。

elytra: 鞘翅的纹理，使用命名空间 ID，地址从 `assets\<命名空间>\textures` 开始计。

model: 玩家模型的类型，有效值 `wide`（普通）和 `slim`（纤细）。

texture: 皮肤的纹理，使用命名空间 ID，地址从 `assets\<命名空间>\textures` 开始计。

例 5.16

用文本组件显示玩家 `Mu_xian` 的头 

解

组件为

```
{object:"player",player:"Mu_xian"}
```

5.41

5.2.8 组件解析

记分板分数组件、实体名称组件和 NBT 组件这三种组件类型并不是在任何情况下都有效的，它们需要从游戏中索取数据，将数据渲染成静态的文本，这一过程被称为**组件解析 (Component resolution)**，解析后游戏才会将组件内容发送至各自的客户端。**这些经过解析产生的文本是一种非动态的文本，简单地说，这些文本不会随着游戏内容的改变而产生更新，它们只反映游戏在组件解析发生那一刻的结果。**

组件解析依赖于一个“触发实体”，这个实体会提供上下文，文本组件中使用的 `@s`、`*` 指代的实体即为这些触发实体。例如，文本展示实体中的文本组件触发实体为这个文本组件本身，故其中所有的 `@s` 都是这个文本展示实体：

```
summon text_display ~ ~ ~ {text:{selector:"@s"}}
```

5.42

无论这个命令在书写的时候是否被修改了执行者，文本组件中的 `@s` 依旧是这个文本展示实体：

```
execute as @a run summon text_display ~ ~ ~ {text:{selector:"@s"}}
```

5.43

此时文本展示实体的内容会被解析为文本展示实体本身的自定义名称。

对于一些没有触发实体的情况，部分组件会行为异常或受限。

所有能够发生组件解析的情况列于表 5.4，同时该表也提供了相应的上下文，作为对表 1.7 的补充。

表 5.4 组件解析上下文

情况	触发实体	权限等级	执行者实体	执行者名称	执行位置	执行朝向	执行锚点	执行维度
<code>/tellraw</code> <code>/title</code>	接收文本的各客户端的玩家	命令执行时的权限等级	命令执行时的执行者实体	命令执行时的执行者名称	命令执行时的执行位置	命令执行时的执行朝向	命令执行时的执行锚点	命令执行时的执行维度
<code>/bossbar</code> <code>/scoreboard</code> <code>/team</code>	此命令的执行者							
告示牌	无							
玩家打开成书	该玩家	玩家的权限等级	玩家	玩家的名称	家所在位置	玩家的朝向	脚部	玩家所在维度
讲台上放入成书	无	2	无	<code>Lectern</code>	讲台所在方块正中心	水平向南	脚部	讲台所在维度
文本展示实体	该文本展示实体	2	该文本展示实体	该文本展示实体的名称	该文本展示实体所在位置	该文本展示实体的朝向	脚部	该文本展示实体所在维度
物品修饰器的 <code>set_lore</code> 和 <code>set_name</code>	<code>entity</code> 指定的实体	2	<code>entity</code> 指定的实体	<code>entity</code> 指定实体的名称	<code>entity</code> 指定实体所在的位置	<code>entity</code> 指定实体的朝向	脚部	<code>entity</code> 指定实体所在的维度

以下情况无法进行组件解析：

1. 自定义物品的名称

例如，执行以下命令以给予一个苹果：

```
give @s minecraft:apple[minecraft:custom_name={selector:"@s"}]
```

5.44

此时苹果的名称显示为 ，它并没有发生解析。

2. 对话框

对话框中的所有记分板分数组件、实体名称组件和 NBT 组件都不能解析，要想显示这些数据，只能用宏函数将数据传递进去。

5.3 文本组件样式

文本组件样式，包括文字的样式、字体和交互事件，主要用于修饰文字。

5.3.1 样式与字体

基本的样式可应用于所有内容类型，其数据结构为：

文本组件

-  **bold**: 是否将文字变为**粗体 (Bold)**。
-  **color**: 文本本身的渲染颜色，可用值见下文的说明。
-  **font**: 渲染文字使用的字体，可用值见下文的说明。
-  **italic**: 是否将文字变为**斜体 (Italic)**。
-  **obfuscated**: 否将文字**模糊化 (Obfuscated)**处理, 使文本渲染为动态的随机字符(乱码)。
-  **shadow_color**: 文本阴影的颜色，可以直接用  整型/  数字形式指定 ARGB 颜色，也可以用  列表/  数组内的浮点数作为不同通道的分量以表示颜色。
 - 若使用  列表/  数组形式，则包含以下字段：**
 -   A 通道分量，这个通道表示不透明度，有效值  ~  (含)。
 -   R 通道分量，这个通道表示红色值，有效值  ~  (含)。
 -   G 通道分量，这个通道表示绿色值，有效值  ~  (含)。
 -   B 通道分量，这个通道表示蓝色值，有效值  ~  (含)。
-  **strikethrough**: 是否为文字添加**删除线 (Strikethrough)**。
-  **underlined**: 是否为文字添加**下划线 (Underline)**。

一、颜色与背景颜色

字段 `color` 可用的值包括 Minecraft 预设的 16 种颜色值、使用 `#<HEX>` 格式的 6 位十六进制颜色值以及用于重置颜色为默认颜色的 `reset`。预设的 16 种颜色如下表所示：

小提示

应用在不同物件上的文本组件默认颜色不同。命令 `/title` 和 `/tellraw` 的默认颜色为白色，成书和告示牌则为黑色。

表 5.5 颜色值表

颜色	值	HEX	颜色	值	HEX
黑色	<code>black</code>	#000000	深灰	<code>dark_gray</code>	#555555
深蓝	<code>dark_blue</code>	#0000AA	蓝色	<code>blue</code>	#5555FF
深绿	<code>dark_green</code>	#00AA00	绿色	<code>green</code>	#55FF55
湖蓝	<code>dark_aqua</code>	#00AAAA	天蓝	<code>aqua</code>	#55FFFF
深红	<code>dark_red</code>	#AA0000	红色	<code>red</code>	#FF5555
紫色	<code>dark_purple</code>	#AA00AA	粉红	<code>light_purple</code>	#FF55FF
金色	<code>gold</code>	#FFAA00	黄色	<code>yellow</code>	#FFFF55
灰色	<code>gray</code>	#AAAAAA	白色	<code>white</code>	#FFFFFF

例 5.17

使用正红色 `#FF0000` 渲染纯文本 `Hello World!`。

解

文本组件为 ①

```
{text:"Hello World!",color:"#FF0000"}
```

5.45

例 5.18

编写一条命令以实现 Herobrine 进入游戏的假象。

解

玩家在进入游戏的时候会在聊天栏中显示黄色的文字 `<玩家>加入了游戏`（中文）或 `<player> joined the game`（英文）。在使用命令制造“玩家进入游戏”的事件时，实际上不是真的有其他玩家加入了游戏，而是使用命令 `/tellraw` 造成的假信息。可以使用 `/tellraw` 在聊天栏中显示一段黄色的文本，并且运用翻译标识符以支持多种语言。在 `en_us.json` 中，玩家加入游戏的翻译文本由以下字段控制：

```
"multiplayer.player.joined": "%s joined the game"
```

5.46

对其中的 `%s` 定义文本 `Herobrine`，并添加样式，则所需的命令为

① 组件中的颜色块仅用于表示文字使用的颜色，实际编写时不存在这个色块。

```
tellraw @a {translate:"multiplayer.player.joined",with:
["Herobrine"],color:"yellow"}
```

5.47

顺着这条思路还可以制造出其他效果。比如，在游戏中传播假死信息：

 从高处摔了下来

玩家的死亡信息为白色不加粗字体，在使用文本组件编写时只需使用默认样式即可，命令可以为：

```
tellraw @a {translate:"death.fell.accident.generic",with:["Mu_xian"]}
```

5.48

背景颜色使用 ARGB 格式，由于它有一个 A 通道，因此可以指定背景颜色的透明度。 列表/ 数组形式可以分别指定各通道，如果是  整型/ 数字形式，这个值可按以下方式计算：

每一个通道都是介于 0 ~ 255（含）之间的值。若使用十六进制表示，则是一个八位数，每两位数为一个通道，从高到低依次是 A、R、G、B 通道。也可以使用二进制表示，如此每个通道都是 8 位，一共 32 位。在 SNBT 格式中可以用二进制和十六进制，但是在 JSON 格式中必须把这个数据转换为十进制。

例 5.19

为文本  添加蓝色背景（FF0000FF ）。

解

在 SNBT 中直接用十六进制即可：

```
{text:"Hello World!",shadow_color:0xff0000ff}
```

5.49

或者使用二进制：

```
{text:"Hello World!",shadow_color:0b11111111000000000000000011111111}
```

5.50

在 JSON 中就只能用十进制了：

```
{"text":"Hello World!","shadow_color":4278190335}
```

5.51

二、字体

字段  `font` 用于渲染文字的字体，值需要为所用字体的命名空间 ID，若不指定，则使用 Minecraft 默认的字体，即 Mojanglas，对于 Mojanglas 没有的码位，则使用 Unifont 字体。



图 5.2 原版使用的 Mojanglas 字体

字体文件夹一般存在于资源包中，地址为 `assets\<命名空间>\font\<字体>.json`。默认渲染的字体为 `minecraft:default`，在原版资源包中还存在其他字体，如 Unifont 字体、alt 字体（附魔台文字）。可以通过在资源包中编写字体 `.json` 文件以添加新的字体。

Minecraft 中的字体本质上是图像,如果设计巧妙,则可以将任意字符的字体设计成图像以实现一些视觉效果。具体内容见《资源包》。

三、 其他文字处理效果

例 5.20

将文本 **Hello World!** 设置为深红色粗体字，并带有下划线。

解

文本组件为

```
{text:"Hello World!",color:"dark_red",bold:true,underlined:true}
```

5.52

例 5.21

将文本 **Hello World** 模糊化处理，并设置删除线。

解

文本组件为

```
{text:"Hello World!", strikethrough:true, obfuscated:true}
```

5.53

这段文本的效果可能为 `FontStyle.Italic`，它是动态的。

5.3.2 格式化代码

在标准的文本组件中,为了给一段文本添加样式,通常的做法是在其后面添加各种样式的字段。其实在 Minecraft 中可以**直接在文本中添加所需的样式**,不一定需要在文本组件样式中定义,这时需要用到的工具便是**格式化代码 (Formatting code)**,其作用是为本文本本身添加格式信息。对于一些不支持文本组件的地方,可以使用格式化代码以定义文本的格式。

格式化代码使用分节符 `§` 作为其标识，介于分节符无法直接从键盘上输入，且原版客户端不支持分节符的输入，可以在允许 Unicode 的地方使用其 Unicode 码作为替代，即 U+00A7。在聊天栏或命令控制台中写成 `\u00a7`，而使用编译软件编写数据包时可以直接写为 `§`。格式化代码大致可分为两类：一类是 `§0 ~ §9`、`§a ~ §f` 的颜色代码，另一类是其余控制文本样式的格式代码。所有的格式化代码及其效果如下表所示：

表 5.6 格式化代码表

格式化代码	效果	HEX	格式化代码	效果	HEX
§0	黑色	#000000	§b	天蓝	#55FFFF
§1	深蓝	#0000AA	§c	红色	#FF5555
§2	深绿	#00AA00	§d	粉红	#FF55FF
§3	湖蓝	#00AAAA	§e	黄色	#FFFF55
§4	深红	#AA0000	§f	白色	#FFFFFF
§5	紫色	#AA00AA	§k	随机字符	-
§6	金色	#FFAA00	§l	粗体	-
§7	灰色	#AAAAAA	§m	删除线	-
§8	深灰	#555555	§n	下划线	-
§9	蓝色	#5555FF	§o	斜体	-
§a	绿色	#55FF55	§r	重制文字样式	-

当想要对一段文本应用样式时，只需在文本前添加相应的格式化代码。例如，下面的命令也可用于输出深红色的 `Hello World!` 文本，其与应用了文本组件相应键的命令效果相同：

§4Hello World!

5.54

在一段文本中间使用的颜色代码会使从此处起至下一个颜色代码之前的文本应用相应的颜色。例如，在下面的文本中，`World!` 为深红色，而前面的文本为默认颜色（聊天栏中为白色）。

Hello §4World!

5.55

下面的文本使 `Wo` 为深红色，`rl!` 为蓝色，前面的文本为默认颜色。

Hello §4Wo§9rl!

5.56

使用 `§r` 能之后的文本恢复为默认颜色。下面的文本中 `Wo` 为深红色，而其余文本均为默认颜色。

Hello §4Wo§rrl!

5.57

一段文本中位于后面的颜色代码会覆盖掉之前的颜色代码，因为一段文本不能同时拥有两种颜色。而其余的文本格式是可以重叠的，因此格式代码不会覆盖。在下面的文本中，`Hello` 为粗体，而 `World!` 既为粗体又为斜体：

```
$1Hello $oWorld!
```

5.58

若要取消这些文本格式，则须要使用 `$r`。接着上面的这个例子，若想要对 `World!` 文本取消粗体，则格式化代码需要这样写：

```
$71Hello $oWo$r$orld!
```

5.59

由于 `$r` 取消了粗体和斜体两种格式，而斜体需要保留，因此需要紧随其后重新加上 `$o`。注意，把 `$r$o` 写成 `$o$r` 是错误的，**格式化代码遵循从左到右的应用准则**，在编写的时候需要确定先后关系。在这个例子中需要先去掉所有格式再加上要保留的格式。

当颜色代码和格式代码发生混用时，**颜色代码会重置文本格式，使上一个格式代码的作用终止**。例如，`$4$oHello $1World!` 显示的文本为 `Hello World!`。格式代码需要在颜色代码重置的时候重新使用，如 `$4$oHello $1$oWorld!` 显示的文本为 `Hello World!`

若要使颜色代码和格式代码在同一个字符处开始产生作用，应该把颜色代码写在前面，例如，一个深红色、粗体的字符 `H` 需要这么写：

```
$4$1H
```

5.60

写成 `$1$4H` 只会使字符 `H` 渲染为深红色，而没有粗体的效果。

例 5.22

编写一段含格式化代码的文本组件，使其输出后实现效果：`Hello World!`

解

首先分析这段文本的颜色：`Hello` 为金色 `$6`，`Wo` 为黑色 `$0`，`rd!` 为绿色 `$a`，`!` 为天蓝色 `$b`。当只考虑颜色时，文本可以这么写：

```
$6Hello $0Wo$arld$b!
```

5.61

接着考虑字符格式，可以看到 `Hello` 为斜体 `$o`，`llo` 和 `!` 为粗体 `$1`，而 `World` 没有格式，不过 `!!` 这里启用了新的颜色代码，故可用颜色代码重置格式，因此完整的文本如下：

```
$6$oHe$1llo $0Wo$arld$b$1!
```

5.62

5.3.3 交互事件

交互事件是文本组件最重要的功能之一。有了这项功能后，文本不再仅限于显示信息，它还具有和玩家产生互动的能力。交互事件一共有三种不同的类型——即填入事件、点击事件和悬停事件，这三种事件分别在玩家按住 **Shift** 点击文字、直接点击文字和将鼠标光标移动至文本之上时发生。

对于命令 `/tellraw`、命令 `/ttitle`、告示牌、成书、文本展示实体、对话框等，并非所有交互事件都是可用的，其中所有的事件对于命令 `/ttitle` 而言均不可用。下文将具体说明各事件的可用性。

一、填入事件

填入事件 (Insertion)：当玩家按住 **Shift** 点击文字时触发，用于将一串特定的文本填入玩家的聊天框中。填入聊天框的内容不会将原有的文本覆盖，只会在光标显示的位置插入。例如，聊天框的文本 `fi` 中，光标位于字符 `i` 之后、`e` 之前，若填入的文本为 `e`，则聊天框中的文本会变为 `fiee`。注意，这些只是填入聊天框的文本，不代表已经发送出去的文本。。

填入事件的数据格式为：

`{} {}` 文本组件

└─ `"" ""` **insertion**: 填入的文本。

例如，命令

```
tellraw @a {"text":"点击这里发生问候","insertion":"Hello World!"}
```

5.63

返回的文本为 `点击这里发生问候`，对这段文本按住 **Shift** 的同时点击，则会在玩家的聊天框中自动填入 `Hello World!` 字样，按下消息发送键后即可将消息以玩家的名义发出。

二、点击事件

点击事件 (Click event) 是最常用的交互事件类型，也是与玩家最直接发生互动的事件类型，玩家只需要直接点击文本就可产生相关的事件。格式为：

`{} {}` 文本组件

└─ `{} {}` **click_event**: 点击事件。

└─ `"" ""` **action**: 玩家点击文本后触发的动作事件，具体见下文。

└─ 各动作事件的额外字段

点击事件一共有以下 8 种不同的动作事件：

1. `change_page`: 将成书翻至特定的页数，**仅在成书中可用**，数据格式为

`{} {}` 文本组件

└─ `{} {}` **click_event**

└─ `"" ""` **action**: `change_page`

└─ `I N` **page**: 指定的页数。

例 5.23

为成书设计一段文本 `[To Page 2]`，使得玩家点击后将成书翻至第 2 页。

解

文本组件为

```
{text:"[To page 2]",color:"green",click_event:
{action:"change_page",page:2}}
```

5.64

2. `copy_to_clipboard`：将指定内容复制至剪切板，在 `/tellraw` 和成书中可用，数据格式为

文本组件

click_event

action: `copy_to_clipboard`

value: 指定要复制的文本内容。

3. `custom`：向服务端发送封包，在 `/tellraw`、告示牌和成书中可用，一般用于自定义的服务端，对原版服务端基本没有实质效果。数据格式为

文本组件

click_event

action: `custom`

id: 需要发送的封包的命名空间 ID。

payload: 需要发送的自定义网络负载，可以是任意类型的数据。嵌套不超过 16 层，序列化后长度不超过 32768 字节。

4. `open_file`：用于打开指定的文件，出于安全原因，这种点击事件禁止玩家使用，仅用于客户端内部，比如截图后在聊天栏中出现的带横线文本，当点击这段文本的时候就会打开截图的图片文件。数据格式为

文本组件

click_event

action: `open_file`

path: 要打开的文件的路径。

5. `open_url`：用于打开对应的网址，在 `/tellraw` 和成书中可用。

若 `options.txt` 中 `chatLinks` 的值为 `false`，此点击事件不生效；若 `chatLinksPrompt` 的值为 `true`，则打开网址时不会有待确认对话框。数据格式为

文本组件

click_event

action: `open_file`

url: 需要打开的网址。

例 5.24

设计文本 `Minecraft`，使玩家在点击该文本的时候打开 Minecraft 的官网。

解

Minecraft 的官网地址为 `https://www.minecraft.net`，只需将其作为字段 `I N` `url` 的值即可。

```
{text:"Minecraft",click_event:{action:"open_url",url:"https://www.minecraft.net"}}
```

5.65

6. `run_command`：在点击文本的时候执行命令，在 `/tellraw`、告示牌、成书和非配置阶段的对话框中可用，数据格式为

`0 0` 文本组件

`0 0` `click_event`

`” ”` `action`: `run_command`

`” ”` `command`: 要执行的命令，可以省略 `/` 前缀。

在命令 `/tellraw`、成书和对话框中使用该点击事件等价于使用此点击事件的玩家在聊天栏输入命令，受到聊天栏中最多输入 256 个字符的限制。各种情况执行命令的上下文参数按表 1.7 确定，需要额外确认命令所需的权限等级以及使用此点击事件的玩家自身的权限等级，若执行权限等级过低，可使用触发器命令以规避，详情参见节 7.3.1 所作的说明。

若在告示牌中使用，则只允许父组件具有该点击事件，子组件不得使用该点击事件，此时可以通过点击告示牌来执行命令。命令上下文依旧按照表 1.7 确定。告示牌一共有四行可用的文本，因此可以应用四个不同的点击事件。当玩家点击告示牌时，不会识别玩家点击告示牌的哪一行文本。所以当玩家点击告示牌时，告示牌上所有的点击事件会按从上到下的顺序同时发生。

例 5.25

设计一段文本 `Minecraft`，使玩家在点击该文本的时候对此玩家显示主标题

`Hello Minecraft!`。

解

主标题 `Hello Minecraft!` 的显示也需要文本组件，因此需要在文本组件内嵌套文本组件。若使用 SNBT 形式的文本组件，则字段 `” ”` `command` 可以用单引号定义字符串，从而在内部的文本组件中使用双引号定义的字符串：

```
{text:"Minecraft",click_event:{action:"run_command",command:'title @s title "Hello Minecraft!"'}}
```

5.66

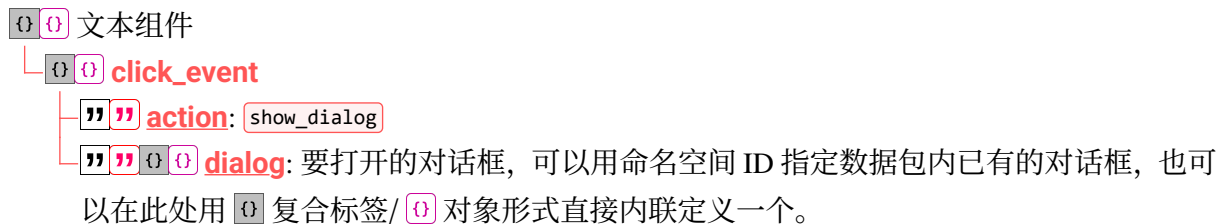
同样也可以在 `” ”` `command` 中用双引号定义字符串，从而在内部的文本组件中使用单引号定义的字符串。

若整个文本组件使用 JSON 格式，则必须使用双引号，此时内部命令中的文本组件可以用单引号规避转义：

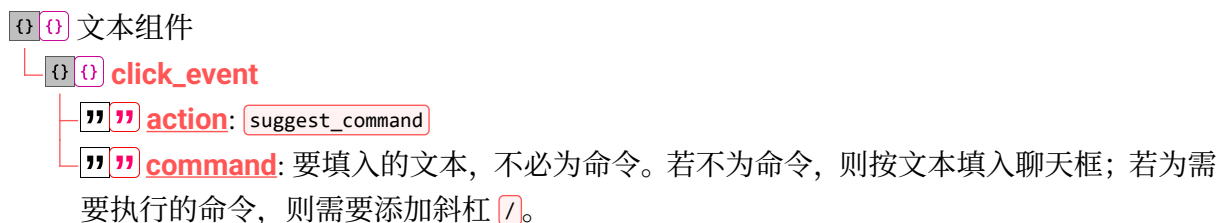
```
{"text":"Minecraft","click_event":
{"action":"run_command","command":"title @s title 'Hello
Minecraft!'"}}
```

5.67

7. `show_dialog`: 向玩家显示一个对话框, 在 `/tellraw`、告示牌和成书中可用, 数据格式为



8. `suggest_command`: 将文本填入聊天框中并覆盖原先的文本, 与填入事件不同。此事件在 `/tellraw` 中可用, 在成书中不可用, 数据格式为



三、悬停事件

悬停事件 (Hover event) 使玩家将他们的鼠标光标移动到文本上时显示悬停在该文本上的提示框, 所有悬浮事件仅在命令 `/tellraw` 和成书中可用, 效果大致如图所示:

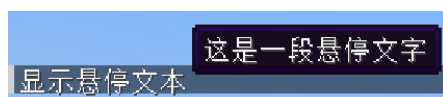
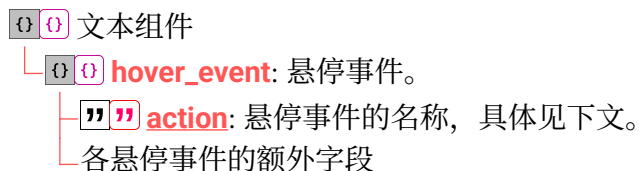


图 5.3 悬停文本

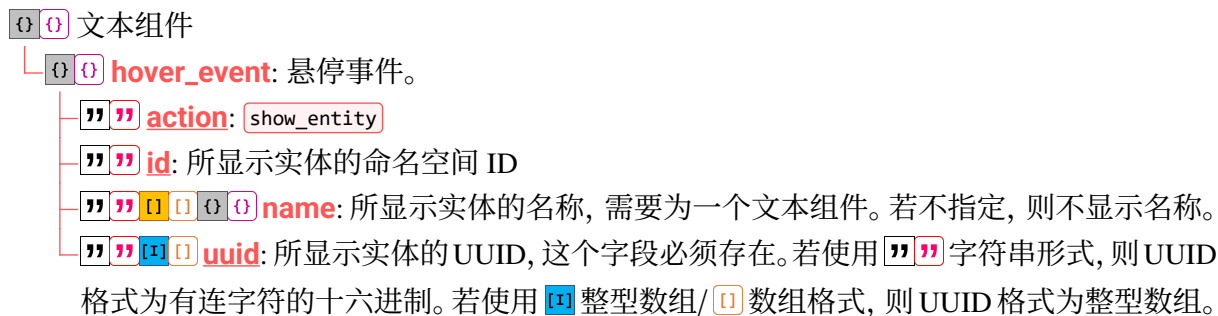
悬停事件一共分为 3 种, 分别可显示文本、物品和实体信息, 数据格式为:



字段 `"" action` 一共有 3 种有效值, 分别对应不同的动作事件。

1. `show_entity`: 用于显示实体提示框。

显示包括实体的名称、类型和 UUID 三条信息, 显示的效果与使用实体名称这种内容类型时将光标移动到返回的实体名称上显示的信息相同。显示的实体不必真实存在, 数据格式为:



2. `show_item`: 用于显示物品提示框。

显示的内容与玩家将鼠标光标移动至物品栏中的物品上时显示的内容相同。比如当显示钻石剑的信息时，提示信息为

```
钻石剑
在主手时：
7 攻击伤害
1.6 攻击速度
minecraft:diamond_sword
9 个组件
```

显示的物品不必真实存在，数据格式为：

  文本组件

└   **hover_event**: 悬停事件。

└   **action**: `show_item`

└   **id**: 所显示物品的命名空间 ID。

└   **components**: 物品堆叠组件，部分组件信息会显示在提示框中。若文本组件为 SNBT 格式，则根据节 6.11 直接使用 SNBT 格式的物品堆叠组件；若文本组件为 JSON 格式，则需要按照节 4.5.1 办法将 SNBT 格式的物品堆叠组件转换为 JSON 格式。

└ 一个物品堆叠组件

└   **count**: 所显示物品的堆叠数量，数量不会直接在提示框中显示。

3. `show_text`: 用于显示文本提示框，数据格式为：

  文本组件

└   **hover_event**: 悬停事件。

└   **action**: `show_text`

└      **value**: 要显示的文本，使用文本组件。此处无法使用点击事件   `click_event` 和悬停事件   `hover_event`。

例 5.26

用命令实现如图 5.3 所示的效果。

 解

这是聊天栏中的文本，需使用 `/tellraw`，命令可以为

```
tellraw @a {text:"显示悬停文字",hover_event:
{action:"show_text",value:"这是一段悬停文字"}}
```

5.68

如果想给悬停文字添一些样式，比如，将图 5.3 中的悬停文字变成红色，则只需在字段 `value` 中使用  复合标签/ 对象，命令可以写为：

```
tellraw @a {text:"显示悬停文字",hover_event:{action:"show_text",value:
{text:"这是一段悬停文字",color:"red"}}}
```

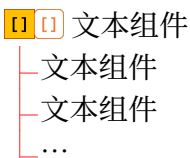
5.69

5.4 子组件

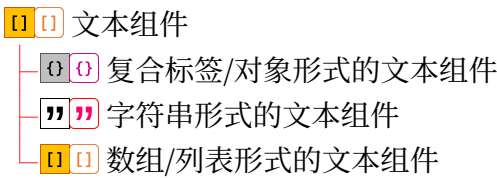
前面所讲到的文本组件类型和文本组件样式, 都只能将它们写在同一个对象中, 虽然不同样式是可以随意组合的。因为一个对象中只能使用一种内容类型, 而组合后的样式只能作用于一个对象中的文本。这意味着, 返回一段文本时, 整段文本使用的样式都是同一种。如果需要在一段文本的不同位置应用多种样式组合, 比如, 如何输出一段文本 `Hello World!`, 使 `Hello` 为红色, 而 `World!` 为蓝色呢? 在不使用格式化代码的情况下, 显然需要使用多个对象并分别对这些对象应用不同的样式。子组件提供了允许多种样式或交互事件同时存在于一段文本内的可能。文本组件中的 `[]` 列表/ `[]` 数组和字段 `[] extra` 有助于实现这一功能。

5.4.1 数组

使用 `[]` 列表/ `[]` 数组形式的文本组件有如下的数据格式:



因此可以将多个文本组件一起放到数组中:



规定: `[]` 列表/ `[]` 数组中第一个元素为父组件, 其他的元素均为子组件。可以在这些对象中使用不同的内容类型, 也可以对不同的组件应用不同的样式。下面的一些例子有助于对数组的理解:

例 5.27

编写一段文本组件, 使之输出一段文本 `Hello World!`, 其中 `Hello` 为红色, 而 `World!` 为蓝色。

解

在这个例子中, 首先需要编写一段纯文本, 并应用样式为红色; 再编写一段应用样式为蓝色的纯文本, 因此需要使用两个文本组件。第 1 个可以写为:

```
{text:"Hello ",color:"red"}
```

5.70

第 2 个可以写为:

```
{text:"World!",color:"blue"}
```

5.71

接着用 `list` 列表/ `array` 数组将它们整合到一起形成一个列表：

```
[{text:"Hello ",color:"red"},{text:"World!",color:"blue"}]
```

5.72

于是就可以输出 `Hello World!`。输出文本时数组中元素与元素之间是不会有空格的，在这个例子中，`Hello` 和 `World!` 之间的空格是直接算在第 1 个文本组件中的。而空格也被赋予了红色的样式，但由于这个字符是个空格，因此红色的样式不会直接表现。

例 5.28

一张冒险地图的过场动画允许玩家按使用键（默认为 `鼠标右键`）以跳过，试设计提示所需的文本。

解

现文本设计为 `按鼠标右键以跳过动画`，显示在动作栏，键位标识符查表 5.3 可得 `key.use`。由于其他文本没有设计样式，使用 `key` 字符串即可。命令为

```
title @a actionBar ["按",{keybind:"key.use"},"以跳过动画"]
```

5.73

例 5.29

在聊天栏中编写一段命令，要求能返回命令执行者自己在记分项 `[personal_score]` 上面的分数。返回文本可以如下所示，其中分数显示为红色：

《玩家》的个人分数为：《分数》【对分数有疑惑？】

若玩家将鼠标光标移至 `【对分数有疑惑？】` 上，则显示悬停的文本提示框，内容为

被刷新的分数也计入个人分数

解

要求的文本内容一共有三种类型：`《玩家》` 很明显是要求输出实体名称，`的个人分数为：` 和 `【对分数有疑惑？】` 是纯文本，其中 `【对分数有疑惑？】` 还带有悬停事件，而 `《分数》` 则要求输出记分板分数。因此需要有 4 个文本组件，所以需要使用数组将这 4 个组件包括起来。对于第 1 个组件，它的类型为实体名称，根据题意要求返回命令执行者自己的名称，因此写为：

```
{selector:"@s"}
```

5.74

对于第 2 个组件，它是无样式的纯文本，因此写为：

```
"的个人分数为："
```

5.75

对于第 3 个组件，它的类型为记分板分数。要求输出命令执行者自己在 `[personal_score]` 上的分数，而命令执行者同时也为文本的观察者。加上题目要求的红色样式，因此可以写为：

```
{score:{objective:"test",name:"*"},color:"red"}
```

5.76

对于第 4 个组件，为它添加悬停事件：

```
{text:" [对分数有疑惑? ]",color:"green",hover_event:
{action:"show_text",value:"被刷新的分数也计入个人分数"}}
```

5.77

将它们整合起来，于是命令可以写为：

```
tellraw @s [{selector:"@s"},"的个人分数为: ",{score:
{objective:"test",name:"*"},color:"red"},{text:" [对分数有疑
惑? ]",color:"green",hover_event:{action:"show_text",value:"被刷新的分
数也计入个人分数"}}]
```

5.78

5.4.2 字段 extra

字段 `[1] [1] extra` 是为一个组件添加附加文本组件的工具。在使用 `[1] [1] extra` 时，必须得保证同一个文本组件中必须存在它所依附的文本，因为 `[1] [1] extra` 不能单独出现在一个文本组件中。若一个对象中只存在一个文本，则可以为它添加 `[1] [1] extra`，作为文本的补充内容使用。数据格式为：

`[1] [1]` 文本组件

├ 文本组件类型

├ 文本组件样式

└ `[1] [1] extra`: 附加的文本组件。

└ 文本组件

规定：**被依附的文本组件为父组件**，`[1] [1] extra` 内的组件均为**子组件**。当文本组件被成功解析时，返回的内容中子组件的内容一定是跟在父组件内容的后面。在写法上，字段 `[1] [1] extra` 不一定必须放在其所依附文本的后面，放在其前面也是可以的，此时它的内容物依旧均为子组件。

如果一个父组件中存在多个 `[1] [1] extra`，则只有位置处于最后面的 `[1] [1] extra` 才会被使用。

例 5.30

下列文本组件分别返回什么文本？

```
{text:"A",extra:["B",{text:"C"}]}
```

5.79

```
{extra:["A",{text:"B"}],text:"C"}
```

5.80

```
{text:"A",extra:["B",{text:"C"}],text:"D"}
```

5.81

```
{text:"A",extra:["B"],text:"C",extra:["D"]}
```

5.82

解

在文本组件 5.79 中，可以清晰地看到一段纯文本 `ñ` 及其子组件内容，即 `extra` 中的文本 `ñC`。因此输出文本为 `ññC`。

文本组件 5.80 是   extra 与文本交换位置的情况，即使文本组件中的子组件在父组件的前面，输出时   extra 的文本也只能跟在父组件文本的后面，因此输出文本 。

文本组件 5.81 的情况有些特殊，一个对象中使用了两个纯文本和一个 `TextExtra`。那么子组件所依附的是哪一个父组件呢？根据节 5.2 对同一个对象的多个文本中判断优先选择输出哪一个文本的讲述，组件会使用类型优先级最高且位置最靠后的那个文本。于是父组件为纯文本 `Text`，`TextExtra` 便作为了它的子组件，因此输出的文本为 `000`。文本 `fi` 被直接忽略，优先级的准则使得它无法作为一个父组件。

在文本组件 5.82 中，文本  为父组件，文本  由最后一个   extra 输出，因此输出文本为 。

5.4.3 继承

子组件使多个文本组件存在于同一段文本中。当存在多个组件时,自然可以为每一个组件分别指定一些样式。但是,这些组件之间并不是相互独立的,它们的样式会产生影响。这种发生在组件之间对样式产生影响的行为,一般称之为样式的**继承 (Inherit)**。

继承发生在组件与组件之间,且通常由一个组件对另一个组件产生作用,这意味着继承具有单向性,组件与组件之间的样式和交互事件不是相互影响的。继承具有以下几条准则:

1. 若子组件没有指定任何样式，则其将继承父组件的所有样式。
2. 子组件和子组件之间相互独立，彼此之间的样式不会相互影响。
3. 若子组件指定了若干种类的样式，则该子组件将使用自己指定的样式，对于自己指定的这些样式，父组件的相应样式会被直接忽略。
4. 若子组件没有指定一些的样式，则父组件将这些没有指定的样式继承到子组件。



图 5.4 继承的规律

继承的实质是，子组件将父组件的所有样式作为它的默认样式（颜色、字体、字体处理方式和交互事件）使用。对于图 5.4 中的四种情况，所有未指定的样式均使用默认样式：父组件使用系统本身的默认样式（所有交互事件均默认不存在），子组件将父组件的样式作为默认样式。但

子组件的样式 2 已指定，因此不使用父组件给它的默认样式；而子组件的样式 1 未指定，因此使用父组件给它的默认样式。

一、数组的继承

❏ 列表/ **❏** 数组中第一个元素为父组件，其他的元素均为子组件。

例 5.31

判断下面的文本组件输出的文本样式。

```
[
  {text:"A",color:"red"},
  "B",
  {text:"C",bold:true},
  {text:"D",color:"blue"}
]
```

5.83

解

列表一共包含了 4 个文本组件，列表中第一个组件即为父组件，现列出这个父组件表现的样式和父组件的系统默认样式：

表 5.7

	纯文本 (text)	颜色 (color)	粗体 (bold)
父组件的系统默认样式	-	默认颜色	否 (false)
父组件表现的样式	A	红色 (red) 	默认 (false)

父组件的样式即为子组件的默认样式。列表中第二个组件为字符串，其等效于 `{text:"B"}`，整理得：

表 5.8

	纯文本 (text)	颜色 (color)	粗体 (bold)
子组件的默认样式	-	红色 (red) 	否 (false)
子组件 1	B	默认 (red) 	默认 (false)
子组件 2	C	默认 (red) 	是 (true)
子组件 3	D	蓝色 (blue) 	默认 (false)

所以输出的文本为 **ABCD**。

有时候，为了取消列表中第一个文本组件的样式对后面文本组件的影响，可以让列表第一个组件不使用任何的样式。但这个时候，第一个文本组件返回的文本就只会使用系统的默认样式了。于是**通常将第一个文本组件设为空值**，采用如下的写法：

```
[ "", <文本组件 2>, <文本组件 3>, ... ]
```

5.84

从第 2 个文本组件开始写起，既可以防止父组件对后面所有子组件样式的影响，又可以让第一个有文本的文本组件拥有特定的样式。

二、extra 的继承

被 `<Text>` `extra` 依附的文本组件为父组件，`<Text>` `extra` 内的组件均为子组件。注意，`<Text>` `extra` 虽然为一个文本组件列表，但是该列表的第一个文本组件不会对后面的文本组件造成影响。

例 5.32

判断下面的文本组件输出的文本样式。

```
{
  text:"A",
  color:"red",
  bold:true,
  hover_event:{action:"show_text",value:"A"},
  extra:[
    {text:"B",color:"green",italic:true},
    "C",
    {text:"D",color:"blue",bold:false},
    {text:"E",hover_event:{action:"show_text",value:"E"}}
  ]
}
```

5.85

解 先整理出父组件表现的样式和父组件的系统默认样式：

表 5.9

	父组件的系统默认样式	父组件表现的样式
纯文本 (<code>text</code>)	-	A
颜色 (<code>color</code>)	默认颜色	红色 (<code>red</code>)
粗体 (<code>bold</code>)	否 (<code>false</code>)	是 (<code>true</code>)
斜体 (<code>italic</code>)	否 (<code>false</code>)	默认 (<code>false</code>)
悬停事件 (<code>hover_event</code>)	无	文本提示框 A

再依次整理出所有子组件的表现格式：

表 5.10

	子组件的系统默认样式	子组件 1	子组件 2	子组件 3	子组件 4
纯文本 (<code>text</code>)	-	B	C	D	E
颜色 (<code>color</code>)	红色 (<code>red</code>)	绿色 (<code>green</code>)	默认 (<code>red</code>)	蓝色 (<code>blue</code>)	默认 (<code>red</code>)
粗体 (<code>bold</code>)	是 (<code>true</code>)	默认 (<code>true</code>)	默认 (<code>true</code>)	否 (<code>false</code>)	默认 (<code>true</code>)
斜体 (<code>italic</code>)	否 (<code>false</code>)	是 (<code>true</code>)	默认 (<code>false</code>)	默认 (<code>false</code>)	默认 (<code>false</code>)
悬 停 事 件 (<code>hover_event</code>)	文本提示框 A	文本提示框 A	文本提示框 A	文本提示框 A	文本提示框 E

所以输出的文本格式为 `ABCDE`，其中 `ABCD` 均有悬停文本 `fi`，`E` 有悬停文本 `E`。

5.4.4 组件序列化

写入数据的时候，文本组件允许使用 `[]` 列表/`[]` 数组格式，但在存储阶段，其中的数据会被统一序列化为 `[]` `extra` 模式。例如，写入的文本组件为：

```
[{text:"Hello ",color:"red"},{text:"World!",color:"blue"}]
```

5.86

存储的形式为：

```
{text:"Hello ",color:"red"},extra:[{text:"World!",color:"blue"}]}
```

5.87

因此，若这个文本组件是一个 NBT 字段 `text` 的值，要用 NBT 路径访问 `World!` 的颜色，不能写成 `text[1].color`，要写成 `text.extra[0].color`。

5.5 应用实例

文本组件专门用于显示文本，因此它在冒险地图或服务中承载了大量的信息，用于剧情推进、提示说明等。在冒险地图中使用文本组件是一项必要的技能。本节提供了若干实例用于说明文本组件的一些应用。

例 5.33

编写一段文本组件，使之在聊天栏中返回如下文本：

1 2 3 4 5 6 7 8 9 0

解

首先观察：这段文本一共有 11 种不同的颜色，其中数字 `1~0` 均为不同的颜色，而竖线为白色（聊天栏文本的默认颜色）。文本本身是可以通过格式化代码赋予一些格式信息的。如果使用格式化代码，则允许只使用一个父组件便实现不同的样式。而这段文本含有聊天栏的默认颜色，则可以使用 `$r` 对样式进行重制。现在按照从左到右的方向依次为文本中的字符带上格式化代码：

```
$41$r | $c2$r | $63$r | $e4$r | $a5$r | $26$r | $37$r | $98$r | $19$r  
| $50
```

5.88

上面便是含有格式化代码的字符串形式文本组件，当然也可以写成对象的格式，这时候只要使用纯文本即可。若因分节符输入困难而不使用格式化代码，则需要用含有多文本组

件的列表。注意，若列表第一个组件内容为深红色的 **1**，则后面的所有组件均会认深红色为其默认颜色，这时候所有白色竖线文本就需要更改颜色至白色，这样会使命令的长度大大增加。因此对父组件使用白色的样式，这样列表中其他组件的格式就不会相互影响：

```
[
  "",
  {text:"1",color:"dark_red"},
  " | ",
  {text:"2",color:"red"},
  " | ",
  {text:"3",color:"gold"},
  " | ",
  {text:"4",color:"yellow"},
  " | ",
  {text:"5",color:"green"},
  " | ",
  {text:"6",color:"dark_green"},
  " | ",
  {text:"7",color:"dark_aqua"},
  " | ",
  {text:"8",color:"blue"},
  " | ",
  {text:"9",color:"dark_blue"},
  " | ",
  {text:"0",color:"dark_purple"},
  " | ",
]
```

5.89

例 5.34

编写一段文本组件使在聊天栏中实现如下文本：

【提示】 在下面的选项中选择其一以推进后续的剧情。

===== ☆ 请 ☆ 选 ☆ 择 ☆ =====

【上前与之对话】 【绕道去学校】

=====

其中 **【上前与之对话】** 有悬停文本提示框 **走上前去与“大仙”对话，说不定能问到些什么**，并设置点击事件为执行命令

令 `function main:story/talk;` **【绕道去学校】** 有悬停文本提示框 **学校的事情急，还是先去学校吧，这个人就不去管他了**，并设置

点击事件为执行命令 `function main:story/school。`

解

对于一段文本中含有多个样式的情况，一律使用文本组件列表进行编写。先将列表中第一个组件设为空值。**【提示】** 为绿色文本，单独使用一个文本组件：

```
{text:"[提示]",color:"green"}
```

5.90

【提示】 在下面的选项中选择其一以推进后续的剧情。

===== ★ 请 ★ 选 ★ 择 ★ =====

这两行文本的样式一致，故使用同一个

文本组件。鉴于五角星的符号不容易输入，故可以用五角星的 Unicode 码代替。不同文本行之间用换行符 `\n` 隔开：

" 在下面的选项中选择其一以推进后续的剧情。

===== \u2605 请 \u2605 选 \u2605 择 \u2605 ====="

5.91

接下来是一个换行符和一段空格，从效果上看不出应用的样式，故可与上一个文本组件进行合并，写入上一个组件，同时换行符不能忘记：

" 在下面的选项中选择其一以推进后续的剧情。 \n===== \u2605 请 \u2605 选
\u2605 择 \u2605 =====\n "

5.92

【上前与之对话】 这段文本不仅有绿色的样式，还有悬停事件和点击事件。注意悬浮的文字中虽然有双引号，但是不需要加转义字符。这是因为文本中的双引号为中文的引号，不会与英文的引号发生匹配，只有英文的引号才需要加转义字符。于是该文本组件可以写为：

```
text:"[上前与之对话]",color:"green",hover_event:
{action:"show_text",value:"走上前去与“大仙”对话，\n 说不定能问到些什
么"},click_event:{action:"run_command",command:"function main:story/
talk"}}
```

5.93

下面是一段空格，在此不做赘述。**【绕道去学校】**与**【上前与之对话】**的编写方式相同，在此也不做赘述。最后是换行符加一段文本。将上面所有的文本组件组合起来，可以得到文本组件：

```
[
  "",
  {text:"[提示]",color:"green"},
  " 在下面的选项中选择其一以推进后续的剧情。 \n===== \u2605 请 \u2605
选 \u2605 择 \u2605 =====\n "
  {
    text:"[上前与之对话]",
    color:"green",
    hover_event:{
      action:"show_text",
      value:"走上前去与“大仙”对话，\n 说不定能问到些什么"
    },
    click_event:{
      action:"run_command",
      command:"function main:story/talk"
    }
  }
]
```

5.94

```
    },
    " ",
    {
      text: "[绕道去学校]",
      color: "■green",
      hover_event: {
        action: "show_text",
        value: "学校的事情急, \n 还是先去学校吧, \n 这个人就不去管他了"
      },
      click_event: {
        action: "run_command",
        command: "function main:story/school"
      }
    },
    "\n=====
  ]
```

5.6 聊天类型

聊天（Chat） 是玩家发送文本内容、输入命令的一项功能。相比于通过 `/tellraw` 在聊天栏中发送内容, 用聊天功能发送的文本在内容和样式上相对单调且固定, 但开发者依然可以像文本组件那样有限地为聊天文本设计内容和样式。

5.6.1 基本聊天类型

Minecraft 一共有以下几种**聊天类型（Chat type）**：

- 1. 玩家发送聊天信息
这种类型只需玩家按 `T` 键呼出聊天栏, 在其中输入文本后发送即可, 默认的格式为:

```
<sender> content
```

- 2. 使用 `/me` 命令发送信息
`/me` 通常用于展示命令执行者的状态或是动作, 成功执行后返回文本的默认格式为 `* sender content`。这个动作需要的字符串可以是随意的, 比如玩家 Steve 向所有玩家公布自己正在制作地图, 则 Steve 在聊天栏中输入的命令可以为 `/me 正在制作地图`, 则返回的结果是 `* Steve 正在制作地图`。该命令所需权限等级为 0, 语法为:

```
me <action>
```

5.95

其中的参数: `<action>` (字符串 `brigadier:string`) —— 所展示的动作。

- 3. 发送私信

`/msg`、`/tell` 和 `/w` 三条命令用于发送私聊信息给指定的玩家，它们的语法完全相同，可相互替代，所需权限等级均为 0：

```
msg <targets> <message>
```

5.96

```
tell <targets> <message>
```

5.97

```
w <targets> <message>
```

5.98

其中的参数：`<message>`（文本 `minecraft:message`）——需要发送的信息。可以接受目标选择器，如果在这些部分中指定了某些实体且权限等级大于等于 2，则返回结果时会显示这些实体的名字。

消息在发送者聊天栏中的默认格式为 `你悄悄地对你说: content`，在接收者处的默认格式为 `sender 悄悄地对你说: content`。

4. 使用 `/say` 命令发送信息

命令 `/say` 用于向所有玩家发送信息，所需权限等级为 2，语法为：

```
say <message>
```

5.99

其中的参数：`<message>`（文本 `minecraft:message`）——贪婪词组，可以接受目标选择器，如果在这些部分中指定了某些实体，则返回结果时会显示这些实体的名字。

在默认情况下执行该命令后游戏会在聊天栏显示 `[sender] content`。在命令方块中执行该命令时，命令执行者是命令方块本身，返回的结果会显示命令方块的名称。如果命令方块没有名称，则会显示 `@` 以代替命令方块的名称。

例 5.35

用 `/say` 命令显示消息 `Hello world!`。

解

命令为

```
say Hello world!
```

5.100

5. 向队内成员发送信息

`/teammsg` 和 `/tm` 这两条命令用于向同一队伍中的所有成员发送消息，可以相互代替，所需权限等级为 0，语法为：

```
teammsg <message>
```

5.101

```
tm <message>
```

5.102

其中的参数：`<message>`（文本 `minecraft:message`）——需要发送的信息。可以接受目标选择器，如果在这些部分中指定了某些实体且权限等级大于等于 2，则返回结果时会显示这些实体的名字。

消息在发送者聊天栏中的默认格式为 `-> target <sender> content`，在接收者处的默认格式为 `target <sender> content`。

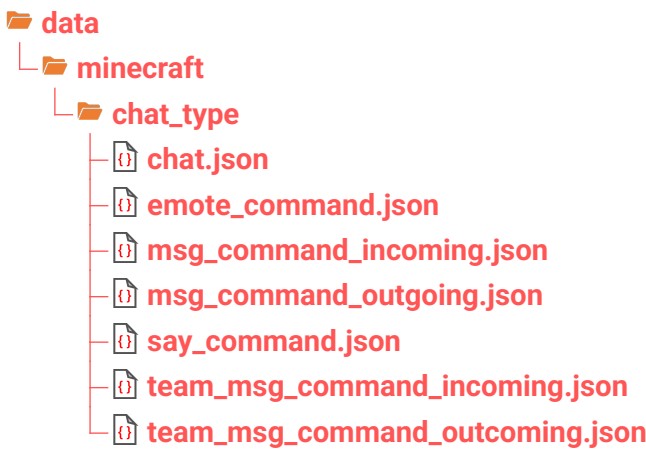
5.6.2 聊天类型的定义

上述聊天类型返回的文本都可以通过数据包修改，即可写注册表 `chat_type`。原版存在七个聊天类型，分别对应的聊天类型如下所示：

表 5.11 原版聊天类型

注册项	聊天类型	默认格式
<code>chat</code>	玩家发送聊天信息	<code><sender> content</code>
<code>emote_command</code>	使用 <code>/me</code> 命令发送信息	<code>* sender content</code>
<code>msg_command_incoming</code>	<code>/msg</code> 、 <code>/tell</code> 和 <code>/w</code> 私信的接收者	<code>sender悄悄地对你说: content</code>
<code>msg_command_outgoing</code>	<code>/msg</code> 、 <code>/tell</code> 和 <code>/w</code> 私信的发送者	<code>你悄悄地对target说: content</code>
<code>say_command</code>	使用 <code>/say</code> 命令发送信息	<code>[sender] content</code>
<code>team_msg_command_incoming</code>	<code>/teammsg</code> 和 <code>/tm</code> 队内消息的接收者	<code>target <sender> content</code>
<code>team_msg_command_outcoming</code>	<code>/teammsg</code> 和 <code>/tm</code> 队内消息的发送者	<code>-> target <sender> content</code>

虽然 `chat_type` 是可写注册表，但游戏实际使用的也就以上 7 种聊天类型，且它们的命名空间 ID 都是 `minecraft`。所以有效的聊天类型配置文件必须使用如下的数据包路径：



其他任何的路径都不会被游戏识别，因此只能对原有的这 7 个配置文件做修改。

一个聊天类型配置文件的格式如下所示：

- ❏ 文件封装
 - ❏ **chat**: 聊天信息在聊天栏内显示的内容和样式。
 - ❏ **translation_key**: 聊天信息的内容，需要是一个翻译标识符或一个允许包含译文变量的文本。

- ① **parameters**: 传入到聊天内容中的参数。
 - ” 一个参数，有效值只能是 `content`、`sender` 或 `target`。三种有效值的意义如下表所示：

表 5.12 聊天信息的可用参数

参数	意义	可用
<code>content</code>	聊天信息的文本	所有聊天类型均可用
<code>sender</code>	聊天信息的发送者	所有聊天类型均可用
<code>target</code>	聊天信息的接收者	仅适用于聊天类型 <code>msg_command_outgoing</code> 、 <code>team_msg_command_incoming</code> 和 <code>team_msg_command_outcoming</code>

- ① **style**: 聊天信息的样式，默认没有样式。
 - 文本组件样式
- ① **narration**: 聊天信息在复述功能中复述的内容。
 - ” **translation_key**: 聊天信息的内容，需要是一个翻译标识符或一个允许包含译文变量的文本。
 - ① **parameters**: 传入到聊天内容中的参数。
 - ” 一个参数，有效值只能是 `content`、`sender` 或 `target`。
 - ① **style**: 聊天信息的样式，对复述功能无效。
 - 文本组件样式

例如，若 ” `translation_key` 的值为 `%s%s`，① `parameters` 为 `["sender", "content", "sender"]`，则返回的内容为 `sender content sender`。① `parameters` 中的参数可以重复。

例 5.36

修改玩家发送的聊天信息，使之呈现的样式为 `sender : content`

解 如下所示：

```
data > minecraft > chat_type > chat.json
1  {
2    "chat": {
3      "parameters": [
4        "sender",
5        "content"
6      ],
7      "translation_key": "%s: %s"
8    },
9    "narration": {
10     "parameters": [
11       "sender",
12       "content"
13     ],
14     "translation_key": "chat.type.text.narrate"
15   }
16 }
```

第五章思考题与习题

1. 针对下面的输出内容，分别写出其不带格式化代码的 SNBT 文本组件。

(1) MC-CMD

(2) 分节符“\”的作用很大

(3) 纯文本 JSON 语法为：{"text":"<文本>"}

(4) 游戏规则 *keep_inventory* 已更新

(5) 错误的语法

(6) \\"Hello World\\"

(7) 

2. 针对下面的输出内容，分别写出其不带格式化代码的 SNBT 文本组件。

(1) 【任务】合成一个钻石镐

(2) 红色粗体斜体

(3) 123456789

(4) \\"Hello World\\"

3. 判断下列说法是否正确。

(1)  translate 字段的值可以包含 %s。

(2) 一个命令方块中有如下的命令：tellraw @a {score:{name:"*",objective:"test"}}, 激活该命令方块，此时不会有任何玩家看到返回的分数。

(3) 触发实体为玩家时，该文本组件无法被解析：{selector:"@e[type=villager]}"}

4. 文本组件

```
{score:"@r",nbt:"Items",block:"~ ~1
~,entity:"@r",translate:"commands.debug.stopped",with:
[4,7,5],text:"OK!",text:"NO",entity:"@e"}
```

5.103

返回的文本内容为_____。

5. 若有一翻译标识符 custom.1，其在  zh_cn.json 的值如下所示：

```
%s%3$s%s%6$s%1$s%s%2$s%
```

5.104

(1) 写出该文本组件返回的文本：

```
{translate:"custom.1",with:[1,true,"7","A",[9,"6"]],
{text:false},"5"]}
```

5.105

- (2) 要想返回文本 `False0rfalse8F290rtext`，写出使用翻译标识符 `custom.1` 的文本组件。
6. 在下表中填写各种事件的可用性，填写的内容在命令 `/tellraw`、命令 `/title`、告示牌、成书和文本展示实体中选择。

表 5.13

事件类型	值	可用性
填入事件		_____
点击事件	<code>change_page</code>	_____
	<code>copy_to_clipboard</code>	_____
	<code>custom</code>	_____
	<code>open_url</code>	_____
	<code>run_command</code>	_____
	<code>show_dialog</code>	_____
	<code>suggest_command</code>	_____
悬停事件	<code>show_entity</code>	_____
	<code>show_item</code>	_____
	<code>show_text</code>	_____

7. 编写一段在 `/tellraw` 中运行的文本组件，使返回的文本 `show` 拥有悬浮文字，悬浮文字显示一个石头的物品信息，其中石头的数据组件为

```
{"minecraft:custom_name":"show"}
```

5.106

8. 编写一段在 `/tellraw` 中运行的文本组件，使返回的文本 `click here` 拥有点击事件，点击该文本后返回一段仍能够被点击的文本 `click again!`，点击这段文本后才返回红色的 `Hello!` 文本。

第六章 存档格式

Minecraft 具体的游戏数据, 如某个位置的箱子里存储的物品、一个实体的位置、世界边界的大小、Boss 栏、玩家已达成的进度……这些数据绝大部分都以 NBT 的形式存储在游戏存档里, 是为**存档格式 (Level format)**。本章将对这些游戏数据的结构、编辑方法做具体的阐述。

6.1 概述

存档 (Level, 又称地图、世界) 是独立运行游戏世界的基本单位, 它以文件夹的形式存储于 `saves` 内。每个存档都可以自由移动, 因此可以在 `saves` 文件夹内增添或删除存档, 这为冒险地图的下载和安装提供了可能, 也允许玩家将他们的存档备份到其他地方。`saves` 的路径在小节 1.4.2 中已有说明, 其基本结构是:

- `saves`
 - `<存档名称>`: 一个存档。

其中存档文件夹的名称会显示在游戏存档页面的第二行, 文件夹的名称允许使用格式化代码以应用样式。例如, 下图所示存档的文件夹名称为 `$b$1跃动晶界$c2 $aby Mu_xian$f$sk`。



图 6.1 存档名称

存档文件夹的结构在 26.1-Snapshot-6 发生过变动, 自 26.1-Snapshot-6 起, 一个存档文件夹的基本结构如下所示:

- `<存档名称>`
 - `data`: 存档数据。
 - `<命名空间>`: 任意命名空间下的存档数据。
 - `maps`: 地图数据。
 - `last_id.dat`: 地图计数文件, 游戏在使用地图时, 会给每张独立的地图分配一个地图编号。
 - `<地图 ID>.dat`: 地图数据文件, 地图数据不在物品数据中存储, 而是在此处存储。使用地图计数文件存储的地图 ID。
 - `command_storage.dat`: 命令存储文件, 数据格式见小节 6.2.1。
 - `custom_boss_events.dat`: 存储自定义 Boss 栏的文件, 数据格式见小节 6.2.2。
 - `game_rules.dat`: 存储当前存档游戏规则的文件。
 - `random_sequences.dat`: 随机序列数据文件, 数据格式见小节 6.2.3。
 - `scheduled_events.dat`: 计划事件数据文件。
 - `scoreboard.dat`: 记分板数据文件, 数据格式见小节 7.2.1。这个文件内还有队伍相关的数据。
 - `stopwatches.dat`: 秒表数据文件, 数据格式见小节 6.2.4。
 - `wandering_trader.dat`: 流浪商人数据文件。
 - `weather.dat`: 天气数据文件, 数据格式见小节 6.2.5。
 - `world_clocks.dat`: 世界时钟数据文件。
 - `world_gen_settings.dat`: 存储世界生成设置的文件。
 - `datapacks`: 世界指定数据包。

- 📁 **<数据包名称>[.zip]**: 一个数据包。文件(夹)结构已在小节 1.5.1 给出, 其中文件的编写方式会在《数据包》教程详细给出。
- 📁 **dimensions**: 维度数据。
 - 📁 **<命名空间>**: 任意命名空间下的维度数据。
 - 📁 **<维度 ID>**: 一个维度, 可以添加路径。内容详见节 6.3。
 - 📁 **data**: 该维度的零散数据。
 - 📁 **minecraft**: 命名空间, 必须为 `minecraft`, 无论自定义维度的自身命名空间。
 - 📄 **chunk_tickets.dat**: 区块标签数据文件。
 - 📄 **raids.dat**: 袭击数据文件。
 - 📄 **ender_dragon_fight.dat**: 末影龙战斗数据文件, 此文件仅存在于末地或其他可发生末影龙战斗的维度。
 - 📄 **world_border.dat**: 该维度的世界边界数据文件。
 - 📁 **entities**: 该维度的实体数据。
 - 📄 **r.<x>.<z>.mca**: 区域文件。
 - 📄 **c.<x>.<z>.mcc**: 区域额外文件。
 - 📁 **poi**: 该维度的兴趣点数据。
 - 📄 **r.<x>.<z>.mca**: 区域文件。
 - 📄 **c.<x>.<z>.mcc**: 区域额外文件。
 - 📁 **region**: 该维度的区块基础数据。
 - 📄 **r.<x>.<z>.mca**: 区域文件。
 - 📄 **c.<x>.<z>.mcc**: 区域额外文件。
 - 📁 **generated**: 世界生成数据。
 - 📁 **<命名空间>**: 任意命名空间下的世界生成数据。
 - 📁 **structure**: 结构模板注册表。这部分数据的使用方法可见附录 I.2。
 - 📄 **<名称>.nbt**: 一个结构模板, 可以添加路径。
 - 📁 **players**: 玩家所有数据。
 - 📁 **advancements**: 存储玩家达成进度的文件夹。
 - 📄 **<玩家 UUID>.json**: 存储进入过该存档的此玩家达成的进度。
 - 📁 **data**: 存储基本的玩家数据的文件夹。
 - 📄 **<玩家 UUID>.json**: 一个进入过该存档的玩家的数据。
 - 📁 **stats**: 存储统计信息的文件夹。
 - 📄 **<玩家 UUID>.json**: 一个进入过该存档的玩家的统计信息数据, 这些数据会显示在菜单的统计信息页面。
 - 📁 **resourcepacks**: 世界指定资源包。
 - 📁 **resources.zip**: 世界指定资源包, 该资源包的内容仅在该存档可用。必须为 `.zip` 格式的压缩文件, 名称必须为 `resources.zip`。
 - 🖼️ **icon.png**: 存档的图标, 必须是 64 × 64 像素大小的图片。
 - 📄 **level.dat[_old]**: 世界全局信息数据文件。只有存在该文件时, 📁 **<存档名称>** 才会被识别为一个有效的存档。
 - 🔒 **session.lock**: 存档会话锁文件, 存储最后一次访问此存档的时间戳和访问权限。

例 6.1

将以下的图片设为存档的图标。

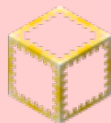


图 6.2

解

将这张图片命名为  `icon.png`，并将其放于路径 `<存档名称> > icon.png`。注意，这张图必须保证是 64×64 像素大小。

例 6.2

将一个资源包  跃动晶界跑酷地图专用资源包 做成存档  `$b$1跃动晶界$c2 $aby Mu_xian$f$K` 的世界专用资源包。

解

首先将文件夹形式的资源包转换成压缩文件，有效压缩文件形式的资源包根目录  跃动晶界跑酷地图专用资源包 和  `assets`、 `pack.mcmeta` 之间不能有额外的文件夹。

其次将压缩文件重命名为  `resources.zip`，并将其放在  `$b$1跃动晶界$c2 $aby Mu_xian$f$K > resourcepacks > resources.zip`。

 `level.dat` 和  `level.dat_old` 是存档的全局信息数据， `level.dat_old` 是数据的实时备份版本，当  `level.dat` 因为各种原因损坏时，这个文件能确保存档全局信息不丢失。在游戏数据发生变动时，两个文件的存在能确保安全更替。 `level.dat` 和  `level.dat_old` 的数据结构一致，都是 NBT 格式的文件，格式如下所示：

根标签

 **Data**: 存档的全局信息数据。

 **allowCommands**: 该存档是否启用命令。若此字段不存在，则检查字段  `GameType`，若其值为 `1`（创造模式），则此字段为 `true`，否则为 `false`。

 **DataPacks**: 存档启用和禁用的数据包。

 **Disabled**: 未启用的数据包列表。

 一个未启用的数据包名称。

 **Enabled**: 已启用的数据包列表，加载顺序为从上到下。

 一个已启用的数据包名称。

 **DataVersion**: 游戏数据版本。

 **difficulty_settings**: 游戏数据版本。

 **difficulty**: 游戏难度，有效值 `peaceful`（和平）、`easy`（简单）、`normal`（普通）、`hard`（困难），默认值为 `normal`。

 **hardcore**: 是否启用极限模式，控制玩家在死亡之后是否自动转变为旁观模式，默认值为 `false`。

 **locked**: 是否在此存档内锁定游戏难度，默认值为 `false`。

- I** **enabled_features**: 启用的功能开关。此字段不一定存在，若不存在，则仅启用 `vanilla` 功能。
 - "** 一项启用的功能开关，有效值有 `vanilla`（原版）、`trade_rebalance`（村民交易平衡性调整）、`redstone_experiments`（红石实验性内容）和 `minecart_improvements`（矿车改进）。
- I** **GameType**: 该存档的默认游戏模式，有效值 `0`（生存模式）、`1`（创造模式）、`2`（冒险模式）、`3`（旁观模式），默认值为 `0`。
- F** **initialized**: 存档是否被正确初始化并生成奖励箱，默认值为 `true`。
- L** **LastPlayed**: 上次保存游戏的时间戳，默认值为 `0`。
- "** **LevelName**: 存档名称，允许使用格式化代码。
- I** **singleplayer_uuid**: 单人游戏的所有者玩家 UUID，只在单人游戏使用，专用服务器不使用这个字段。
- I** **removed_features**: 用于崩溃中的 `Removed feature flags` 部分记录。
 - "** 一条记录。
- I** **ServerBrands**: 打开过此存档的服务端的铭牌列表。
 - "** 一个服务端铭牌。
- O** **spawn**: 世界出生点数据。
 - "** **dimension**: 世界出生点的维度，默认值为 `minecraft:overworld`（主世界）。
 - I** **pos**: 世界出生点的坐标，依次为 x 、 y 、 z 坐标，默认值为 `[I;0,0,0]`。
 - F** **pitch**: 在世界出生点出生时玩家的俯仰角，默认值为 `0.0f`。
 - F** **yaw**: 在世界出生点出生时玩家的偏航角，默认值为 `0.0f`。
- L** **Time**: 存档的游戏时间，单位为游戏刻，默认值为 `0`。
- I** **version**: 存档区块文件的版本，对于 Anvil 文件格式（当前）为 `19133`，对于 MCRegion 格式为 `19132`。
- O** **Version**: 该存档的详细版本信息。
 - I** **Id**: 游戏数据版本。
 - "** **Name**: 游戏版本名称。
 - "** **Series**: 开发系列，正式版和快照为 `main`。
 - F** **Snapshot**: 此版本是否为快照。
- F** **WasModded**: 存档是否被修改过的客户端或服务端加载并保存，默认值为 `false`。

6.2 存档数据

自 26.1-Snapshot-6 起，存档数据，即位于存档文件夹 `data` 中的数据都按照带命名空间父目录的格式存储。但是，大部分数据都只存储在 `minecraft` 命名空间，无论这些资源本身使用的命名空间。

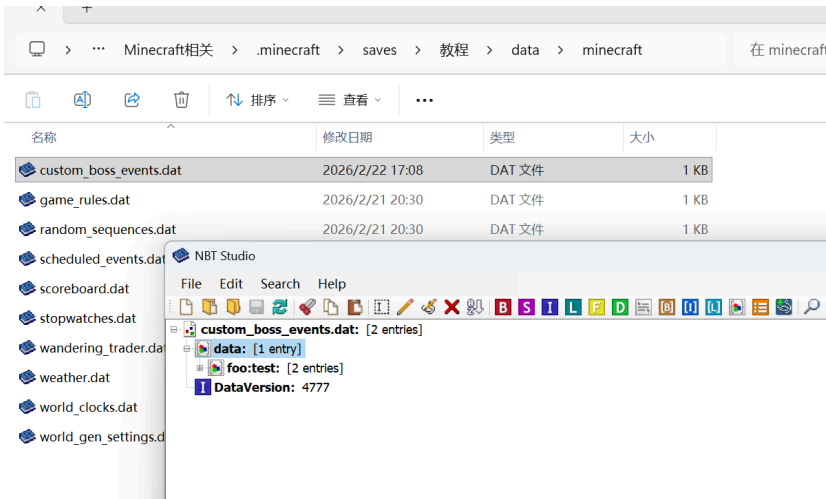


图 6.3 Boss 栏 `foo:test` 的命名空间为 `foo`，但其数据存储于 `minecraft` 命名空间

6.2.1 命令存储

对于大部分的游戏数据，玩家可以修改这些字段的值，但不能自由添加或删去这些字段，而命令存储（Command storage）允许玩家在文件中写入、修改任意的数据，便于存储、处理适用于全局游戏的自定义数据。它与方块实体、实体构成三种能使用命令访问的数据。使用命令存储数据可以摆脱方块实体和实体的限制，可以制定任意的标签、数据类型。每一个命令存储由不同的命名空间 ID 区分。

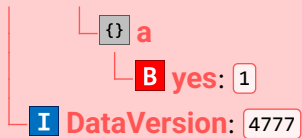
命令存储文件位于 `<存档名称> > data > <命名空间> > command_storage.dat`，这里的 `.dat` 文件是 NBT 格式的文件。`<命名空间>` 即为命令存储的命名空间（不包含 ID），默认为 `minecraft`，允许存在多个不同命名空间的 `.dat` 文件。一个 `command_storage.dat` 文件的数据树如下所示：

- 根标签
 - `data`: 存储该命名空间下不同 ID 的命令存储数据。
 - `contents`: 允许存在多个不同的标签 `<ID>` 以支持同一个命名空间下的不同 ID。
 - `<ID>`: 标签名为一个命令存储的 ID。命令 `/data` 可以读取、编辑或写入一个命令存储，这些数据都会被存储到标签 `<ID>` 内。如果一个命名空间 ID 的命令存储地址不存在，则会自动创建该命名空间 ID。因此一个命令存储文件下可以包含多个相同命名空间而 ID 不同的具体的命令存储。此时一般称这个标签为拥有该命名空间 ID 的命令存储的根标签。
 - 任意 NBT 数据
 - `DataVersion`: 游戏数据版本。

例 6.3

一个命令存储文件 `data > tutorial > command_storage.dat` 的数据结构如下所示：

- 根标签
 - `data`
 - `contents`



回答下列问题:

- (1) 该命令存储的命名空间 ID 为_____。
- (2) 假设该命令存储的初始值为空, 若要使该命令存储的所存储的数据如上所示, 写出可行的命令。

解

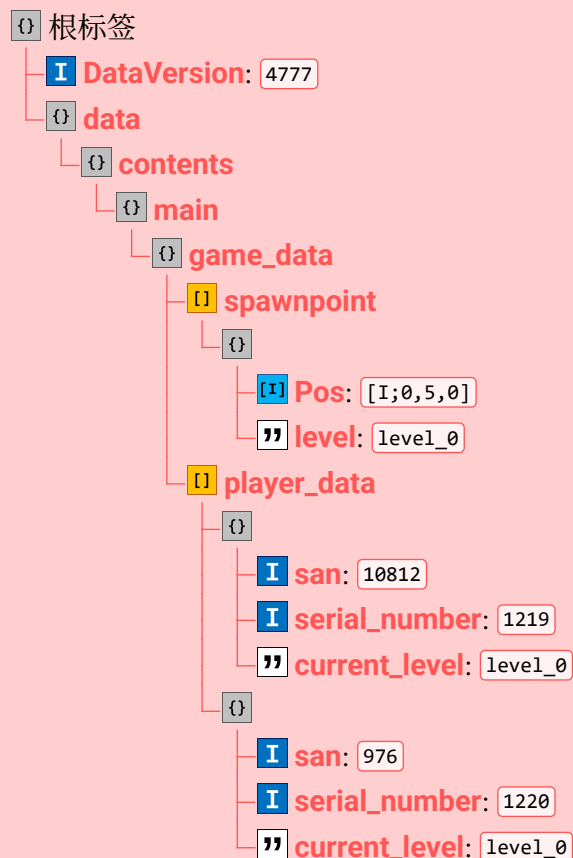
- (1) 此命令存储的命名空间为 `tutorial`, 而 ID 为标签 `{}` `contents` 子标签的标签名。所以该命令存储的命名空间 ID 为 `tutorial:a`。
- (2) 写入一个命令存储需要命令 `/data merge` 或 `/data modify`, 注意命令存储 `tutorial:a` 的数据从标签 `{}` `a` 开始。使用 `/data merge` 的命令可以为

```
data merge storage minecraft:a {yes:1b}
```

6.1

例 6.4

在一张后室主题的冒险地图中, 命令存储文件 `data > the_backrooms > command_storage.dat` 的数据如下所示:



其中复合标签的列表 `I spawnpoint` 存储了后室各层级的玩家出生点，子标签 `I Pos` 按 x 、 y 、 z 坐标的顺序存储了出生点坐标，`I level` 是该出生点坐标所处的层级。`I player_data` 存储了与玩家有关的数据，其中的每个复合标签均为一个具体玩家的数据：`I san` 是玩家当前的 SAN 值，`I serial_number` 是该玩家的编号，`I current_level` 是该玩家当前所处的层级。按要求对其中的命令存储内容进行操作：

- (1) 获取编号为 1219 的玩家的 SAN 值；
- (2) 将层级 Level 0（命令存储格式中为 `level_0`）的玩家出生点 y 坐标改为 10。

解

- (1) 显然获取值的操作需要使用命令 `/data get`。编号为 1219 的玩家数据可以从数据树上找到：

```

{
  I san: 10812
  I serial_number: 1219
  I current_level: level_0

```

这个复合标签是列表 `I player_data` 的第一个元素，该玩家的 SAN 值也可以从这个复合标签中获取。虽然使用节点 `player_data[0]` 可以直接定位到这个元素，但是这样的做法不值得提倡，因为在数据的处理过程中无法保证这个复合标签一定位于列表的第 1 个。因此不妨使用当前列表或数组中的复合标签元素，节点写成 `player_data[{serial_number:1219}]`，这样无论该复合标签在列表的什么位置，都可以精确地定位到这个复合标签。

`/data get` 中的完整 NBT 路径为 `game_data.player_data[{serial_number:1219}].san`，而命令存储的命名空间 ID 为 `backrooms:main`，因此完整的命令为

```
data get storage backrooms:main
game_data.player_data[{serial_number:1219}].san
```

6.2

- (2) 该小题适用命令 `/data modify`。由题意，Level 0 的玩家出生点数据存储在下述的数据树中：

```

{
  I Pos: [I;0,5,0]
  I level: level_0

```

同第 (1) 小题之理，定位这个复合标签时也应该使用当前列表或数组中的复合标签元素。定位到 `I Pos` 中的 y 坐标时，由于题目已经规定了三个坐标值在数组中的顺序，因此可直接使用节点 `Pos[1]`。完整的命令为：

```
data modify storage backrooms:main
game_data.spawnpoint[{level:"level_0"}].Pos[1] set value 10
```

6.3

命令存储一般用于存储全局性的数据,因为它自身不具备将数据绑定至各个玩家的能力,因此在实际编写时必须手动维护一套将数据映射至各个玩家的算法。显然记分板更适用于存储玩家各自的数据。

6.2.2 Boss 栏

Boss 栏 (Boss bar) 是显示在 HUD^①顶端中央的一个可被填充的条式图形,通常用于显示末影龙和凋零的血量。除了用于显示 Boss 的血量,Boss 栏在多人游戏和冒险地图中也被用于显示倒计时、游戏进展等。如果设计得当,可以用 Boss 栏制造出很好的效果。

一个 Boss 栏拥有一个**最大值**和一个**当前值**,这两个值决定了 Boss 栏被填充的比例。在 GUI 和 HUD 大小不变的情况下,一个 Boss 栏的长度是不变的,只有 Boss 栏中填充的内容发生变化,填充的比例为当前值与最大值的比。例如,最大值为 20、当前值为 10 的 Boss 栏填充占比为 50%。当前值不必比最大值小,若当前值大于或等于最大值,则 Boss 栏中的填充固定为 100%。

一、命令 `/bossbar`

手动创建和修改 BOSS 栏可以通过命令 `/bossbar` 完成,它需要的权限等级为 2,下面是命令 `/bossbar` 的所有用法:

1. 添加一个 Boss 栏,语法为

```
bossbar add <id> <name>
```

6.4

其中的参数: `<id>` (命名空间 ID `minecraft:resource_location`) —— 被添加 Boss 栏的命名空间 ID,可自由指定。若不填写命名空间,则默认命名空间为 `minecraft`。Boss 栏的 ID 在系统内部使用,作为与其他 Boss 栏区分的凭证。

`<name>` (文本组件 `minecraft:component`) —— Boss 栏的显示名称,必须为文本组件。显示名称会显示在 HUD 的 Boss 栏上方。

2. 查询一个 Boss 栏的信息,语法为

```
bossbar get <id> (max|players|value|visible)
```

6.5

其中的参数: `(max|players|value|visible)` —— 查询的信息,有效值如下。

`max`: Boss 栏的最大值。

`players`: 这个 Boss 栏对哪些玩家显示。

`value`: Boss 栏的当前值。

`visible`: Boss 栏是否可见。

3. 列出所有的 Boss 栏,语法为

```
bossbar list
```

6.6

^① 即平视显示器 (Hheads-up display), 是叠加在游戏视野上的画面。

4. 移除一个 Boss 栏，语法为

```
bossbar remove <id>
```

6.7

5. 编辑 Boss 栏

(1) 编辑 Boss 栏填充的颜色，语法为

```
bossbar set <id> color (blue|green|pink|purple|red|white|yellow)
```

6.8

其中的参数：(blue|green|pink|purple|red|white|yellow) —— 填充的颜色。

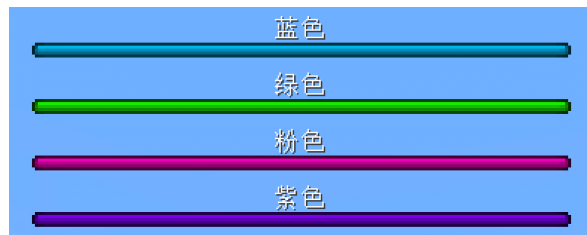


图 6.4 Boss 栏的不同颜色

(2) 编辑 Boss 栏的最大值，语法为

```
bossbar set <id> max <max>
```

6.9

其中的参数：<max> (整型 brigadier:integer) —— 设置的最大值，默认值为 100。

(3) 编辑 Boss 栏的显示名字，语法为

```
bossbar set <id> name <name>
```

6.10

(4) 编辑 Boss 栏可被哪些玩家看见，语法为

```
bossbar set <id> players [<targets>]
```

6.11

其中的参数：[<targets>] (实体 minecraft:entity) —— 可选，指定 Boss 栏可以被哪些玩家看到，必须指定玩家。若不定义，则对所有玩家显示此 Boss 栏。

(5) 编辑 Boss 栏显示的样式，语法为

```
bossbar set <id> style (notched_6|notched_10|notched_12|notched_20|progress)
```

6.12

其中的参数：(notched_6|notched_10|notched_12|notched_20|progress) —— 显示样式，有效值如下。

notched_6：分为 6 段。

notched_10：分为 10 段。

notched_12：分为 12 段。

notched_20：分为 20 段。

progress：不做任何分段，为默认样式。



图 6.5 Boss 栏的不同样式

(6) 编辑 Boss 栏的当前值，语法为

```
bossbar set <id> value <value>
```

6.13

其中的参数：<value>（整型 `brigadier:integer`）——设置的当前值。

(7) 编辑 Boss 栏的可见性，语法为

```
bossbar set <id> visible <visible>
```

6.14

其中的参数：<visible>（布尔值 `brigadier:bool`）——设置 Boss 栏是否可见。

二、Boss 栏 NBT 格式 *

Boss 栏数据被存储在 `data > minecraft > custom_boss_events.dat`。无论 Boss 栏自身的命名空间为何，`custom_boss_events.dat` 只存储于 `minecraft` 命名空间。这部分数据格式如下：

- 根标签
 - data**: 存储的多个自定义 Boss 栏数据。
 - <Boss 栏命名空间 ID>**: 一项 Boss 栏。
 - Color**: Boss 栏的填充颜色，有效值 `blue`、`green`、`pink`、`purple`、`red`、`white`、`yellow`。
 - CreateWorldFog**: 该 Boss 栏是否使天空暗淡。
 - DarkenScreen**: 该 Boss 栏是否创建迷雾效果。
 - Max**: Boss 栏最大值。
 - Name**: Boss 栏的显示名字，是一个文本组件。
 - Overlay**: Boss 栏的显示样式，有效值 `notched_6`、`notched_10`、`notched_12`、`notched_20`、`progress`。
 - PlayBossMusic**: 是否播放 Boss 音乐。
 - Players**: 可以看见该 Boss 栏的玩家列表。
 - 一个玩家的 UUID。
 - Value**: Boss 栏当前值。
 - Visible**: 该 Boss 栏是否对玩家可见。
 - DataVersion**: 游戏数据版本。

6.2.3 随机序列

随机序列 (Random sequences) 是游戏中可控的随机数发生器，在游戏中的应用主要在战利品表上，如实体、方块的掉落物、猪灵的以物易物等。

一、命令 `/random`

`/random` 是用于生成随机数以及修改随机序列的命令，以下是所有用法。

1. 获取一个随机数

```
random (value|roll) <range> [<sequence>]
```

6.15

其中的参数： `(value|roll)` —— 产生随机数时，设为 `value` 会将结果仅显示给执行命令的玩家；设为 `roll` 会将结果通知给所有玩家。

`<range>`（整数范围 `minecraft:int_range`）—— 生成的随机数在该参数指定的范围内，范围可以包含负数。此范围能够产生的数值个数必须介于 `2` 和 `2147483646` 之间（含），比如参数 `1` 只能产生 `1` 这个随机数，因此无效；参数 `1..2` 能产生 `1`、`2` 两个随机数，因此是有效的。

`[<sequence>]`（命名空间 ID `minecraft:resource_location`）—— 可选。每一个随机数的产生都使用了一个随机序列，这些序列是独一无二的，因此可以自定义命名空间 ID 来指定这些序列，这样就可以用命名空间 ID 唯一地指定特定地序列。如果指定的序列不存在，则游戏会现场创建一个随机序列使用。如指定了该参数，则命令 `/random` 所需权限等级为 `2`；不指定则为 `0`。

2. 重制随机数规则

```
random reset (*|<sequence>) [<seed>] [<includeWorldSeed>]
[<includeSequenceId>]
```

6.16

其中的参数： `(*|<sequence>)` —— 指定要重制规则的随机数序列。若设为 `*`，则重制所有的随机序列；若使用参数 `<sequence>`，则是要重制规则的随机数序列的命名空间 ID。

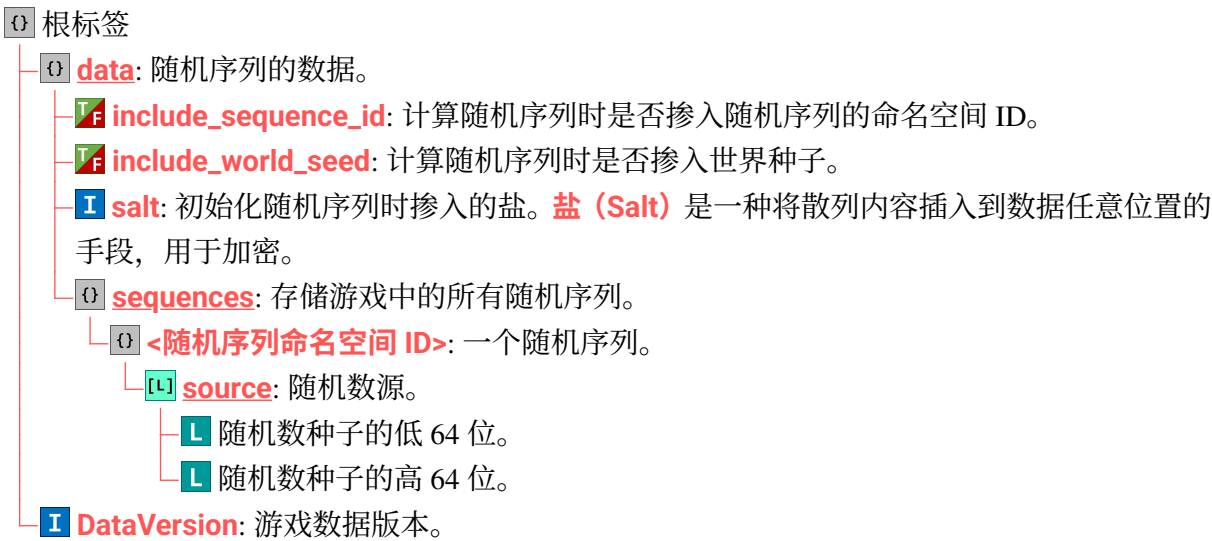
`[<seed>]`（长整型 `brigadier:long`）—— 用于重制随机序列的种子。在同一个序列中使用相同的种子得到的随机数是一样的。

`[<includeWorldSeed>]`（布尔值 `brigadier:bool`）—— 可选，用于决定在重制随机序列的种子中是否掺入世界种子。

`[<includeSequenceId>]`（布尔值 `brigadier:bool`）—— 可选，用于决定在重制随机序列的种子中是否掺入随机序列 ID。

二、随机序列数据文件 *

随机序列的数据存储于 `data > minecraft > random_sequences.dat`。无论随机序列自身的命名空间为何，`random_sequences.dat` 只存储于 `minecraft` 命名空间。随机序列的数据格式为



6.2.4 秒表

秒表 (Stopwatch) 用于记录真实的时间流逝而非游戏刻，因此秒表的计时与游戏刻并不完全一致。计时的单位为**毫秒**。只要服务端开启，这些秒表就会开始计时。只有当服务端被关闭时，秒表才会停止计时。因此，以下情况都会让秒表持续计时：

- 1. 在单人或多人游戏进入游戏暂停界面。
- 2. 服务端发生掉刻。
- 3. 使用 `/tick freeze` 冻结游戏刻。
- 4. 没有玩家在独立服务端内进行游戏。
- 5. 计算机休眠。

一、命令 `/stopwatch`

命令 `/stopwatch` 用于管理游戏中存在的秒表，它需要的权限等级为 2，以下是所有用法。

- 1. 创建一个新的秒表

`stopwatch create <id>`

6.17

其中的参数：`<id>`（命名空间 ID `minecraft:resource_location`）——秒表的命名空间 ID。如果指定的秒表不存在，则游戏会现场创建一个秒表使用。

- 2. 查询特定秒表的时间

`stopwatch query <id> [<scale>]`

6.18

其中的参数: `[<scale>]` (双精度浮点数 `brigadier:double`) —— 可选, 将返回的值进行缩放的倍率。缩放操作为: 先乘以此值, 再向下取整。

此命令查询得到的时间单位为秒, 但精确到小数点后 3 位, 所以实际上能够得到精确的毫秒数据。

3. 重置指定的秒表

```
stopwatch restart <id>
```

6.19

此命令会让指定命名空间 ID 的秒表归零, 但由于它不能让秒表停止计时, 因此秒表归零后会立即开始从零计时。

4. 移除指定的秒表

```
stopwatch remove <id>
```

6.20

二、秒表数据文件 *

秒表的数据存储于 `data > minecraft > stopwatches.dat`。无论秒表自身的命名空间为何, `stopwatches.dat` 只存储于 `minecraft` 命名空间。秒表的数据格式为

-  根标签
 -  **data**: 秒表的数据。
 -  **stopwatches**: 存储游戏中的所有秒表。
 -  **<秒表命名空间 ID>**: 一个秒表的当前计时, 单位为毫秒。
 -  **DataVersion**: 游戏数据版本。

6.2.5 天气

天气 (Weather) 是游戏中的全局性事件, 所有维度的天气会保持一致。游戏中一共有 3 种天气: 晴天、降雨和雷暴。

一、命令 `/weather`

命令 `/weather` 可用于直接更改游戏内天气, 也可用于重置降雨计时器和雷暴计时器, 所需权限等级为 2, 语法为:

```
weather (clear|rain|thunder) [<duration>]
```

6.21

其中的参数: `(clear|rain|thunder)` —— 用于指定天气为晴天、雨天 (温度值低于 0.15 的区域会下雪) 或雷暴。

`[<time>]` (时间 `minecraft:time`) —— 可选, 指定天气的持续时间, 格式为: `<单精度浮点数>[<单位>]`, 单位可以为: `t` (游戏刻), `s` (秒) 或 `d` (游戏日), 无单位默认为游戏刻。如不填写, 则按指定的天气随机取值: 对于 `clear`: 随机取 `12000` 到 `180000` 之间 (含) 的值。

对于 `rain`：随机取 `12000` 到 `24000` 之间（含）的值。

对于 `thunder`：随机取 `3600` 到 `15600` 之间（含）的值。

使用 `/weather` 改变天气会导致降雨计时器和雷暴计时器被重置为相同的值，因此晴天过后总是为雷暴。

若游戏规则 `advance_weather` 为 `false`，则游戏中的天气只能通过此命令更改。

二、天气数据文件 *

天气的数据存储于  `data > minecraft > weather.dat`，数据格式为

 根标签

 **data**: 天气的数据。

 **clear_weather_time**: 存档内世界晴天剩余时间，默认值为 `0`。

 **raining**: 当前世界是否为降雨天气，默认值为 `false`。

 **rain_time**: 如果世界当前不处于降雨天气，此值代表距离下一次降雨的时间；如果当前正处于降雨天气，此值代表距离降雨结束的时间。

 **thundering**: 当前世界是否为雷暴天气，默认值为 `false`。

 **rain_time**: 如果世界当前不处于雷暴天气，此值代表距离下一次雷暴的时间；如果当前正处于雷暴天气，此值代表距离雷暴结束的时间。

 **DataVersion**: 游戏数据版本。

6.3 维度与区域文件

一个游戏世界存在多个维度，因此游戏存储数据时首先需要对维度进行区分。Minecraft 原版存在三个维度：主世界、下界和末地，使用数据包可以添加一些自定义维度。所有维度的数据都按该维度的命名空间 ID 存储于  `<存档名称> > dimensions > <命名空间> > ... > <ID>` 中。

维度的命名空间 ID 允许使用任意的路径。例如，`minecraft:overworld`（主世界）维度的文件夹路径为

```

dimensions
├── minecraft
│   └── overworld
    
```

自定义维度 `the_backrooms:level_0/red_rooms` 的文件夹路径为

```

dimensions
├── the_backrooms
│   ├── level_0
│   │   └── red_rooms
    
```

维度数据可以分为区块数据和零散数据，其中存储区块信息的文件被称为**区域文件 (Region file)**，或称**Anvil 文件 (Anvil file)**，这些文件均使用 NBT 格式，其后缀为 `.mca`。Anvil 文件以 `r.<x>.<z>.mca` 的名字命名，一个 Anvil 文件存储了一个**区域 (Region)** 内所有区块的相关信息，

其中一个区域包含 32×32 个区块。如果一个区域的信息量过大，则游戏会创建一个对应的区域额外文件，这个文件被命名为  `c.<x>.<z>.mcc`。游戏存储区块数据时，不会将这个区块的所有数据都存储在一个 Anvil 文件中，而是将这些数据拆分为区块基础数据、实体数据和兴趣点数据，并将它们分配到不同的文件夹内存储。其中  `region` 文件夹存储区块全局信息； `entities` 文件夹存储实体信息； `poi` 文件夹存储兴趣点信息。维度零散数据则存储于  `data` 中。因此，完整的维度数据文件结构如下所示：

 **dimensions**: 维度数据。

└  **<命名空间>**: 任意命名空间下的维度数据。

└  **<维度 ID>**: 一个维度，可以添加路径。

└  **data**: 该维度的零散数据。

└  **minecraft**: 命名空间，必须为 `minecraft`，无论自定义维度的自身命名空间。

└  **chunk_tickets.dat**: 区块标签数据文件。

└  **raids.dat**: 袭击数据文件。

└  **ender_dragon_fight.dat**: 末影龙战斗数据文件，此文件仅存在于末地或其他可发生末影龙战斗的维度。

└  **world_border.dat**: 该维度的世界边界数据文件。

└  **entities**: 该维度的实体数据。

└  **r.<x>.<z>.mca**: 区域文件。

└  **c.<x>.<z>.mcc**: 区域额外文件。

└  **poi**: 该维度的兴趣点数据。

└  **r.<x>.<z>.mca**: 区域文件。

└  **c.<x>.<z>.mcc**: 区域额外文件。

└  **region**: 该维度的区块基础数据。

└  **r.<x>.<z>.mca**: 区域文件。

└  **c.<x>.<z>.mcc**: 区域额外文件。

6.3.1 Anvil 文件

对于一个 Anvil 文件  `r.<x>.<z>.mca`，文件名中的 x 和 z 均为常量 ($x, z \in \mathbb{Z}$)，现定义 (x, z) 为  `r.<x>.<z>.mca` 所存储区域 Ω 的区域坐标，记该区域内某个区块的(绝对)区块坐标为 $[x_c, z_c]$ ($x_c, z_c \in \mathbb{Z}$)，则满足

$$\begin{cases} 32x < x_c < 32x + 31 \\ 32z < z_c < 32z + 31 \end{cases} \quad (6.1)$$

因此有

$$\begin{cases} x = \lfloor x_c / 32 \rfloor \\ z = \lfloor z_c / 32 \rfloor \end{cases} \quad (6.2)$$

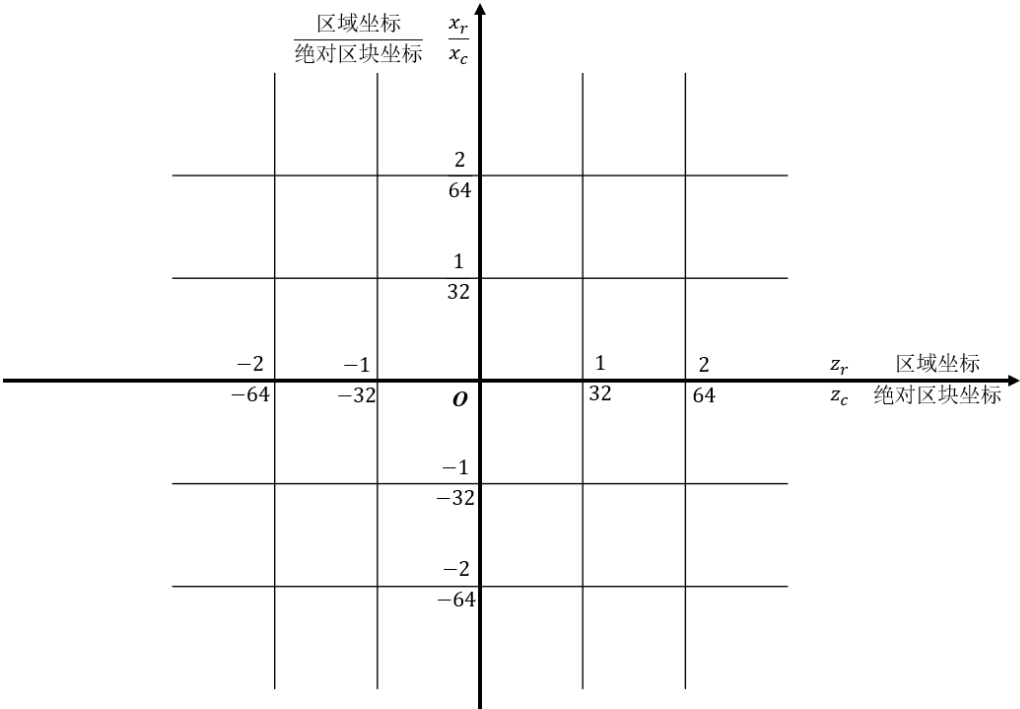


图 6.6 区域和区块坐标，其中每一个方格均代表一个区域，内包含 32×32 个区块

根据这个式子，仅凭一个区块坐标就能找到存储该区块或方块的 Anvil 文件和该区块所在的标签。

例 6.5

已知一个区块的区块坐标为 $[312, -109]$ ，计算这个区块的数据所处的区域文件。

解

直接代入式 (6.2)，得 $x = \lfloor 312/32 \rfloor = 9$ ， $z = \lfloor -109/32 \rfloor = -4$ ，因此区域坐标为 $(9, -4)$ ，相应的区域文件为  `r.9.-4.mca`。

由于区块位于区域 Ω 内，其数据都被存储在  `r.<x>.<z>.mca` 内，而这些 Anvil 文件又是相互独立的，内部的标签名可以相同。因此可以不必使用绝对的区块坐标，仅表示该区块位于区域 Ω 内的位置即可。现定义 $[x'_c, z'_c]$ ($x'_c, z'_c \in \mathbb{Z}$) 为相对于区域 Ω 的区块坐标，称为相对区块坐标，则满足

$$\begin{cases} x_c - 32x = x'_c \\ z_c - 32z = z'_c \end{cases} \tag{6.3}$$

将式 (6.2) 代入，得相对区块坐标

$$\begin{cases} x'_c = x_c - 32\lfloor x_c/32 \rfloor \\ z'_c = z_c - 32\lfloor z_c/32 \rfloor \end{cases} \tag{6.4}$$

如果使用  NBTStudio 打开 Anvil 文件，可以看到其中有很多名为  `Chunk  $[x'_c, z'_c]$  in world at  $(x_c, z_c)$` 的复合标签，一个复合标签存储一个区块的信息，其中该复合标签存储区块的绝对区块坐标为 $[x_c, z_c]$ ，相对区块坐标为 $[x'_c, z'_c]$ 。

对于任意一个二维的方块坐标 (x_0, z_0) ，将式 (2.16) 的计算结果代入式 (6.2)，它所在的区域坐标可计算为：

$$\begin{cases} x = \lfloor \lfloor x_0/16 \rfloor / 32 \rfloor \\ z = \lfloor \lfloor z_0/16 \rfloor / 32 \rfloor \end{cases} \quad (6.5)$$

将式 (2.16) 代入式 (6.4), 相应地、该方块坐标所在区块的在区域内的相对区块坐标 $[x'_c, z'_c]$ 为

$$\begin{cases} x'_c = \lfloor x_0/16 \rfloor - 32 \lfloor \lfloor x_0/16 \rfloor / 32 \rfloor \\ z'_c = \lfloor z_0/16 \rfloor - 32 \lfloor \lfloor z_0/16 \rfloor / 32 \rfloor \end{cases} \quad (6.6)$$

例 6.6

已知一个方块的方块坐标为 (3247, 98, -1030), 求存储该方块的数据的 Anvil 文件名和区块标签名

解

首先代入式 (2.16), 计算此方块坐标所属绝对区块坐标

$$x_c = \lfloor 3247/16 \rfloor = 202, \quad z_c = \lfloor -1030/16 \rfloor = -65$$

接下来代入式 (6.5), 计算此方块坐标所属区域坐标

$$x = \lfloor \lfloor 3247/16 \rfloor / 32 \rfloor = 6, \quad z = \lfloor \lfloor -1030/16 \rfloor / 32 \rfloor = -3$$

因此对应的 Anvil 文件名为  `r.6.-3.mca`。

又根据式 (6.6), 可得局部区块坐标

$$x'_c = \lfloor 3247/16 \rfloor - 32 \lfloor \lfloor 3247/16 \rfloor / 32 \rfloor = 10$$

$$z'_c = \lfloor -1030/16 \rfloor - 32 \lfloor \lfloor -1030/16 \rfloor / 32 \rfloor = 31$$

于是存储区块的复合标签的标签名为 `Chunk [10,31] in world at (202,-65)`。

6.3.2 维度数据 *

一个维度拥有以下的数据格式。

一、区块基础数据

区块基础数据存储的是这个区块的方块、地形、高度图、光照等基本信息。存储位置为  `region`, 例 6.6 所述的 Anvil 文件路径就为  `dimensions > <命名空间> > <维度ID> > region > r.6.-3.mca`。

区块基础数据的格式如下:

根标签

 **below_zero_retrogen**: 仅在 1.18 之前的世界更新为 1.18 及之后的世界时使用, 用于在高度为负的区域生成方块。

 **missing_bedrock**: 从区块内部 (0,0) 位置起先沿 x 轴正方向、再沿 z 轴正方向遍历所有水平位置以检查是否缺失基岩, 从而决定是否重新生成此位置下的方块。由于一个区块有 256 个可用的水平位置, 每一个位置都用一个二进制位表示是否缺失基岩, 这些二进制位会被转换为 4 个长整型整数, 依次存入该数组内。

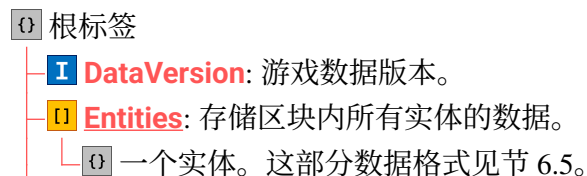
- └─ **target_status**: 重新生成方块时区块的目标状态, 可用值有 **empty** (尚未进行区块生成)、**structure_starts** (生成结构范围)、**structure_references** (计算结构引用)、**biomes** (填充生物群系)、**noise** (填充初始噪声地形)、**surface** (应用表面规则)、**carvers** (地形雕刻)、**features** (生成地物)、**initialize_light** (初始化光照计算)、**light** (计算初始光照)、**spawn** (生成初始实体) 和 **full** (区块生成完毕)。
- └─ **blending_data**: 用于新旧区块平滑过渡的混合数据。
 - └─ **heights**: 混合数据中元胞的高度值, 有 16 个值。
 - └─ **D** 一个高度值。
 - └─ **max_section**: 混合数据中最高区段的 y 坐标。
 - └─ **min_section**: 混合数据中最低区段的 y 坐标。
- └─ **block_entities**: 存储区块内所有方块实体信息。
 - └─ **Entity** 一个方块实体。这部分数据格式见节 6.4。
 - └─ 方块实体格式
- └─ **block_ticks**: 存储区块中的方块计划刻。
 - └─ **BlockTick** 一项方块计划刻。
 - └─ **i**: 方块的命名空间 ID。
 - └─ **p**: 计划刻的处理优先级, 该值越低, 则此计划刻会被优先处理。
 - └─ **t**: 此计划刻将要执行的倒计时。
 - └─ **x**: 方块的 x 坐标。
 - └─ **y**: 方块的 y 坐标。
 - └─ **z**: 方块的 z 坐标。
- └─ **carving_masks**: 区块雕刻时的标记, 仅在区块生成时使用。位置使用 YZX 编码。YZX 编码的计算方式为 $y \ll 8 | z \ll 4 | x$ 。
- └─ **DataVersion**: 游戏数据版本。
- └─ **entities**: 生成实体时使用的信息, 实体生成完毕后该字段被删除。
 - └─ **Entity** 一个实体。这部分数据格式见节 6.5。
 - └─ 实体格式
- └─ **fluid_ticks**: 存储区块中的流体计划刻。
 - └─ **FluidTick** 一项流体计划刻。
 - └─ **i**: 流体的命名空间 ID。
 - └─ **p**: 计划刻的处理优先级, 该值越低, 则此计划刻会被优先处理。
 - └─ **t**: 此计划刻将要执行的倒计时。
 - └─ **x**: 流体的 x 坐标。
 - └─ **y**: 流体的 y 坐标。
 - └─ **z**: 流体的 z 坐标。
- └─ **HeightMaps**: 从区块内部 (0,0) 位置起先沿 x 轴正方向、再沿 z 轴正方向遍历所有水平位置以存储区块的高度图信息。
 - └─ **MOTION_BLOCKING**: 最高的能阻挡移动的方块, 包括流体方块。
 - └─ **MOTION_BLOCKING_NO_LEAVES**: 最高的能阻挡移动的方块, 不包括树叶。
 - └─ **OCEAN_FLOOR**: 最高的能阻挡移动的非流体方块。

- L OCEAN_FLOOR_WG**: 最高的能阻挡移动的非流体方块，仅用于世界生成，生成完毕即被删除。
 - L WORLD_SURFACE**: 最高的非空气方块。
 - L WORLD_SURFACE_WG**: 最高的非空气方块，仅用于世界生成，生成完毕即被删除。
- L InhabitedTime**: 所有玩家在此区块停留的总时间之和，用于区域难度计算。
- Tf isLightOn**: 区块是否已经正常完成光照计算。
- L LastUpdate**: 此区块最后一次保存时的游戏时间。
- I PostProcessing**: 存储区块生成完毕后需要进行更新的位置。
 - I** 一个区段内需要进行更新的位置，存储区段的顺序从低到高。
 - S** 一个需要更新的方块的位置，使用 ZYX 编码。ZYX 编码计算同上文 YZX 编码的计算方式类似。
- I sections**: 区块中所有区段的信息。
 - I** 一个区段的信息。
 - I biome**: 此区段的生物群系信息。
 - [L] data**: 以元胞为单位存储生物群系信息。存储方式为建立“调色板”以保存方块状态与数字的映射，即以下的 **I palette** 字段，**I palette** 内元素的索引作为对应生物群系的 ID，此 ID 仅在本子区块有效。并将数字按照 YZX 编码按顺序存储到此字段。为节省空间，如果该区段只存在一种生物群系，则此项不存在。
 - I palette**: 生物群系的集合。
 - ”** 一个生物群系的命名空间 ID。
 - I block_states**: 此区段的方块状态信息。
 - [L] data**: 存储区段内所有方块的方块状态。存储方式与上文生物群系存储方式完全一致。如果该区段只存在一种方块状态，则此项不存在。
 - [L] palette**: 方块状态的集合。
 - I** 一个方块状态。
 - ” Name**: 方块的命名空间 ID。
 - I Properties**: 可选，由若干方块属性组成的方块状态。
 - ” <方块属性>**: 标签名为方块状态的属性，值使用字符串表示。
 - [B] BlockLight**: 存储区段内所有方块光照亮度信息。使用 YZX 编码。
 - [B] SkyLight**: 存储区段内所有天空光照亮度信息。使用 YZX 编码。
 - B Y**: 区段的 y 坐标。
 - Tf shouldSave**: 在加载区块后是否要标记此区块已被修改。
 - ” Status**: 存储该区块当前在世界生成中的状态。可用值有 **empty**（尚未进行区块生成）、**structure_starts**（生成结构范围）、**structure_references**（计算结构引用）、**biomes**（填充生物群系）、**noise**（填充初始噪声地形）、**surface**（应用表面规则）、**carvers**（地形雕刻）、**features**（生成地物）、**initialize_light**（初始化光照计算）、**light**（计算初始光照）、**spawn**（生成初始实体）和 **full**（区块生成完毕）。
 - I structures**: 存储该区块内的结构。
 - I References**: 包含 **I starts** 中结构的区块坐标。
 - [L] <结构命名空间 ID>**: 包含此结构生成点的区块坐标。
 - I starts**: 未生成完毕的结构，生成完毕后该字段会被删除。



二、实体数据

实体数据存储的是这个区块的实体信息。存储位置为 `entities`，文件内的数据格式如下：

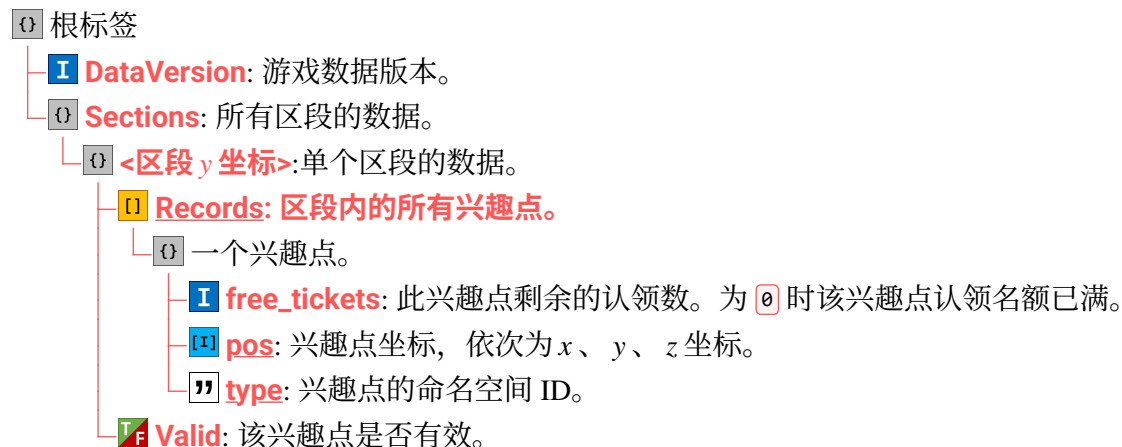


└─ 实体格式

❏ **Position**: 该区块的区块坐标, 依次为 x 坐标、 z 坐标。

三、兴趣点数据

兴趣点 (Point of Interest, POI) 是指对部分实体有吸引力的一个点, 通常由单个方块构成。兴趣点方块可以是村民的工作站方块, 也可以是蜜蜂的工作方块、避雷针等。兴趣点数据存储于 `poi` 文件夹内的 Anvil 文件, 其数据格式为



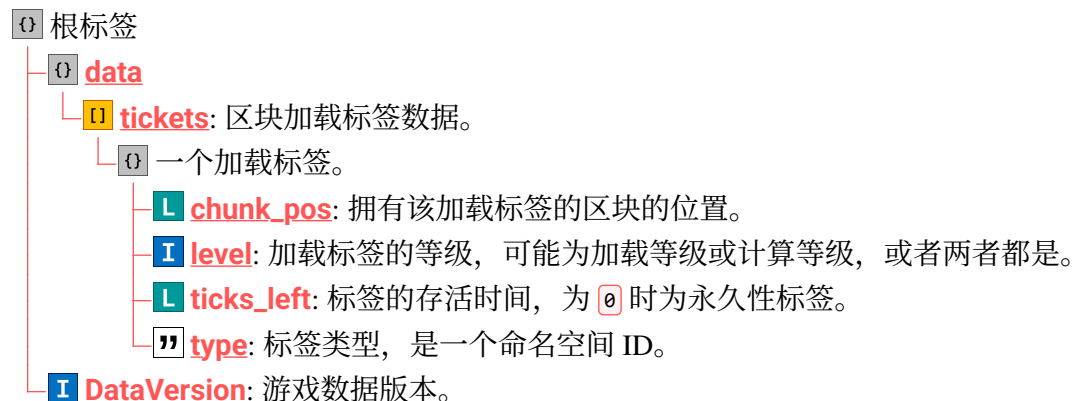
四、维度零散数据

这一部分数据均存储于存储于 `data > minecraft` 文件夹内, 被放置在命名空间之下。但是无论维度自身使用什么命名空间, 其零散数据只存储于 `minecraft` 命名空间。例如, 维度 `the_backrooms:level_0` 的零散数据位于 `dimensions > the_backrooms > level_0 > data > minecraft`。

零散数据包括区块标签数据、袭击数据和世界边界数据。

1. 区块标签数据

区块标签数据文件, 即 `data > minecraft > chunk_tickets.dat`, 是存储区块加载标签的文件, 其数据格式为



2. 袭击数据

袭击数据文件是存储袭击事件相关数据的文件, 文件位于 `data > minecraft > raids.dat`。其数据格式为

❏ 根标签

{} **data**: 袭击数据。

I next_id: 下一次袭击的编号，每次袭击发生后就增加 1，未发生过袭击时此值为 1。

- [] raids:** 此维度存在的袭击。

{} 一个袭击事件。

TF active: 该袭击是否处于激活状态。

[I] center: 袭击中心的坐标, 依次为 x 、 y 、 z 坐标。

I cooldown_ticks: 距离下一波袭击的游戏刻数。上一波袭击结束或袭击开始时此值为初始值 300。

- I groups_spawned**: 袭击已进行的波数。

I group_count: 该袭击的总波数。

[] heroes_of_the_village: 袭击胜利后要给予村庄英雄效果的玩家。

[I] 一个玩家的 UUID。

I id: 袭击的编号。

I post_raid_ticks: 袭击所有波次结束后至游戏宣布袭击胜利并给予玩家村庄英雄效果的冷却时间。

I raid omen level: 该袭击所用的袭击之兆等级。

TF started: 袭击是否已经开始。

” **status**: 该袭击当前的状态, 有效值有 `ongoing` (正在进行)、`victory` (胜利)、`loss` (失败) 和 `stopped` (已结束)。

I ticks_active: 袭击从开始到此文件保存时运行的非冻结游戏刻数, 此值超过 48000 时袭击被判定为平局。

I total health: 本袭击波次中所有生物的最大生命值总量。

I tick: 存档从创建至此运行的游戏刻数, 不计入游戏刻冻结的时间。

I DataVersion: 游戏数据版本。

3. 世界边界数据

世界边界数据文件位于 `data > minecraft > world_border.dat`，其数据格式为

{} 根标签

{} **data**: 世界边界数据。

D **center_x**: 世界边界中心 x 坐标。

D **center_z**: 世界边界中心 z 坐标。

D damage_per_block: 缓冲区外每增加一层伤害区每秒钟对玩家造成的伤害增量。

- D lerp_target**: 世界边界直径的更改量。

L lerp_time: 世界边界直径发生变化所需的时间。

D safe_zone: 缓冲区的宽度。

D size: 世界边界直径。

warning_blocks: 产生警告时玩家距离边界墙的距离。

I warning_time: 距离边界墙到达该位置的倒计时, 若玩家在该位置, 则产生警告。

I DataVersion: 游戏数据版本。

6.4 方块实体

方块实体数据以 NBT 的格式存储在  `regions` 文件夹下的 `Anvil` 文件中。其中有一个字段名为  `block_entities`，是一个复合标签的列表，这便是专门用于存储方块实体的标签。每一个复合标签承载一个方块的方块实体信息，视复合标签为该方块实体的根标签。

所有拥有方块实体的数据列举于附录 II.3 中，供读者参考。

6.4.1 方块实体共通标签 *

一般来说，不同的方块实体拥有不同的标签数据，如告示牌的文本、箱子容纳的物品等，但是所有方块实体一定包含几种特定的标签，这些标签是所有方块实体所共同拥有的，于是称这些标签为**方块实体共通标签（Tags common to all block entities）**。这些标签无法被命令 `/data` 所修改，使用像 `/setblock` 这样的命令放置方块时也无法指定这些共通标签的值。但是除标签  `keepPacked` 外，其他标签均可被命令 `/data get` 查询。下面列出了所有的方块实体共有的字段：

-  根标签
 -  **components**: 使用此方块实体对应的物品放置此方块实体时，如果物品带有非默认的且不会被继承处理的数据组件，则数据会被复制存储入此标签内。
 - └ 一个特定的物品堆叠组件，使用与之匹配的数据类型。
 -  **id**: 该方块的命名空间 ID。
 -  **keepPacked**: 该方块是否“有效”。“有效”的判定条件是，在区块加载的时候被立即放置。这个标签无法被 `/data get` 查询。
 -  **x**: 该方块的 x 坐标。
 -  **y**: 该方块的 y 坐标。
 -  **z**: 该方块的 z 坐标。

每个方块实体的数据格式由共通标签和各自的特有字段组成：

-  根标签
 - └ 方块实体共通标签
 - └ 各方块实体的特有字段

6.4.2 方块实体数据的应用实例

不是所有方块都有方块实体，下面的例子仅仅列举了其中一些方块实体的数据应用。

例 6.7

灾厄旗帜由于其图案的样式超出了合成次数的上限，因此无法被自然合成，但它可以通过命令来获取。写出放置一个灾厄旗帜需要的命令（方块实体的名称不需要）。

解

查阅附录 II.3 的数据知旗帜的特有标签为

{} 根标签

” [] { } **CustomName:** 可选，该旗帜的自定义名称，需要为文本组件。

1 patterns: 可选，按顺序使用的旗帜图案。列表中复合标签的次序越靠前，其对应的图案在旗帜上的层级就越往下。

{ } 一个单独的旗帜图案。

color: 图案的颜色。有效值有 **black** (黑色)、**blue** (蓝色)、**brown** (棕色)、**cyan** (青色)、**gray** (灰色)、**green** (绿色)、**light_blue** (淡蓝色)、**light_gray** (淡灰色)、**lime** (黄绿色)、**magenta** (品红色)、**orange** (橙色)、**pink** (粉红色)、**purple** (紫色)、**red** (红色)、**white** (白色)、**yellow** (黄色)。

pattern: 图案的类型，当使用 **”** 字符串形式时，值为图案的命名空间 ID。旗帜图案可由数据包自定义，在数据包内的相应文件为 **data > <命名空间> > banner_pattern > <ID>.json**。也可以以内联 SNBT 的形式直接在此处定义一个图案类型，此时使用 **复合** 标签形式。

若使用 `{ }` 复合标签形式，则以下字段：

” **asset_id**: 资源包内旗帜图案纹理的命名空间 ID，文件路径为 
assets > <命名空间> > textures > entity > banner > <路径>.png。

” **translation_key**: 该旗帜图案的翻译标识符前缀，游戏解析时会加上 ” **color** 字段的值作为后缀。

灾厄旗帜为白色旗帜，其图案样式为：青色菱形、底淡灰横条、中灰竖条、淡灰色方框边、中黑横条、淡灰色上半方形、淡灰色圆形和黑色方框边。图案样式的顺序不可变化，则

[1] patterns 的数据为

```
patterns:[
  {color:"cyan",pattern:"minecraft:rhombus"},
  {color:"light_gray",pattern:"minecraft:stripe_bottom"},
  {color:"gray",pattern:"minecraft:stripe_center"},
  {color:"light_gray",pattern:"minecraft:border"},
  {color:"black",pattern:"minecraft:stripe_middle"},
  {color:"light_gray",pattern:"minecraft:half_horizontal"},
  {color:"light_gray",pattern:"minecraft:circle"},
  {color:"black",pattern:"minecraft:border"}
]
```

6.22

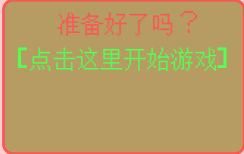
在实际编写命令时，不要使用换行符和进位符。本题要求放置一个灾厄旗帜，则在本地坐标放置灾厄旗帜的命令为

```
setblock ~ ~ ~ white_banner{patterns:
[{color:"cyan",pattern:"minecraft:rhombus"},
{color:"light_gray",pattern:"minecraft:stripe_bottom"},
{color:"gray",pattern:"minecraft:stripe_center"},
{color:"light_gray",pattern:"minecraft:border"},
{color:"black",pattern:"minecraft:stripe_middle"},
{color:"light gray",pattern:"minecraft:half horizontal"}],
```

6.23

```
{color:"light_gray",pattern:"minecraft:circle"},  
{color:"black",pattern:"minecraft:border"}}}]}
```

例 6.8

在 (0, 56, 0) 放置一个站立的橡木告示牌，使之正面呈现的文本为 。点击

该告示牌后，将告示牌替换为文本 。

解 查阅附录 II.3，告示牌的方块实体数据如下：

- 根标签
 - back_text**: 告示牌背面的文本信息。
 - color**: 文本的颜色，相当于用染料为告示牌文本染色。
 - filtered_messages**: 告示牌被过滤的文字，含有四个元素，按顺序分别存储告示牌第一行、第二行、第三行和第四行的文本。
 - 一行文本，必须使用文本组件。
 - has_glowing_text**: 告示牌文本是否发光。
 - messages**: 告示牌的文字，含有四个元素，按顺序分别存储告示牌第一行、第二行、第三行和第四行的文本。
 - 一行文本，必须使用文本组件。
 - front_text**: 告示牌正面的文本信息。
 - color**: 文本的颜色，相当于用染料为告示牌文本染色。
 - filtered_messages**: 告示牌被过滤的文字，含有四个元素，按顺序分别存储告示牌第一行、第二行、第三行和第四行的文本。
 - 一行文本，必须使用文本组件。
 - has_glowing_text**: 告示牌文本是否发光。
 - messages**: 告示牌的文字，含有四个元素，按顺序分别存储告示牌第一行、第二行、第三行和第四行的文本。
 - 一行文本，必须使用文本组件。
 - is_waxed**: 告示牌是否被涂蜡。被涂蜡后告示牌文本不能修改，但交互事件中的命令仍可以执行。

不考虑点击事件的情况下，原告示牌第一和第二行的文本分别为

```
{text:"准备好了吗? ",color:"red"}
```

6.24

```
{text:"[点击这里开始游戏]",color:"green"}
```

6.25

告示牌上的文本组件可以添加点击事件, 点击告示牌的任意位置均可触发点击事件, 无论拥有点击事件的文本组件所处的行数。当一个告示牌上有多个点击事件时, 点击告示牌会触发所有事件。这里的点击事件所处的位置随意, 可以选择在第二行的文本组件添加点击事件, `command` 值先空着:

```
{text:"[点击这里开始游戏]",color:"green",click_event:
{action:"run_command",command:""}}
```

6.26

点击事件会将该告示牌的内容替换掉。告示牌的文本是存储在方块实体中的, 可以很容易想到用命令 `/data` 修改告示牌上的文本:

```
data merge block ...
```

6.27

玩家在点击告示牌的时候, 命令执行者为玩家, 执行的位置为告示牌所处的位置, 因此用 `~~~` 表示修改数据的告示牌为当前位置上的告示牌。告示牌修改文本后, 第一行内容为空, 第二行为默认样式的 `Loading...`, 于是整个 `/data` 命令可以写为

```
data merge block ~ ~ ~ {front_text:{messages:["","loading...","",""]}}
```

6.28

这里 `/data` 使用到了双引号, 不妨将定义 `command` 的引号写成单引号以规避转义, 所以告示牌第二行的文本组件为

```
{text:"[点击这里开始游戏]",color:"green",click_event:
{action:"run_command",command:'data merge block ~ ~ ~ {front_text:
{messages:["","loading...","",""]}}'}}
```

6.29

综上所述, 放置告示牌需要的完整命令为

```
setblock 0 56 0 oak_sign{front_text:{message:[{text:"准备好了
吗?",color:"red"},{text:"[点击这里开始游戏]",color:"
green",click_event:{action:"run_command",command:'data merge block ~ ~
~ {front_text:{messages:["","loading...","",""]}}'}]},,""]}}
```

6.30

例 6.9

在游戏中获取计算机上的现实时间。

解

命令方块的方块实体数据有这样一条字段: `LastOutput`, 这条字段会存储上一个命令的输出。当上一条命令执行有误的时候, 会有如下的输出: `[时:分:秒]错误内容`, 其中 `[时:分:秒]` 便是计算机系统上的现实时间。

因此需要放置一个命令方块, 在其中执行错误的命令, 随后获取命令方块的 `LastOutput` 数据, 将时间截取出来。放置的命令方块可以为保持开启的循环型命令方块以每刻获取当前时间。

首先不妨在 (0,0,0) (确保该位置已被加载) 放置一个保持开启的循环型命令方块, 控制台中含有以下的错误命令:

```
advancement
```

6.31

命令方块是否保持开启由字段  `auto` 控制, 保持开启需要的值为 `true`。控制台命令由 `”` 存储。放置此命令方块所需的命令为

```
setblock 0 0 0  
repeating_command_block{auto:true,Command:"advancement"}
```

6.32

然后高频获取 `”`  `LastOutput`, 此时的 `”`  `LastOutput` 内容大致如下所示:

```
{extra: [{color: "red", extra: [{click_event: {action:  
"suggest_command", command: "/advancement"}, color: "gray", extra:  
["...", "dvancement", {color: "red", italic: 1b, translate:  
"command.context.here"}], text: ""}], text: ""}], text: "[16:40:25] "}
```

6.33

其中的 `[16:40:25]` 需要被截取, 这是一个字符串, 可以用字符串切片得到任意的形式。比如, 将整个时间切出来、去除外侧的方括号, 将结果存入命令存储 `foo:test` 的自定义字段 `”` `system_time`, 显然需要从第 2 个字符切到第 9 个, 命令为:

```
data modify storage foo:test system_time set string block 0 0 0  
LastOutput.text 1 9
```

6.34

现在就可以用文本组件将这个结果显示出来:

```
{source:"storage",nbt:"system_time",storage:"foo:test\"}
```

6.35

或者也可以单独将时、分、秒截取出来:

```
data modify storage foo:clock system_time.hour set string block 0 0 0  
LastOutput.text 1 3
```

6.36

```
data modify storage foo:clock system_time.minute set string block 0 0  
0 LastOutput.text 4 6
```

6.37

```
data modify storage foo:clock system_time.second set string block 0 0  
0 LastOutput.text 7 9
```

6.38

不过, 字符串切片得到的结果都是字符串, 如果需要对这些时间数据进行运算, 则需要使用宏将它们转换成数值:

```
data > foo > function > clock.mcfuction  
1 function foo:clock_macro with storage foo:clock system_time
```

```
data > foo > function > clock_macro.mcfuction  
1 $scoreboard players set #system_time_hour var $(hour)  
2 $scoreboard players set #system_time_minute var $(minute)
```

```
3 $scoreboard players set #system_time_second var $(second)
```

6.5 实体

存储实体格式的 Anvil 文件位于文件夹 `entities` 下，相应字段能够存储绝大部分种类的实体数据，只有玩家的数据不在此处，因此本节所讲述的实体格式不会包含任何与玩家有关的信息。

6.5.1 处理实体的命令

一、命令 `/summon`

通常地、如果需要生成一个实体，除了用创造模式物品栏里的刷怪蛋外，还可以用命令 `/summon` 生成一个自定义的实体，这使得实体的性质更丰富。命令 `/summon` 需要的参数等级为 2，语法为：

```
summon <entity> [<pos>] [<nbt>]
```

6.39

- 其中的参数：
- `<entity>`（召唤实体 `minecraft:entity_summon`）——生成实体的命名空间 ID，必须为可召唤实体的命名空间 ID。
 - `[<pos>]`（三维坐标 `minecraft:vec3`）——可选，生成该实体的坐标，坐标使用中心点校准。如不指定，则使用命令执行位置。命令中坐标的位置必须已经加载。
 - `[<nbt>]`（NBT 复合标签 `minecraft:nbt_compound_tag`）——可选，该实体的 NBT 数据，必须为 SNBT，是根标签的值。如不指定，则会在坐标的位置生成一个默认的实体。

例如，在命令执行者的位置生成一只猪：

```
summon pig
```

6.40

或

```
summon pig ~ ~ ~
```

6.41

当然也可以在命令后面添加 SNBT 以自定义这只猪的各项性质，下面的内容将重点介绍实体 SNBT 的写法。

二、命令 `/kill`

命令用于清除实体，所需权限等级为 2，语法为：

```
kill [<targets>]
```

6.42

其中的参数：`<targets>`（实体 `minecraft:entity`）——可选，不指定则清除命令执行者自身。

三、命令 `/ride`

命令 `/ride` 用于控制实体的骑乘关系，它需要的权限等级为 2，以下是所有用法：

1. 让指定骑手骑乘指定坐骑，语法为

```
ride <target> mount <vehicle>
```

6.43

其中的参数：`<targets>`（实体 `minecraft:entity`）——骑手，可以为玩家名称、UUID 或目标选择器。

`<vehicle>`（实体 `minecraft:entity`）——坐骑，可以为玩家名称、UUID 或目标选择器。

注意，遇到下列情况即命令执行失败：

- (1) **坐骑为玩家或标记**；
- (2) 骑手已骑乘了其他坐骑；
- (3) 骑手和坐骑为同一实体；
- (4) 骑手和坐骑有直接或间接的骑乘关系。

例 6.10

让最近的物品展示实体骑乘最近的狼。

解

命令为

```
ride @n[type=item_display] mount @n[type=wolf]
```

6.44

2. 让指定骑手取消骑乘当前的坐骑，语法为

```
ride <target> dismount
```

6.45

小提示

1. 可以多层嵌套骑乘实体，如：让狼作为坐骑，物品展示实体骑乘狼，标记骑乘物品展示实体。
2. 常用的坐骑为狼，常用的骑手为展示实体。狼提供移动方式，也可以跟随玩家；展示实体能提供自定义的外观和动画。
3. 标记不能作为坐骑！如果需要用到标记，请将它作为骑手使用。

对原版技术性开发而言，骑乘是常用的组合实体的手段。通常是将一个能够提供移动方式的实体作为坐骑，提供外观的实体作为骑手，这样就能创建自定义实体。在编辑其数据的时候，也可以用 `/execute on` 子命令精确地指向具有骑乘关系的实体。

6.5.2 实体共通标签

与方块实体类似, 所有实体的根标签下一定有一些共同存在的子标签, 这些标签不会因为实体种类的变化而有所改变, 于是称这类标签为**实体共通标签 (Tags common to all entities)**。下面列举了所有的实体共有的字段, 注意, 不是所有的实体共通标签都能被命令 `/data` 所修改。

根标签

- S Air**: 该实体剩余的空气值。
- CustomName**: 该实体的自定义名称, 未指定名称时该标签不存在。需要是一个文本组件。
- CustomNameVisible**: 该实体的自定义名称能否总是显示在该实体的上方, 未指定名称时不存在。
- data**: 任意自定义数据。写入数据的时候可以使用 `"` 字符串形式, 但存储的时候一律存储为 `{ }` 复合标签形式。
 - 自定义数据结构
- fall_distance**: 该实体已下坠的距离。
- Fire**: 该实体身上的火焰距熄灭的剩余游戏刻时长。对于未着火的玩家, 此值为 `-20s`, 对于未着火的其他实体, 此值为 `-1s`。
- Glowing**: 该实体是否发光。
- HasVisualFire**: 该实体是否在外观上有着火效果。
- id**: 该实体的命名空间 ID, 此值不可被修改。
- Invulnerable**: 该实体是否会受到伤害。这里的伤害是指除创造模式玩家造成的伤害和属于伤害类型标签 `#bypasses_invulnerability` 以外的其他伤害。
- Motion**: 列表中存在三个元素, 均为双精度浮点数, 用于表示该实体在当前游戏刻的速度。在三维空间中速度矢量分解为沿三个坐标轴的速度矢量, 即 dx 、 dy 和 dz , 分别用此列表第一、二、三个元素表示。

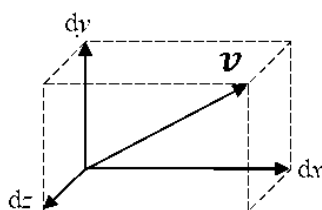


图 6.7 速度矢量的分解

- NoGravity**: 该实体是否会受到重力影响。
- OnGround**: 该实体是否接触地面。
- Passengers**: 骑乘该实体的实体。
 - 一个骑乘该实体的实体。
 - 实体格式
- PortalCooldown**: 该实体距下次被允许穿过下界传送门前往另一维度的游戏刻。
- Pos**: 该实体的三维坐标, 列表内元素顺序不可改变。
 - x 坐标。
 - y 坐标。

- └─ **D** **z** 坐标。
- └─ **U** **Rotation**: 该实体的朝向，列表内元素顺序不可改变。
 - └─ **F** 偏航角。
 - └─ **F** 俯仰角。
- └─ **TF** **Silent**: 该实体是否会发出声音。
- └─ **U** **Tags**: 该实体的所有记分板标签。
 - └─ **U** 一个记分板标签。
- └─ **I** **TicksFrozen**: 可能不存在，表示实体已冷冻的时间。当实体在细雪中时每游戏刻增加 **1**，离开细雪则每游戏刻减少 **2**。为 **0** 时此标签不存在。
- └─ **UI** **UUID**: 该实体的 UUID。

如果一个实体属于生物一类（Java 类名 `LivingEntity`），则它拥有如下的共同标签，这些标签被称为**生物共通标签**：

- U** 根标签
 - └─ **F** **AbsorptionAmount**: 此生物的伤害吸收值。
 - └─ **U** **active_effects**: 该生物拥有的状态效果列表。
 - └─ **U** 一个状态效果。
 - └─ 格式见节 6.6。
 - └─ **U** **attributes**: 该实体的属性列表。
 - └─ **U** 一个属性。
 - └─ 格式见节 6.7。
 - └─ **U** **Brain**: 生物记忆的相关信息，用于 AI 计算。不使用生物记忆的实体该项标签为空。具体格式见附录 II.4 的相关说明。
 - └─ **U** **memories**: 生物记忆。此处存储长期记忆，不存储短期记忆，短期记忆仅存在于内存中临时使用。
 - └─ **U** **<生物记忆类型命名空间 ID>**: 一项长期记忆。
 - └─ **L** **ttl**: 该记忆距离过期的游戏刻数。每种生物记忆的初始过期时间都不同，此值为 **01** 时该记忆被删除。
 - └─ **TFILUIU** **value**: 记忆的值，每种生物记忆的值类型都不同。
 - └─ **U** **current_explosion_impact_pos**: 此生物被所有形式的爆炸击退时的坐标，列表内元素顺序不可改变。
 - └─ **I** **current_impulse_context_reset_grace_time** 为 **0** 时此字段被删除。
 - └─ **D** **x** 坐标。
 - └─ **D** **y** 坐标。
 - └─ **D** **z** 坐标。
 - └─ **I** **current_impulse_context_reset_grace_time**: 爆炸击退减少摔落伤害的最长时间，单位为游戏刻。
 - └─ **S** **DeathTime**: 距离生物死亡而被删除的时间。生物存活时该值为 **0s**，死亡后每游戏刻增加 **1s**，直到该值达到 **20s** 时生物被删除。
 - └─ **U** **equipment**: 生物的装备。
 - └─ **U** **<槽位>**: 在此槽位上装备的物品。有效的槽位有 **mainhand**、**offhand**、**feet**、**legs**、**chest**、**head**、**body** 和 **saddle**。

- └ 无槽位物品格式，见节 6.10。
- └ **TF FallFlying**: 该生物是否处于滑翔状态。
- └ **F Health**: 该生物的生命值。
- └ **I HurtByTimestamp**: 以该生物生成的时间为准，存储该生物上次被伤害的时间。
- └ **S HurtTime**: 该生物受伤后伤害免疫阶段（变红）的剩余时间，初始为 10 gt，每游戏刻减 1。
- └ **I last_hurt_by_mob**: 当前最后一次攻击该生物的生物的 UUID。
- └ **I last_hurt_by_player**: 当前最后一次攻击该生物的玩家的 UUID。
- └ **I last_hurt_by_player_memory_time**: 生物被玩家攻击后该值为 **100**（单位为游戏刻），每游戏刻减少 1，此值降到 **0** 以下后清除 **I last_hurt_by_player** 的数据，并删除此字段。
- └ **I locator_bar_icon**: 生物的路径点图标。
 - └ **I I color**: 此生物路径点的图标颜色，可以用 **I** 整型指定 RGB 颜色，对二进制而言，每八位表示一个通道，从高到低依次为 R、G、B 通道。也可以用 **I** 列表分别指定每个通道。游戏在存储时一律存储为 **I** 整型。
 - └ 若使用 **I** 列表形式，则包含以下字段：
 - └ **F** R 通道分量，表示红色值。
 - └ **F** G 通道分量，表示绿色值。
 - └ **F** B 通道分量，表示蓝色值。
 - └ **I style**: 路径点样式的命名空间 ID。
 - └ **I sleeping_pos**: 该生物所睡床的坐标，依次为 x、y、z 坐标。该生物不睡觉时此标签不存在。
 - └ **I Team**: 可选，该生物所在的队伍。
 - └ **I ticks_since_last_hurt_by_mob**: 生物最近一次被生物攻击后距离上次攻击的游戏刻数。

具体到每一个实体时，除了上述的共通标签，还可以使用实体自身特有的标签，由于篇幅限制，本章无法对所有实体的所有标签进行分析，故将它们列举于附录 II.4 中，供读者查阅。

6.5.3 实体数据的应用实例

实体种类、可用字段繁多，以下列举一些应用实例以说明实体数据的使用。

例 6.11

生成一个有双臂、没有基座的盔甲架。

解

查阅附录 II.4，盔甲架的两种标签：**TF ShowArms** 和 **TF NoBasePlate** 分别控制盔甲架是否显示双臂、是否使基座不可见。易知生成有双臂、没有基座的盔甲架的命令可以为

```
summon minecraft:armor_stand ~ ~ ~ {ShowArms:true,NoBasePlate:true}
```

6.46

例 6.12

对于世界中所有的棕色绵羊，使用命令将其中任意一只变成粉红色绵羊。

解

在这个例子中，需要用命令进行两个行为：一是选择目标，二是对目标的数据进行更改。前者即为第三章所讲的目标选择器，显然选择世界中任意一只绵羊的目标选择器为

@e[type=sheep,sort=random,limit=1]

6.47

其次要对选择的绵羊进行进一步的限定。根据附录 II.4，控制绵羊外观颜色的标签为 **B Color**，使用颜色的数字 ID，这也是扁平化后少数还存留数字 ID 的地方，其对应关系如下表所示：

表 6.1 颜色数字 ID 表

数值	颜色	数值	颜色
0	白色	8	淡灰
1	橙色	9	青色
2	品红	10	紫色
3	淡蓝	11	蓝色
4	黄色	12	棕色
5	黄绿	13	绿色
6	粉红	14	红色
7	灰色	15	黑色

其中棕色对应数据值 12。根据选择器 NBT 参数的语法，将这个目标选择器写为

@e[type=sheep,sort=random,limit=1,nbt={Color:12b}]

6.48

第二步为更改目标实体的数据，很明显需要使用命令 `/data merge` 或 `/data modify`。本例使用 `/data merge` 的写法，粉红色的数据值为 6，于是最终的命令为

data merge entity @e[type=sheep,sort=random,limit=1,nbt={Color:12b}]{Color:6b}

6.49

例 6.13

数据包 AMR Bot 的无人机是由多个实体通过骑乘拼接起来的自定义实体。在设计上无人机需要跟随玩家，因此根实体选用狼，现生成这个自定义实体，要求狼有以下的性质：

- (1) 隐形；
- (2) 静音；
- (3) 无法受到伤害；
- (4) 被一个物品展示实体骑乘，这个物品展示实体又被一个交互实体骑乘。



图 6.8 AMR Bot 数据包

解

首先确定这个自定义实体的类型为狼，其次对于这个自定义实体的特征，可以逐个分析出实现这些特征所需要的标签：

- (1) 隐形：狼的可用字段中并没有直接控制它是否隐形的，因此需要使用状态效果。在 `active_effects` 直接给狼定义无限时长的隐身状态效果，不显示粒子效果。

```
active_effects:
  [{id:"minecraft:invisibility",duration:-1,show_particles:false}]
```

6.50

- (2) 静音：由字段 `T F` `Silent` 定义。

```
Silent:true
```

6.51

- (3) 无法受到伤害：由字段 `T F` `Invulnerable` 定义。

```
Invulnerable:true
```

6.52

- (4) 被一个物品展示实体骑乘，这个物品展示实体又被一个交互实体骑乘：实体的骑乘由实体共通标签 `L` `Passengers` 存储，内部每一个元素都是一个完整的实体格式，写法如下所示。

```
Passengers:[{id:"minecraft:item_display",Passengers:
  [{id:"minecraft:interaction"}]}]
```

6.53

将上面的所有标签整合到一起，放在命令 `/summon` 中，可得到

```
summon wolf ~ ~ ~ {
  active_effects:[
    {
      id:"minecraft:invisibility",
      duration:-1,
      show_particles:false
    }
  ],
  Silent:true,
  Invulnerable:true,
  Passengers:[
    {
      id:"minecraft:item_display",
      Passengers:[
        {
          id:"minecraft:interaction"
        }
      ]
    }
  ]
}
```

6.54

例 6.14

生成一个潜行的 Mu_xian 玩家模型。



图 6.9

解

参考附录 II.4，命令为

```
summon mannequin ~ ~ ~ {profile:{name:"Mu_xian"},pose:"crouching"} 6.55
```

例 6.15

生成一个物品展示框，使之朝向上方，里面存储了一个被旋转了 7 次的铁镐，且该物品展示框无法被冒险模式的玩家破坏。



图 6.10

解

查阅附录 II.4，除了实体共通标签外，物品展示框还拥有下列标签：

根标签

- B Facing**: 物品展示框的朝向，用 0b 表示朝向下方，1b 表示上方，2b 表示北方，3b 表示南方，4b 表示西方，5b 表示东方。
- TF Fixed**: 物品展示框是否为一个锁定的物品展示框。锁定的物品展示框意思为，该物品展示框无法被绝大部分伤害源所破坏（特例：处于创造模式的玩家），并且里面物品无法被旋转、放置或移除；此外该物品展示框无论是否有支撑方块，均不会被破坏。
- TF Invisible**: 物品展示框是否隐形。
- Item**: 物品展示框中的物品。
 - 无槽位物品格式
- F ItemDropChance**: 物品展示框被破坏时内部物品掉落的概率。
- B ItemRotation**: 物品被旋转的次数。

详细的物品格式概念将在节 6.10 中讲解，这里先给出一个铁镐的物品格式：

```
{id:"minecraft:iron_pickaxe",count:1} 6.56
```

物品展示框的其他一些标签在此用不到, 故不单独介绍它们。要想生成一个题设要求的物品展示框, 可以不难写出如下的命令:

```
summon item_frame ~ ~ ~ {
  Facing:1b,
  Item:{
    id:"minecraft:iron_pickaxe",
    count:1
  },
  ItemRotation:7b,
  Invisible:1b,
  Fixed:1b
}
```

6.57

例 6.16

为标签 **I DisabledSlots** 编写数据, 使之禁用盔甲架如下部位的特定交互事件:

- (1) 添加和改变靴子栏的物品;
- (2) 添加、移除和改变主手的手持物品。

解 标签 **I DisabledSlots** 用于禁止玩家与盔甲架某些部位的交互。虽然它使用整型数据, 但在内部其实是使用二进制存储盔甲架物品栏可交互信息的。该二进制数据一共有 22 位数字, 每一位都用于存储禁用的部位与禁用的交互动作。对于每一位数字, 使用 **1** 以表示禁用与特定部位物品的特定交互动作, 使用 **0** 以解除禁用。每一位的禁用部位和禁用动作如下表所示:

表 6.2 盔甲架禁用部位与禁用动作

位数	禁用部位与禁用动作	位数	禁用部位与禁用动作
0	添加和改变主手的手持物品	11	移除和改变胸甲栏的物品
1	添加和改变靴子栏的物品	12	移除和改变头盔栏的物品
2	添加和改变护腿栏的物品	13	移除和改变副手的手持物品
3	添加和改变胸甲栏的物品	14	空位
4	添加和改变头盔栏的物品	15	空位
5	添加和改变副手的手持物品	16	添加主手的手持物品
6	空位	17	添加靴子栏的物品
7	空位	18	添加护腿栏的物品
8	移除和改变主手的手持物品	19	添加胸甲栏的物品
9	移除和改变靴子栏的物品	20	添加头盔栏的物品
10	移除和改变护腿栏的物品	21	添加副手的手持物品

禁用特定部位和特定交互动作时, 需要先确定禁用的部位和动作分属哪些位数, 再将这些位上的数据组合成二进制数据。第 (1) 小题禁用添加和改变靴子栏的物品, 控制这项禁用为二进制数据的第 1 位 (注意与第 0 位作区分), 所以二进制数据为 **10**, 即:

```
DisabledSlots:0b10
```

6.58

第 (2) 小题的禁用项使用了多个位数, 分别为第 0 位禁用添加和改变主手的手持物品和第 8 位移除和改变主手的手持物品, 得到二进制数据 `100000001`, 即:

```
DisabledSlots:0b100000001
```

6.59

例 6.17

有一个位于坐标 (0, 70, 0) 的朝向为南、左内角形状的橡木楼梯, 将这个橡木楼梯变成受到重力影响的下落方块, 要求石头每多下落一格, 就对下方的实体多造成 2 点伤害。

解

有一类实体被称作**下落的方块 (Falling block)**, 顾名思义, 这类实体由方块转化而来, 受到重力的影响。一般下落的方块可由游戏中原本受重力影响的方块, 如沙子、沙砾、铁砧等转化, 使用 NBT 可以让其他不受重力影响的方块也转化为下落的方块。不过, 下落的方块并不能直接由命令 `/summon` 凭空生成。只有当下落的方块生成的位置有命名空间 ID 与之代表的方块命名空间 ID 相同的方块时, 该下落的方块才会正常生成。在其他情况下, 下落的方块会在其生成之后 1gt 内被删除。因此生成这个下落的石头方块时, 一定要在原石头方块的位置生成。

在这个题设中, 需要用到的下落的方块的标签如下:

根标签

BlockState: 该下落的方块代表的方块。

Name: 方块的命名空间 ID。

Properties: 可选, 由若干方块属性组成的方块状态。

<方块属性>: 标签名为方块状态的属性, 值使用字符串表示。

FallHurtAmount: 下落的方块每多下落一格对下方实体造成伤害的增量, 仅当 `HurtEntities` 存在时有效。

HurtEntities: 是否对砸中的实体造成伤害。

首先指定一个朝向为南、左内角形状的橡木楼梯:

```
minecraft:oak_stairs[facing=south,shape=inner_left]
```

6.60

现在使这个橡木楼梯变成下落的方块, 则标签 `BlockState` 为

```
BlockState:{Name:"minecraft:oak_stairs",Properties:
{facing:"south",shape:"inner_left"}}
```

6.61

综上所述, 首先放置这个橡木楼梯:

```
setblock 0 70 0 minecraft:oak_stairs[facing=south,shape=inner_left]
```

6.62

其次将它变为下落的方块:


```

summon falling_block 0 70 0 {
  BlockState:{
    Name:"minecraft:oak_stairs",
    Properties:{
      facing:"south",
      shape:"inner_left"
    }
  },
  FallHurtAmount:2.0f,
  HurtEntities:true
}

```

6.63

例 6.18

生成一个掉落物钻石，使之永远不会自行消失。

解

物品虽然在物品栏中严格使用物品格式，但是掉落物形式的物品却是一种实体，使用实体格式，物品本身的信息被打包在物品堆叠组件中成为实体格式的一部分。掉落物物品的实体格式中有一个标签名为 `{Item}`，复合标签内即为该物品的物品堆叠组件。现在先提供一个钻石的物品格式：

```
{id:"minecraft:diamond"}
```

6.64

题设还有一个要求：物品永远不会自行消失。掉落物物品的实体格式有一个名为 `S Age` 的标签，存储该掉落物物品已存在的游戏刻长度，即从生成至未被捡起的时长。该标签的值会随着游戏过程而增加，掉落物物品刚生成时值为 `0s`，一旦值达到 `6000s`，该物品就会消失，在游戏中表现为 5 分钟内掉落物未被捡起就自行消失。如果值被设为 `-32768s`，则该值不会随着游戏过程而增加，也就可以实现物品永远不会自行消失的目地。最终的命令为：

```
summon item ~ ~ ~ {Item:{id:"minecraft:diamond"},Age:-32768s}
```

6.65

6.6 状态效果

有时候需要对实体施加一定的增益或减益，这种给予增益或减益的手段便是**状态效果 (Status effect)**，这是一种从实体外部施加作用于实体的、在限定时间内给实体提供正面或负面效果的游戏机制。

一个状态效果包含有三个要素，即命名空间 ID、作用时间和倍率。命名空间 ID 决定了施加于实体的是何种状态效果，对实体起正面作用还是负面作用；作用时间决定了该状态效果持续的时间；倍率决定了该状态效果的等级：**等级比倍率大 1**，例如，若一个状态效果等级为 I，则其倍率为 0。

大多数的状态效果都随着其倍率（或等级）增大而效果增强，少数如夜视在任何倍率下效果都是一样的。一个状态效果的持续时间可以是有限的，也可以是无限的。**一个实体可以同时拥有多个不同种类的状态效果，但不能拥有相同的状态效果，即使其倍率不同。**当一个实体拥有某种状态效果，其持续时间和倍率都是确定的，则无法为这个实体的施加倍率更低的同种状态效果，但可以施加倍率更高的同种状态效果，此时原先低倍率的状态效果会被隐藏，但持续时间会继续扣除，**若高倍率的状态效果终止，则低倍率的状态效果会重新出现。**

下表列举了所有可用状态效果的 ID，其中命名空间前缀 `minecraft:` 已省略：

表 6.3 状态效果表

状态效果	英文 ID	状态效果	英文 ID	状态效果	英文 ID
速度	<code>speed</code>	失明	<code>blindness</code>	潮涌能量	<code>conduit_power</code>
缓慢	<code>slowness</code>	夜视	<code>night_vision</code>	海豚的恩惠	<code>dolphins_grace</code>
急迫	<code>haste</code>	饥饿	<code>hunger</code>	不祥之兆	<code>bad_omen</code>
挖掘疲劳	<code>mining_fatigue</code>	虚弱	<code>weakness</code>	村庄英雄	<code>hero_of_the_village</code>
力量	<code>strength</code>	中毒	<code>poison</code>	黑暗	<code>darkness</code>
瞬间治疗	<code>instant_health</code>	凋零	<code>wither</code>	试炼之兆	<code>trial_omen</code>
瞬间伤害	<code>instant_damage</code>	生命提升	<code>health_boost</code>	袭击之兆	<code>raid_omen</code>
跳跃提升	<code>jump_boost</code>	伤害吸收	<code>absorption</code>	蓄风	<code>wind_charged</code>
反胃	<code>nausea</code>	饱和	<code>saturation</code>	盘丝	<code>weaving</code>
生命恢复	<code>regeneration</code>	发光	<code>glowing</code>	渗浆	<code>oozing</code>
抗性提升	<code>resistance</code>	漂浮	<code>levitation</code>	寄生	<code>infested</code>
防火	<code>fire_resistance</code>	幸运	<code>luck</code>	鹦鹉螺之息	<code>breath_of_the_nautilus</code>
水下呼吸	<code>water_breathing</code>	霉运	<code>unluck</code>		
隐身	<code>invisibility</code>	缓降	<code>slow_falling</code>		

命令 `/effect` 用于移除或施加实体的状态效果，它需要的权限等级为 2，以下是所有用法：

1. 移除状态效果，语法为

```
effect clear [<targets>] [<effect>]
```

6.66

其中的参数：`<targets>`（实体 `minecraft:entity`）——可选，指定的目标实体，必须为玩家名称、UUID 或目标选择器，不填写则清除命令执行者自身的所有状态效果。

`[<effect>]`（注册项 `minecraft:resource`）——可选，指定要移除的状态效果的命名空间 ID，不填写则清除指定实体的所有状态效果。

2. 施加状态效果，语法为

```
effect give <targets> <effect> [<seconds>] [<amplifier>] [<hideParticles>]
```

6.67

其中的参数：`[<seconds>]`（整型 `brigadier:integer`）——状态效果持续时间，为不大于 `1000000` 的正整数。对于除瞬间伤害（`instant_damage`）、瞬间治疗

(`instant_health`) 和饱和 (`saturation`) 外的状态效果而言单位均为秒，而这三者的单位则为游戏刻。如果该参数不指定，则将除了除瞬间伤害、瞬间治疗和饱和外的状态效果默认设定为 30 秒，瞬间伤害、瞬间治疗和饱和则被设为 1gt。

[`<amplifier>`] (整型 `brigadier:integer`) —— 可选，状态效果的倍率，必须为不大于 255 的非负整数。倍率是比等级小 1 的参数，倍率设为 0 时，等级为 I；设为 1 时，等级为 II。默认为 0。

[`<hideParticles>`] (布尔值 `brigadier:bool`) —— 是否显示粒子效果和 HUD 上显示的状态效果图标。该参数在冒险地图中较为常用。

3. 施加无限时长的状态效果，语法为

```
effect give <targets> <effect> infinite [<amplifier>] [<hideParticles>]
```

6.68

例 6.19

给所有玩家施加 2 秒的速度提升 II 效果。

解

命令如下所示，注意等级为 II，则倍率为 1：

```
effect give @a speed 2 1
```

6.69

存储实体状态效果的 NBT 数据结构如下所示：

根标签

active_effects: 该生物拥有的状态效果列表。

一个状态效果。

ambient: 该状态效果是否由信标施加。

amplifier: 该状态效果的倍率。若实际倍率大于 127，设实际倍率为 s ，此标签存储的值为 $127 - s$ 。

duration: 状态效果的持续时间，单位为游戏刻。如果持续时间为无尽，则值为 -1。

hidden_effect: 存储类型、命名空间 ID 相同但倍率较低的状态效果，在高倍率状态效果结束后该状态效果会出现，是为低倍率状态效果提供隐藏功能的标签。

不包含字段 `id` 的状态效果格式

id: 该状态效果的 ID。

show_icon: 是否显示图标。

show_particles: 是否显示粒子效果。

例 6.20

直接生成一个村民，使之带有 10 秒的跳跃提升 II 和 10 秒的速度 I 效果。

解

命令如下所示，注意标签 **I** `duration` 数据的单位为游戏刻，因此要把 10 秒化为 200gt:

```
summon villager ~ ~ ~ {active_effects:
[{"id":"minecraft:jump_boost",duration:200,amplifier:1b},
{"id":"minecraft:speed",duration:200,amplifier:0b}]\}
```

6.70

6.7 属性

相对于状态效果这样由外部施加给实体的机制，决定了诸如攻击伤害、移动速度这样的实体本身的“能力”，游戏有另一套机制来描述之，即属性。**属性（Attribute）是一套用于决定玩家和生物本身能力的机制。**

一个属性一定存在一个**属性名称**，这个名称需要是一个命名空间 ID。属性名称决定了该属性控制玩家和生物哪方面的能力，如移动速度、生命值上限。在属性的机制中使用特定的值以表示一方面能力的大小，即**属性的值**，为了与属性的基值做区分，通常称之为属性的最终值。这个值可以为浮点数。属性的值由两部分决定，一个是**属性基值（Attribute base）**，另一个是**属性修饰符（Attribute modifiers）**。**若存在属性修饰符，则属性的最终值等于在基值上进行修饰后的值；若不存在属性修饰符，则属性的最终值等于基值。**下面举一个例子以说明这个机制运作的原理。

属性的基值是一个属性在不加任何修饰符、处于原本状态下的值，例如决定玩家攻击力的属性基值为 1，表示玩家的基础攻击力为 1，即不带任何装备、武器的情况下造成的伤害为 1 点（ $0.5 \times \heartsuit$ ）。而玩家一旦手持武器，则造成的攻击会发生变化，但这并不意味着是玩家本身的攻击力得到了提升，而是因为手持的武器提供了一个属性修饰符，该修饰符在攻击力基值的基础上增加了一定的数值。因此玩家一次攻击造成的伤害值实际上是由两部分组成的，一部分是玩家本身的攻击力，另一部分是武器提供的加成。若一个玩家手持钻石剑，其单次攻击的伤害值为玩家攻击力的 1 点基值加上钻石剑的修饰符提供的 6 点加成值，总和为 7 点。

虽然修饰符可以自由地对属性的值进行修改，而对于特定名称的属性，其值必须处于一个规定的**值域**内，每种属性的值域各不相同。在实际修改属性的值时，一定要注意修改后的值是否超限。此外，每种属性各有一个**默认值**，若属性未定义基值，则使用默认值作为基值计算。

不同属性适用生物也不同，各自使用的基值也不相同。下面列举了所有的属性，包括它们的名称、值域、默认值和基值，其中名称的命名空间前缀 `minecraft:` 已省略。

表 6.4 可用属性表

属性名称	ID	值域	默认值	适用生物	基值
护甲值	<code>armor</code>	0 ~ 30	0	杀手兔	8
				凋零	4
				岩浆怪	3 × 体型 + 1

(续表)

属性名称	ID	值域	默认值	适用生物	基值
				僵尸、尸壳、溺尸、僵尸村民、僵尸猪灵	2
				其他所有生物	0
盔甲韧性	armor_toughness	0 ~ 20	0	所有生物	0
攻击伤害	attack_damage	0 ~ 2048	2	巨人	50
				监守者	30
				铁傀儡	15
				劫掠兽	12
				青蛙	10
				远古守卫者	8
				末影人、猪灵蛮兵	7
				幻翼	$\begin{cases} 2, s = 0 \\ 6 + s, s > 0 \end{cases}$ ①
				烈焰人、守卫者、成年疣猪兽、熊猫、北极熊、成年僵尸疣猪兽	6
				猪灵、掠夺者、卫道士、僵尸猪灵	5
				恼鬼、狼	4
				旋风人、猫、嘎吱、海豚、溺尸、尸壳、豹猫、兔子、僵尸、僵尸村民	3
				悦灵、美西螈、蜜蜂、沼骸、洞穴蜘蛛、苦力怕、末影螨、唤魔者、狐狸、成年山羊、幻术师、骷髅、史莱姆、蜘蛛、流浪者、女巫、凋灵、凋灵骷髅	2

① s 为幻翼的体型，由 `I size` 字段存储。

(续表)

属性名称	ID	值域	默认值	适用生物	基值
				岩浆怪、史莱姆	1 + 体型
				幼年山羊、玩家、蠹虫	1
				幼年疣猪兽、幼年僵尸疣猪兽	0.5
击退	attack_knockback	0 ~ 5	0	劫掠兽、监守者	1.5
				疣猪兽、僵尸疣猪兽	1
				其他所有生物	0
攻击速度	attack_speed	0 ~ 1024	4	玩家	4
方块破坏速度	block_break_speed	0 ~ 1024	1	玩家	1
方块交互距离	block_interaction_range	0 ~ 64	4.5	玩家	4.5
着火时间	burning_time	0 ~ 1024	1	所有生物	1
镜头距离	camera_distance	0 ~ 32	4	末影龙、巨人	16
				快乐恶魂、恶魂	8
				其他所有生物	4
实体交互距离	entity_interaction_range	0 ~ 64	3	玩家	3
爆炸击退抗性	explosion_knockback_resistance	0 ~ 1	0	所有生物	0
摔落伤害倍数	fall_damage_multiplier	0 ~ 100	1	骆驼、驴、马、羊驼、骡、骷髅马、行商羊驼、僵尸马	0.5
				其他所有生物	0
飞行速度	flying_speed	0 ~ 1024	0.4	蜜蜂、凋灵	0.6
				鹦鹉	0.4
				悦灵	0.1
				恶魂	0.06
				快乐恶魂	0.05
生物跟随距离	follow_range	0 ~ 2048	32	恶魂	100
				末影人	64
				烈焰人	48
				凋灵	40
				僵尸、尸壳、溺尸、僵尸村民、僵尸猪灵	35
				嘎吱、狐狸、掠夺者、劫掠兽	32

(续表)

属性名称	ID	值域	默认值	适用生物	基值
				旋风人、监守者	24
				北极熊	20
				幻术师	18
				唤魔者、猪灵蛮兵、卫道士	12
				其他所有 AI 生物	16
重力	gravity	-1 ~ 1	0.08	所有生物	0.08
跳跃力度	jump_strength	0 ~ 32	0.42	马、骷髅马、僵尸马	$U[0.4, 1]$ ^①
				驴、骡、羊驼、行商羊驼	0.5
				其他所有生物	0.42
击退抗性	knockback_resistance	0 ~ 1	0	铁傀儡、监守者	1
				劫掠兽	0.75
				疣猪兽、僵尸疣猪兽	0.6
				其他所有生物	0
幸运值	luck	-1024 ~ 1024	0	玩家	0
最大伤害吸收值	max_absorption	0 ~ 2048	0	所有生物	0
最大生命值 ^②	max_health	0 ~ 1024	0	监守者	500
				凋灵	300
				末影龙	200
				铁傀儡、巨人、劫掠兽	100
				远古守卫者	80
				猪灵蛮兵	50
				末影人、疣猪兽、驯服的狼、僵尸疣猪兽	40
				骆驼、骆驼尸壳、幻术师	32

① 服从均匀分布。
② 大多数生物在生成时的生命值会自动设为最大生命值。

(续表)

属性名称	ID	值域	默认值	适用生物	基值
				旋风人、守卫者、潜影贝、海龟、北极熊	30
				女巫	26
				僵尸马	25
				唤魔者、掠夺者、卫道士	24
				驴、马、羊驼、骡、行商羊驼	$U[15,30]$
				岩浆怪、史莱姆	尺寸 ²
				悦灵、盔甲架、烈焰人、苦力怕、溺尸、快乐恶魂、尸壳、非体弱熊猫、幻翼、玩家、骷髅、流浪者、炽足兽、村民、流浪商人、凋灵骷髅、僵尸村民、僵尸、僵尸猪灵	20
				沼骸、焦骸、猪灵、蜘蛛	16
				鹦鹉螺、骷髅马、僵尸鹦鹉螺	15
				美西螈、嗅探兽、恼鬼	14
				狢狢、洞穴蜘蛛、铜傀儡	142
				蜜蜂、猫、牛、海豚、狐狸、青蛙、恶魂、发光鱿鱼、山羊、哞菇、豹猫、体弱熊猫、猪、鱿鱼	10
				杀手兔、末影螨、绵羊、蠹虫、野生狼	8
				蝙蝠、鹦鹉、蝌蚪	6
				鸡、雪傀儡	4

(续表)

属性名称	ID	值域	默认值	适用生物	基值
				鳕鱼、河豚、兔子、鲑鱼、热带鱼	3
				嘎吱	1
挖掘效率	mining_efficiency	0 ~ 1024	0	玩家	0
移动效率	movement_efficiency	0 ~ 1	0	所有生物	0
速度 ^①	movement_speed	0 ~ 1024	0.7	海豚	1.2
				美西螈、青蛙、蝌蚪	1
				蝙蝠、鳕鱼、末影龙、恶魂、发光鱿鱼、幻翼、河豚、鲑鱼、潜影贝、鱿鱼、热带鱼、恼鬼、流浪商人	0.7
				旋风人、凋灵	0.6
				唤魔者、巨人、守卫者、幻术师	0.5
				嘎吱	0.4
				猪灵、猪灵蛮兵、掠夺者、卫道士	0.35
				蜜蜂、猫、洞穴蜘蛛、远古守卫者、末影人、狐狸、疣猪兽、豹猫、兔子、蜘蛛、监守者、狼、僵尸疣猪兽、劫掠兽	0.3
				鸡、苦力怕、末影螨、铁傀儡、猪、北极熊、蠹虫、骷髅、流浪者、沼骸、海龟、女巫、凋灵骷髅	0.25
				烈焰人、溺尸、尸壳、绵羊、僵	0.23

① 设该值为 a ，生物脚下的方块阻力为 f ，则最高移动速度为 $v = 0.21168 \frac{a}{1 - 0.91f} f^3$ 。

(续表)

属性名称	ID	值域	默认值	适用生物	基值
				尸、僵尸村民、僵尸猪灵	
				马	$U[0.1125, 0.3375]$
				史莱姆	$0.2 + 0.1 \times \text{尺寸}$
				铜傀儡、牛、山羊、岩浆怪、哱菇、鹦鹉、骷髅马、僵尸马	0.2
				驴、羊驼、骡、炽足兽、行商羊驼	0.175
				非懒惰的熊猫	0.15
				玃狢	0.14
				悦灵、嗅探兽、玩家	0.1
				骆驼	0.09
				懒惰的熊猫	0.07
				快乐恶魂	0.05
额外氧气 ^①	oxygen_bonus	0 ~ 1024	0	所有生物	0
安全摔落高度	safe_fall_distance	-1024 ~ 1024	3	骆驼、驴、马、羊驼、骡、骷髅马、行商羊驼、僵尸马	6
				狐狸	5
				其他所有生物	3
尺寸	scale	0.0625 ~ 16	1	所有生物	1
潜行速度	sneaking_speed	0 ~ 1	0.3	玩家	0.3
僵尸增援	spawn_reinforcements	0 ~ 1	0	僵尸、尸壳、溺尸、僵尸村民	$U[0, 0.1]$
				僵尸猪灵	0
最大行走高度	step_height	0 ~ 10	0.6	骆驼	1.5
				嘎吱	1.0625
				美西螈、铜傀儡、驴、溺尸、末影人、青蛙、	1

① 设该属性值为 o ，若 $o = 0$ ，则生物在水中每游戏刻氧气值减 1；若 $o > 0$ ，则生物每游戏刻有 $\frac{o}{o+1}$ 的概率不消耗氧气值。

(续表)

属性名称	ID	值域	默认值	适用生物	基值
				马、铁傀儡、羊驼、骡、劫掠兽、骷髅马、行商羊驼、海龟、僵尸马	
				盔甲架	0
				其他所有生物	0.6
水下挖掘速度	submerged_mining_speed	0 ~ 20	0.2	玩家	0.2
横扫伤害比率 ^①	sweeping_damage_ratio	0 ~ 1	0	玩家	0
生物引诱范围	tempt_range	0 ~ 2048	10	玃狢、美西螈、蜜蜂、鸡、狐狸、青蛙、山羊、疣猪兽、豹猫、熊猫、猪、北极熊、兔子、绵羊、嗅探兽、炽足兽、海龟、骆驼、马、骷髅马、僵尸马、驴、骡、羊驼、行商羊驼、牛、哞菇、猫、狼、鸮鹉	10
水中移动效率	water_movement_efficiency	0 ~ 1	0	所有生物	0
路径点传输距离	waypoint_transmit_range	0 ~ 60000000	0	玩家	60000000
				其他生物	0
路径点接收距离	waypoint_receive_range	0 ~ 60000000	0	玩家	60000000

6.7.1 属性修饰符

在一个属性上可以使用多个属性修饰符，则属性的最终值为基值和这些属性修饰符共同修饰后的结果。在数据的存储上，每一个修饰符都需要作区分，用命名空间 ID 唯一地标识每个属性修饰符，充当修饰符“身份证”的作用。不同修饰符中的不同属性允许使用相同的 ID。

一个属性修饰符在对基值进行修饰时，会采用一定的**运算模式（Operation）**。修饰符一共有三种运算模式：**属性增量（Add value）**、**倍率增量（Add multiplied base）**和**最终倍乘（Add multiplied total）**，每一个修饰符只会采用其中一种运算方式。

① 设该值为 r ，当 $r = 0$ 时，横扫攻击伤害为 1；当 $r > 0$ 时，横扫攻击伤害为 $1 + rd$ ，其中 d 为近战攻击伤害。

一、属性增量

属性增量即在基值的基础上进行加减运算。若一个属性同时存在多个运算模式为属性增量的修饰符, 则属性的最终值为基值与这些修饰符值得代数和。设 x_1 、 x_2 、 $\cdots$ 、 x_n 是运算模式为属性增量的属性修饰符 M_1 、 M_2 、 $\cdots$ 、 M_n 的修饰量, 而被修饰的属性基值为常数 B , 则该属性的最终值 $f(x_1, x_2, \cdots, x_n)$ 可以表示为

$$f(x_1, x_2, \cdots, x_n) = B + \sum_{i=1}^n x_i \quad (6.7)$$

例 6.21

用两个属性增量的修饰符修饰同一属性, 已知属性基值为 1, 修饰量分别为 2、3, 求属性的最终值。

解

属性的最终值为 $f = 1 + (2 + 3) = 6$ 。

二、倍率增量

倍率增量在属性增量修饰完成后进行, 用于将属性增量修饰完成后的属性值乘以一定倍率。对于同一个属性中多个倍率增量的修饰符, 修饰后的值为倍率增量的修饰量代数和与 1 的和与属性增量修饰完成后属性值的乘积, 设 y_1 、 y_2 、 $\cdots$ 、 y_m 是倍率增量的修饰符 M_1 、 M_2 、 $\cdots$ 、 M_m 的修饰量, 而 $f(x_1, x_2, \cdots, x_n)$ 为属性增量的修饰符修饰完成后的属性值, 则该属性的最终值 $g(y_1, y_2, \cdots, y_m)$ 可以表示为

$$g(y_1, y_2, \cdots, y_m) = f(x_1, x_2, \cdots, x_n) \left(1 + \sum_{i=1}^m y_i \right) \quad (6.8)$$

将式 (6.7) 代入, 得

$$g(y_1, y_2, \cdots, y_m) = \left(B + \sum_{i=1}^n x_i \right) \left(1 + \sum_{i=1}^m y_i \right) \quad (6.9)$$

例 6.22

在例 6.21 的基础上, 追加两个修饰量分别为 2、4 的倍率增量修饰符, 求属性的最终值。

解

属性的最终值为 $g = 6 \times (1 + 2 + 4) = 42$ 。

三、最终倍乘

最终倍乘在所有的属性增量和倍率增量修饰符修饰完成后进行。最终倍乘的意义为: 将属性增量和倍率增量修饰符修饰完成后的属性值增加一定的倍数。若一个属性存在一个最终倍乘的修饰符, 记属性增量和倍率增量修饰完成后的属性值为 g , 而该最终倍乘的修饰量为 z , 则属性的最终值 $h(z)$ 可以表示为

$$h(z) = g \cdot (z + 1) \quad (6.10)$$

可以看到, 对原值 g 倍乘的倍率实际上为修饰量加 1 后的值。若存在多个最终倍乘的修饰符, 则在上一个最终倍乘修饰完毕后, 在当前属性值的基础上再进行倍乘。设 z_1 、 z_2 、 $\cdots$ 、 z_p 是倍率增量的修饰符 M_1 、 M_2 、 $\cdots$ 、 M_p 的修饰量, 则属性的最终值 $h(z_1, z_2, \cdots, z_p)$ 可以表示为

$$h(z_1, z_2, \cdots, z_p) = g \cdot \prod_{i=1}^p (z_i + 1) \quad (6.11)$$

例 6.23

在例 6.22 的基础上, 追加两个修饰量分别为 2、3 的最终倍乘修饰符, 求属性的最终值。

解

属性的最终值为 $h = 42 \times (2 + 1) \times (3 + 1) = 504$ 。

四、修饰符公式

修饰符进行修饰运算时, 遵循“先属性增量, 再倍率增量, 后最终倍乘”的运算顺序。设 x_1 、 x_2 、 $\cdots$ 、 x_n 是属性增量的修饰量, y_1 、 y_2 、 $\cdots$ 、 y_m 是倍率增量的修饰量, z_1 、 z_2 、 $\cdots$ 、 z_p 是最终倍乘的修饰量, 将式 (6.9) 和式 (6.11) 结合, 可得属性最终值的多元函数:

$$\varphi = \varphi(x_1, x_2, \cdots, x_n, y_1, y_2, \cdots, y_m, z_1, z_2, \cdots, z_p) \quad (6.12)$$

通常记 Op0 为所有属性增量修饰值的代数和, Op1 为所有倍率增量修饰值的代数和, Op2 为所有最终倍率修饰值加 1 后的乘积, 即:

$$\text{Op0} = \sum_{i=1}^n x_i \quad (6.13)$$

$$\text{Op1} = \sum_{i=1}^m y_i \quad (6.14)$$

$$\text{Op2} = \prod_{i=1}^p (z_i + 1) \quad (6.15)$$

于是式 (6.12) 又可以写成如下的形式, 即**修饰符一般公式**:

$$\varphi = (B + \text{Op0})(\text{Op1} + 1)\text{Op2} \quad (6.16)$$

式中: φ ——属性经过修饰后的最终值。

B ——属性的基值。

例 6.24

一个属性的基值为 0.3, 已知该属性拥有修饰量为 0.2 的一个属性增量修饰符, 修饰量为 5 的一个倍率增量修饰符和两个修饰量分别为 1 和 4 的最终倍乘修饰符, 求该属性的最终值。

解

根据题意，不难得出 $Op0 = 0.2$ ， $Op1 = 5$ ， $Op2 = (1 + 1) \times (4 + 1) = 10$ ，属性基值 $B = 0.3$ ，则根据式 (6.16)，计算得到属性的最终值为 $\varphi = (0.3 + 0.2) \times (5 + 1) \times 10 = 30$ 。

6.7.2 命令/attribute 的语法

命令 `/attribute` 是用于查询、修改属性的命令，玩家的属性也是可以被修改的。它需要的权限等级为 2，以下是所有用法：

1. 返回目标实体指定属性的基值或最终值，语法为

```
attribute <target> <attribute> [base] get [<scale>]
```

6.71

其中的参数：`<targets>`（实体 `minecraft:entity`）——指定的目标实体，必须为玩家名称、UUID 或目标选择器。

`<attribute>`（命名空间 ID `minecraft:resource_location`）——指定属性的命名空间 ID。

`[base]`——可选，若填写该参数，则返回属性的基值，不填写则返回最终值。

`[<scale>]`（双精度浮点数 `brigadier:double`）——可选，定义返回的属性基值或最终值的缩放倍率。

2. 修改目标实体属性的基值，语法为

```
attribute <target> <attribute> base set <value>
```

6.72

其中的参数：`<value>`（双精度浮点数 `brigadier:double`）——指定的基值。

3. 重置目标实体属性的默认值，语法为

```
attribute <target> <attribute> base reset
```

6.73

4. 为目标实体的指定属性添加一个属性修饰符，语法为

```
attribute <target> <attribute> modifier add <id> <value> (add_value|
add_multiplied_total|add_multiplied_base)
```

6.74

其中的参数：`<value>`（双精度浮点数 `brigadier:double`）——修饰符的修饰量。

`(add_value|add_multiplied_total|add_multiplied_base)`——运算模式，分别为：

`add_value` 属性增量

`add_multiplied_total` 倍率增量

`add_multiplied_base` 最终倍乘

5. 删除特定 UUID 的属性修饰符，语法为

```
attribute <target> <attribute> modifier remove <id>
```

6.75

6. 返回指定 UUID 修饰符的修饰量，语法为

```
attribute <target> <attribute> modifier value get <id> [<scale>]
```

6.76

例 6.25

将当前玩家的最大生命值设置为 40。

解

如果直接修改最大生命值属性的基值，则命令可以为

```
attribute @s max_health base set 40
```

6.77

如果使用修饰符，则可以用修饰量为 20 的属性增量修饰基值，命令为

```
attribute @s max_health modifier add minecraft:health_boost 20 add
```

6.78

其中命名空间 ID 可以自由指定。

6.7.3 属性 NBT 格式

属性是实体数据的一部分，属性 NBT 隶属于实体格式。属性修饰符也可以存在于物品堆叠组件中，当实体持有这些有修饰符的物品时，即对实体的属性值进行修饰。当属性存在于实体数据时，使用标签 **I** `attributes`，其数据结构为

I 根标签

I `attributes`: 该实体的属性列表。

I 一个属性。

D `base`: 该属性的基值。

" `id`: 该属性的名称，使用命名空间 ID。

I `modifiers`: 该实体的属性列表。

I 一个属性修饰符。

D `amount`: 该修饰符的修饰量。

" `id`: 该修饰符的命名空间 ID。

" `operation`: 运算模式，可用值为 `add_value`（属性增量）、`add_multiplied_total`（速率增量）和 `add_multiplied_base`（最终倍乘）。

例 6.26

生成一个一次普通攻击会造成 10 点伤害、最大生命值为 100 的僵尸。

解

下面的命令直接修改了基值：

```
summon ~ ~ ~ zombie {attributes:
  [{base:10.0d,id:"minecraft:attack_damage"},
  {base:100.0d,id:"minecraft:max_health"}]}
```

6.79

6.8 技术性实体

命令系统处理的实体一般可分为三类：一是**标记实体**、二是**展示用实体**、三是**可交互实体**。

相对于直接作用对象实体而言，搭建系统时通常需要处理另外一些实体，这些实体不直接参与游戏进程，主要充当游戏后台的角色，作为命令运行的辅助工具，对游戏过程产生间接的影响。例如，当一个玩家经过某一特定位置，现在需要对这个位置做一个标记以方便该玩家此后某一时间通过命令回到该位置，这时可以使用到标记实体，当玩家第一次经过该点时在该位置放置一个标记实体，当玩家需要回到该位置时，就可以使用命令 `/tp` 将玩家传送至该标记实体。这些标记实体的其主要作用是辅助命令的执行。由于无法直接通过标记静态的方块来实现这一功能，因此标记实体的存在极为必要。**为了尽量规避这些实体做出多余的行为以干扰到游戏的正常运行，一般选择行动较少的、对游戏本身的影响较少的实体。**

展示用实体，顾名思义，就是用于展示图像、文字之类的实体。由于它们的作用主要体现在视觉方面，因此也需要有尽量少的干扰行为。

可以直接与之发生交互的、对游戏进程有着直接影响的实体，比如玩家、与玩家发生互动的生物等，可称之为“可交互实体”，这类实体一般拥有特定的实体边界框。

对于这三类实体，Minecraft 各有一种专门适用于命令系统的实体，即标记、展示实体和交互实体。**盔甲架 (Armor stand)** 作为一种传统的兼具标记、展示和交互功能的实体，在这些技术性实体未加入之前被广泛使用，其主要特点是显形，可以直观地显示标记实体的状态。但是随着标记、展示实体和交互实体的加入，盔甲架基本上不再参与到这几项任务中，本教程不再讲述盔甲架的使用。

6.8.1 标记

于 21w15a 加入的**标记 (Marker)** 是完全无行为、不会产生更新、没有碰撞箱、不会干扰玩家的一种隐形实体，相对于盔甲架而言对游戏的多余影响更小。标记**只存在于服务端，不在客户端中渲染**，只能使用命令检查标记的存在。

标记只能通过命令 `/summon` 生成：

```
summon minecraft:marker
```

6.80

例 6.27

生成一个标记，使其 `{data}` 标签拥有一个子标签 `marker:true`。

解

自定义标签是 `{data}` 的子标签，注意标签 `{data}` 不能遗漏。

```
summon minecraft:marker ~ ~ ~ {data:{marker:true}}
```

6.81

如此标记的数据树为：

`{}` 根标签

```

{ data
  TF marker: true

```

6.8.2 展示实体

展示实体 (Display) 是一类用于展示内容的实体, 包括**物品展示实体 (Item display)**、**方块展示实体 (Block display)** 和**文本展示实体 (Text display)** 三种。这些展示实体没有碰撞箱, 没有任何自主行为, 只能通过命令 `/summon` 生成。在生成时如果不指定 NBT, 则不会显示任何内容。

一、展示实体共通标签

下面这些字段是三种展示实体共同拥有的, 即**展示实体共通标签 (Tags common to all display entities)**。它们指定了展示内容的形式、渐变动画和插值。

根标签

billboard: 展示内容跟随玩家视角的变换形式。有效值 `fixed` (固定不发生旋转)、`vertical` (跟随玩家视角在水平方向上转动)、`horizontal` (跟随玩家视角在竖直方向上转动) 和 `center` (跟随玩家视角绕中心自由转动)。

brightness: 该实体渲染使用的亮度值, 若不存在则使用当前位置使用的亮度值。一个位置的亮度值由**天空光照 (Sky light)** 和**方块光照 (Block light)** 决定。

block: 方块光照等级, 有效值 `0 ~ 15`。

sky: 天空光照等级, 有效值 `0 ~ 15`。

glow_color_override: 覆盖发光边框的颜色, 使用 RGB 格式, 对二进制来说从高到底依次是: R 通道 8 位、G 通道 8 位、B 通道 8 位。若设为 `-1`, 则使用该实体所在队伍的颜色。默认为 `-1`。只有当 `TF Glowing` 的值为 `true` 时才会显示发光边框。

height: 定义**剔除边界框 (Rendering culling bounding box)** 的高度, 剔除边界框的作用是, 当剔除边界框不在玩家的视角范围内, 则该实体不在该玩家的客户端渲染。若 `F height` 值定义为 h , 则剔除边界框自实体位置 (使用实际坐标) 向上延伸 h , 如图 6.11 所示。

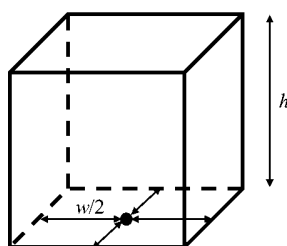


图 6.11 剔除边界框的范围

width: 定义剔除边界框的宽度。若 `F width` 值定义为 w , 则剔除边界框自实体位置 (使用实际坐标) 横向扩展 $\frac{w}{2}$, 如图 6.11 所示。`F height` 和 `F width` 默认值均为 `0.0f`, 即不使用剔除边界框, 只要实体位于玩家视角范围内即渲染。

I start_interpolation: 此**插值 (Interpolation)** 方式用于制造渲染上的渐变动画而非跳跃式的突变, 由该字段定义插值开始的时间, 插值会在定义后第 `start_interpolation` 刻开始。单位为游戏刻, 使用游戏内时间。若设为 `-1`, 则使用当前游戏时间。**制作插值动画时此标签是必须的。**

I interpolation_duration: 实体动画渲染意义上的插值持续的时长, 单位为游戏刻。插值会在第 **I** `start_interpolation` + **I** `interpolation_duration` 游戏刻终止。在插值过程中, 所有标记为可插值的字段都作为单个**插值集 (Interpolation set)** 的一部分。当插值集内的任意值被修改时, 修改后可插值字段的**所有值**都会被视作**当前值**, 修改前的值被视作**先前值**。在插值的持续时间内, **实体会由先前值定义的外观过渡至当前值定义的外观**。若同一游戏刻内出现值的多次变更, 则只会计算一次变更。下面标注了所有的可插值字段。

F shadow_strength: 可插值, 控制实体阴影的透明度, 默认值为 `1.0f`。

I teleport_duration: 应用于实体本身位置和朝向的插值。例如, 若更改了某展示实体的位置 (如使用 `/tp` 命令), 定义 **I** `teleport_duration` 插值可制造展示实体从原本位置移动到新位置的动画而非突然的传送。若该标签的值设为 `0`, 则为突然移动。大于 `0` 的值可制造移动动画。此值必须介于 `0` 和 `59` 之间 (含)。

I transformation: 可插值。此字段数据结构较为复杂, 用于表示模型的渲染变换。变换仅在渲染上有意义, 实体本身的位置和朝向不作变换, 因此只受 **I** `interpolation_duration` 插值作用, 不受 **I** `teleport_duration` 插值的影响。所有渲染变换一律以展示实体的实际位置为变换的原点。

└ 渲染变换的数据

F view_range: 展示实体的最大可视范围。记该值为 v , 在选项中设置的实体渲染距离的最大值为 e , 若玩家与该实体的距离超过 $64ve$, 则该实体不会被渲染。默认值为 `1.0f`。

例 6.28

一张冒险地图需要制作运镜, 要求镜头从 $(0, 70, 0)$ 移动至 $(5, 72, 0)$, 同时视角从面向 $(5, 70, 0)$ 变换至面向 $(5, 70, 5)$, 整个运镜在 2 秒内完成。

解

运镜可以通过展示实体完成, 当玩家旁观展示实体时, 其视角会随着展示实体的视角移动。因此可以生成一个物品展示实体, 初始状态: 位于 $(0, 70, 0)$ 、朝向 $(5, 70, 0)$; 最终状态: 位于 $(5, 72, 0)$ 、朝向 $(5, 70, 5)$ 。这中间的运镜可以通过长达两秒 (40 游戏刻) 的 **I** `teleport_duration` 以实现。

首先生成这个展示实体并设置初始化的状态, 由 $(0, 70, 0)$ 面向 $(5, 70, 0)$ 实际上是水平朝向正东方, 偏航角为 `-90`, 俯仰角为 `0`:

```
summon item_display 0 70 0 {Rotation:
[-90.0f,0.0f],teleport_duration:40}
```

6.82

其次必须保证玩家 (这里用 `@s` 指代这个玩家) 处于旁观模式且旁观这个展示实体, 依次执行:

```
gamemode spectator @s
```

6.83

```
spectate @n[type=item_display,x=0.5,y=70,z=0.5] @s
```

6.84

最后设置终止状态、并立即开始插值，以下两条命令应在同一游戏刻内依次执行：

```
tp @n[type=item_display,x=0.5,y=70,z=0.5] 5 72 0 facing 5 70 5
```

6.85

```
data modify entity @n[type=item_display,x=0.5,y=70,z=0.5]
start_interpolation set value 0
```

6.86

将以上命令按顺序整合进函数内：

```
data > tutorial > function > display_camera.mcfuction
1 summon item_display 0 70 0 {Rotation:[-90.0f,0.0f],teleport_duration:40}
2 gamemode spectator @s
3 spectate @n[type=item_display,x=0.5,y=70,z=0.5] @s
4 tp @n[type=item_display,x=0.5,y=70,z=0.5] 5 72 0 facing 5 70 5
5 data modify entity @n[type=item_display,x=0.5,y=70,z=0.5] start_interpolation set
  value 0
```

玩家在聊天栏中运行此函数，运镜即可生效。

二、各类展示实体特有标签

1. 方块展示实体

方块展示实体用于展示一个方块，被展示的方块仅有渲染意义，不具备具体使用价值。默认的实体锚点位于被展示方块的西北下角，若在 `/summon` 中使用整数形式的坐标，因为命令中坐标参数使用中心点校准，因此生成的方块展示实体不会贴合方格；贴合方格应使用小数坐标。下面是方块展示实体的特有字段：

❏ 根标签

❏ **block_state**

” **Name**: 方块的命名空间 ID。

❏ **Properties**: 可选，由若干方块属性组成的方块状态。

” **<方块属性>**: 标签名为方块状态的属性，值使用字符串表示。

例 6.29

在 (0, 70, 0) 位置贴合方格展示一个开启的、朝向为东的铁门。

解

命令为

```
summon block_display 0.0 70.0 0.0 block_state:
{Name:"minecraft:iron_door",Properties:{open:"true",face:"east"}}
```

6.87

2. 物品展示实体

物品展示用于展示一个物品，同样，被展示的物品仅有渲染意义。被展示的物品可以带有特定的堆叠组件，拥有自定义物品模型的物品也可以被渲染。允许使用 `/item` 或 `/loot` 变更和获取其中的物品数据。下面是物品展示特有的字段：

根标签

item: 一个要展示的物品，使用物品格式，见节 6.10。默认为空气。

└ 无槽位物品格式

item_display: 描述展示物品的方式，有效值：`none`（无变换）、`thirdperson_lefthand`（第三人称视角左手）、`thirdperson_righthand`（第三人称视角右手）、`firstperson_lefthand`（第一人称视角左手）、`firstperson_righthand`（第一人称视角右手）、`head`（放置于头部）、`gui`（GUI 视图）、`ground`（平铺于地面）、`fixed`（默认变换）和 `on_shelf`（在展示架中）。默认值为 `none`。

3. 文本展示实体

顾名思义，文本展示实体用于展示一段文本。下面是文本展示的特有字段：

根标签

alignment: 规定文本对齐方向，可用值有 `center`（居中对齐）、`left`（左对齐）和 `right`（右对齐）。

background: 可插值。文本的背景颜色，使用 ARGB 颜色格式，默认值为 `0x40000000`。渲染时会丢弃 Alpha（透明度）通道小于 10% 的片段，所以当 A 小于 `0x1A` 时背景会完全透明。

default_background: 是否使用默认的文本背景，此项设置会覆盖字段 **background** 的值，默认为 `false`。

line_width: 一行文本的最大宽度，超过此数值即换行，默认值为 `200`。

see_through: 是否能透过方块渲染，默认为 `false`。

shadow: 文本是否渲染阴影，默认为 `false`。

text: 显示的文本内容，使用文本组件。

text_opacity: 可插值，用 -128 与 127 之间的值表示透明度。透明度需要是 0 ~ 255 之间的数，故大于 127 的值一律使用负数替代，换算公式为 真正透明度 = `<text\_opacity> + 256`。例如，完全不透明（255）使用 -1 这个值来表示（`<text\_opacity> = 255 - 256 = -1`）。类似于字段 **background**，透明度小于 26（即通道小于 10%）时会丢弃片段，故文本将不可见。默认值为 `-1b`（完全不透明）。

例 6.30

将最近的玩家的名字存入存储 `tutorial:test` 的字段 **player_name**。

解

玩家的名字可以由实体名称组件得到，经过动态解析之后，文本内容会变成不需要解析的静态类组件。虽然在聊天栏、标题中只能看到解析的结果，但文本展示实体可以存储 SNBT 格式的解析结果。以上结果是由实体名称组件解析而来。因此可以生成一个解析最近玩家名字的文本展示实体：

```
summon minecraft:text_display ~ ~ ~ {text:{selector:"@p"}}
```

6.88

文本展示实体生成完毕后，用

```
data get entity @n[type=minecraft:text_display] text
```

6.89

获取 **text** 的值，其中的内容大致如下所示：


```
{click_event: {action: "suggest_command", command: "/tell Mu_xian "},
insertion: "Mu_xian", text: "Mu_xian", hover_event: {name: "Mu_xian",
action: "show_entity", id: "minecraft:player", uuid: [I; 1699506248,
-677556729, -1141634424, -430286865]}}
```

6.90

可见玩家名字存储于 `text` 中，按以下命令即可获取玩家名字并存入存储：

```
data modify storage tutorial:test player_name set from entity
@n[type=minecraft:text_display] text.text
```

6.91

三、渲染变换

对于展示实体的渲染变换，字段 `transformation` 拥有两种数据形式：**矩阵形式**和**分解形式**。编写 NBT 时可用矩阵形式，但实体被保存时一律使用分解形式。

1. 矩阵形式

使用矩阵形式时，字段 `transformation` 的数据类型为列表：

transformation: 含有十六个元素的仿射变换矩阵。

└ **F** 矩阵中的一个元素。

使用一个 4×4 的矩阵以表示变换方式 A ：

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \quad (6.17)$$

这是一个 **行主序矩阵 (Row-major matrix)**，每一个元素都是浮点数，在 NBT 中写成列表的形式即

小提示

列表中每一个元素后面不要忘记附加 `f` 以表明单精度浮点数的数据类型。

```
[<a11>, <a12>, <a13>, <a14>, <a21>, <a22>, <a23>, <a24>, <a31>, <a32>,
<a33>, <a34>, <a41>, <a42>, <a43>, <a44>]
```

6.92

这个矩阵用于描述一个**仿射变换 (Affine transform)**。为了以矩阵形式表示三维空间中点的变换，将原空间映射至仿射空间。对于三维空间内每一个点 (x_0, y_0, z_0) ，在其尾部添加一个 1 以在仿射空间内表示一个点，即 $(x_0, y_0, z_0, 1)$ 。令该点经过一定仿射变换 A 后位于 $(x', y', z', 1)$ ，则写成矩阵乘法的形式：

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (6.18)$$

基础变换形式有平移、旋转、缩放（镜像）、剪切，**所有的变换均基于实体的实际坐标进行**，下面依次讲解之。

(1) 平移

设展示实体上任意一点 $(x_0, y_0, z_0, 1)$ 在 x 、 y 、 z 轴分别平移 a 、 b 、 c 后得到点 $(x', y', z', 1)$ ，则

$$\begin{cases} x' = x_0 & + a \\ y' = & y_0 & + b \\ z' = & & z_0 + c \\ 1 = & & & 1 \end{cases} \quad (6.19)$$

则平移矩阵 T 为

$$T(a, b, c) = \begin{pmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (6.20)$$

(2) 旋转

一共有三种旋转方式，即绕 x 轴、绕 y 轴和绕 z 轴旋转。旋转矩阵的推导过程在第二章对朝向和局部坐标的推导中已给出，同理可得绕 x 轴旋转 α 的矩阵形式

6.8.3 交互实体

6.9 玩家

6.10 物品堆叠

6.11 数据组件

6.12 教程：NBT Studio 的使用 ✱

第七章 记分板

7.1 队伍与标签

7.2 记分板的基本概念

7.2.1 记分板 NBT 格式

7.3 记分板命令 触发器

7.3.1 触发器

第八章 命令/execute

附录 I 命令方块与红石电路

命令系统程序化运行的载体一般有两种：一是传统的命令方块电路；二是数据包。命令方块电路是具象化、实体化的命令系统构建模式，在命令系统未完善的早期版本，冒险地图作者就已经能够通过纯粹的红石电路搭建整个地图的机关。在命令方块随骇人更新加入游戏后，命令就能借助红石电路程序化执行了。在往后很长一段时间，命令通常被视为红石电路的一部分，社区中的命令玩家通常称呼自己为“CBer”，意味围绕命令方块玩游戏的玩家。而随着数据包的加入和完善，命令系统最终脱离了红石电路，拥有了更程序化的运行载体。

当今主流的原版技术性开发应使用数据包作为开发工具。命令方块因其性能不佳、不便于数据备份和维护基本被淘汰。不过，由于命令方块仍然在某些地方有独特用处，于是本教程依旧保留了命令方块电路的内容。

广义的命令方块电路除了包含核心的几种命令方块元件，还包括结构方块这种较为实用的管理员用品，因此本章还囊括了结构方块的教程。

I.1 红石电路基础

命令方块是一种红石机械元件，放在红石电路中对红石信号做出一定的响应，因此在介绍命令方块电路前，有必要先了解一些红石电路的基础知识。**红石电路（Redstone circuits）是为玩家建造的，可以用于控制或激活其他机械的结构。**

I.1.1 红石信号

红石信号（Redstone signal）是红石电路中由电源元件产生的，能够由传输元件进行传输并使机械元件作出一定响应的信号。红石信号有两种状态：**有信号**和**无信号**，或简单地表示为**1**和**0**。信号从无到有的瞬间被称为**上升沿（Rising edge）**，从有到无的瞬间被称为**下降沿（Falling edge）**。

当一个红石信号经过暂时性的改变而最终回到了起始状态，这种暂时性的改变被称为**脉冲（Pulse）**。若红石信号从无到有，再从有到无，即形成了“0-1-0”过程，则称这个脉冲为**正脉冲（On-pulse，或简称为脉冲）**；若红石信号从有到无，再从无到有，即形成了“1-0-1”过程，则称这个脉冲为**负脉冲（Off-pulse）**。脉冲的长度一般由红石刻度量，且无论持续时间的长短，满足上述定义的过程均可以被称为脉冲。不同红石元件对脉冲长度的响应要求不同，例如，红石灯无法因短于 2 rt 的负脉冲而熄灭，红石比较器无法传导所有短于 1 rt 和大部分等于 1 rt 的脉冲。

红石信号具有**信号强度（Signal strength）**，通常是介于 0 和 15 之间（含）的整数。当处于无信号状态时，强度一般为 0；处于有信号状态时，强度一般为 1 ~ 15。0 到 15 是大部分红石元件可接受的强度范围，但红石比较器却能够处理大于 15 或小于 0 的信号强度。

事实上，任何大于 0 的信号强度不会引起脉冲长度的变化，也不会对机械元件的运作造成影响，更不会影响充能和激活。

I.1.2 充能与激活理论

为了进一步研究红石信号对电路中各方块的作用机制，社区玩家提出了**充能与激活理论**。此理论作出如下定义：

1. 方块在接收到红石信号后，若方块本身作出一定响应，如门打开、红石灯亮、命令方块执行命令等，则称这个方块被**激活**了。
2. 方块在接收到红石信号后，若方块能向所有毗邻方块输出红石信号，则称这个方块被**充能**了，这个方块也被称为**红石导体（Redstone conductor）**，或**充能方块**。并非所有方块都能作为红石导体，仅有部分固体方块能作为红石导体使用，如泥土、石头。^①一个红石导体被多少强度的信号充能，就称该方块有多少**充能等级**。充能又分为强充能和弱充能：
 - (1) 若该红石导体能够激活毗邻的红石粉和其他红石元件，则称这种充能行为为**强充能（Strongly powered）**。
 - (2) 若该红石导体只能激活毗邻其他红石元件，而不能激活红石粉，则称这种充能行为为**弱充能（Weakly powered）**。

^① 注意方块没有透明度的属性，不能以方块的透明度去判断一个方块是否能作为红石导体。

一次充能行为是否为强弱充能与充能等级无关。

注意，上述说法仅作为理论存在，游戏中并不直接存在这种机制，能作为红石导体的方块均没有相关的方块状态。充能行为实际上是游戏中的 NC 更新，与之相关的理论贴合游戏机制，便于理解和分析，至今仍被社区接纳。

I.1.3 红石元件

红石元件（Redstone components），即用于构成红石电路的方块。通常分为以下几类：

一、电源元件

电源元件是一类可以产生红石信号的元件。

1. 红石块

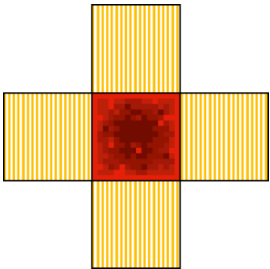


图 I.1 红石块的信号输出^①

作为永久性电源使用，持续输出强度为 15 的信号。红石块共有六个毗邻位置可用于输出信号，**可以激活所有毗邻元件，但无法充能毗邻的红石导体。**

2. 按钮



图 I.2 按钮：木质按钮（左）与石质按钮（右）

可附着于其他方块表面碰撞箱的完整面，用于手动输出脉冲信号。当按钮被打开时，其会向六个毗邻位置输出强度为 15 的信号。不同种类的按钮开启并输出信号的持续时间不同，其中石质按钮的信号持续时间为 10 rt；木质则为 15 rt。**按钮可以激活所有毗邻的元件，同时强充能其依附的方块。**

① 本教程使用的红石图例：表示能够被强充能的位置，表示能够被弱充能的位置，表示只能被激活的位置，表示不会做出任何响应的位置。

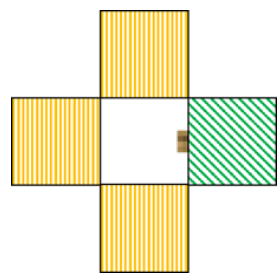


图 I.3 按钮的信号输出

3. 压力板



图 I.4 压力板：从左到右依次为：木质压力板、石质压力板、轻质测重压力板和重质测重压力板

压力板能够用于探测位于其上的实体。当压力板上有实体时，压力板被激活，并向其毗邻的位置输出信号，并将它下方的方块强充能。

不同类型的压力板需要不同条件才能开启——木质压力板可以被除了掉落中的方块和投掷物外的所有实体开启，石质压力板只能由玩家和生物开启，测重压力板可用于探测所有实体。同时，不同的压力板输出的信号强度有所不同：木质和石质压力板输出的信号强度总是为 15。测重压力板会探测位于其上的实体数量，然后决定输出的信号强度，具体情况列于下表：

表 I.1 测重压力板信号强度表

信号强度	需要的实体数量	
	轻质	重质
0	0	0
1	1	1 ~ 10
2	2	11 ~ 20
3	3	21 ~ 30
4	4	31 ~ 40
5	5	41 ~ 50
6	6	51 ~ 60
7	7	61 ~ 70
8	8	71 ~ 80
9	9	81 ~ 90
10	10	91 ~ 100
11	11	101 ~ 110
12	12	111 ~ 120
13	13	121 ~ 130
14	14	131 ~ 140
15	≥ 15	≥ 141

压力板可以激活所有的毗邻元件，同时强充能其下方的毗邻方块。

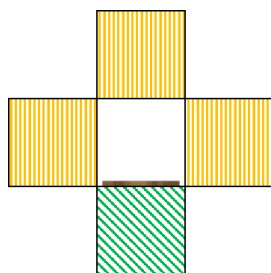


图 I.5 压力板的信号输出

1.2 结构方块

附录 II 数据库

II.1 数据包和资源包版本号

表 II.1 数据包和资源包版本号总表

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
13w41a	-	1	1.15.2	5	5
13w41b	-	1	Combat Test 4	5	5
13w42a	-	1	Combat Test 5	5	5
13w42b	-	1	Snapshot 20w06a	5	5
13w43a	-	1	20w07a	5	5
1.7	-	1	20w08a	5	5
1.7.1	-	1	20w09a	5	5
1.7.2	-	1	20w10a	5	5
13w47a	-	1	20w11a	5	5
13w47b	-	1	20w12a	5	5
13w47c	-	1	20w13a	5	5
13w47d	-	1	20w13b	5	5
13w47e	-	1	20w15a	5	5
13w48a	-	1	20w16a	5	5
13w48b	-	1	20w17a	5	5
13w49a	-	1	20w18a	5	5

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
1.7.3	-	1	20w19a	5	5
1.7.4	-	1	20w20a	5	5
1.7.5	-	1	20w20b	5	5
1.7.6-pre1	-	1	20w21a	5	5
1.7.6-pre2	-	1	20w22a	5	5
1.7.6	-	1	1.16 Pre-release 1	5	5
1.7.7	-	1	1.16 Pre-release 2	5	5
1.7.8	-	1	1.16 Pre-release 3	5	5
1.7.9	-	1	1.16 Pre-release 4	5	5
1.7.10-pre1	-	1	1.16 Pre-release 5	5	5
1.7.10-pre2	-	1	1.16 Pre-release 6	5	5
1.7.10-pre3	-	1	1.16 Pre-release 7	5	5
1.7.10-pre4	-	1	1.16 Pre-release 8	5	5
1.7.10	-	1	1.16 Release Candidate 1	5	5
14w02a	-	1	1.16	5	5
14w02b	-	1	1.16.1	5	5
14w02c	-	1	20w27a	5	5
14w03a	-	1	20w28a	5	5
14w03b	-	1	20w29a	5	5
14w04a	-	1	20w30a	5	5
14w04b	-	1	1.16.2 Pre-release 1	5	5
14w05a	-	1	1.16.2 Pre-release 2	5	5
14w05b	-	1	1.16.2 Pre-release 3	5	5
14w06a	-	1	Combat Test 6	5	5
14w06b	-	1	1.16.2 Release Candidate 1	6	6
14w07a	-	1	1.16.2 Release Candidate 2	6	6
14w08a	-	1	1.16.2	6	6
14w10a	-	1	Combat Test 7	6	6
14w10b	-	1	Combat Test 7b	6	6
14w10c	-	1	Combat Test 7c	6	6
14w11a	-	1	Combat Test 8	6	6
14w11b	-	1	Combat Test 8b	6	6
14w17a	-	1	Combat Test 8c	6	6
14w18a	-	1	1.16.3 Release Candidate 1	6	6
14w18b	-	1	1.16.3	6	6

(续表)

游戏版本名称	数据包版本号	资源包版本号	游戏版本名称	数据包版本号	资源包版本号
14w19a	-	1	1.16.4 Pre-release 1	6	6
14w20a	-	1	1.16.4 Pre-release 2	6	6
14w20b	-	1	1.16.4 Release Candidate 1	6	6
14w21a	-	1	1.16.4	6	6
14w21b	-	1	1.16.5 Release Candidate 1	6	6
14w25a	-	1	1.16.5	6	6
14w25b	-	1	20w45a	6	7
14w26a	-	1	20w46a	7	7
14w26b	-	1	20w48a	7	7
14w26c	-	1	20w49a	7	7
14w27a	-	1	20w51a	7	7
14w27b	-	1	21w03a	7	7
14w28a	-	1	21w05a	7	7
14w28b	-	1	21w05b	7	7
14w29a	-	1	21w06a	7	7
14w29b	-	1	21w07a	7	7
14w30a	-	1	21w08a	7	7
14w30b	-	1	21w08b	7	7
14w30c	-	1	21w10a	7	7
14w31a	-	1	21w11a	7	7
14w32a	-	1	21w13a	7	7
14w32b	-	1	21w14a	7	7
14w32c	-	1	21w15a	7	7
14w32d	-	1	21w16a	7	7
14w33a	-	1	21w17a	7	7
14w33b	-	1	21w18a	7	7
14w33c	-	1	21w19a	7	7
14w34a	-	1	21w20a	7	7
14w34b	-	1	1.17 Pre-release 1	7	7
14w34c	-	1	1.17 Pre-release 2	7	7
14w34d	-	1	1.17 Pre-release 3	7	7
1.8-pre1	-	1	1.17 Pre-release 4	7	7
1.8-pre2	-	1	1.17 Pre-release 5	7	7
1.8-pre3	-	1	1.17 Release Candidate 1	7	7
1.8	-	1	1.17 Release Candidate 2	7	7

(续表)

游戏版本名称	数据包 版本号	资源包 版本号	游戏版本名称	数据包 版本号	资源包 版本号
1.8.1-pre1	-	1	1.17	7	7
1.8.1-pre2	-	1	1.17.1 Pre-release 1	7	7
1.8.1-pre3	-	1	1.17.1 Pre-release 2	7	7
1.8.1-pre4	-	1	1.17.1 Pre-release 3	7	7
1.8.1-pre5	-	1	1.17.1 Release Candidate 1	7	7
1.8.1	-	1	1.17.1 Release Candidate 2	7	7
1.8.2-pre1	-	1	1.17.1	7	7
1.8.2-pre2	-	1	1.18 Experimental Snapshot 1	7	7
1.8.2-pre3	-	1	1.18 experimental snapshot 2	7	7
1.8.2-pre4	-	1	1.18 experimental snapshot 3	7	7
1.8.2-pre5	-	1	1.18 experimental snapshot 4	7	7
1.8.2-pre6	-	1	1.18 experimental snapshot 5	7	7
1.8.2-pre7	-	1	1.18 experimental snapshot 6	7	7
1.8.2	-	1	1.18 experimental snapshot 7	7	7
1.8.3	-	1	21w37a	8	7
1.8.4	-	1	21w38a	8	7
1.8.5	-	1	21w39a	8	8
1.8.6	-	1	21w40a	8	8
1.8.7	-	1	21w41a	8	8
1.8.8	-	1	21w42a	8	8
1.8.9	-	1	21w43a	8	8
15w31a	-	2	21w44a	8	8
15w31b	-	2	1.18 Pre-release 1	8	8
15w31c	-	2	1.18 Pre-release 2	8	8
15w32a	-	2	1.18 Pre-release 3	8	8
15w32b	-	2	1.18 Pre-release 4	8	8
15w32c	-	2	1.18 Pre-release 5	8	8
15w33a	-	2	1.18 Pre-release 6	8	8
15w33b	-	2	1.18 Pre-release 7	8	8
15w33c	-	2	1.18 Pre-release 8	8	8
15w34a	-	2	1.18 Release Candidate 1	8	8
15w34b	-	2	1.18 Release Candidate 2	8	8
15w34c	-	2	1.18 Release Candidate 3	8	8
15w34d	-	2	1.18 Release Candidate 4	8	8
15w35a	-	2	1.18	8	8

(续表)

游戏版本名称	数据包版本号	资源包版本号	游戏版本名称	数据包版本号	资源包版本号
15w35b	-	2	1.18.1 Pre-release 1	8	8
15w35c	-	2	1.18.1 Release Candidate 1	8	8
15w35d	-	2	1.18.1 Release Candidate 2	8	8
15w35e	-	2	1.18.1 Release Candidate 3	8	8
15w36a	-	2	1.18.1	8	8
15w36b	-	2	22w03a	8	8
15w36c	-	2	22w05a	8	8
15w36d	-	2	22w06a	8	8
15w37a	-	2	22w07a	8	8
15w38a	-	2	Deep Dark Experimental Snapshot 1	8	8
15w38b	-	2	1.18.2 Pre-release 1	9	8
15w39a	-	2	1.18.2 Pre-release 2	9	8
15w39b	-	2	1.18.2 Pre-release 3	9	8
15w39c	-	2	1.18.2 Release Candidate 1	9	8
15w40a	-	2	1.18.2	9	8
15w40b	-	2	22w11a	10	9
15w41a	-	2	22w12a	10	9
15w41b	-	2	22w13a	10	9
15w42a	-	2	22w14a	10	9
15w43a	-	2	22w15a	10	9
15w43b	-	2	22w16a	10	9
15w43c	-	2	22w16b	10	9
15w44a	-	2	22w17a	10	9
15w44b	-	2	22w18a	10	9
15w45a	-	2	22w19a	10	9
15w46a	-	2	1.19 Pre-release 1	10	9
15w47a	-	2	1.19 Pre-release 2	10	9
15w47b	-	2	1.19 Pre-release 3	10	9
15w47c	-	2	1.19 Pre-release 4	10	9
15w49a	-	2	1.19 Pre-release 5	10	9
15w49b	-	2	1.19 Release Candidate 1	10	9
15w50a	-	2	1.19 Release Candidate 2	10	9
15w51a	-	2	1.19	10	9
15w51b	-	2	22w24a	10	9

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
16w02a	-	2	1.19.1 Pre-release 1	10	9
16w03a	-	2	1.19.1 Release Candidate 1	10	9
16w04a	-	2	1.19.1 Pre-release 2	10	9
16w05a	-	2	1.19.1 Pre-release 3	10	9
16w05b	-	2	1.19.1 Pre-release 4	10	9
16w06a	-	2	1.19.1 Pre-release 5	10	9
16w07a	-	2	1.19.1 Pre-release 6	10	9
16w07b	-	2	1.19.1 Release Candidate 2	10	9
1.9-pre1	-	2	1.19.1 Release Candidate 3	10	9
1.9-pre2	-	2	1.19.1	10	9
1.9-pre3	-	2	1.19.2 Release Candidate 2	10	9
1.9-pre4	-	2	1.19.2 Release Candidate 3	10	9
1.9	-	2	1.19.2	10	9
1.9.1-pre1	-	2	22w42a	10	11
1.9.1-pre2	-	2	22w43a	10	11
1.9.1-pre3	-	2	22w44a	10	11
1.9.1	-	2	22w45a	10	12
1.9.2	-	2	22w46a	10	12
16w14a	-	2	1.19.3 Pre-release 1	10	12
16w15a	-	2	1.19.3 Pre-release 2	10	12
16w15b	-	2	1.19.3 Pre-release 3	10	12
1.9.3-pre1	-	2	1.19.3 Release Candidate 1	10	12
1.9.3-pre2	-	2	1.19.3 Release Candidate 2	10	12
1.9.3-pre3	-	2	1.19.3 Release Candidate 3	10	12
1.9.3	-	2	1.19.3	10	12
1.9.4	-	2	23w03a	11	12
16w20a	-	2	23w04a	11	12
16w21a	-	2	23w05a	11	12
16w21b	-	2	23w06a	12	12
1.10-pre1	-	2	23w07a	12	12
1.10-pre2	-	2	1.19.4 Pre-release 1	12	13
1.10	-	2	1.19.4 Pre-release 2	12	13
1.10.1	-	2	1.19.4 Pre-release 3	12	13
1.10.2	-	2	1.19.4 Pre-release 4	12	13
16w32a	-	3	1.19.4 Release Candidate 1	12	13

(续表)

游戏版本名称	数据包版本号	资源包版本号	游戏版本名称	数据包版本号	资源包版本号
16w32b	-	3	1.19.4 Release Candidate 2	12	13
16w33a	-	3	1.19.4 Release Candidate 3	12	13
16w35a	-	3	1.19.4	12	13
16w36a	-	3	23w12a	13	13
16w38a	-	3	23w13a	13	13
16w39a	-	3	23w14a	13	14
16w39b	-	3	23w16a	14	14
16w39c	-	3	23w17a	14	15
16w40a	-	3	23w18a	15	15
16w41a	-	3	1.20 Pre-release 1	15	15
16w42a	-	3	1.20 Pre-release 2	15	15
16w43a	-	3	1.20 Pre-release 3	15	15
16w44a	-	3	1.20 Pre-release 4	15	15
1.11-pre1	-	3	1.20 Pre-release 5	15	15
1.11	-	3	1.20 Pre-release 6	15	15
16w50a	-	3	1.20 Pre-release 7	15	15
1.11.1	-	3	1.20 Release Candidate 1	15	15
1.11.2	-	3	1.20	15	15
17w06a	-	3	1.20.1 Release Candidate 1	15	15
17w13a	-	3	1.20.1	15	15
17w13b	-	3	23w31a	16	16
17w14a	-	3	23w32a	17	17
17w15a	-	3	23w33a	17	17
17w16a	-	3	23w35a	17	17
17w16b	-	3	1.20.2 Pre-release 1	18	17
17w17a	-	3	1.20.2 Pre-release 2	18	18
17w17b	-	3	1.20.2 Pre-Release 3	18	18
17w18a	-	3	1.20.2 Pre-Release 4	18	18
17w18b	-	3	1.20.2 Release Candidate 1	18	18
1.12-pre1	-	3	1.20.2 Release Candidate 2	18	18
1.12-pre2	-	3	1.20.2	18	18
1.12-pre3	-	3	23w40a	19	18
1.12-pre4	-	3	23w41a	20	18
1.12-pre5	-	3	23w42a	21	19
1.12-pre6	-	3	23w43a	22	20

(续表)

游戏版本名称	数据包 版本号	资源包 版本号	游戏版本名称	数据包 版本号	资源包 版本号
1.12-pre7	-	3	23w43b	22	20
1.12	-	3	23w44a	23	20
17w31a	-	3	23w45a	24	21
1.12.1-pre1	-	3	23w46a	25	21
1.12.1	-	3	1.20.3 Pre-Release 1	26	22
1.12.2-pre1	-	3	1.20.3 Pre-Release 2	26	22
1.12.2-pre2	-	3	1.20.3 Pre-Release 3	26	22
1.12.2	-	3	1.20.3 Pre-Release 4	26	22
17w43a	-	3	1.20.3 Release Candidate 1	26	22
17w43b	-	3	1.20.3	26	22
17w45a	-	3	1.20.4 Release Candidate 1	26	22
17w45b	-	3	1.20.4	26	22
17w46a	-	3	23w51a	27	22
17w47a	-	3	23w51b	27	22
17w47b	-	3	24w03a	28	24
17w48a	4	4	24w03b	28	24
17w49a	4	4	24w04a	29	24
17w49b	4	4	24w05a	30	25
17w50a	4	4	24w05b	30	25
18w01a	4	4	24w06a	31	26
18w02a	4	4	24w07a	32	26
18w03a	4	4	24w09a	33	28
18w03b	4	4	24w10a	34	28
18w05a	4	4	24w11a	35	29
18w06a	4	4	24w12a	36	30
18w07a	4	4	24w13a	37	31
18w07b	4	4	24w14a	38	31
18w07c	4	4	1.20.5 Pre-Release 1	39	31
18w08a	4	4	1.20.5 Pre-Release 2	40	31
18w08b	4	4	1.20.5 Pre-Release 3	41	31
18w09a	4	4	1.20.5 Pre-Release 4	41	32
18w10a	4	4	1.20.5 Release Candidate 1	41	32
18w10b	4	4	1.20.5 Release Candidate 2	41	32
18w10c	4	4	1.20.5 Release Candidate 3	41	32
18w10d	4	4	1.20.5	41	32

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
18w11a	4	4	1.20.6 Release Candidate 1	41	32
18w14a	4	4	1.20.6	41	32
18w14b	4	4	24w18a	42	33
18w15a	4	4	24w19a	43	33
18w16a	4	4	24w20a	44	33
18w19a	4	4	24w21a	45	34
18w19b	4	4	1.21 Pre-Release 1	46	34
18w20a	4	4	1.21 Pre-Release 2	47	34
18w20b	4	4	1.21 Pre-Release 3	48	34
18w20c	4	4	1.21 Pre-Release 4	48	34
18w21a	4	4	1.21 Release Candidate 1	48	34
18w21b	4	4	1.21	48	34
18w22a	4	4	1.21.1 Release Candidate 1	48	34
18w22b	4	4	1.21.1	48	34
18w22c	4	4	24w33a	49	35
1.13-pre1	4	4	24w34a	50	36
1.13-pre2	4	4	24w35a	51	36
1.13-pre3	4	4	24w36a	52	37
1.13-pre4	4	4	24w37a	53	38
1.13-pre5	4	4	24w38a	54	39
1.13-pre6	4	4	24w39a	55	39
1.13-pre7	4	4	24w40a	56	40
1.13-pre8	4	4	1.21.2 Pre-Release 1	57	41
1.13-pre9	4	4	1.21.2 Pre-Release 2	57	41
1.13-pre10	4	4	1.21.2 Pre-Release 3	57	42
1.13	4	4	1.21.2 Pre-Release 4	57	42
18w30a	4	4	1.21.2 Pre-Release 5	57	42
18w30b	4	4	1.21.2 Release Candidate 1	57	42
18w31a	4	4	1.21.2 Release Candidate 2	57	42
18w32a	4	4	1.21.2	57	42
18w33a	4	4	1.21.3	57	42
1.13.1-pre1	4	4	24w44a	58	43
1.13.1-pre2	4	4	24w45a	59	44
1.13.1	4	4	24w46a	60	45
1.13.2-pre1	4	4	1.21.4 Pre-Release 1	60	46

(续表)

游戏版本名称	数 据 包 版本号	资 源 包 版本号	游戏版本名称	数 据 包 版本号	资 源 包 版本号
1.13.2-pre2	4	4	1.21.4 Pre-Release 2	61	46
1.13.2	4	4	1.21.4 Pre-Release 3	61	46
18w43a	4	4	1.21.4 Release Candidate 1	61	46
18w43b	4	4	1.21.4 Release Candidate 2	61	46
18w43c	4	4	1.21.4 Release Candidate 3	61	46
18w44a	4	4	1.21.4	61	46
18w45a	4	4	25w02a	62	47
18w46a	4	4	25w03a	63	48
18w47a	4	4	25w04a	64	49
18w47b	4	4	25w05a	65	50
18w48a	4	4	25w06a	66	51
18w48b	4	4	25w07a	67	52
18w49a	4	4	25w08a	68	53
18w50a	4	4	25w09a	69	53
19w02a	4	4	25w10a	70	54
19w03a	4	4	1.21.5 Pre-Release 1	70	55
19w03b	4	4	1.21.5 Pre-Release 2	71	55
19w03c	4	4	1.21.5 Pre-Release 3	71	55
19w04a	4	4	1.21.5 Release Candidate 1	71	55
19w04b	4	4	1.21.5 Release Candidate 2	71	55
19w05a	4	4	1.21.5	71	55
19w06a	4	4	25w15a	72	56
19w07a	4	4	25w16a	73	57
19w08a	4	4	25w17a	74	58
19w08b	4	4	25w18a	75	59
19w09a	4	4	25w19a	76	60
19w11a	4	4	25w20a	77	61
19w11b	4	4	25w21a	78	62
19w12a	4	4	1.21.6 Pre-Release 1	79	63
19w12b	4	4	1.21.6 Pre-Release 2	79	63
19w13a	4	4	1.21.6 Pre-Release 3	80	63
19w13b	4	4	1.21.6 Pre-Release 4	80	63
19w14a	4	4	1.21.6 Release Candidate 1	80	63
19w14b	4	4	1.21.6	80	63
1.14 Pre-Release 1	4	4	1.21.7 Release Candidate 1	80	63

(续表)

游戏版本名称	数据包 版本号	资源包 版本号	游戏版本名称	数据包 版本号	资源包 版本号
1.14 Pre-Release 2	4	4	1.21.7 Release Candidate 2	81	64
1.14 Pre-Release 3	4	4	1.21.7	81	64
1.14 Pre-Release 4	4	4	1.21.8 Release Candidate 1	81	64
1.14 Pre-Release 5	4	4	1.21.8	81	64
1.14	4	4	25w31a	82.0	65.0
1.14.1 Pre-Release 1	4	4	25w32a	83.0	65.1
1.14.1 Pre-Release 2	4	4	25w33a	83.1	65.2
1.14.1	4	4	25w34a	84.0	66.0
1.14.2 Pre-Release 1	4	4	25w34b	84.0	66.0
1.14.2 Pre-Release 2	4	4	25w35a	85.0	67.0
1.14.2 Pre-Release 3	4	4	25w36a	86.0	68.0
1.14.2 Pre-Release 4	4	4	25w36b	86.0	68.0
1.14.2	4	4	25w37a	87.0	69.0
1.14.3 Pre-Release 1	4	4	1.21.9 Pre-Release 1	87.1	69.0
1.14.3 Pre-Release 2	4	4	1.21.9 Pre-Release 2	88.0	69.0
1.14.3 Pre-Release 3	4	4	1.21.9 Pre-Release 3	88.0	69.0
1.14.3 Pre-Release 4	4	4	1.21.9 Pre-Release 4	88.0	69.0
1.14.3	4	4	1.21.9 Release Candidate 1	88.0	69.0
1.14.4 Pre-Release 1	4	4	1.21.9	88.0	69.0
1.14.4 Pre-Release 2	4	4	1.21.10 Release Candidate 1	88.0	69.0
1.14.4 Pre-Release 3	4	4	1.21.10	88.0	69.0
1.14.4 Pre-Release 4	4	4	25w41a	89.0	70.0
1.14.4 Pre-Release 5	4	4	25w42a	90.0	70.1
1.14.4 Pre-Release 6	4	4	25w43a	91.0	71.0
1.14.4 Pre-Release 7	4	4	25w44a	92.0	72.0
1.14.4	4	4	25w45a	93.0	73.0
1.14.3 - Combat Test	4	4	25w45a Unobfuscated	93.0	73.0
Combat Test 2	4	4	25w45b	93.1	74.0
Combat Test 3	4	4	25w45b Unobfuscated	93.1	743.0
19w34a	4	4	1.21.11 Pre-Release 1	94.0	75.0
19w35a	4	4	1.21.11 Pre-Release 1 Unobfuscated	94.0	75.0
19w36a	4	4	1.21.11 Pre-Release 2	94.0	75.0
19w37a	4	4	1.21.11 Pre-Release 2 Unobfuscated	94.0	75.0

(续表)

游戏版本名称	数据包 版本号	资源包 版本号	游戏版本名称	数据包 版本号	资源包 版本号
19w38a	4	4	1.21.11 Pre-Release 3	94.0	75.0
19w38b	4	4	1.21.11 Pre-Release 3 Unobfuscated	94.0	75.0
19w39a	4	4	1.21.11 Pre-Release 4	94.1	75.0
19w40a	4	4	1.21.11 Pre-Release 4 Unobfuscated	94.1	75.0
19w41a	4	4	1.21.11 Pre-Release 5	94.1	75.0
19w42a	4	4	1.21.11 Pre-Release 5 Unobfuscated	94.1	75.0
19w44a	4	4	1.21.11 Release Candidate 1	94.1	75.0
19w45a	4	4	1.21.11 Release Candidate 1 Unobfuscated	94.1	75.0
19w45b	4	4	1.21.11 Release Candidate 2	94.1	75.0
19w46a	4	4	1.21.11 Release Candidate 2 Unobfuscated	94.1	75.0
19w46b	4	4	1.21.11 Release Candidate 3	94.1	75.0
1.15 Pre-release 1	5	5	1.21.11 Release Candidate 3 Unobfuscated	94.1	75.0
1.15 Pre-Release 2	5	5	1.21.11	94.1	75.0
1.15 Pre-release 3	5	5	1.21.11 Unobfuscated	94.1	75.0
1.15 Pre-release 4	5	5	26.1-Snapshot-1	95.0	76.0
1.15 Pre-release 5	5	5	26.1-Snapshot-2	96.0	77.0
1.15 Pre-release 6	5	5	26.1-Snapshot-3	97.0	78.0
1.15 Pre-release 7	5	5	26.1-Snapshot-4	97.1	78.1
1.15	5	5	26.1-Snapshot-5	98.0	79.0
1.15.1 Pre-release 1	5	5	26.1-Snapshot-6	99.0	80.0
1.15.1	5	5	26.1-Snapshot-7	99.1	81.0
1.15.2 Pre-Release 1	5	5	26.1-Snapshot-8	99.2	81.1
1.15.2 Pre-release 1	5	5	26.1-Snapshot-9	99.2	81.1

II.2 方块状态

II.3 方块实体数据

II.4 实体数据

II.5 数据包标签

附录 III 索引

III.1 本书命令

A

attribute 264

C

clone 104

B

ban 80
ban-ip 80
banlist 80
bossbar 221

D

data 156
datapack 34
defaultgamemode 72
deop 81
difficulty 72

kill 242

E

effect 252

L

list 83

F

fill 101
forceload 113

H

me 207
msg 208

G

gamemode 72
gamerule 73

O

op 81

H

help 14

P

pardon 80
pardon-ip 81
perf 83
publish 57

J

jfr 32

R

K

random 224
ride 242
rotate 95

kick 83

S

save-all	82
save-off	82
save-on	82
say	208
setblock	100
setidletimeout	82
setworldspawn	105
spawnpoint	105
spectate	72
spreadplayers	107
stop	82
summon	241

T

teammsg	208
teleport	106
tell	208
tellraw	170
tick	59
title	170
tm	208
tp	106
transfer	82

W

w	208
weather	226
whitelist	81
worldborder	108

III.2 专有名词（汉语拼音顺序）

次要版本号 (Minor versions)	39
粗体 (Bold)	187
存档 (Level, 地图, 世界)	214
存档格式 (Level format)	213

安全模式 (Safe mode)	36
按键绑定组件 (Keybind)	177
Anvil 文件 (Anvil file)	227

倍率增量 (Add multiplied base)	261
崩溃 (Crash)	29
崩溃报告 (Crash Report)	29
边界箱 (Boundary box)	69
边界中心 (World border's center)	108
扁平化 (The flattening)	8
标记 (Marker)	266
标签 (Tag)	12, 135
Boss 栏 (Boss bar)	221
布尔值 (Bool)	14, 22, 136

大驼峰命名法 (Upper camel case)	135
单个词 (Single word)	14
单精度浮点数 (Float)	14, 138
当前列表或数组中的复合标签元素 (Compound elements of named list tag)	153
当前列表或数组中的某个元素 (Element of named list or array tag)	152
当前列表或数组中的所有元素 (All elements of named list or array tag)	152
点击事件 (Click event)	193
动作栏 (Action bar)	171
端 (Sides)	56
短整型 (Short)	136
对象 (Object)	23

测试 NBT 标签 (Tsetting NBT Tags)	147
插值 (Interpolation)	268
插值集 (Interpolation set)	268
长整数 (Long)	14
长整型 (Long)	137
长整型数组 (Long array)	139
成功次数 (Success)	20
纯文本组件 (Plain text)	174
词组 (Quotable phrase)	14

二维坐标 (Two-dimensional coordinates) 90

F

翻译标识符 (Translation identifier, 本地化键名) 175

翻译文本组件 (Translated text, 本地化文本组件)	174	红石元件 (Redstone components)	277
方块 (Block)	67		
方块更新 (Block update)	69		
方块光照 (Block light)	267		
方块实体 (Block entity)	68		
方块实体共通标签 (Tags common to all block entities)	236		
方块属性 (Block property)	68	继承 (Inherit)	201
方块展示实体 (Block display)	267	记分板分数组件 (Scoreboard value)	180
方块状态 (Block states)	68	计划刻 (Schedule tick)	67
方块坐标 (Block position)	89	计算等级	62
仿射变换 (Affine transform)	271	加载标签 (Load ticket)	62
封包 (Packet)	56	加载等级 (Load level)	62
服务 (器) 端 (Server)	56	键位标识符 (Keybind identifier)	178
俯仰角 (Pitch)	93	键值对 (Name-value pair)	21
父标签 (Parent tag)	140	节点 (Node)	150
副标题 (Subtitle)	171	结果 (Result)	20
复合标签 (Compound)	140	精灵图组件 (Object)	182
负脉冲 (Off-pulse)	276	JSON (JavaScript Object Notation, JavaScript 对象表示法)	21
		局部坐标 (Local coordinates)	91
		绝对坐标 (Absolute world coordinates)	91

G

格式化代码 (Formatting code)	190
根标签 (Root tag)	141
根复合标签 (Root compound tag)	150
功能开关 (Feature Flag)	45
功能数据包 (Feature datapack)	45
功能元素 (Feature Element)	45
固有注册表 (Built-in registry)	2

K

可写注册表 (Writable registry)	2
刻 (Tick)	58
客户端 (Client)	56
控制台命令 (Console command)	12
盔甲架 (Armor stand)	266

H

哈希值 (Hash value, 散列值)	28
行主序矩阵 (Row-major matrix)	271
红石导体 (Redstone conductor)	276
红石电路 (Redstone circuits)	276
红石刻 (Redstone tick, rt)	67
红石信号 (Redstone signal)	276

L

聊天 (Chat)	207
聊天类型 (Chat type)	207
列表 (List)	140
流体状态 (Fluid states)	68
逻辑端 (Logical sides)	56

逻辑服务端 (Logical server)	56
逻辑客户端 (Logical client)	56



欧几里得距离 (Euclidean distance)	121
-----------------------------	-----

脉冲 (Pulse)	276
MC-CMD	12
每刻毫秒数 (Milliseconds per tick, MSPT)	58
每秒刻度数 (Ticks per second, TPS)	58
秒表 (Stopwatch)	225
命令 (Command)	12
命令存储 (Command storage)	218
命令来源堆叠 (Command source stack)	19
命令历史 (Command history)	15
命令上下文 (Command context)	19
命令源 (Command origin)	19
(赋) 命名空间 ID (Namespaced identifier)	9, 10
命名空间字符串 (Namespaced string)	9
模糊化 (Obfuscated)	187
模拟距离 (Simulation distance)	63
某名称的标签 (Named tag)	151
某名称的复合标签 (Named compound tag)	151
目标区域 (Destination region)	103
目标选择器 (Target selectors)	119
目标选择器参数 (Target selector arguments)	120



排序算法 (Sorting algorithm)	128
判定箱 (Hitbox)	69
偏航角 (Yaw)	93



前置数据包 (Library datapack)	40
强充能 (Strongly powered)	276
切比雪夫距离 (Chebyshev distance)	64
区段 (Chunk section)	61
区块 (Chunk)	61
区块刻 (Chunk tick)	66
区域 (Region)	227
区域文件 (Region file)	227
权限等级 (Permission level)	16



难度 (Difficulty)	72
NBT 路径 (NBT path)	150
NBT 组件 (NBT values)	181
NBT (Named Binary Tags, 二进制命名标签)	135
内置服务器 (Integrated server)	56



弱充能 (Weakly powered)	276
----------------------	-----

S

三维坐标 (Three-dimensional coordinates)	89
删除线 (Strikethrough)	187
上升沿 (Rising edge)	276
蛇形命名法 (Snake case)	11, 135
十六进制 (Hexadecimal)	118
实体 (Entity)	69
实体共通标签 (Tags common to all entities)	243
实体锚点 (Entity anchor)	19
实体名称组件 (Entity names)	181
实验性内容 (Experiments)	45
实验性设置 (Experimental settings)	36
世界边界直径 (World border diameter)	108
视平线 (Eye level)	70
世界指定资源包 (World specific resources)	49
属性 (Attribute)	254
属性基值 (Attribute base)	254
属性修饰符 (Attribute modifiers)	254
属性增量 (Add value)	261
数据包 (Data pack)	33
数据包版本号 (Data pack format)	39
数据包标签 (Tags in data packs)	12, 46
数据树 (Data tree)	140
数值 (Number)	22
数组 (Array)	23
双精度浮点数 (Double)	14, 138
SNBT 操作 (SNBT Operations)	143
SNBT (Stringified NBT, 字符串化的二进制命名标签)	135
随机刻 (Random tick)	66
随机序列 (Random sequences)	223

T

贪婪词组 (Greedy phrase)	14
剔除边界框 (Rendering culling bounding box)	267
天空光照 (Sky light)	267

天气 (Weather)	226
填入事件 (Insertion)	193
调试棒 (Debug stick)	68

U

UUID (Universally Unique Identifier, 通用唯一识别码)	118
---	-----

H

玩家计算标签 (Player simulation ticket)	63
玩家加载标签 (Player loading ticket)	63
微时序 (Microtiming, 微观延迟)	60
文本展示实体 (Text display)	267
文本组件 (Text component)	169
文本组件类型 (Text component content)	169
文本组件样式 (Text component style)	169
物理端 (Physical sides)	56
物理服务端 (Physical server)	58
物理客户端 (Physical client)	56
物品展示实体 (Item display)	267

X

下划线 (Underline)	187
下降沿 (Falling edge)	276
下落的方块 (Falling block)	250
闲置超时 (Idle timeout)	66
相对坐标 (Relative world coordinates)	91
小驼峰命名法 (Lower camel case)	135
斜杠命令 (Slash command)	12
斜体 (Italic)	187
信号强度 (Signal strength)	276
兴趣点 (Point of Interest, POI)	234

悬停事件 (Hover event)	196	状态效果 (Status effect)	251
渲染距离 (Render distance)	63	资源包 (Resource pack)	49
		资源包版本号 (Resource pack format)	51
		资源标识符 (Resource identifier)	9
		资源路径 (Resource location)	9
		子标签 (Children tag)	140
		子区块 (Sub chunk)	61
		子组件 (Text component children)	169
		字段 (Field)	21
盐 (Salt)	225	字符串 (String)	14, 22, 138
译文变量 (Slots)	175	字节型 (Byte)	136
游戏规则 (Game rule)	73	字节型数组 (Byte array)	139
游戏刻 (Game tick, gt)	58	组件解析 (Component resolution)	185
游戏模式 (Game mode)	72	最终倍乘 (Add multiplied total)	261
有连字符的十六进制 (Hyphenated hexadecimal)	118	坐标 (Coordinate)	87
源区域 (Source region)	101		
原始 JSON 文本 (Raw JSON text)	169		
元数据 (Metadata)	36, 50		
运算模式 (Operation)	261		

Z

展示实体 (Display)	267
展示实体共通标签 (Tags common to all display entities)	267
帧率 (Frame per second, FPS)	58
整数 (Integer)	14
整型 (Int)	137
整型数组 (Int array)	118, 139
正脉冲 (On-pulse)	276
执行朝向 (Execution rotation)	19
执行锚点 (Execution anchor)	19
执行上下文 (Execution context)	19
执行维度 (Execution dimension)	19
执行位置 (Execution position)	19
执行者 (Executor)	19
中心校准 (Center correct)	90
主标题 (Screen title)	171
主要版本号 (Major versions)	39
注册表 (Registry)	2
专用服务器 (Dedicated server, 独立服务端)	58
转义字符 (Escape character)	24

III.3 重要方法

H

获取玩家的名字	270
获取现实时间	239

J

检测资源包是否安装并显示不同的文本	177
禁用原版游戏资源	38

S

数据包多版本兼容	42
----------	----

T

用展示实体实现运镜	268
-----------	-----

Z

在聊天栏显示虚假的广播信息	188
---------------	-----

参考文献

- [1] 中文 Minecraft Wiki[EB/OL]. (2026)[2026-01-14]. <https://zh.minecraft.wiki/>.
- [2] Minecraft Wiki[EB/OL]. (2026)[2026-01-25]. https://minecraft.wiki/w/Minecraft_Wiki.
- [3] DREAMY_BLAZE. 如何合并多个版本的数据包? [J/OL]. Feature, 2025, 1(4). <https://cr-019.github.io/datapack-index/feature/archive/202504/2/content.html>.
- [4] DAHESOR. 数据包优化原则以及分析方式简述[J/OL]. Feature, 2025, 1(4). <https://cr-019.github.io/datapack-index/feature/archive/202504/3/content.html>.
- [5] DAHESOR. MC 命令教程“真”从零开始: (十六) 有关 NBT 选择器匹配条件的一切, 大概 [M/OL]. 2024. https://www.bilibili.com/read/cv39434373/?spm_id_from=333.1387.0.0&opus_fallback=1.
- [6] RUHUASIYU. 原版模组入门教程 [M/OL]. 2022. <https://zhangshenxing.github.io/VanillaModTutorial/>.
- [7] 皮革剑. 从 /stopwatch 开始: 与时间检测有关的一些胡思乱想[J/OL]. Feature, 2025, 1(10). <https://cr-019.github.io/datapack-index/feature/archive/202510/5/content.html>.
- [8] 如何获取现实时间? 【命令快闪 #4】 [EB/OL]. (2025)[2025-12-19]. https://www.bilibili.com/video/BV1agq7BFefQ/?spm_id_from=333.337.search-card.all.click&vd_source=322a906315baac6344e85da7ab57e764.
- [9] 展示实体还能这么玩? [EB/OL]. (2025)[2025-02-13]. https://www.bilibili.com/video/BV1VKKFehE9X/?spm_id_from=333.1387.upload.video_card.click&vd_source=322a906315baac6344e85da7ab57e764.
- [10] 轩宇 1725. 为什么不推荐使用命令方块开发 [R/OL]. (2025). <https://cr-019.github.io/datapack-index/resources/%E4%B8%BA%E4%BB%80%E4%B9%88%E4%B8%8D%E6%8E%A8%E8%8D%90%E4%BD%BF%E7%94%A8%E5%91%BD%E4%BB%A4%E6%96%B9%E5%9D%97%E5%BC%80%E5%8F%91.html>.